www.ieeeturkiye.wordpress.com adına yüklenmiş olup toplumları geliştiren bilginin herhangi bir şekilde ulaşılaşılamaz olmasını kabullenemeyen kişi veya kişiler tarafından upload edilmiştir.saygılarımzla...

Herkes İçin

PIC

Programlama

Nursel Ak

Editör: Selçuk Tüzel

ALFA

Alfa Yayınları **1824**
Bilgisayar Dizisi **363**

Herkes İçin PIC Programlama

Nursel Ak
Editör Selçuk Tüzel
Yeni Basımı Yayına Hazırlayan Suat Koyuncu

1. Basım : Eylül 2007
2. Basım : Ekim 2009

ISBN : 978-975-297-926-0

Sertifika No: 10905

Yayıncı ve Genel Yayın Yönetmeni M. Faruk Bayrak
Yayın Yönetmeni ve Editör Rana Gürtuna
Pazarlama ve Satış Müdürü Vedat Bayrak

© 2007, ALFA Basım Yayım Dağıtım Ltd. Şti.

Kitabın tüm yayın hakları Alfa Basım Yayım Dağ. Ltd. Şti.'ne aittir. Yayınevinden yazılı izin alınmadan kısmen ya da tamamen alıntı yapılamaz, hiçbir şekilde kopya edilemez, çoğaltılamaz ve yayımlanamaz.

Baskı ve Cilt
Melisa Matbaacılık
Tel: (212) 674 97 23 Faks: (212) 674 97 29

Alfa Basım Yayım Dağıtım Ltd. Şti.
Ticarethane Sokak No: 53 34110 Cağaloğlu İstanbul, Turkey
Tel: (212) 511 53 03 - 513 87 51 - 512 30 46 Faks: (212) 519 33 00
www.alfakitap.com
info@alfakitap.com

Kızım Bilge'ye...

Yazar Hakkında

Nursel Ak, 1979 İstanbul doğumludur. Ortaöğrenimini Haydarpaşa Anadolu Teknik Lisesi Elektronik Bölümü'nde tamamladıktan sonra Marmara Üniversitesi Teknik Eğitim Fakültesi Elektronik ve Haberleşme Öğretmenliği Bölümü'nden 2000 yılında mezun oldu. Bir yıl İngilizce öğretmenliği yaptıktan sonra 2002 yılında Handan Hayrettin Yelkikanat Anadolu Meslek Lisesi'nde Elektronik öğretmeni olarak göreve başladı. 2004-2008 yılları arasında Elektronik Bölüm Şefliği görevini yürüttü. Halen aynı okulda Elektrik-Elektronik Teknolojileri Alan Şefi olarak görev yapmaktadır. Yazar evli ve bir çocuk sahibidir. Kendisine e-posta ile ulaşmak için `ak_nursel@mynet.com` adresini kullanabilirsiniz.

Önsöz

Bu kitabı yazmaktaki başlıca amacım konu ile ilgilenen herkese PIC programlamanın zor bir iş olmadığını, aksine, ilerledikçe daha zevkli hale gelen bir uğraş olduğunu göstermekti. Özellikle öğrencilere, elektroniği sevdirmek adına bir eser oluşturmak istedim. Sadece teorik bilgilere dayalı bir eğitim anlayışına karşı uygulamalı eğitimi destekleyen bu kitapta en temelden başlayarak PIC programlamayı öğreneceksiniz.

Daha önceden pek çok malzeme gerektiren, kurması uzun zaman alan karmaşık devrelerle gerçekleştirebileceğiniz uygulamaları PIC kullanarak kısa sürede ve kolayca yapabilirsiniz. Örneğin, engellerden kaçan bir robot yaptığınızı düşünün. Engelin yerine göre robotun farklı yerlere hareket etmesi, yani nereye gideceğine karar vermesi gerekir. Mikrodenetleyici kullanmadan bu tür devreleri yapmak çok zordu. Ancak günümüzde Internet'te kısa bir gezinti ile mikrodenetleyicilerle yapılmış milyonlarca robot devresi bulabilirsiniz.

Kitaptaki konuların çoğu birbirine dayalı olduğundan, konuları sırasıyla takip etmenizi, verilen programları iyice kavradıktan sonra benzer programları kendi kendinize yazmaya çalışmanızı tavsiye ederim. Programlama öğrenmenin tek yolu programı kendi kendinize yazmaktır. Uygulama yapma ya da en azından simülasyon programları kullanarak yazdığınız programı deneme şansınız olursa, kitaptan elde edeceğiniz verim maksimum olacaktır.

Kitabımın PIC programlama öğrenmek isteyen herkese faydalı olmasını dilerim. Çalışmamda benden manevi desteklerini esirgemeyen aileme ve Handan Hayrettin Yelkikanat Teknik ve Endüstri Meslek Lisesi Elektronik Bölümü öğretmenlerine teşekkür ederim.

Nursel AK

Handan Hayrettin Yelkikanat Teknik ve Endüstri Meslek Lisesi
Elektrik-Elektronik Teknolojileri Alan Şefi

İçindekiler

Bölüm 1: Mikrodenetleyiciler 1

Mikroişlemci Nedir? 2
Mikrodenetleyici Nedir? 3
PIC Nasıl Programlanır? 4

Bölüm 2: PIC16F84 Mikrodenetleyici Genel Özellikleri 7

I/O (Giriş-Çıkış) Portları 9
Besleme Devresi 11
Reset Devresi 12
Osilatör Devresi 12
RC Osilatör 13
XT Osilatör 13
PIC16F84'ün Bellek Organizasyonu 15
Program Belleği 15
Veri Belleği 15
Program Sayıcı (Program Counter - PC) 21

Bölüm 3: PIC Assembly Dili 23

Assembly Nedir? 24
Komut Seti 24
Sayıların Yazılış Formatı 40
Açıklamalar 40
Etiketler 41
Adres Atanan Etiketler 41
Adres Atanmayan Etiketler 42
Program Formatı 42
PIC'in Tanıtılması 43
Include Dosyasının Yüklenmesi 43
Portların Giriş-Çıkış Olarak Yönlendirilmesi 46
Çıkış Portunun Temizlenmesi 49
Ana Programın Yazılması 49
Alt Programlar 49
Programın Sonlandırılması 50
Konfigürasyon Bitleri 51
Akış Diyagramları 52
Akış Diyagramı Nedir? 52
Akış Diyagramı Şekilleri 52
Akış Diyagramı Kullanarak Program Yazma 53
Lojik Kapılar 54
VE (AND) Kapısı 54

VEYA (OR) Kapısı 54
ÖZEL VEYA (EXOR) Kapısı 55
Sayı Sistemlerinin Birbirine Çevrilmesi 55
Decimal Sayının Binary Sayıya Çevrimi 55
Binary Sayının Decimal Sayıya Çevrimi 56
Decimal Sayının Hexadecimal Sayıya Çevrimi 56
Hexadecimal Sayının Decimal Sayıya Çevrimi 57
Hexadecimal Sayının Binary Sayıya Çevrimi 57
Binary Sayının Hexadecimal Sayıya Çevrimi 58

Bölüm 4: Portlara Veri Gönderme 59

Sabit Veriyi Register'a Yüklemek 60
Bir Register'daki Bilgiyi Başka Bir Register'a Yüklemek 60

Bölüm 5: Buton Kontrolü 65

Tek Butonlu Uygulamalar 66
Çok Butonlu Uygulamalar 68

Bölüm 6: Sayıcılar 71

Geri Sayıcılar 72
Gecikme Alt Programı 74
İleri Sayıcılar 78
Zero Flag (Sıfır Bayrağı) 81
7-Segment Display 82

Bölüm 7: Portlardan Veri Girişi 89

Bölüm 8: Bit Kaydırma 95

Bölüm 9: Motor Kontrolü 101

Step Motorlar 102
Step Motor Uçlarının Bulunması 102
Uygulanacak Darbe Sırasının Tespiti 103
Step Motor Sürücü Devreleri 103
Step Motorların Sürülmesi 104
Step Motoru Belirlenen Adım Sayısı Kadar Döndürme 112
Step Motoru Belirlenen Tur Sayısı Kadar Döndürme 115
DC Motorlar 118
DC Motor Yön Kontrolü 119
DC Motor Hız Kontrolü 121

Bölüm 10: Aritmetik İşlemler 127

Toplama İşlemi 128
8 Bitlik Toplama 128
16 Bitlik Toplama 130

Çıkarma İşlemi 133
8 Bitlik Çıkarma 133
16 Bitlik Çıkarma 135
Çarpma İşlemi 137

Bölüm 11: Kesmeler 139

Kesme (Interrupt) Nedir? 140
Kesme Kullanılan Programların Yapısı 140
Kesme Kaynakları ve INTCON Register'ı 141
Kesme Türleri 142
RB0/INT Pini Harici Kesmesi (Int Kesmesi) 142
PORTB Değişim Kesmesi 145
EEPROM Kesmesi 146
TMR0 Kesmesi 146
Kesme Sırasındaki Bilgileri Saklamak 146

Bölüm 12: Tuş Takımı Uygulamaları 149

Tarama Metodu 150
Klavye Entegresi Kullanma 157

Bölüm 13: LCD Uygulamaları 161

LCD'nin Yapısı 162
LCD Komutları 163
Ekranı Temizle 164
Kursörü Başa Getir 165
Giriş Modu 166
Ekran On/Off Kontrolü 166
Kursör/Display Kaydırma 167
Fonksiyon Seti 167
CGRAM Adres Ayarı 167
DDRAM Adres Ayarı 167
Meşgul Bayrağını ve Adresi Okuma 169
CGRAM veya DDRAM'e Yazma 169
CGRAM veya DDRAM'den Okuma 169
LCD'nin Kullanıma Hazırlanması 170

Bölüm 14: MPLAB Programı 179

Bölüm 15: ICPROG Programı 187

Bölüm 16: PROTEUS Programı 195

Bölüm 17: Cep Telefonu ile Uzaktan Kontrol 207

Kaynaklar 214

www.ieeeturkiye.wordpress.com adına yüklenmiş olup toplumları geliştiren bilginin herhangi bir şekilde ulaşılaşılamaz olmasını kabullenemeyen kişi veya kişiler tarafından upload edilmiştir.saygılarımzla...

Kitapta Yer Alan Programlar

Program Formatı

- PROGRAM 1: PIC'e enerji verildiği zaman PORTB'nin 0. bitine bağlı ledi yakan program.

Portlara Veri Gönderme

- PROGRAM 2: PORTB'ye bağlı olan tüm ledleri yakan program.
- PROGRAM 3: PORTB'ye bağlı ledleri ●○●○●○●○ şeklinde yakan program.
- PROGRAM 4: PORTB'ye bağlı ledleri sürekli yakıp söndüren program (Flaşör programı)

Buton Kontrolü

- PROGRAM 5: PORTA'nın 1. bitine bağlı butona basınca PORTB'nin 1. bitine bağlı ledi yakan program.
- PROGRAM 6: Turn-off devresine ait program. (A0 butonuna basılınca B0 ledini yakacak, bir süre sonra ledi kendiliğinden söndürecek.)
- PROGRAM 7: PORTA'nın 0. ve 1. bitlerine bağlı butonlara sırasıyla basıldığında PORTB'ye bağlı ledlerin ilk dördünü yakan program.
- PROGRAM 8: PORTA'nın 0. bitine bağlı butona basınca PORTB'deki ledlerin ilk dördünü, 1. bitine bağlı butona basınca son dördünü yakan program.
- PROGRAM 9: PORTA'nın 2. bitine bağlı butona basılıp bırakıldığında PORTB'nin 4. bitine bağlı ledi yakan program.

Sayıcılar

- PROGRAM 10: PORTB'ye bağlı ledler üzerinde binary 9'dan 0'a kadar azaltarak sürekli saydıran program.
- PROGRAM 11: PORTA'nın 0. bitine bağlı butona her basışta PORTB'ye bağlı ledler üzerinde 15'den 0'a kadar sayıp 0'da duran program.
- PROGRAM 12: PORTB'ye bağlı ledleri 5 kez yakıp söndüren program.
- PROGRAM 13: PORTB'ye bağlı ledler üzerinde 0-255 arasında sürekli sayan program.
- PROGRAM 14: PORTB'ye bağlı ledler üzerinde 0'dan 9'a kadar sayıp 9'da duran program.
- PROGRAM 15: Ortak katotlu display'de 3 rakamını yakan program.
- PROGRAM 16: PORTB'ye bağlı 7-segment display üzerinde 0-15 arasında sürekli sayan program.

Portlardan Veri Girişi

- PROGRAM 17: PORTA'dan B '00011001' bilgisi girildiği zaman PORTB'nin 0. bitine bağlı ledi yakan program.
- PROGRAM 18: PORTA'nın 0, 1, 2 ve 3 ve 4 numaralı uçlarına bağlı olan butonlardan girilen sayının değerini PORTB'ye bağlı ledlerde gösteren program.

Bit Kaydırma

- PROGRAM 19: PORTB'ye bağlı ledler üzerinde bir ledin yanışını 0. bitten 7. bite doğru sürekli kaydıran program.
- PROGRAM 20: PORTB'ye bağlı yanık ledler üzerinde bir sönük ledi 7. bitten 0. bite doğru kaydıran, sönük led 0. bite geldiğinde duran program.
- PROGRAM 21: PORTB'ye bağlı ledler üzerinde bir yanık ledi sürekli olarak sağa-sola kaydıran program (Karaşimşek)

Motor Kontrolü

- PROGRAM 22: Bir step motoru sürekli sağa doğru döndüren program.
- PROGRAM 23: A1 butonuna basınca step motoru sürekli sağa, A2 butonuna basınca sürekli sola döndüren, bu arada A4 butonuna basılırsa motoru olduğu yerde durduran program.
- PROGRAM 24: PORTA'ya bağlı step motorun sağa doğru her adım atışında PORTB'ye bağlı display'deki sayıyı 1 artıran, adım sayısı 15 olunca motoru durduran program.
- PROGRAM 25: Step motoru bir tam tur (360°) döndürüp durduran program.
- PROGRAM 26: Bir step motoru önce 5 tur sağa, hemen ardından 2 tur sola döndürüp durduran program.
- PROGRAM 27: Bir DC motoru bir yöne doğru sürekli döndüren, bu arada A1 butonuna basılırsa motoru durduran program.
- PROGRAM 28: A0 butonuna basılınca DC motoru sağa, A1 butonuna basılınca sola döndüren, A2 butonuna basınca motoru durduran program.
- PROGRAM 29: PWM metodu ile DC motorun hızını kontrol eden program.
- PROGRAM 30: Bir DC motoru A1 butonuna basınca yavaş, A2 butonuna basınca ise hızlı döndüren, motor hızlı dönerken 7-segment display'de H harfini, yavaş dönerken ise Y harfini gösteren program.

Aritmetik İşlemler

- PROGRAM 31: H'12' sayısı ile H'AD' sayısını toplayan ve sonucu PORTB'deki ledlerde gösteren program.
- PROGRAM 32: H'5D' sayısı ile H'D0' sayısını toplayıp sonucu PORTB'de gösteren, bu arada A1 butonuna basılırsa elde olup olmadığını gösteren program.
- PROGRAM 33: H'26' sayısı ile PORTA'dan girilecek sayıyı toplayan program.
- PROGRAM 34: H'25C8' ve H'1F69' sayılarını toplayarak sonucun alt byte'ını PORTB'de gösteren, A1 butonuna basıldığında ise üst byte'ını gösteren program.
- PROGRAM 35: H'F2' sayısından H'2A' sayısını çıkarıp sonucu PORTB'de gösteren program.
- PROGRAM 36: H'2A' sayısından H'F2' sayısını çıkarıp sonucu PORTB'de gösteren program.
- PROGRAM 37: H'FF' sayısından PORTA'ya bağlı olan butonlardan girilecek sayıyı çıkaran ve sonucu PORTB'de gösteren program.

- PROGRAM 38: H'F569' sayısından H'23E1' sayısını çıkararak sonucun alt byte'ını PORTB'de gösteren, A1 butonuna basıldığında ise üst byte'ını gösteren program.
- PROGRAM 39: 7 x 14 işlemini yaparak sonucu PORTB'ye bağlı ledlerde gösteren program.

Kesmeler

- PROGRAM 40: RBO/INT ucuna bağlı butona basıldığında PORTB'nin diğer uçlarına bağlı yanık ledleri bir süre söndüren program.
- PROGRAM 41: RB4 ~ RB7 uçlarına bağlı butonlardan bir veya birkaçına basıldığında PORTA'nın ilk iki bitine bağlı ledleri bir süre boyunca yakan program.

Tuş Takımı Uygulamaları

- PROGRAM 42: 4x3 tuş takımından girilen sayıyı PORTA'ya bağlı ledler üzerinde gösteren program.
- PROGRAM 43: 4x3 tuş takımından girilen sayıyı PORTA'ya bağlı ledler üzerinde gösteren program (kısa yoldan yapılışı).
- PROGRAM 44: 4x4 klavyeden girilen sayıyı ledler üzerinde binary olarak gösteren program (74C922 klavye entegresi ve RB0 interrupt kullanılarak).
- PROGRAM 45: 4x4 klavyeden girilen sayıyı ledler üzerinde binary olarak gösteren program (74C922 klavye entegresi – RBHigh interrupt kullanılarak).

LCD Uygulamaları

- PROGRAM 46: 8 bit ile LCD'nin ilk satırına ELEKTRONİK yazan program. (1. YOL)
- PROGRAM 47: 8 bit ile LCD'nin ilk satırına ELEKTRONİK yazan program.(2. YOL)
- PROGRAM 48: 8 bit ile LCD'nin ilk satırının ortasına ELEKTRONİK ikinci satırın ortasına BOLUMU yazan program.

Cep Telefonu ile Uzaktan Kontrol

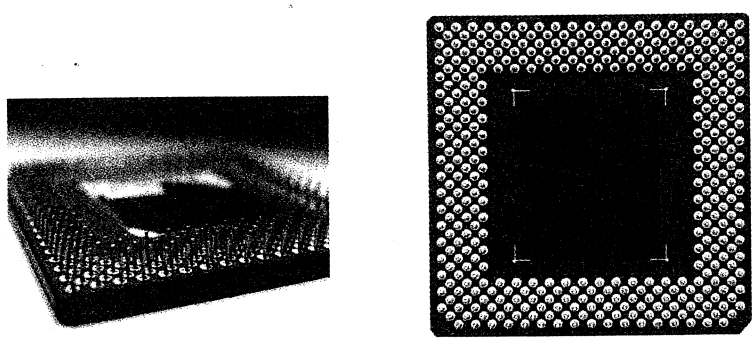
- PROGRAM 49: Cep telefonundan 1 tuşuna basıldığında PORTB'nin 0. bitine bağlı ledi yakan, 7 tuşuna basıldığında ise aynı ledi söndüren program.
- PROGRAM 50: Cep telefonundan 4 tuşuna basıldığında bir DC motoru sola, 6 tuşuna basıldığında sağa doğru döndüren, 0 tuşuna basıldığında ise motoru durduran program.

Bölüm 1

Mikrodenetleyiciler

Mikroişlemci Nedir?

Mikrodenetleyicileri anlayabilmek için önce mikroişlemci kavramını açıklamak yerinde olur. Mikroişlemci, bir bilgisayar sisteminde tüm aritmetik ve lojik işlemleri gerçekleştiren ve yapılan tüm işlemleri denetleyen entegredir. Kısaca, bilgisayarın beynidir de diyebiliriz. Bilgisayarlardan bahsedilirken duyduğumuz Intel Pentium-III, AMD, vs. birer mikroişlemcidir. Farklı kaynaklarda mikroişlemci yerine Merkezi İşlem Birimi, İşlemci, CPU (Central Processing Unit) gibi isimler de kullanılmaktadır.

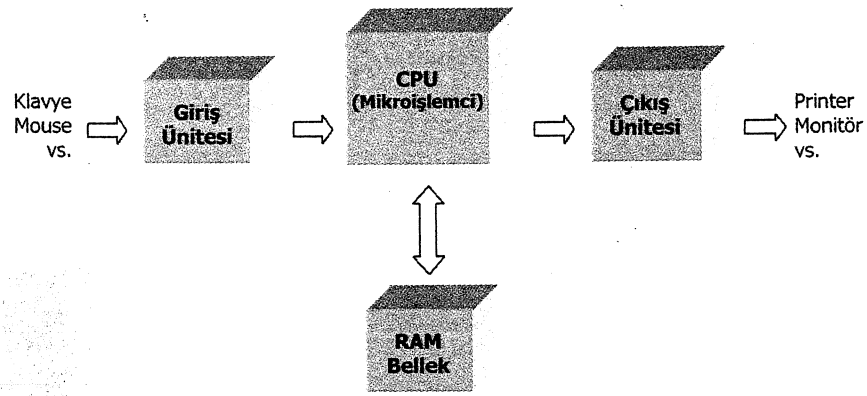


ŞEKİL 1: Bir mikroişlemcinin görünümü.

Bir mikroişlemcinin tek başına herhangi bir sistemi kontrol etmesi mümkün değildir. Tüm mikroişlemciler işlevlerini yerine getirebilmek için aşağıdaki yardımcı elemanlara ihtiyaç duyar:

1. Giriş ünitesi (Input)
2. Çıkış ünitesi (Output)
3. Bellek ünitesi (Memory)

Bu üniteler mikroişlemci entegresinin dışında yer alan harici elektronik elemanlardan veya çiplerden oluşur. Örneğin bir kişisel bilgisayarda anakart üzerinde mikroişlemcinin etrafında bu elemanları görebilirsiniz. Mikroişlemci ile yardımcı üniteler arasındaki iletişim, veri yolu ve adres yolu denilen hatlar üzerinden sağlanır. Demek ki mikroişlemcili bir sistemde bir baskı devre olması da şarttır.

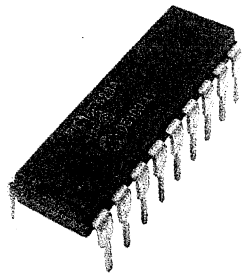


ŞEKİL 2: Bir mikroişlemci sisteminin blok diyagramı.

Mikrodenetleyici Nedir?

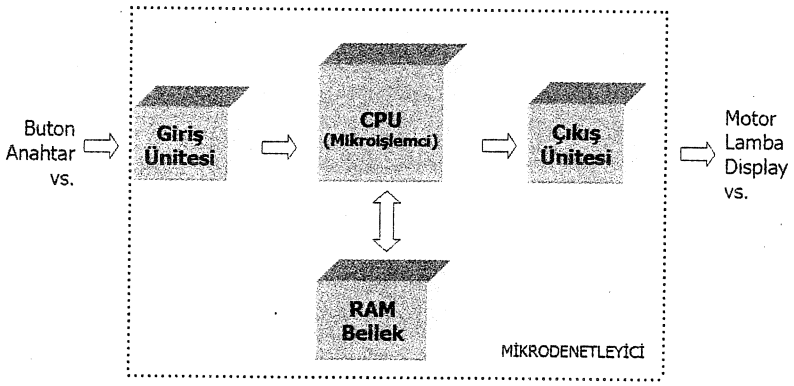
Görüldüğü gibi bir sistemi kontrol etmek için mikroişlemci kullanmak hem karışık bir tasarım aşaması gerektirmekte, hem de maliyeti oldukça yüksek olmaktadır. Bu sebeple kontrol gerektiren elektronik uygulamalarında son zamanlarda mikroişlemcilerin yerine mikrodenetleyiciler tercih edilmektedir.

Mikrodenetleyici, bir mikroişlemci ile yukarıda yardımcı üniteler olarak ifade edilen bellek ve giriş-çıkış (I/O) ünitelerinin tek bir entegre içerisine yerleştirilmiş halidir. Günlük hayatımızda kullandığımız televizyon, çamaşır makinesi gibi cihazlarda, otomobillerde, oyuncaklarda ve daha pek çok alanda mikrodenetleyiciler kullanılmaktadır. Hem az yer kaplaması, hem kullanım kolaylığı, hem de ucuzluğu mikrodenetleyici kullanımının oldukça yaygınlaşmasına neden olmuştur.



ŞEKİL 3: Bir mikrodenetleyicinin görünümü.

Günlük hayatımızda kullandığımız kişisel bilgisayarlar yüklediğimiz programa göre pek çok işlevi yerine getirebilecek olan genel amaçlı bilgisayarlardır. Örneğin, bir PC'de yazı yazabilir, çizim yapabilir veya oyun oynayabilirsiniz. Mikrodenetleyici için ise kısaca **belli bir işe tahsis edilmiş özel amaçlı bilgisayar** diyebiliriz. Yani mikrodenetleyici, yüklediğimiz programa göre sadece tek bir işlevi iyi bir şekilde yerine getirir.



ŞEKİL 4: Bir mikrodenetleyici sisteminin blok diyagramı.

Görüldüğü gibi tek bir entegre içerisinde bir sistemi kontrol etmek için gereken tüm bileşenler mevcuttur. Mikrodenetleyiciye sistemi çalıştıracak programı yükledikten sonra yapılacak tek işlem giriş ve çıkış elemanlarını bağlamaktan ibarettir.

Blok diyagramdaki elemanlardan CPU yazdığımız programı yürütür, RAM bellek programda kullanılacak değişkenleri tutar, giriş ve çıkış üniteleri ise sistemin kullanıcı ile iletişimini sağlar. Yani dış dünyadan sisteme veri girişini ve sistemden dış dünyaya veri aktarılmasını sağlar.

Mikroişlemci üreten pek çok firma mikrodenetleyici de üretmektedir; Intel, Motorola, Microchip gibi. Microchip firması ürettiği mikrodenetleyicileri Çevre Birimleri Denetleyici anlamındaki İngilizce kelimelerin baş harflerinden oluşan PIC (Peripheral Interface Controller) kısaltması ile isimlendirmektedir. PIC16F84, PIC16F877, gibi. Bu kitapta anlatılan PIC16F84 mikrodenetleyicisi Microchip firmasının ürettiği en popüler denetleyicilerden biridir. Temel seviye uygulamalar yapmak ve programlama becerinizi geliştirmek için PIC16F84 entegresi en doğru seçim olarak karşımıza çıkmaktadır.

PIC Nasıl Programlanır?

PIC programlamaya karar verdiğiniz andan programlanmış PIC'in çalışmasını izlediğiniz ana kadar geçmeniz gereken bir dizi aşamalar vardır. Bu aşamalar aşağıdaki şekilde sıralanabilir:

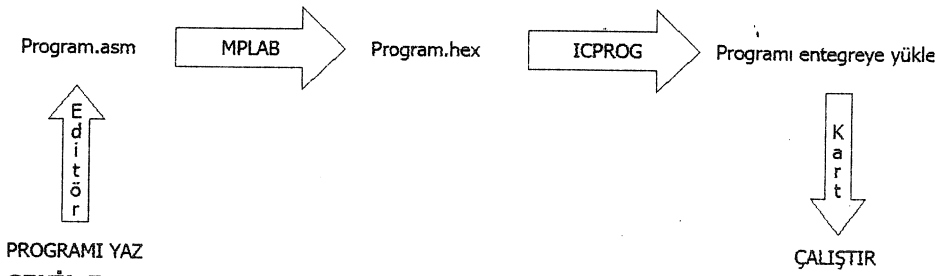
1. Bir text editöründe veya assembler programı içerisinde programı yazmak.
2. Programı assembler (derleyici) ile hexadecimal kodlara dönüştürmek.
3. Yükleme programı ile programlama kartı üzerine yerleştirdiğiniz entegreye programı yüklemek.
4. Programlanmış entegreyi deneme kartı üzerinde denemek.

Bu aşamaların kısa açıklamaları aşağıda verilmiştir:

1. **Programın Yazılması:** Öncelikle programı yazmak için bir metin editörüne ihtiyacımız var. Bunun için Windows içerisindeki Not Defteri'ni kullanabiliriz. Eğer derleme işlemini MPLAB programı ile yapacaksak programı MPLAB içerisinde yazmamız da mümkündür. Yazdığımız programı `.asm` uzantılı olarak kaydetmemiz gerekir.
2. **Programın Derlenmesi:** Programı yazdıktan sonra PIC'in anlayacağı hexadecimal kodlara dönüştürmemiz gerekir. Bu işi yapan programlar derleyici (assembler) olarak adlandırılır. Bu kitapta assembler olarak MPLAB programının kullanılışı anlatılmıştır.
3. **Programın PIC'e Yüklenmesi:** Programı hex kodlara dönüştürdükten sonra entegreye yüklemek için ayrı bir yükleme programına ihtiyacımız olacaktır. Bu kitapta ICPROG programının kullanılışı anlatılmıştır. Programlama işlemi için bir programlama kartına sahip olmanız gerekmektedir.
4. **Programın Denenmesi:** Entegreye programı yükledikten sonra programın çalışıp çalışmadığını bir deneme kartı üzerinde deneyebilirsiniz. Deneme kartı olarak piyasadan hazır kartlar alabileceğiniz gibi, bunlardan çok daha düşük bir maliyetle kendiniz de bir deneme kartı yapabilirsiniz.

Yazdığınız programı PIC'e yükleyip çalışmasını gerçek elektronik elemanlarla denemeden önce simülasyon programları kullanarak da gözlemleyebilirsiniz. Proteus programı bu iş için kullanabileceğimiz güzel bir programdır. Bu program sayesinde programları yazdıkça her defasında entegreye yükleme zahmetinden kurtulacak, sadece Play tuşuna basarak yaptığınız değişikliklerin programınızı nasıl etkileyeceğini görebileceksiniz.

Yukarıdaki dört aşama ile Proteus'un kullanılışı ilerideki bölümlerde ayrıntılı olarak açıklanacaktır. Bu aşamada sadece PIC'in nasıl programlandığı konusunda bir fikir sahibi olmanız amaçlanmıştır.

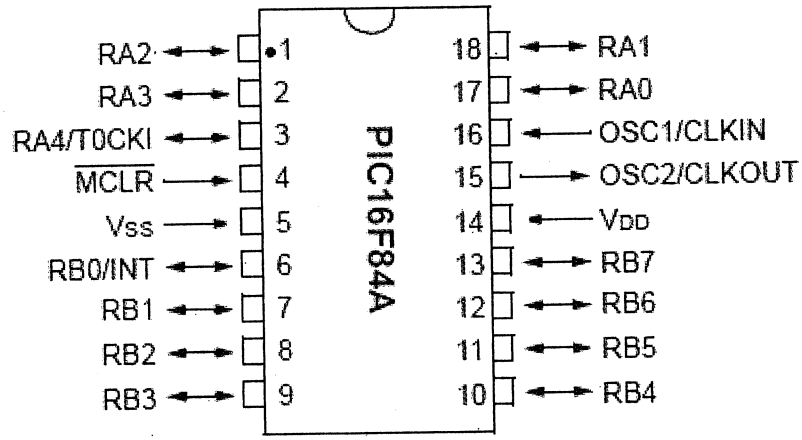


ŞEKİL 5: PIC programlama aşamaları.

Bölüm 2

PIC16F84 Mikrodenetleyici Genel Özellikleri

PIC16F84; 18 pinli, flash program belleğine sahip 8 bitlik bir mikrodenetleyicidir. CMOS teknolojisi ile üretilmiş olup 2.0 V ile 5.5 V arasında geniş bir çalışma aralığı vardır. 5V besleme gerilimi ve 4 MHz osilatör frekansında 2 mA'den daha az akım çeker.



ŞEKİL 6: PIC16F84'ün pin görünüşü.

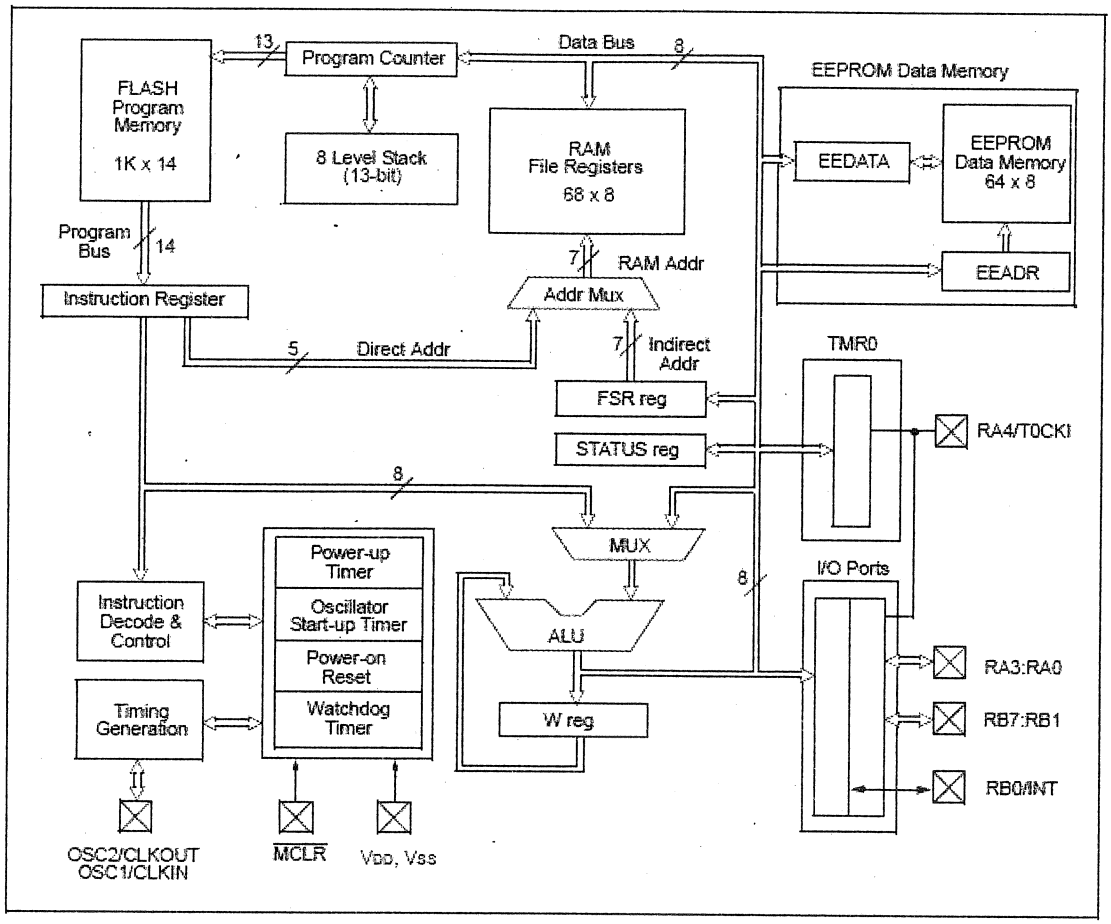
PIC16F84 veri alışverişlerini 8 bit ile yaptığı için 8-bitlik mikrodenetleyici olarak adlandırılır. Bir mikrodenetleyicinin bir defada işleyebileceği en uzun verinin bit sayısına ise kelime uzunluğu (word length) denir. PIC16F84'ün kelime uzunluğu 14 bittir. Şekil 7'deki blok diyagramda hangi birimler arasında kaç bitlik iletişim yapıldığı görülmektedir.

PIC16F84 Harvard mimarisi ile üretilmiş RISC mikrodenetleyiciler sınıfına girer. Harvard mimarisi, adres yolu ve veri yolunun birbirinden ayrı olması, yani CPU'nun daha hızlı çalışması anlamına gelir. Daha önceleri kullanılan Von Neumann mimarisinde hem veri, hem de adres yolu aynı hat olduğu için CPU'nun çalışması daha yavaştı. RISC yani Azaltılmış Komut Seti (Reduced Instruction Set Computer) kavramı ise mikrodenetleyicinin az sayıda komut ile programlanabildiğini ifade eder. Örneğin PIC16F84 denetleyicisi toplam 35 komut ile programlanabilir. CISC (Complex Instruction Set Computer) denetleyiciler ise çok sayıda komutla programlanabilen denetleyicilerdir. Örneğin Motorola ve Intel mikrodenetleyicilerde yüzlerce komut kullanılmaktadır.

PIC16F84'ün genel özelliklerini şu şekilde özetleyebiliriz:

- Sadece 35 komutla programlanabilme
- 1024 byte program belleği (Flash bellek)
- 68 byte veri belleği (RAM bellek)
- 64 byte EEPROM bellek
- 8 bit veri yolu
- 14 bit komut uzunluğu
- 15 özel amaçlı donanım register'ı
- Sekiz seviye derinliğinde stack (yığın) register
- Dört farklı kaynaktan kesme alabilme

- 8 bit zamanlayıcı/sayıcı (TMR0)
- Watchdog Timer (WDT)
- Kod koruması
- SLEEP (düşük enerji) modu



ŞEKİL 7: PIC16F84A blok diyagramı.

NOT **PIC16F84, PIC16F84A ve PIC16C84 arasında hiçbir fark yoktur. Microchip firması farklı yıllarda ürettiği entegrelere farklı isimler vermiştir.**

PIC16F84'ün en önemli özelliği EEPROM (flash) program belleğine sahip olmasıdır. EEPROM (Electrically Erasable Programmable Memory) bellek elektriksel olarak silinebilir ve yeniden programlanabilir (PIC16F84 için yaklaşık 10000 defa). Bu özelliğinden dolayı program geliştirmeye elverişlidir.

I/O (Giriş-Çıkış) Portları

Giriş/çıkış portları mikrodenetleyicinin dış dünyadan veri almasını veya dış dünyaya veri aktarmasını sağlayan pinlerdir. Programın çalışmasına giriş portuna girdiğimiz bilgi ile müda-

hale edebiliriz. Yapılan bir işlemin sonucunu görmek için ise sonuç bilgisinin çıkış portuna atılmış olması gerekir. Portlar da birer register'dır ve PIC16F84'ün Bellek Organizasyonu bölümünde göreceğiniz gibi Bank 0'da yer alırlar. Bir porta bilgi göndermek aslında bir register'a bilgi yüklemekten başka bir şey değildir.

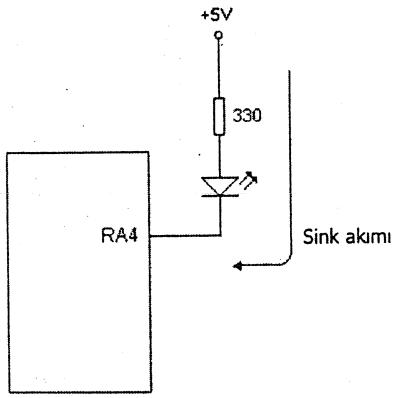
PIC16F84'ün toplam 13 adet giriş-çıkış pini vardır. Bunların RA0~RA4 arasındaki beş tanesine A Portu (PORTA), RB0~RB7 arasındaki sekiz tanesine ise B Portu (PORTB) adı verilir.

	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit0
PORTB	RB7	RB6	RB5	RB4	RB3	RB2	RB1	RB0

	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit0
PORTA	-	-	-	RA4	RA3	RA2	RA1	RA0

ŞEKİL 8: PIC16F84'ün A ve B Portları.

Portların tüm pinleri hem giriş, hem de çıkış olarak yönlendirilebilir. Ancak RA4 pini open drain (açık kanal) özelliğinden dolayı çıkış olarak yönlendirildiğinde mutlaka harici bir pull-up direncine bağlanmalıdır. Bunun anlamı şudur; RA4 çıkış olarak kullanıldığında yük sink akımı yoluyla çalıştırılır. Yani akım beslemeden PIC'e doğru akar. Şekil 9'daki ledin yakılabilmesi için RA4 pinine lojik-0, söndürülebilmesi için ise lojik-1 bilgisi gönderilmelidir. RA4 giriş olarak yönlendirildiğinde diğer pinlerden farkı yoktur.



ŞEKİL 9: RA4 pininin çıkış olarak kullanılması.

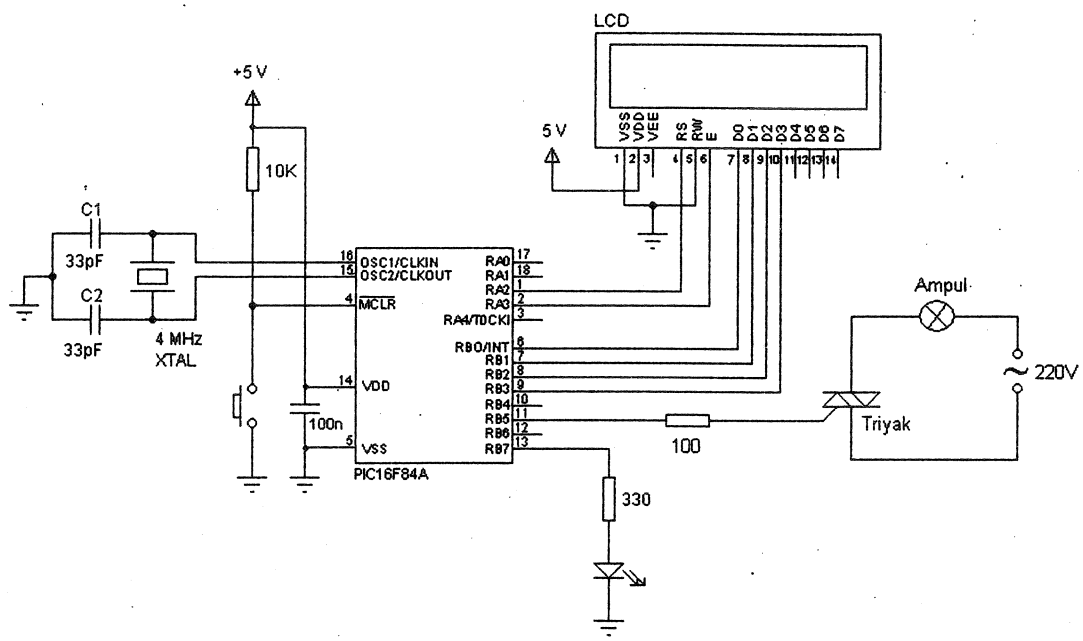
NOT **Sink akımı gerilim kaynağından PIC'in çıkış portuna doğru akan akım, source akımı ise PIC'ten şaseye doğru akan akımdır. Bu kitaptaki uygulamalarda yükler source akımı ile çalıştırılmaktadır. (RA4 pini hiçbir uygulamada çıkış olarak kullanılmamıştır.)**

Örneğin PORTB'nin sekiz pinine de birer led bağlı olduğunu ve bu ledlerden RB1, RB3, RB5 ve RB7 pinlerine bağlı ledleri yakmak, diğerlerini söndürmek istediğimizi düşünelim. Yakacağımız ledlere +5V yani lojik-1, söndüreceğimiz ledlere ise lojik-0 göndermemiz gerekir. Bu durumda PORTB register'ına yüklememiz gereken bilgi şu şekilde olur:

	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
PORTB	1	0	1	0	1	0	1	0

Gönderilen bilgi B'10101010' veya başka bir deyişle H'AA' sayısıdır.

Pinlerin her birinden maksimum 25 mA akım çekilebilir. Bunun anlamı şudur: Çıkışa bağlanan LED, 7-segment display, LCD display gibi elemanlar direkt olarak sürülebilir. Ayrıca bu akımla transistör, triyak gibi elemanlar tetiklenerek motor, lamba, vs. yükler de çalıştırılabilir.



ŞEKİL 10: PIC16F84 ile farklı yüklerin çalıştırılması.

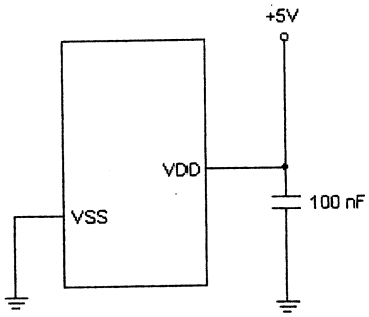
Genelde çıkış portu olarak PORTB, giriş portu olarak ise PORTA kullanılmaktadır. Bu kitaptaki uygulamaların çoğu bu şekildedir. Ancak bu bir zorunluluk değildir.

Şekil 6'da bazı pinlerin çift fonksiyonlu olduğu görülmektedir; RB0/INT, RA4/T0CKI gibi. Pinlerin ikincil fonksiyonları yeri geldikçe anlatılacaktır. Herhangi bir pinin ikincil fonksiyonu aktif hale getirildiğinde artık o pin genel amaçlı bir I/O pini olarak kullanılamaz.

PIC'e enerji verildiğinde PORTA ve PORTB'nin değeri belirsizdir (Tablo 2). Yani pinlerin seviyesi lojik-0 veya lojik-1 olabilir. Bu yüzden tüm programların başında çıkış olarak kullanılan portun temizlenmesi gerekir.

Besleme Devresi

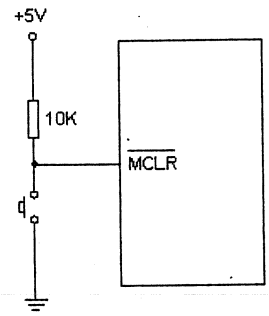
PIC16F84'ün 14 numaralı ucu besleme (VDD), 5 numaralı ucu ise şasedir (VSS). Besleme bağlantısı yapılırken şekilde görüldüğü gibi VDD ucuna 100 nF'lık bir kondansatör bağlanır. Bu şekilde devreye enerji uygulandığı anda meydana gelebilecek gerilim dalgalanmalarından PIC'in etkilenmemesi sağlanmış olur.



ŞEKİL 11: PIC16F84 besleme bağlantısı.

Reset Devresi

$\overline{MCLR}$ (Master Clear) ucu reset işlemini gerçekleştirir. Reset işlemi programın çalışmasını durdurup başa döndürmek, yani program sayıcıyı 00h adresine konumlandırmak demektir. Reset ucu 0 aktif çalışır. Şekil 12'deki reset butonuna basıldığı zaman $\overline{MCLR}$ ucu şase seviyesine iner, program resetlenir. Buton basılı değilken program normal çalışmasını sürdürür. Reset butonu kullanılmayacaksa $\overline{MCLR}$ ucu kesinlikle boşta bırakılmaz, +5V'a bağlanır. Aksi halde program çalışmayabilir.



ŞEKİL 12: PIC16F84 reset butonu bağlantısı.

Osilatör Devresi

Mikrodenetleyiciler clock sinyali olmadan çalışmaz. Bu yüzden ihtiyacımız olan çalışma hızına göre aşağıdaki dört faklı osilatör tipinden birini seçmemiz gerekir.

- LP (Low Power Crystal) – Düşük Güçlü Kristal
- XT (Crystal/Resonator) – Kristal/Rezonatör
- HS (High Speed Crystal/Resonator) – Yüksek Hizlı Kristal/Rezonatör
- RC (Resistor/Capacitor) – Direnç/Kondansatör

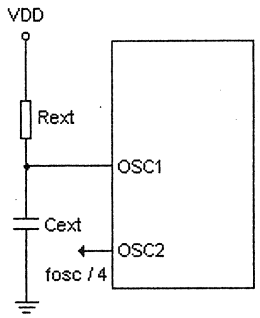
Devre kurarken osilatörün mikrodenetleyiciye mümkün olduğunca yakın yerleştirilmesi gerekir. Böylece clock alınan uçta herhangi bir girişim olması engellenmiş olur.

Aldığımız PIC entegresinin üzerinde uygulanabilecek frekansın maksimum değeri yazar. Örneğin, 10/P mikrodenetleyicinin 10 Mhz'e kadar, 4/P 4 MHz'e kadar olan clock sinyali ile kullanılabileceğini ifade eder.

Aşağıda dört osilatör tipinden yaygın olarak kullanılan RC ve XT osilatör tipleri incelenmiştir.

RC Osilatör

RC osilatörler KHz'ler seviyesinde sinyal üretirler. Bu yüzden zamanlamanın çok hassas bir şekilde yapılması gerekmeyen uygulamalarda, daha ucuz olması dolayısıyla tercih edilirler. RC osilatörün frekansı kaynak gerilimine, R_{ext} ve C_{ext} değerlerine bağlıdır.

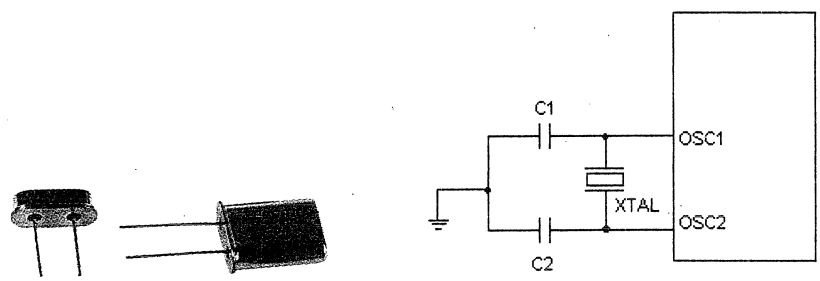


ŞEKİL 13: PIC16F84 RC osilatör bağlantısı.

Tavsiye edilen R_{ext} değeri 5K-100K arasında, C_{ext} değeri ise >20 pF'dır. Direnç değeri tavsiye edilenin altında olursa osilatör kararsızlaşır, üzerinde olursa gürültü ve neme karşı çok duyarlı olur. OSC1 clock giriş ucu, OSC2 ise clock çıkış ucudur. OSC1'e uygulanan osilatör frekansının ¼'ü OSC2 ucunda meydana gelir. Bu frekans istenilen başka bir yerde kullanılabilir.

XT Osilatör

Kristal osilatörler zamanlamanın hassas bir şekilde yapılması gereken uygulamalarda kullanılır. Bu osilatör tipi iki kondansatör ile bir kristalden meydana gelir.

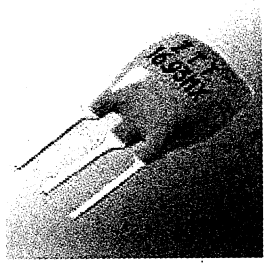


ŞEKİL 14: Çeşitli kristaller ve PIC16F84'ün kristal osilatör bağlantısı.

Kristalin frekansına göre C1 ve C2'nin alması gereken değerler aşağıdaki tabloda gösterilmiştir:

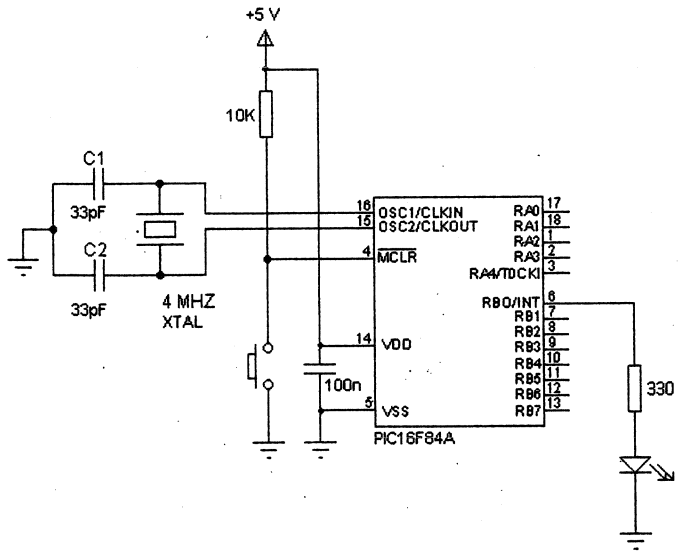
Frekans	Kondansatör
500 KHz	20-68 pF
1 MHz	15-68 pF
2 MHz	15-47 pF
4 MHz	15-33 pF

Yukarıdaki osilatörde bulunan iki kondansatörle kristalin tek kılıf içerisine alınması ile seramik rezonatörler elde edilir. Rezonatörün orta ucu şaseye, diğer iki uç ise OSC1 ve OSC2 uçlarına bağlanır. OSC uçlarına bağlanan uçların sırası farketmez.



ŞEKİL 15: Seramik rezonatör.

Görüldüğü gibi PIC16F84 için yazılan bir programın denenmesi için gerekli olan donanım oldukça basittir. Örneğin PORTB'nin 0. bitine bağlı olan bir ledi yakan programı denemek için kurulması gereken devre Şekil 16'daki gibidir.



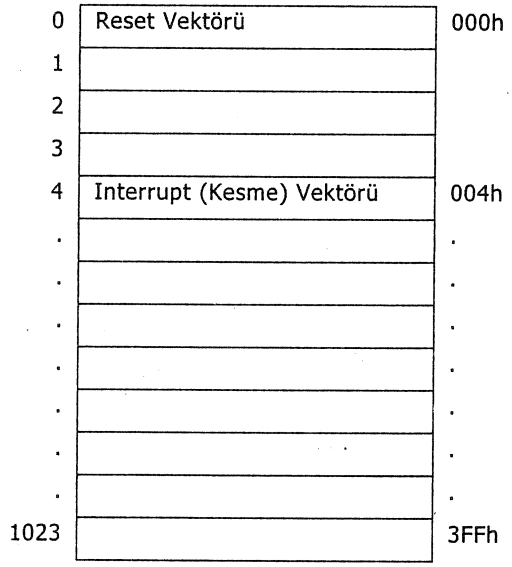
ŞEKİL 16: Bir ledi yakmak için kurulması gereken devre.

PIC16F84'ün Bellek Organizasyonu

Bellek mikrodenetleyicide bilgileri depolayan kısımdır. PIC16F84'ün program belleği ve veri belleği olmak üzere iki adet bellek bloğu vardır. Bu blokların her birinin kendine ait veri yolu olduğundan aynı osilatör periyotu içerisinde her iki bloğa da erişmek mümkündür.

Program Belleği

PIC16F84'ün 1 KB'lık program belleği vardır, yani toplam 1024 adet komut satırı yazılabilir. Yazdığımız program, program belleği içerisinde saklanır. Bu bellek flash bellek tipinde olduğu için defalarca silip yeniden program yazmamız mümkündür.



00h başlangıç adresidir. Program bu adresten itibaren komutları işler. Bir programa reset atıldığı zaman program 00h adresine döner. 04h adresi ise interrupt (kesme) vektörünün adresidir. Yani bir kesme meydana geldiğinde program bu adrese dallanarak kesme alt programını çalıştırmaya başlar.

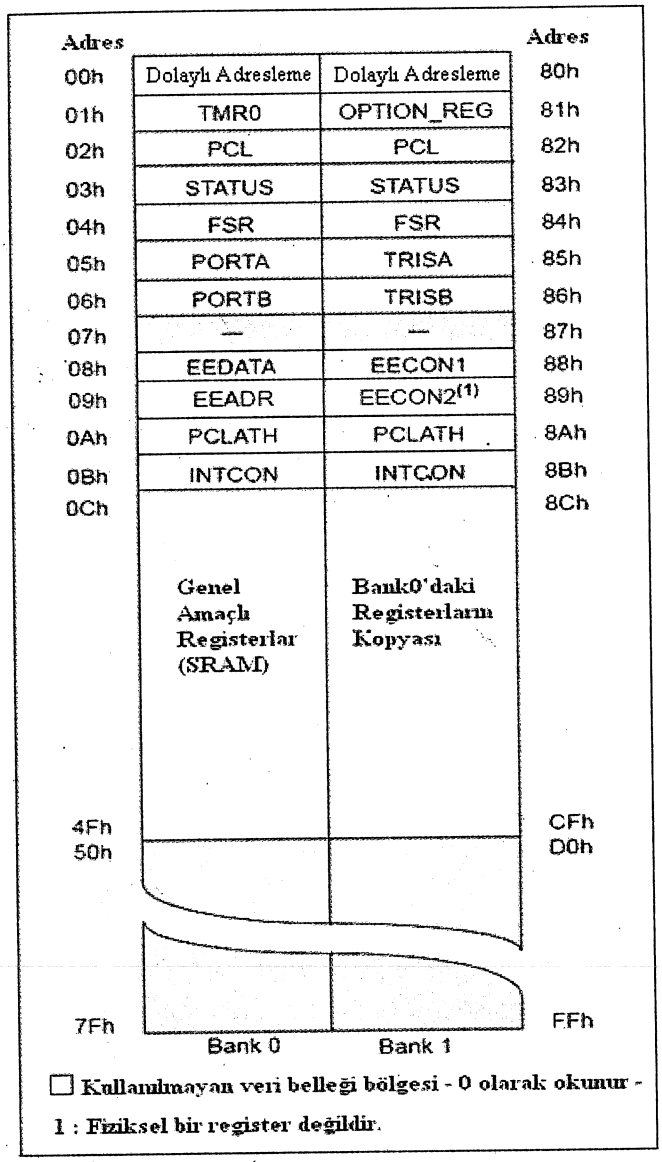
Veri Belleği

Veri belleği program içerisindeki değişkenler ile mikrodenetleyicinin çalışmasını kontrol eden bazı değişkenlerin tutulduğu hafıza bölgesidir. PIC16F84'ün veri belleği statik RAM bellektir. Yani enerji kesildiğinde içerisindeki tüm bilgiler kaybolur.

RAM bellek 00h – 4Fh adresleri arasında, her biri 8 bitten meydana gelen bloklardan oluşmuştur. Bu bloklar kaydedici (file register) olarak isimlendirilir.

Register haritasında bazı adresler TMR0, STATUS, PORTB gibi register'lara tahsis edilmiş durumdadır. Bunlara Özel Amaçlı Register'lar (SFR) adı verilir. Özel Amaçlı Register'lar mikrodenetleyicinin çalışmasını kontrol etmek amacıyla kullanılırlar. Giriş-çıkış işlemleri, portların giriş veya çıkış olarak yönlendirilmesi, kesme işlemleri gibi.

TABLO 1: PIC16F84'ün Veri Belleği



0Ch-4Fh adresleri arasındaki bellek bölgesinde programcının kullanımına ayrılmış register'lar bulunur. Bunlara Genel Amaçlı Register'lar (GPR) adı verilir.

Tablo 1'de görüldüğü gibi PIC16F84'ün veri belleği iki banka ayrılmıştır: Bank0 ve Bank1. Erişmek istediğimiz register hangi bankta ise o banka geçmemiz gerekir. Bank değiştirme işlemi Portların Giriş-Çıkış Olarak Yönlendirilmesi başlığı altında anlatılacaktır. Bank0, 00h-4Fh adresleri arasındaki 80 register'ı, Bank1 ise 80h-CFh adresleri arasındaki 80 register'ı kapsar. İlk bakışta RAM bellekte 160 adet kullanılabilir register olduğu sanılır. Ancak aslında durum faklıdır. Bank0'da bulunan Genel Amaçlı Kaydediciler Bank1'e kopyalanır. Örneğin 8C adresinde bulunan register 0C adresindeki register'ın kopyasıdır. Bu register'a hem 8C, hem de 0C

adresine konumlanarak ulaşılabilir. Kopyalama işlemi verilere bank değiştirme işlemine gerek kalmadan erişebilmeyi sağlar. Özel amaçlı register'lardan bazılarının da (STATUS, INTCON gibi) her iki bankta yer aldığı görülmektedir. Bu durum çok kullanılan register'lara daha hızlı bir biçimde ulaşma imkanı sağlar.

Register haritasında 07h (= 87h) adresinin de boş olduğu görülse de bu adrese yerleştirilen register mikrodenetleyici tarafından tanınmadığı için kullanılmamalıdır.

Sonuç olarak diyebiliriz ki PIC16F84'te programcı tarafından kullanılabilecek toplam 68 tane genel amaçlı register bulunur (0Ch – 4Fh adresleri arasında). Programımızda bir register oluşturacağımız zaman adresini bu aralık içerisinde tanımlamamız gerekir.

TABLO 2: Özel Amaçlı Register'lar

ADRES	İSİM	BİT7	BİT6	BİT5	BİT4	BİT3	BİT2	BİT1	BİT0	Power-On Resette Alacağı Değer
BANK 0										
00h	INDF	Veri belleğini adreslemek için FSR'nin içeriğini kullanır (Fiziksel bir register değildir)								---- ----
01h	TMR0	8-bit gerçek zamanlı clock / sayıcı								xxxx xxxx
02h	PCL	Program sayıcının (PC) düşük değerlikli 8 biti								0000 0000
03h	STATUS	IRP	RP1	RP0	TO	PD	Z	DC	C	0001 1xxx
04h	FSR	Veri belleğini dolaylı adresleme işaretçisi (pointer 0)								xxxx xxxx
05h	PORTA	-	-	-	RA4/T0CKI	RA3	RA2	RA1	RA0	---x xxxx
06h	PORTB	RB7	RB6	RB5	RB4	RB3	RB2	RB1	RB0	xxxx xxxx
07h	-	Tanımsız bölge (0 olarak okunur)								-
08h	EEDATA	EEPROM Data Register								xxxx xxxx
09h	EEADR	EEPROM Address Register								xxxx xxxx
0Ah	PCLATH	-	-	-	Program sayıcının yüksek değerlikli 5 biti için yazma tamponu					---0 0000
0Bh	INTCON	GIE	EEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF	0000 000x
BANK 1										
80h	INDF	Veri belleğini adreslemek için FSR'nin içeriğini kullanır (Fiziksel bir register değildir)								---- ----
81h	OPTION_REG	RBPU	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0	1111 1111
82h	PCL	Program sayıcının (PC) düşük değerlikli 8 biti								0000 0000
83h	STATUS	IRP	RP1	RP0	TO	PD	Z	DC	C	0001 1xxx
84h	FSR	Veri belleğini dolaylı adresleme işaretçisi (pointer 0)								xxxx xxxx
85h	TRISA	-	-	-	PORTA yönlendirme registeri					---1 1111
86h	TRISB	PORTB yönlendirme registeri								1111 1111
87h	-	Tanımsız bölge (0 olarak okunur)								-
88h	EECON1	-	-	-	EEIF	WRERR	WREN	WR	RD	---0 x000
89h	EECON2	EEPROM Kontrol Registeri 2 (Fiziksel bir register değildir)								---- ----
0Ah	PCLATH	-	-	-	Program sayıcının yüksek değerlikli 5 biti için yazma tamponu					---0 0000
0Bh	INTCON	GIE	EEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF	0000 000x

x = bilinmeyen u = değişmez - = tanımsız (0 olarak okunur)

Özel Amaçlı Register'lar Tablo 2'de verilmiştir. Bu register'ların işlevi ve bitlerinin görevleri yeri geldikçe anlatılacaktır.

Özel Amaçlı Register'lardan bazılarında farklı görevleri yerine getiren bitler bulunur. Bunlar STATUS, INTCON, OPTION_REG ve EECON1 register'larıdır. Aşağıda bu register'larda bulunan bitlerin görevleri açıklanmıştır. Bu kitapta EEPROM'a yazma ve okuma uygulamalarına değinilmediğinden EECON1 register'ı açıklanmamıştır.

Status Register

STATUS register içerisinde ALU'nun (Aritmetik Lojik Ünite) aritmetiksel durumunu ve reset durumunu gösteren bitler ile bank seçme bitleri bulunur.

TABLO 3: STATUS Register

Bit 7							Bit 0
IRP	RP1	RP0	TO	PD	Z	DC	C

Bit 7 : IRP : Bank seçme biti (Dolaylı adresleme için kullanılır)

Bit 6-5 : RP1 - RP0 : Bank seçme bitleri

00 - Bank 0 (00h - 7Fh)
01 - Bank 1 (80h - FFh)

Bit 4 : TO : Time out (zaman aşımı) biti

1 : PIC'e enerji verildiğinde, CLRWDT veya SLEEP komutları çalıştığında
0 : WDT zaman aşımı oluştuğunda

Bit 3 : PD : Power down (düşük enerji) biti

1 : PIC'e enerji verildiğinde veya CLRWDT komutu çalıştığında
0 : SLEEP komutu çalıştığında

Bit 2 : Z : Sıfır (zero) biti

1 : Bir aritmetik veya lojik işlemin sonucu 0 ise
0 : Bir aritmetik veya lojik işlemin sonucu 0 değilse

Bit 1 : DC : Basamak elde/borç biti (Digit carry / borrow)

1 : Sonucun alt dört bitinden üst dört bitine taşma olduğunda
0 : Sonucun alt dört bitinden üst dört bitine taşma olmadığında

Bit 0 : C : Elde / borç (carry / borrow) biti

1 : Elde olduğunda (Sonucun 7. bitinden taşma meydana geldiğinde)
0 : Elde olmadığında (Sonucun 7. bitinden taşma meydana gelmediğinde)

IRP biti PIC16F84 tarafından kullanılmaz. Değeri her zaman 0'dır. PIC16F84'te Bank 0 ve Bank 1 olmak üzere iki adet bank olduğu için bank seçme işlemi için tek bit yeterlidir. Bu yüzden RP1 biti de kullanılmaz. Değeri her zaman 0'dır. Seçme işlemi sadece RP0 biti ile yapılır.

Bit 0, yani Carry bitinin yukarıdaki değerleri toplama işlemi için geçerlidir. Çıkarma işleminde bitlerin değeri terstir. Yani bir borç durumu meydana geldiğinde C=0, gelmediğinde C=1'dir.

STATUS register içerisindeki diğer bitler hem yazılabilir, hem de okunabilir olduğu halde TO ve PD bitlerine yazılamaz, sadece okunabilir. Bu yüzden STATUS register'ın hedef olarak kullanıldığı komutlarda elde edilecek sonuç beklenenden farklı olabilir. Örneğin CLRF STATUS komut satırı işlendiğinde IRP, RP1 ve RP0 bitleri 0, Z biti 1 olur. Diğer bitler ise değişmeden kalır. STATUS register'ının bu durumda değeri 000u u1uu olur. (u = değişmez)

STATUS register'ındaki bitlerin değerini değiştirmeyen (etkilemeyen) dört adet komut vardır. Bunlar: BCF, BSF, SWAPF ve MOVWF komutlarıdır.

INTCON Register

TABLO 4: INTCON Register

Bit 7							Bit 0
GIE	EEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF

Bit 7 : GIE : Tüm kesmeleri etkinleştirme biti

1 : Tüm kesmelere izin verilir
0 : Hiçbir kesmeye izin verilmez

Bit 6 : EEIE : EEPROM kesmesi etkinleştirme biti

1 : EEPROM kesmesine izin verilir
0 : EEPROM kesmesine izin verilmez

Bit 5 : T0IE : TMR0 kesmesi etkinleştirme biti

1 : TMR0 kesmesine izin verilir
0 : TMR0 kesmesine izin verilmez

Bit 4 : INTE : RB0/INT kesmesi etkinleştirme biti

1 : RB0/INT kesmesine izin verilir
0 : RB0/INT kesmesine izin verilmez

Bit 3 : RBIE : PORTB değişim kesmesi etkinleştirme biti

1 : PORTB değişim kesmesine izin verilir
0 : PORTB değişim kesmesine izin verilmez

Bit 2 : T0IF : TMR0 kesme bayrağı

1 : TMR0 kesmesi meydana geldi
0 : TMR0 kesmesi meydana gelmedi

Bit 1 : INTF : RB0/INT kesme bayrağı

1 : RB0/INT kesmesi meydana geldi
0 : RB0/INT kesmesi meydana gelmedi

Bit 0 : RBIF : PORTB değişim kesmesi bayrağı

1 : PORTB değişim kesmesi meydana geldi
0 : PORTB değişim kesmesi meydana gelmedi

OPTION_REG Register

TABLO 5: OPTION_REG Register

Bit 7							Bit 0
RBPU	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0

Bit 7 : RBPU : PORTB pull-up dirençlerini etkinleştirme biti

1 : PORTB pull-up dirençleri iptal edilir (disable)
0 : PORTB pull-up dirençleri geçerli yapılır (enable)

Bit 6 : INTEDG : RB0/INT kesmesi kenar seçme biti

1 : Yükselen kenarda kesme meydana gelir
0 : Düşen kenarda kesme meydana gelir

Bit 5 : T0CS : TMR0 clock kaynağı seçme biti

1 : RA4/T0CKI pinindeki harici clock seçilir
0 : Dahili komut saykıl clock'u (CLKOUT) seçilir

Bit 4 : T0SE : TMR0 kaynağı kenar seçme biti

1 : RA4/T0CKI pininden düşen kenar tetiklemesi
0 : RA4/T0CKI pininden yükselen kenar tetiklemesi

Bit 3 : PSA : Frekans bölücü seçme biti

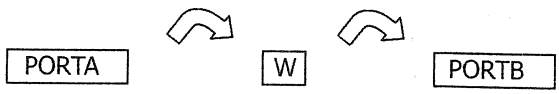
1 : Frekans bölme sayısı WDT için geçerli
0 : Frekans bölme sayısı TMR0 için geçerli

Bit 2 - 0: PS2 - PS1 - PS0 : Frekans bölme sayısı

Bit Değeri	TMR0 Oranı	WDT Oranı
000	1 : 2	1 : 1
001	1 : 4	1 : 2
010	1 : 8	1 : 4
011	1 : 16	1 : 8
100	1 : 32	1 : 16
101	1 : 64	1 : 32
110	1 : 128	1 : 64
111	1 : 256	1 : 128

PIC16F84'ün bellek organizasyonu ile ilgili buraya kadar verdiğimiz bilgileri özetlersek; PIC16F84 için yazdığımız program, program belleğinde yer alır. Program belleği flash bellek olduğundan enerji kesilse bile entegre içerisindeki program silinmez. Veri belleği statik RAM bellek olduğundan enerji kesildiğinde bu bellekteki bilgiler kaybolur. Eğer programımızda enerji kesilse dahi silinmemesini istediğimiz veriler varsa – örneğin, bir şifreli kilit programının şifresi gibi – bu durumda kullanabileceğimiz 64 byte'lık bir EEPROM belleği mevcuttur.

Bu üç tür belleğin dışında W (working register) adı verilen bir register bulunur. W işlemlerin çoğunda kullanılan geçici bir kaydedicidir. Örneğin, bir register'dan diğerine doğrudan veri aktarımı mümkün olmadığından bu işlem W üzerinden yapılır.



Program Sayıcı (Program Counter - PC)

Program sayıcı işlenecek olan komutun adresini tutan 13 bitlik bir register'dır. Adresin üst 5 bitini PCLATH register'ında, alt 8 bitini ise PCL register'ında tutar.

Bir atlama veya dallanma komutu olmadığı sürece mikrodenetleyici komutları satır satır sırasıyla işler. Yani program sayıcının değeri her yeni komut satırına geçişte bir artar.

Bölüm 3

PIC Assembly Dili

Assembly Nedir?

Bildiğiniz gibi tüm sayısal sistemlerin anladığı tek dil vardır: 0 ve 1. 0 ve 1'lerden oluşan sayılar binary sayı olarak adlandırılır.

B'11001101' gibi.

Hexadecimal sayılar ise 0-F arasındaki 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E ve F rakamlarından oluşur. Bir binary sayıyı hexadecimal sayıya çevirmek için sayı dörderli gruplara ayrılır ve her gruba karşılık gelen hexadecimal rakam yazılır.

B' 11001010 ' = H'CA'

C A

Makina dili denildiği zaman hexadecimal sayılarla yazılan programlar akla gelir. Örneğin bir komut satırı;

```
DF AC 12 1B 3C
```

şeklinde olur.

Makina dili ile yazılan programlar çok hızlı çalışır, çünkü yapılan tek işlem hexadecimal'i binary'ye çevirmektir.

Assembly dili makina dilinin İngilizce kısaltmalara dönüştürülmesiyle oluşan bir dildir. Hangi dilde yazılmış olursa olsun tüm programlar assembly'ye dönüştürülebilir. Bu dilde yazılan programlar diğer dillerde yazılanlara göre önemsenmeyecek bir hız farkıyla çalışır.

Binary sistem	**00000101010100011001 ...**
Hexadecimal sistem	**DF AC 12 1B 3C 47 90 ...**
Assembly dili	**CLRF PORTB**

Komutun gerçekleştireceği işlevi anlatan İngilizce kelimelerin baş harfleri birleştirilerek oluşturulan kısaltmalara mnemonic denir. Örneğin bir file register'ın belli bir bitini 0 yapmak için kullanılan BCF (Bit Clear File Register) komutunda olduğu gibi.

Komut Seti

Tüm Assembly dili komutları 14 bitten oluşur. Bu 14 bit, komutun ne olduğunu bildiren OPCODE ve komutun işlevini bildiren bir veya birkaç OPERAND'dan meydana gelir.

CLRF f — OPCODE OPERAND

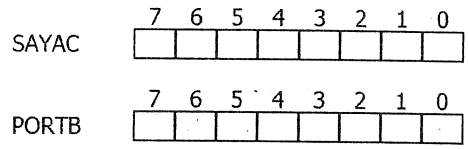
DECF f, d — OPCODE OPERANDLAR

Komutlar üç ana grupta incelenebilir:

- Byte Yönlendirmeli Komutlar
- Bit Yönlendirmeli Komutlar
- Sabit İşleyen Komutlar ve Kontrol Komutları

Bu komut türlerinden önce komut formatında yer alan kısaltmaları açıklamak faydalı olacaktır.

f (File Register): 8 bitlik kaydedici. Bir file register bilgilerin depolandığı 8 bitlik bir blok olarak düşünülebilir. **f** Özel Amaçlı veya Genel Amaçlı Register olabilir. **f** yerine komut tarafından kullanılacak olan register yazılır.



Yukarıdaki örneklerde **SAYAC** programcı tarafından oluşturulan genel amaçlı bir register, **PORTB** ise Bank 0'da 06h adresinde yer alan özel amaçlı bir register'dır.

```
INCF  f, d
INCF  SAYAC, f          ; SAYAC register'ındaki sayıyı 1 artır.
```

W (Working Register): Geçici bellek olarak kullanılan 8 bitlik register.

```
MOVF   f, d

MOVF    PORTA,W         ; PORTA'nın içerisindeki bilgiyi W'ya yükle.
MOVWF    PORTB          ; W'daki bilgiyi PORTB'ye yükle.
```

b (Bit Address): 8 bitlik register içerisinde hangi bitle çalışılacağını belirtir. Bit adresi 0-7 arasında bir sayı olmalıdır.

```
BSF   f, b
BSF    STATUS, 5        ; STATUS register'ının 5. bitini 1 yap.
```

k (Sabit): Sabit sayı veya etiket ifade eder.

```
MOVLW   k
MOVLW   D'12'           ; W register'ına 12 sabit sayısını yükler.

GOTO    k
GOTO    DONGU           ; DONGU etiketinin bulunduğu adrese dallanır.
```

d (Destination): İşlem sonucunun yükleneceği hedefi ifade eder.

d = 0 ise bulunan sonuç **W** içerisine yazılır.

d = 1 ise bulunan sonuç komut satırı içerisinde yer alan register içerisine yazılır.

MPLAB derleyicisi **d** yerine doğrudan **W** veya register ismi yazılmasına izin verir. Örneğin, **INCF SAYAC, 0** yerine **INCF SAYAC, W** yazabiliriz.Böylece komutlar daha anlaşılır hale gelir.

```
ADDWF   f, d
ADDWF   SAYAC, W  ; SAYAC'taki sayı ile W'daki sayıyı topla. Sonucu W'ya yükle.
ADDWF   SAYAC, F  ; SAYAC'taki sayı ile W'daki sayıyı topla. Sonucu SAYAC'a yükle.
```

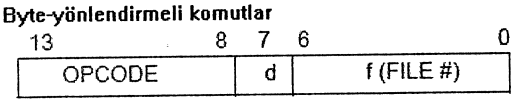
Hedef belirtilmezse derleyici **d**=1 kabul eder. Yani sonucu **f** ile belirtilmiş olan register'a yükler.

```
ADDWF SAYAC, F = ADDWF SAYAC
```

Kısaltmaların anlamlarını öğrendiğimize göre komutların formatını anlamak daha kolay olacaktır.

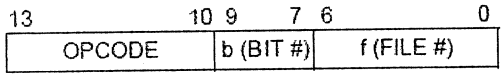
TABLO 6: Komut Formatı

Byte-yönlendirmeli komutlar



d = 0 hedef W ise
d = 1 hedef f ise
f = 7-bit file register adresi

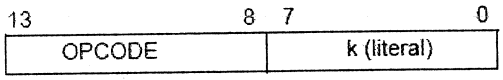
Bit - yönlendirmeli komutlar



b = 3-bit bit adresi
f = 7-bit file register adresi

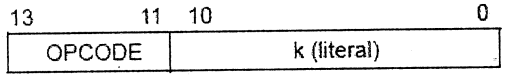
Sabit işleyen ve kontrol komutları

Genel



k = 8-bitlik ani değer

Sadece CALL ve GOTO komutları için



k = 11 bitlik ani değer

PIC16F84'ün komut seti 35 komuttan meydana gelir. Aşağıdaki tabloda tüm komutlar, komutların açıklaması ve 14 bitlik makina kodu listelenmiştir. Tablonun hemen altında ise bu 35 komutun ayrıntılı açıklamasını bulacaksınız.

TABLO 7: PIC16F84 Komut Seti

Komut	Açıklama	14 bit makina kodu MSB LSB
Byte-Yönlendirmeli Komutlar		
ADDWF f, d	W ile F'i topla	00 0111 dfff ffff
ANDWF f, d	W ile F'i AND'le	00 0101 dfff ffff
CLRF f	F'i temizle	00 0001 1fff ffff
CLRW	W'yu temizle	00 0001 0xxx xxxx
COMF f, d	F'in tersini al	00 1001 dfff ffff
DECF f, d	F'i 1 azalt	00 0011 dfff ffff
DECFSZ f, d	F'i 1 azalt, sonuç 0 ise bir komut atla	00 1011 dfff ffff
INCF f, d	F'i 1 artır	00 1010 dfff ffff
INCFSZ f, d	F'i 1 artır, sonuç 0 ise bir komut atla	00 1111 dfff ffff
IORWF f, d	W ile F'i OR'la	00 0100 dfff ffff
MOVF f, d	F'i yükle	00 1000 dfff ffff
MOVWF f	W'yu F'e yükle	00 0000 1fff ffff
NOP	İşlem yok	00 0000 0xx0 0000
RLF f, d	F'i sola kaydır	00 1101 dfff ffff
RRF f, d	F'i sağa kaydır	00 1100 dfff ffff
SUBWF f, d	F'den W'yu çıkar	00 0010 dfff ffff
SWAPF f, d	F'in alt dört biti ile üst dört bitinin yerini değiştir	00 1110 dfff ffff
XORWF f, d	W ile F'i XOR'la	00 0110 dfff ffff
Bit-Yönlendirmeli Komutlar		
BCF f, b	F'deki b. biti 0 yap	01 00bb bfff ffff
BSF f, b	F'deki b. biti 1 yap	01 01bb bfff ffff
BTFSC f, b	F'deki b. biti test et, 0 ise bir komut atla	01 10bb bfff ffff
BTFSS f, b	F'deki b. biti test et, 1 ise bir komut atla	01 11bb bfff ffff
Sabit İşleyen Komutlar ve Kontrol Komutları		
ADDLW k	W ile sabit sayıyı topla	11 111x kkkk kkkk
ANDLW k	W ile sabit sayıyı AND'le	11 1001 kkkk kkkk
CALL k	Altprogram çağır	10 0kkk kkkk kkkk
CLRWDT	Watchdog Timer'ı sıfırla	00 0000 0110 0100
GOTO k	k adresine git	10 1kkk kkkk kkkk
IORLW k	W ile sabit sayıyı OR'la	11 1000 kkkk kkkk
MOVLW k	Sabit sayıyı W'ya yükle	11 00xx kkkk kkkk
RETFIE	Kesmeden (interrupt'tan) geri dön	00 0000 0000 1001
RETLW k	W'daki sabit sayı ile geri dön	11 01xx kkkk kkkk

TABLO 7: PIC16F84 Komut Seti

Komut	Açıklama	14 bit makina kodu MSB LSB
RETURN	Altprogramdan geri dön	**00 0000 0000 1000**
SLEEP	Düşük enerji moduna geç	**00 0000 0110 0011**
SUBLW k	Sabit sayıdan **W**'yu çıkar	**11 110x kkkk kkkk**
XORLW k	**W** ile sabit sayıyı XOR'la	**11 1010 kkkk kkkk**

1. ADDWF Add **W** with **f**

Format: **ADDWF f, d**

Açıklama: **W** ile file register'ın içeriğini toplar.
d = **W** ise sonucu **W**'ya yükler
d = **f** ise sonucu file register'a yükler

İşlem: **W** + **f** ⇒ **d**

Flag: **C**, **DC**, **Z**

Örnekler:

a) ADDWF REG, W

Komuttan önce:	**W** = H'0B'	**REG** = H'56'	0101 0110	H'56'
Komuttan sonra:	**W** = H'61'	**REG** = H'56'	+ 0000 1011	+ H'0B'
			0110 0001	H'61'

b) ADDWF REG, F

Komuttan önce:	**W** = H'0B'	**REG** = H'56'
Komuttan sonra:	**W** = H'0B'	**REG** = H'61'

2. ANDWF AND **W** with **f**

Format: **ANDWF f, d**

Açıklama: **W** ile file register'ın içeriği arasında lojik AND (VE) işlemi yapar
d = **W** ise sonucu **W**'ya yükler
d = **f** ise sonucu file register'a yükler

İşlem: **W** (AND) **f** ⇒ **d**

Flag: **Z**

Örnekler:

a) ANDWF REG, W

Komuttan önce:	**W** = H'7C'	**REG** = H'45'	0111 1100	H'7C'
Komuttan sonra:	**W** = H'44'	**REG** = H'45'	(AND) 0100 0101	(AND) H'45'
			0100 0100	H'44'

b) ANDWF REG, F

Komuttan önce: **W** = H'7C' **REG** = H'45'

Komuttan sonra: **W** = H'7C' **REG** = H'44'

3. CLRF Clear File Register

Format: **CLRF f**

Açıklama: File register'ın tüm bitlerini sıfırlar

İşlem: **0** ⇒ **f**

Flag: **Z**

Örnekler:

a) CLRF REG

Komuttan önce: **REG** = H'2A'

Komuttan sonra: **REG** = H'00'

4. CLRW Clear **W**

Format: **CLRW**

Açıklama: **W** register'ının tüm bitlerini sıfırlar

İşlem: 0 ⇒ **W**

Flag: **Z**

Örnekler:

a) CLRW

Komuttan önce: **W** = H'96'

Komuttan sonra: **W** = H'00'

5. COMF Complement **f**

Format: **COMF f, d**

Açıklama: File register'ın tüm bitlerini tersler (0 ise 1, 1 ise 0 yapar)

d = **W** ise sonucu **W**'ya yükler

d = **f** ise sonucu file register'a yükler

İşlem: $\bar{f}$ ⇒ **d**

Flag: **Z**

Örnekler:

a) COMF REG, W

Komuttan önce: **W** = H'D2' **REG** = H'81'

Komuttan sonra: **W** = H'7E' **REG** = H'81'

b) COMF REG, F

Komuttan önce: **W** = H'D2' **REG** = H'81'

Komuttan sonra: **W** = H'D2' **REG** = H'7E'

6. DECF Decrement **f**

Format: **DECF f, d**

Açıklama: File register'ın içeriğini 1 azaltır
d = **W** ise sonucu **W**'ya yükler
d = **f** ise sonucu file register'a yükler

İşlem: **f** - 1 ⇒ **d**

Flag: **Z**

Örnekler:

a) DECF REG, W

Komuttan önce: **W** = H'89' **REG** = H'65'

Komuttan sonra: **W** = H'64' **REG** = H'65'

b) DECF REG, F

Komuttan önce: **W** = H'89' **REG** = H'65'

Komuttan sonra: **W** = H'89' **REG** = H'64'

7. DECFSZ Decrement **f** Skip If Zero

Format: **DECFSZ f, d**

Açıklama: File register'ın içeriğini 1 azaltır, sonuç 0 ise bir komut satırı atlar, değilse bir alt satırdan komutları işlemeye devam eder.
d = **W** ise sonucu **W**'ya yükler
d = **f** ise sonucu file register'a yükler

İşlem: **f** - 1 ⇒ **d**

Flag: -

Örnekler:

a)
```
DECFSZ REG, W
GOTO X
GOTO Y
```

Komuttan önce: **W** = H'87' **REG** = H'6A'

Komuttan sonra: **W** = H'69' **REG** = H'6A' **X** etiketine dallanır

b)
```
DECFSZ REG, F
GOTO X
GOTO Y
```

Komuttan önce: **W** = H'87' **REG** = H'01'

Komuttan sonra: **W** = H'87' **REG** = H'00' **Y** etiketine dallanır

8. INCF Increment **f**

Format: **INCF f, d**

Açıklama: File register'ın içeriğini 1 artırır
d = **W** ise sonucu **W**'ya yükler
d = **f** ise sonucu file register'a yükler

İşlem: **f** + 1 ⇒ **d**

Flag: **Z**

Örnekler:

a) INCF REG, W

Komuttan önce: **W** = H'43' **REG** = H'94'

Komuttan sonra: **W** = H'95' **REG** = H'94'

b) INCF REG, F

Komuttan önce: **W** = H'43' **REG** = H'94'

Komuttan sonra: **W** = H'43' **REG** = H'95'

9. INCFSZ Increment **f** Skip If Zero

Format: **INCFSZ f, d**

Açıklama: File register'ın içeriğini 1 artırır, sonuç 0 ise bir komut satırı atlar, değilse bir alt satırdan komutları işlemeye devam eder.
d = **W** ise sonucu **W**'ya yükler
d = **f** ise sonucu file register'a yükler

İşlem: **f** + 1 ⇒ **d**

Flag: -

Örnekler:

a)
```
INCFSZ REG, W
GOTO X
GOTO Y
```

Komuttan önce: **W** = H'5A' **REG** = H'60'

Komuttan sonra: **W** = H'61' **REG** = H'60' **X** etiketine dallanır

b)
```
INCFSZ REG, F
GOTO X
GOTO Y
```

Komuttan önce: **W** = H'5A' **REG** = H'FF'

Komuttan sonra: **W** = H'5A' **REG** = H'00' **Y** etiketine dallanır

10. IORWF OR **W** With **f**

Format: **IORWF f, d**

Açıklama: **W** ile file register'ın içeriği arasında lojik OR (VEYA) işlemi yapar
d = **W** ise sonucu **W**'ya yükler
d = **f** ise sonucu file register'a yükler

İşlem: **W** (OR) **f** ⇒ **d**

Flag: **Z**

Örnekler:

a) IORWF REG, W

Komuttan önce:	**W** = H'F5'	**REG** = H'41'	1111 0101	H'F5'
Komuttan sonra:	**W** = H'F5'	**REG** = H'41'	(OR) 0100 0001	(OR) H'41'
			1111 0101	H'F5'

b) IORWF REG, F

Komuttan önce:	**W** = H'F5'	**REG** = H'41'
Komuttan sonra:	**W** = H'F5'	**REG** = H'F5'

11. MOVF Move **f** To **d**

Format: **MOVF f, d**

Açıklama: File register'ın içeriğini hedefe kopyalar
d = **W** ise sonucu **W**'ya yükler
d = **f** ise sonucu file register'a yükler

İşlem: **f** ⇒ **d**

Flag: **Z**

Örnekler:

a) MOVF REG, W

Komuttan önce:	**W** = H'78'	**REG** = H'B5'
Komuttan sonra:	**W** = H'B5'	**REG** = H'B5'

12. MOVWF Move **W** To **f**

Format: **MOVWF f**

Açıklama: **W**'nun içeriğini file register'a kopyalar

İşlem: **W** ⇒ **f**

Flag:

Örnekler:

a) MOVWF REG

Komuttan önce:	**W** = H'9E'	**REG** = H'83'
Komuttan sonra:	**W** = H'9E'	**REG** = H'9E'

13. NOP No Operation
Format: **NOP**
Açıklama: Hiçbir işlem yapmaz
İşlem: -
Flag: -

Örnekler:
a) NOP
Komuttan önce: PC (Program Sayıcı) = x
Komuttan sonra: PC (Program Sayıcı) = x + 1

14. RLF Rotate Left **f**
Format: **RLF f, d**
Açıklama: File register'ın içerisindeki tüm bitleri bir pozisyon sola kaydırır
d = **W** ise sonucu **W**'ya yükler
d = **f** ise sonucu file register'a yükler
İşlem: **f<n>** ⇒ **d<n+1>**, **f<7>** ⇒ **C**, **C** ⇒ **d<0>**
Flag: **C**

Örnekler:
a) RLF REG, W
Komuttan önce: **W** = H'53' **REG** = H'E7' **C** = 0
Komuttan sonra: **W** = H'CE' **REG** = H'E7' **C** = 1

b) RLF REG, F
Komuttan önce: **W** = H'53' **REG** = H'E7' **C** = 0
Komuttan sonra: **W** = H'53' **REG** = H'CE' **C** = 1

15. RRF Rotate Right **f**
Format: **RRF f, d**
Açıklama: File register'ın içerisindeki tüm bitleri bir pozisyon sağa kaydırır
d = **W** ise sonucu **W**'ya yükler
d = **f** ise sonucu file register'a yükler
İşlem: **f<n>** ⇒ **d<n-1>**, **f<0>** ⇒ **C**, **C** ⇒ **d<7>**
Flag: **C**

Örnekler:
a) RRF REG, W
Komuttan önce: **W** = H'71' **REG** = H'95' **C** = 0
Komuttan sonra: **W** = H'4A' **REG** = H'95' **C** = 1

b) RRF REG, F

Komuttan önce: W = H'71' REG = H'95' C = 0

Komuttan sonra: W = H'71' REG = H'4A' C = 1

16. SUBWF Subtract W From f

Format: **SUBWF f, d**

Açıklama: File register'ın içerisindeki bilgiden W'daki bilgiyi çıkartır
d = W ise sonucu W'ya yükler
d = f ise sonucu file register'a yükler

İşlem: f – W ⇒ d

Flag: C, DC, Z

Örnekler:

a) SUBWF REG, F

Komuttan önce: W = H'12' REG = H'13' C = x Z = x

Komuttan sonra: W = H'12' REG = H'01' C = 1 Z = 0 Sonuç > 0

Komuttan önce: W = H'12' REG = H'12' C = x Z = x

Komuttan sonra: W = H'12' REG = H'00' C = 1 Z = 1 Sonuç = 0

Komuttan önce: W = H'13' REG = H'12' C = x Z = x

Komuttan sonra: W = H'13' REG = H'FF' C = 0 Z = 0 Sonuç < 0

17. SWAPF Swap Nibbles in f

Format: **SWAPF f, d**

Açıklama: File register'ın alt dört biti ile üst dört bitini yer değiştirir
d = W ise sonucu W'ya yükler
d = f ise sonucu file register'a yükler

İşlem: f <3:0> ⇒ f <7:4>, f <7:4> ⇒ f <3:0>

Flag: -

Örnekler:

a) SWAPF REG, W

Komuttan önce: W = H'6B' REG = H'14'

Komuttan sonra: W = H'41' REG = H'14'

b) SWAPF REG, F

Komuttan önce: W = H'6B' REG = H'14'

Komuttan sonra: W = H'6B' REG = H'41'

18. XORWF Exclusive OR **W** with **f**

Format: **XORW f, d**

Açıklama: **W** ile file register'ın içeriği arasında lojik EXOR (ÖZEL VEYA) işlemi yapar
d = **W** ise sonucu **W**'ya yükler
d = **f** ise sonucu file register'a yükler

İşlem: **W** (XOR) **f** ⇒ **d**

Flag: **Z**

Örnekler:

a) XORWF REG, W

Komuttan önce:	**W** = H'6E'	**REG** = H'37'	0110 1110	H'6E'
Komuttan sonra:	**W** = H'59'	**REG** = H'37'	(XOR) 0011 0111	(XOR) H'37'
			0101 1001	H'59'

b) XORWF REG, F

Komuttan önce:	**W** = H'6E'	**REG** = H'37'
Komuttan sonra:	**W** = H'6E'	**REG** = H'59'

19. BCF Bit Clear **f**

Format: **BCF f, b**

Açıklama: File register içerisindeki **b** bitini 0 yapar

İşlem: 0 ⇒ **f <b>**

Flag: -

Örnekler:

a) BCF REG, 7

Komuttan önce: **REG** = H'F6'

Komuttan sonra: **REG** = H'76'

20. BSF Bit Set **f**

Format: **BSF f, b**

Açıklama: File register içerisindeki **b** bitini 1 yapar

İşlem: 1 ⇒ **f <b>**

Flag: -

Örnekler:

a) BSF REG, 4

Komuttan önce: **REG** = H'E6'

Komuttan sonra: **REG** = H'F6'

21. BTFSC f, b Bit Test **f** Skip If Clear

Format: **BTFSC f, b**

Açıklama: **f** register'ı içerisindeki **b** bitini test eder. Bit 0 ise bir komut satırı atlar, 1 ise bir alt satırdan komutları işlemeye devam eder.

İşlem: **f<b>** = 0 ise bir komut atla

Flag:

Örnekler:

a)
```
BTFSC REG, 3
GOTO X
GOTO Y
```

Komuttan önce: **REG** = H'37'

Komuttan sonra: **Y** etiketine dallanır

Komuttan önce: **REG** = H'3F'

Komuttan sonra: **X** etiketine dallanır

22. BTFSS f, b Bit Test **f** Skip If Set

Format: **BTFSS f, b**

Açıklama: **f** register'ı içerisindeki **b** bitini test eder. Bit 1 ise bir komut satırı atlar, 0 ise bir alt satırdan komutları işlemeye devam eder.

İşlem: **f<b>** = 1 ise bir komut atla

Flag:

Örnekler:

a)
```
BTFSS REG, 3
GOTO X
GOTO Y
```

Komuttan önce: **REG** = H'37'

Komuttan sonra: **X** etiketine dallanır

Komuttan önce: **REG** = H'3F'

Komuttan sonra: **Y** etiketine dallanır

23. ADDLW Add Literal and **W**

Format: **ADDLW k**

Açıklama: **W** ile sabit sayıyı toplar. Sonucu **W**'ya yükler.

İşlem: **W + k ⇒ W**

Flag: **C, DC, Z**

Örnekler:

a) ADDLW H'23'

Komuttan önce:	W = H'6E'	0010 0011	H'23'
Komuttan sonra:	W = H'91'	+ 0110 1110	+ H'6E'
		1001 0001	H'91'

24. ANDLW AND Literal and W

Format: **ANDLW k**

Açıklama: **W** ile sabit sayı arasında lojik AND (VE) işlemi yapar. Sonucu **W**'ya yükler.

İşlem: **W** (AND) **k** ⇒ **W**

Flag: **Z**

Örnekler:

a) ANDLW H'84'

Komuttan önce:	W = H'2A'	1000 0100	H'84'
Komuttan sonra:	W = H'00'	(AND) 0010 1010	(AND) H'2A'
		0000 0000	H'00'

25. CALL CALL a program

Format: **CALL k**

Açıklama: Alt program çağırır. Önce dönüş adresi (PC+1) **STACK**'e yüklenir. Daha sonra alt programın adresi olan 11 bitlik k sayısı **PC**'a yüklenir.

İşlem: **PC** + 1 ⇒ **STACK**, **k** ⇒ **PC**

Flag: -

26. CLRWDT Clear Watchdog Timer

Format: **CLRWDT**

Açıklama: **WDT** resetlenir (Prescaler = 0, TO ve PD = 1)

İşlem: 0 ⇒ **WDT**

0 ⇒ **WDT Prescaler**

1 ⇒ **TO**

1 ⇒ **PD**

Flag: **TO**, **PD**

27. GOTO GO to an address

Format: **GOTO k**

Açıklama: **k** adresine atlar

İşlem: **k** ⇒ **PC**

Flag: -

28. IORLW Inclusive OR Literal with **W**

Format: **IORLW k**

Açıklama: **W** ile sabit sayı arasında lojik OR (VEYA) işlemi yapar. Sonucu **W**'ya yükler.

İşlem: **W** (OR) **k** ⇒ **W**

Flag: **Z**

Örnekler:

a) IORLW H'6B'

Komuttan önce:	**W** = H'15'	0110 1011	H'6B'
Komuttan sonra:	**W** = H'7F'	(OR) 0001 0101	(OR) H'15'
		0111 1111	H'7F'

29. MOVLW Move Literal To **W**

Format: **MOVLW k**

Açıklama: **k** sabit sayısını **W**'ya yükler

İşlem: **k** ⇒ **W**

Flag:

Örnekler:

a) MOVLW H'88'

Komuttan önce: **W** = H'9C'

Komuttan sonra: **W** = H'88'

30. RETFIE Return From Interrupt

Format: **RETFIE**

Açıklama: Kesme alt programından ana programa döndürür. **STACK**'teki adres bilgisini **PC**'a yükler. **GIE** =1 yapar.

İşlem: **STACK** ⇒ **PC** , 1 ⇒ **GIE**

Flag: -

31. RETLW Return With Literal in **W**

Format: **RETLW k**

Açıklama: **k** sabit sayısını **W**'ya yükler. **STACK**'teki adres bilgisini **PC**'a yükler.

İşlem: **k** ⇒ **W**, **STACK** ⇒ **PC**

Flag: -

32. RETURN Return From A Subprogram

Format: **RETURN**

Açıklama: Alt programdan ana programa döndürür. **STACK**'teki adres bilgisini **PC**'a yükler.

İşlem: **STACK ⇒ PC**
Flag: -

33. SLEEP Stand by mode
Format: **SLEEP**
Açıklama: İşlemci düşük enerji tüketim moduna geçer. Osilatör durdurulur. **WDT** ve **WDT** prescaler resetlenir.
İşlem: 0 ⇒ **WDT**
0 ⇒ **WDT Prescaler**
1 ⇒ **TO**
0 ⇒ **PD**
Flag: **TO, PD**

34. SUBLW Subtract **W** From Literal
Format: **SUBLW k**
Açıklama: Sabit sayıdan **W**'daki sayıyı çıkarır. Sonucu **W**'ya yükler.
İşlem: **k - W ⇒ W**
Flag: **C, DC, Z**

Örnekler:
a) SUBLW H'79'

Komuttan önce:	W = H'78'	C = x Z = x	
Komuttan sonra:	W = H'01'	C = 1 Z = 0	Sonuç > 0
Komuttan önce:	W = H'79'	C = x Z = x	
Komuttan sonra:	W = H'00'	C = 1 Z = 1	Sonuç = 0
Komuttan önce:	W = H'80'	C = x Z = x	
Komuttan sonra:	W = H'FF'	C = 0 Z = 0	Sonuç < 0

35. XORLW Exclusive OR Literal with **W**
Format: **XORLW k**
Açıklama: **W** ile sabit sayı arasında lojik XOR (ÖZEL VEYA) işlemi yapar. Sonucu **W**'ya yükler.
İşlem: **W** (XOR) **k ⇒ W**
Flag: **Z**

Örnekler:
a) XORLW H'27'

Komuttan önce:	W = H'90'	0010 0111	H'27'
Komuttan sonra:	W = H'B7'	(XOR) 1001 0000	(XOR) H'90'
		1011 0111	H'B7'

Komutların çoğu 1 komut saykılı içerisinde çalıştırılır. Ancak yapılan bir test işleminin sonucu doğru ise veya program sayıcının gösterdiği adres komutun işlenmesi sonucu değişiyorsa çalıştırma işlemi 2 saykılda tamamlanır. Bir komut saykılı dört osilatör periyotudur. 4 MHZ'lik bir osilatör kullanılırsa, normal bir komutun icra süresi 1 μs olur. İki saykıllık komutlar ise 2 μs alacaktır.

f = 4 MHz ise;

f = 1/T $\Rightarrow$ 4 MHz = 1 / T $\Rightarrow$ T = 0,25 μs $\Rightarrow$ Bir komut saykılı 4T = 1 μs

Örneğin **BTFSC PORTB,2** komut satırı işlendiğinde **PORTB**'nin 2. biti 0 olursa program sayıcı bir komut satırı atlayacağı, yani değeri değişeceği için icra süresi 2 μs olur. Bitin değeri 1 olursa program bir alt satırdan devam edeceği için icra süresi 1 μs olur.

Sayıların Yazılış Formatı

Binary sayılar hexadecimal sayılara dönüştürülürken dörderli gruplar yapılarak bu dört bite karşılık gelen hexadecimal rakam yazılır. Örneğin;

```
B'11000101' = H'C5'
```

Hexadecimal sayılar yazılırken en çok kullanılan formatlar;

```
0x3F            3Fh             H'3F'
```

şeklindedir. Bu kitapta H' ' formatı kullanılacaktır. **MOVLW H'2C'** gibi.

Decimal sayılar yazılırken başına D, binary sayılar yazılırken B, Ascii karakterler yazılırken A harfi eklenir. D'25', B'11000001', A'Z' gibi.

Ascii karakterler yazılırken doğrudan tırnak içinde de yazılabilir. 'Z' gibi.

Açıklamalar

PIC Assembly dilinde açıklamaların başına ; işareti konur. Programı yazdığımız tarih, komut satırlarının ne amaçla yazıldığı gibi bilgiler açıklama satırı olarak yazılabilir. Bu şekilde uzun programlarda, programın çalışmasını anlamak kolaylaşır. ; işareti satırın başına veya bir komut satırının yanına konulabilir.

```
; --------------16.01.2007------------

LIST P=16F84        ; Kullanacağımız entegreyi tanıtır.
```

Etiketler

PIC Assembly dilinde iki tip etiket vardır; adres atanan etiketler ve adres atanmayan etiketler.

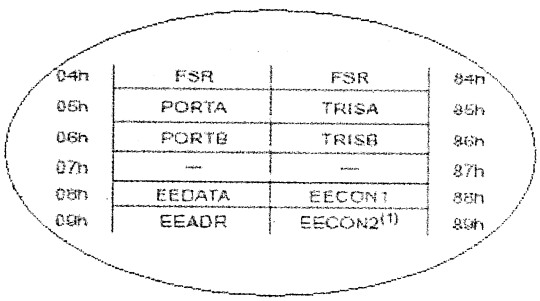
Adres Atanan Etiketler

Adres atanan etiketler bir adreste bulunan register'ı tanımlamak için kullanılır.

Önceden tanımlanmış olan **PORTB**, **TRISA**, vs. veya programcının tanımladığı **SAYAC**, **REG**, vs. birer etikettir.

Etikete adres atama işlemi **EQU** direktifi ile yapılır.

```
PORTB EQU H'06'
PORTA EQU H'05'
EECON1 EQU H'88'
```



EQU direktifi ile tanımlanan bu register'lar daha sonra kullanıldığında hangi adreste oldukları PIC tarafından bilinir. Böylece program yazarken register'ların adreslerini ezberlememize gerek kalmadan isimlerini yazarak kullanabiliriz.

```
BSF H'06', 2    =    BSF PORTB, 2
```

Yani **PORTB** aslında H'06' adresine verilmiş bir özel isimdir. Bu adres bölgesine bilgi depolamak istediğimizde H'06' sayısı yerine **PORTB** ismini kullanmak görüldüğü gibi işimizi oldukça kolaylaştırmaktadır.

PIC16F84'ün INCLUDE dosyasında Özel Amaçlı Register'ların tümünün adresi tanımlanmıştır. Programımıza bu dosyayı dahil edersek bu register'ların adreslerini tek tek **EQU** direktifi ile tanımlamamıza gerek kalmayacaktır. INCLUDE dosyasının içeriği Program Formatı başlığı altında verilmiştir.

Kullanıcının tanımladığı register'lar INCLUDE direktifi ile dahil edilemeyeceğinden mutlaka adres tanımlama işleminin yapılması gerekir. Aksi halde program bu register'ları tanımayacağından hata verecektir.

```
SAYAC   EQU   H'0C'
ADIM    EQU   H'0D'
```

Demek ki, programda kullanacağımız özel amaçlı register'ların adreslerini INCLUDE dosyasını programımıza dahil ederek, kendi tanımladığımız genel amaçlı register'ların adreslerini ise **EQU** direktifi ile tanımlayarak programa bildirmemiz gerekiyor.

Kullanıcının tanımladığı register'lara adres verirken, eğer register sayısı çok fazla ise tek tek **EQU** komutu kullanmak yerine **CBLOCK – ENDC** direktifleri kullanılabilir.

```
CBLOCK  H'0C'
   SAYAC1,SAYAC2,TUR,ADIM
ENDC
```

Bu kullanımda **CBLOCK** direktifinin yanında başlangıç adresi belirtilir, register isimleri yazılarak **ENDC** direktifi ile sonlandırılır. Örnekte sırasıyla H'0C' adresinde **SAYAC1**, H'0D' adresinde **SAYAC2**, H'0E' adresinde **TUR** ve H'0F' adresinde **ADIM** register'ı tanımlanmıştır.

Adres Atanmayan Etiketler

Adres atanmayan etiketler PIC'in çalışması sırasında döneceği program satırını gösteren etiketlerdir. Bu etiketler programcı tarafından oluşturulurlar. Bu etiketlerin adresi assembler tarafından atanır ve biz bu adresi bilmeyiz.

Her iki tür etiket için de bazı yazım kuralları vardır:

- Etiketler 1. sütuna yazılır.
- Etiketlerde Türkçe karakter kullanılmaz.
- Assembly komutları etiket olamaz.
- Etiketler en fazla 31 karakter olabilir.
- Etiketler harf veya alt çizgi ile başlamalıdır.
- Etiket içinde alt çizgi, rakam ve soru işareti kullanılabilir.

Program yazarken küçük veya büyük harf kullanmak programcıya bağlıdır. Yani bir satırda **movlw** diğer satırda **MOVLW** şeklinde bir kullanım mümkündür. Ancak etiketler için aynı şey söz konusu değildir. Örneğin aşağıdaki kullanım yanlıştır:

```
YUKLE
   MOVLW H'FF'
   MOVWF SAYAC
   .
   .
   .
   GOTO yukle
```

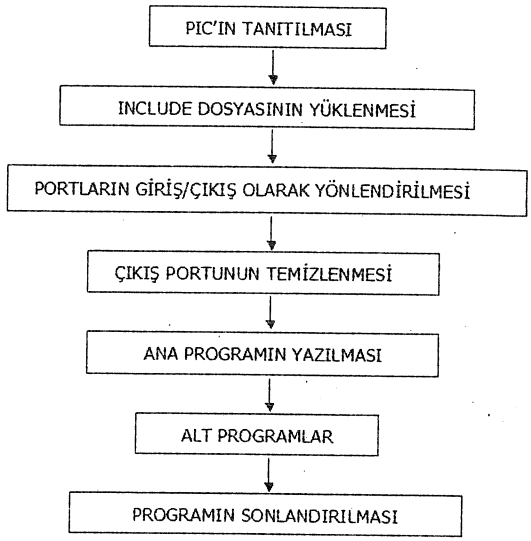
Etikete yazılan karakter program içerisinde yazılanla bire bir aynı olmalıdır.

INCLUDE dosyası ile özel amaçlı register'ları programımıza dahil etmişsek, program içerisinde register isimlerini yazarken bu dosyada yazıldığı şekliyle kullanmamız gerekir. Örneğin INCLUDE dosyası içerisinde **PORTB** yazarken programımıza **portb** yazarsak derleyici hata mesajı verecektir.

Program Formatı

PIC Assembly dilinde yazacağımız tüm programlarda işlem basamakları hemen hemen aynıdır. Bu basamakları teker teker inceleyerek programların temel yapısını öğreneceğiz. Yapının işleyişini iyice kavradıktan sonra kitaptaki programların çalışmasını anlamanız oldukça kolaylaşacaktır.

PROGRAM 1: PIC'e Enerji Verildiği Zaman PORTB'nin 0. Bitine Bağlı Ledi Yakan Program



Şimdi bu program üzerinde yukarıdaki aşamaları teker teker inceleyelim.

PIC'in Tanıtılması

Assembly dilinde yazacağımız programların ilk satırında mutlaka kullanacağımız PIC'i tanıtmamız gerekir. Tanıtma işlemi **LIST** direktifi ile yapılır.

```
LIST P=16F84
LIST P=16F877
```

Include Dosyasının Yüklenmesi

Assembler olarak kullanacağımız MPLAB programının klasörü içerisine girdiğinizde farklı PIC entegrelerine ait INCLUDE dosyalarının bulunduğunu göreceksiniz. Bu dosyalardan, uygulamalarımızda kullanacağımız PIC16F84 entegresine ait olanı – **P16F84.INC** – açarsanız aşağıdaki bilgilerle karşılaşacaksınız:

```
        LIST
; P16F84.INC  Standard Header File, Version 2.00    Microchip Technology, Inc.
        NOLIST

; This header file defines configurations, registers, and other useful bits of
; information for the PIC16F84 microcontroller.  These names are taken to match
; the data sheets as closely as possible.

; Note that the processor must be selected before this file is
; included.  The processor may be selected the following ways:
```

```
;       1. Command line switch:
;               C:\ MPASM MYFILE.ASM /PIC16F84
;       2. LIST directive in the source file
;               LIST   P=PIC16F84
;       3. Processor Type entry in the MPASM full-screen interface

;==========================================================================
;
;       Revision History
;
;==========================================================================

;Rev:   Date:    Reason:

;2.00   07/24/96 Renamed to reflect the name change to PIC16F84.
;1.01   05/17/96 Corrected BADRAM map
;1.00   10/31/95 Initial Release

;==========================================================================
;
;       Verify Processor
;
;==========================================================================

        IFNDEF __16F84
           MESSG "Processor-header file mismatch.  Verify selected processor."
        ENDIF

;==========================================================================
;
;       Register Definitions
;
;==========================================================================

W                               EQU     H'0000'
F                               EQU     H'0001'

;----- Register Files------------------------------------------------------

INDF            EQU     H'0000'
TMR0            EQU     H'0001'
PCL             EQU     H'0002'
STATUS          EQU     H'0003'
FSR             EQU     H'0004'
PORTA           EQU     H'0005'
PORTB           EQU     H'0006'
EEDATA          EQU     H'0008'
EEADR           EQU     H'0009'
PCLATH          EQU     H'000A'
INTCON          EQU     H'000B'

OPTION_REG      EQU     H'0081'
TRISA           EQU     H'0085'
```

```
TRISB           EQU     H'0086'
EECON1          EQU     H'0088'
EECON2          EQU     H'0089'

;----- STATUS Bits --------------------------------------------------

IRP             EQU     H'0007'
RP1             EQU     H'0006'
RP0             EQU     H'0005'
NOT_TO          EQU     H'0004'
NOT_PD          EQU     H'0003'
Z               EQU     H'0002'
DC              EQU     H'0001'
C               EQU     H'0000'

;----- INTCON Bits --------------------------------------------------

GIE             EQU     H'0007'
EEIE            EQU     H'0006'
TOIE            EQU     H'0005'
INTE            EQU     H'0004'
RBIE            EQU     H'0003'
TOIF            EQU     H'0002'
INTF            EQU     H'0001'
RBIF            EQU     H'0000'

;----- OPTION Bits --------------------------------------------------

NOT_RBPU        EQU     H'0007'
INTEDG          EQU     H'0006'
TOCS            EQU     H'0005'
TOSE            EQU     H'0004'
PSA             EQU     H'0003'
PS2             EQU     H'0002'
PS1             EQU     H'0001'
PS0             EQU     H'0000'

;----- EECON1 Bits --------------------------------------------------

EEIF            EQU     H'0004'
WRERR           EQU     H'0003'
WREN            EQU     H'0002'
WR              EQU     H'0001'
RD              EQU     H'0000'

;=========================================================================
;
;       RAM Definition
;
;=========================================================================

        __MAXRAM H'CF'
        __BADRAM H'07', H'50'-H'7F', H'87'
```

```
;==========================================================================
;
;       Configuration Bits
;
;==========================================================================

_CP_ON          EQU     H'000F'
_CP_OFF         EQU     H'3FFF'
_PWRTE_ON       EQU     H'3FF7'
_PWRTE_OFF      EQU     H'3FFF'
_WDT_ON         EQU     H'3FFF'
_WDT_OFF        EQU     H'3FFB'
_LP_OSC         EQU     H'3FFC'
_XT_OSC         EQU     H'3FFD'
_HS_OSC         EQU     H'3FFE'
_RC_OSC         EQU     H'3FFF'
```

INCLUDE dosyasının içerisinde Özel Amaçlı Register'ların adresleri, bu register'lar içerisindeki bitlerin numaraları ve konfigürasyon tanımlamaları vardır.

Program yazarken kullanılan register'ların adreslerinin tanımlı olması gerekmektedir. Etiketler konusunda adres tanımlama işleminin **EQU** komutu ile yapıldığını görmüştük. INCLUDE dosyasının içerisinde tüm özel amaçlı register'ların adresleri **EQU** komutu ile tanımlanmıştır. Özel amaçlı register'ların adresleri sabit olduğuna göre her programda tek tek adres tanımlamak yerine kullanacağımız mikrodenetleyiciye ait INCLUDE dosyasını yükleriz. Böylece programımızda bu register'ların adreslerini tek tek tanıtma zorunluluğu ortadan kalkacak, böylece programımız oldukça kısalacaktır. Yükleme işlemi **INCLUDE** direktifi ile aşağıdaki şekillerde yapılabilir.

```
INCLUDE "P16F84.INC"
#INCLUDE P16F84.INC
```

Dosya içerisinde ayrıca bazı özel amaçlı register'lar içerisindeki bitlerin numaraları da tanımlanmıştır. Örneğin **STATUS** register içerisindeki bitlerden **RP0** bitinin **RP0 EQU H'0005'** şeklinde bir tanımlama ile 5. bit olduğu belirtilmiştir. O halde programlarımızda aşağıdaki iki yolu da kullanmamız mümkündür.

```
BSF STATUS,5        VEYA        BSF STATUS,RP0
```

Programa **INCLUDE** direktifi ile bir dosyayı dahil ettiğimizde o dosyadaki komutlar oraya yazılmış gibi işlem görür.

INCLUDE dosyası içerisinde yer alan konfigürasyon bitlerinin görevleri ilerleyen sayfalarda açıklanacaktır.

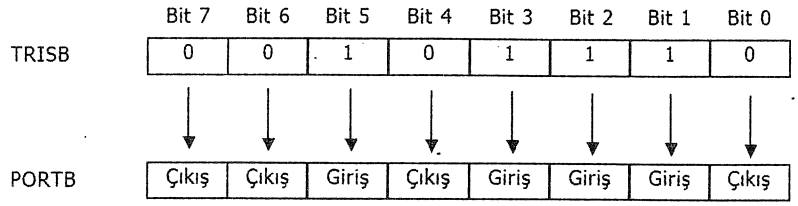
Portların Giriş-Çıkış Olarak Yönlendirilmesi

PIC16F84'ün 8 adet **PORTB**'de, 5 adet de **PORTA**'da olmak üzere toplam 13 adet giriş/çıkış (I/O) pini vardır. Kuracağımız devreye göre bu pinlerin giriş veya çıkış olarak yönlendirilmesi gerekebilir. Portları giriş veya çıkış olarak seçmek için **TRIS** adı verilen özel register'lar kullanılır.

TRISA PORTA'nın giriş/çıkış olma durumunu belirler

TRISB PORTB'nin giriş/çıkış olma durumunu belirler

TRIS register'larında bir bitin 0 yapılması portta ona karşılık gelen pinin çıkış olacağını, 1 yapılması ise giriş olacağını ifade eder.



PORTA register'ının sadece alt beş biti kullanıldığından son üç bitin değeri önemli değildir.

TRISA ve **TRISB** register'larının default değeri 1'dir. Yani programda herhangi bir ayarlama yapılmazsa **PORTA** ve **PORTB** giriş olarak yönlendirilmiş durumdadır. Bu yüzden programlarımızda giriş olarak kullanacağımız pinler için bir yönlendirme yapmadık. Örneğin **PORTA**'nın tüm bitlerine buton bağlayacağız, yani hepsi giriş olarak yönlendirilecek. Bu durumda **TRISA**'ya H'FF' bilgisi yüklememiz gerekir. Ancak zaten **TRISA**'nın default değeri 1 olduğundan bu bilgiyi yüklemesek de program çalışacaktır.

PIC16F84'ün veri belleğinin iki banka ayrılmış olduğunu ve kullanacağımız register hangi bankta ise o banka geçmemiz gerektiğini daha önce görmüştük. Register haritası incelenirse **TRISA** ve **TRISB** register'larının Bank1'de yer aldığı görülür. Programın başında Bank0'da bulunduğumuz için **TRISA** ve **TRISB** register'larına erişebilmek için Bank Değiştirme işlemi yapmamız gerekir.

03h	STATUS	STATUS	83h
04h	FSR	FSR	84h
05h	PORTA	TRISA	85h
06h	PORTB	TRISB	86h
07h	—	—	87h
08h	EEDATA	EECON1	88h
09h	EEADR	EECON2(1)	89h
0Ah	PCLATH	PCLATH	8Ah
0Bh	INTCON	INTCON	8Bh
0Ch			8Ch
	Bank 0	Bank 1	

Status Register'ın 5. ve 6. bitleri (RP0 ve RP1) bank değiştirme bitleridir. Bu bitlerin değerine göre hangi bankın seçileceği aşağıdaki tabloda verilmiştir.

	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
STATUS	IRP	**RP1**	**RP0**	TO	PD	Z	DC	C

RP1	RP0	
0	0	→ Bank0
0	1	→ Bank1
1	0	→ Bank2
1	1	→ Bank3

PIC16F84'de Bank0 ve Bank1 olmak üzere 2 adet bank olduğundan seçme işlemi için tek bit yeterlidir. Bu yüzden sadece RP0 bitini değiştirerek bank değiştirme işlemini gerçekleştirebiliriz.

RP0	
0	→ Bank0
1	→ Bank1

O halde PIC16F84 entegresi için yazılan programlarda RP1 biti kullanılmayacaktır.
Sonuç olarak portların giriş/çıkış olarak yönlendirilmesi için izleyeceğimiz yol şudur:

- Bank1'e geç.
- Portlarda çıkış olmasını istediğin pinlere karşılık gelen **TRIS** register'ı bitlerini 0, giriş olmasını istediğin pinlere karşılık gelen TRIS register'ı bitlerini 1 yap.
- Bank0'a geri dön.

Bir register içerisindeki belirli bir biti 1 yapmak için **BSF** komutu, 0 yapmak içinse **BCF** komutu kullanılır.

```
BSF f, b            ; Register içerisindeki b. biti 1 yap.

BSF PORTB, 3        ; PORTB'nin 3. bitini 1 yap.

BCF f, b            ; Register içerisindeki b. biti 0 yap.

BCF SAYAC, 1        ; SAYAC'ın 1. bitini 0 yap.
```

BSF STATUS, 5 → BANK 1

BCF STATUS, 5 → BANK 0

Programımızda led **PORTB**'nin 0. bitine bağlıdır. Demek ki bu biti çıkış olarak yönlendireceğiz.

```
BSF STATUS,5        ; STATUS register'ının 5. bitini 1 yaparak Bank1'e geç.
BCF TRISB,0         ; TRISB'nin 0. bitini 0 yaparak PORTB'nin 0. bitini çıkış
                    ; yap.
BCF STATUS,5        ; STATUS register'ının 5. bitini 0 yaparak Bank0'a geç.
```

Aynı porta hem giriş, hem çıkış elemanı bağlanmayacaksa genel yöntem portun tamamını giriş veya çıkış yapmaktır. Yani bu uygulamada **PORTB**'nin tamamını çıkış yapabiliriz.

```
BSF STATUS,5
CLRF TRISB
BCF STATUS,5
```

CLRF komutu bir register içerisindeki 8 biti birden 0 yapar. Yani register'ın içeriğini temizler.

```
CLRF f          ; Register'ın tüm bitlerini 0 yap.

CLRF TUR        ; TUR register'ı içerisindeki tüm bitleri 0 yap.
```

Yukarıda **CLRF TRISB** komut satırı ile **TRISB** register'ının tüm bitleri 0 yapılmış, yani **PORTB**'nin tamamı çıkış olarak yönlendirilmiştir.

Çıkış Portunun Temizlenmesi

Daha önce de belirttiğimiz gibi devreye enerji verildiğinde **PORTA** ve **PORTB** pinlerinin lojik seviyesi belirsizdir; yani 1 veya 0 olabilir. Bu yüzden programın başında çıkış portunun temizlenmesi gerekir. Temizleme işlemi **CLRF** komutu ile yapılır.

```
CLRF PORTB
```

Ana Programın Yazılması

Örnek olarak incelediğimiz bir ledi yakma programının ana programı tek satırdan ibarettir. Ledin yanması için **PORTB**'nin 0. bitine +5V yani lojik 1 göndermemiz gerekir. Bu işlemi yine **BSF** komutu ile yapacağız.

```
BSF PORTB,0
```

Bu kadar kısa süren programlar olduğu gibi, sayfalarca sürecek programlar da olabilir. Programın boyutu çok uzun olduğu zaman izlenen yol; ana programı bir blok halinde yazmak yerine, yapılan temel işlemleri küçük program parçaları halinde yazmak (alt program) sonra da ana programdan bunları tek tek çağırıp çalıştırmaktır.

```
BASLA
   CALL ALT_PROG1
   CALL ALT_PROG2
   ...
   ...
ALT_PROG1
   ...
ALT_PROG2
   ...

   END
```

Alt Programlar

Yazdığımız bir programda aynı işlem birden çok kez yapılabilir. Örneğin bir flaşör programını düşünün. Yapılan işlem şu şekildedir: ledi yak-bekle-ledi söndür-bekle-ledi yak...

Gördüğünüz gibi bekleme işleminin birden çok yerde yapılması gerekiyor. Böyle bir durumda kısa bir gecikme sağlayan bir alt program yazılır. İhtiyaç duyulduğu zaman bu program çağrılır. Bir müddet gecikme sağlandıktan sonra program çalışmasına kaldığı yerden devam eder. Alt programlar **CALL** komutu ile çağrılır.

```
CALL k          ; k alt programını çağırır.
```

Alt programlar **RETURN** komutu ile sonlandırılır.

```
RETURN          ; Alt programdan ana programda kaldığın yere geri dön.
```

Bildiğiniz gibi her program satırının bir adresi vardır. **CALL** komutu ile bir alt program çağrıldığı zaman, programın alt programı çalıştırdıktan sonra döneceği adresi bilmesi gerekir. Bu adres bilgisi STACK REGISTER adı verilen özel bir register'da tutulur. Alt programların çalışma mantığını şu şekilde özetleyebiliriz.

- Ana program çalışırken **CALL ALT_PROG** komutunu görür.
- Bu andaki adres bilgisini STACK REGISTER'a kaydeder.
- **ALT_PROG** alt programına dallanarak buradaki komutları işler.
- **RETURN** komutunu görünce STACK register'dan döneceği yerin adres bilgisini alır.
- Kaldığı adrese geri dönerek ana program devam eder.

Örnek programımızda alt program kullanmaya gerek yoktur.

Programın Sonlandırılması

Assembly dilinde programı bir yerde durdurma komutu olmadığından durdurma işlemi için sonsuz döngü kullanılır.

```
DONGU
   GOTO DONGU
```

```
GOTO k          ; k etiketine dallanır.
```

Program sürekli **DONGU** etiketine dallanacağı için bu döngüden çıkamaz, program durdurulmuş olur.

GOTO komutunun **CALL** komutundan farkı şudur: **CALL** komutunda programın o noktasında standart birtakım işlemler yapılması gerektiği için, sonradan aynı noktaya geri dönmek üzere bir alt program çağrılır. Çünkü tüm **CALL** komutları **RETURN** ile biter, yani geri dönüş kesindir. **GOTO** komutunda ise program başka bir adrese dallandırılıp işleyiş oradan itibaren devam eder. Kalınan yere yeni bir **GOTO** komutu ile geri dönülebilir, ancak bu bir mecburiyet değildir.

Bu programda ledi yaktıktan sonra programı bitirirsek, yani sonsuz döngü yazmazsak ledin sürekli yanıp söndüğü (titreştiği) görülür. Çünkü mikrodenetleyici programı sürekli olarak tarar, yani sürekli baştan okur. Bu yüzden her defasında **CLRF PORTB** komutunu gördüğünde ledi söndürecek, **BSF PORTB,0** komutunu gördüğünde ise ledi yakacaktır. Devreyi kurup ledin durumunu incelediğimizde titreşme şeklinde ledin yanıp söndüğünü görürüz.

Assembly dilinde yazılan tüm programlar mutlaka **END** direktifi ile sonlandırılmalıdır. Şimdi yazdığımız satırları birleştirerek ilk programımızın nasıl olduğunu görelim:

```
   LIST P=16F84            ; PIC16F84 kullanılacak
   INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
   BSF STATUS,5            ; Bank 1'e geç
   CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
   BCF STATUS,5            ; Bank 0'a geç
   CLRF PORTB              ; PORTB'yi temizle
   BSF PORTB,0             ; PORTB'nin 0. bitine bağlı ledi yak
DONGU
   GOTO DONGU              ; Ledin sürekli yanmasını sağla
   END                     ; Programı bitir
```

Konfigürasyon Bitleri

INCLUDE dosyasını açtığımız zaman konfigürasyon bitleri başlığı altında aşağıdaki bilgilerle karşılaşırız.

```
;       Configuration Bits
;
;==========================================================================

_CP_ON                  EQU     H'000F'
_CP_OFF                 EQU     H'3FFF'
_PWRTE_ON               EQU     H'3FF7'
_PWRTE_OFF              EQU     H'3FFF'
_WDT_ON                 EQU     H'3FFF'
_WDT_OFF                EQU     H'3FFB'
_LP_OSC                 EQU     H'3FFC'
_XT_OSC                 EQU     H'3FFD'
_HS_OSC                 EQU     H'3FFE'
_RC_OSC                 EQU     H'3FFF'
```

Konfigürasyon bitleri PIC'in çalıştırılmasında kullanılacak osilatör tipi, Watchdog Timer kullanılıp kullanılmadığı, kod koruması olup olmadığı gibi bilgileri program içerisinde belirtmemize yarayan bitlerdir. Şimdi bunları sırasıyla inceleyelim.

CP (Kod koruması): PIC içerisine yüklediğimiz programın başkalarınca okunmasını engelleyen bir özelliktir. Eğer kod koruması ON (açık) yapılırsa PIC'i okuttuğunuz zaman içerisindeki program kodları görülmeyecektir.

WDT (Watchdog Timer): Eğer WDT özelliği kullanılacaksa ON yapılması gerekir.

PWRT (Power-up Timer): Bu özellik PIC'e enerji verildikten sonra 72 ms'lik bir bekleme süresi sağlayarak besleme geriliminin kararlı bir seviyeye çıkmasını, böylece PIC'in gerilim dalgalanmalarından etkilenmemesini sağlamak için kullanılır.

OSC (Osilatör): Kuracağınız devrede PIC'i çalıştırmak için seçeceğiniz osilatör tipini belirler. LP düşük güçlü kristal, XT kristal/rezonatör, HS yüksek hızlı kristal/rezonatör, RC ise direnç/kondansatör osilatör anlamına gelir.

Yazıp derlediğimiz programı entegreye yüklemek için kullanacağımız ICPROG programı ve diğer yükleme programlarının hemen hemen hepsi konfigürasyon ayarlarının yükleme esnasında yapılmasına izin verir. Yapılacak işlem ekranda istediğimiz özellikleri işaretlemekten ibarettir. Ancak yazdığımız programda da konfigürasyon tanımlamalarını yapabiliriz. Bunun için **__CONFIG** ifadesini kullanıyoruz.

```
__CONFIG _CP_ON & _WDT_OFF & _XT_OSC & _PWRTE_ON
```

Yukarıdaki komut satırına göre programda Watchdog Timer kullanılmayacak, Power-up Timer devrede olacak, kristal osilatör veya rezonatör kullanılacak ve program kodları okumaya karşı korunacaktır. İfadeler arasında & işareti kullanmamız gerekir. Konfigürasyon tanımlamaları – eğer yapılacaksa – programda **INCLUDE** komut satırının hemen altına yazılmalıdır.

```
LIST P=16F84
INCLUDE "P16F84.INC"
__CONFIG _CP_ON & _WDT_OFF & _XT_OSC & _PWRTE_ON
BSF STATUS,5
    ...
    ...
```

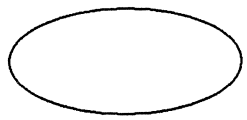
Akış Diyagramları

Akış Diyagramı Nedir?

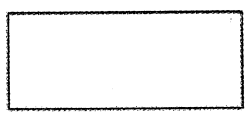
Akış diyagramı; bilgisayar programlarının işlem basamaklarını geometrik şekillerle gösteren şemadır. Akış diyagramının en büyük avantajı, programın görsel olarak takip edilebilmesini, dolayısıyla daha kolay anlaşılabilmesini sağlamasıdır. Çok uzun programlarda işlem basamakları akış diyagramı üzerinde gösterildiğinde programa sonradan müdahale etmek, hangi komutun neden kullanıldığını anlamak oldukça kolaylaşır.

Aşağıda akış diyagramlarında kullanılan şekiller ve açıklamaları verilmiştir.

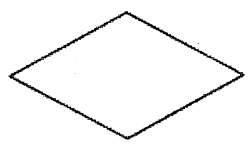
Akış Diyagramı Şekilleri



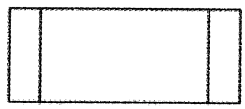
BAŞLA/SON: Tüm akış diyagramları "BAŞLA" şekli ile başlar, "SON" şekli ile biter. Bu iki şekil arasında diğer işlemler yapılır.



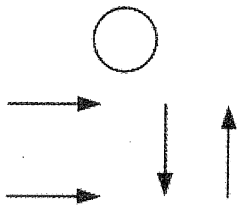
İŞLEM: Programın çalışması sırasında yapılacak işlemler bu şekil içerisine yazılır.



KARŞILAŞTIRMA/KARAR: Karşılaştırma işlemi yaparak sonuçta verilen kararın Evet ve Hayır oluşuna göre farklı yerlere dallanır.



ALT PROGRAM: Alt programlar bu şekil içerisinde belirtilir.

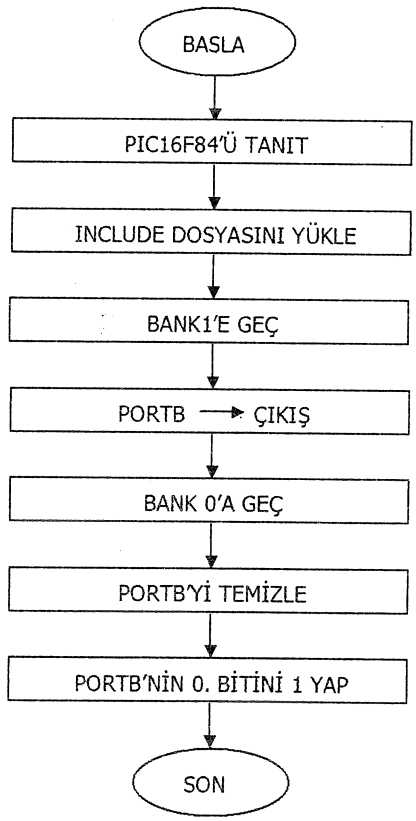


BAĞLANTI: İki akış noktasını birbirine bağlar. Genellikle karışık programlarda kullanılır.

AKIŞ YÖNÜ: Akış diyagramındaki şekilleri birbirine bağlar. Programın akış yönünü gösterir.

Akış Diyagramı Kullanarak Program Yazma

Yukarıdaki ilk programımızda **PORTB**'nin 0. bitine bağlı ledi yakmayı öğrendik. Bu basit bir program olduğundan işlem basamaklarını rahatça program üzerinde takip edebiliyorsunuz. Ancak kitabın daha ileriki programlarında önce akış diyagramı çizip programı ona göre yazmak işimizi daha da kolaylaştıracaktır. Şimdi yukarıdaki programın akış diyagramının nasıl olacağını inceleyelim.



Lojik Kapılar

Dijital elektroniğin temel taşı olan lojik kapı fonksiyonları, programlamada da bazı işlemlerin gerçekleştirilmesinde büyük önem taşır. Bu yüzden kitap içerisinde geçen lojik fonksiyonları kısaca hatırlatmakta fayda var.

VE (AND) Kapısı

VE kapısı aritmetik çarpma işlemi yapar. Yani girişlerden herhangi biri 0 olduğunda çıkış 0'dır. Çıkışın 1 olması için her iki girişin de lojik-1 olması gerekir.

A	B	Q
0	0	0
0	1	0
1	0	0
1	1	1

ŞEKİL 17: VE kapısının doğruluk tablosu ve simgesi.

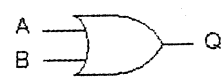
VE kapısı programlamada istenilen bitlerin 0 yapılmasında kullanılır. Örneğin B'10110011' sayısının 0. ve 7. bitlerini 0 yapmak, diğer bitlerini ise değerlerini değiştirmeden korumak istiyoruz. Bu durumda sayıyı B'01111110' bilgisi ile VE işlemine tabi tutarız. Yani 0 yapmak istediğimiz bitleri 0, değişmemesini istediğimiz bitleri ise 1 ile çarparız.

```
   10110011
(x) 01111110
   --------
   00110010
```

VEYA (OR) Kapısı

VEYA kapısı aritmetik toplama işlemi yapar. Yani girişlerden herhangi biri 1 olduğunda çıkış 1'dir. Çıkışın 0 olması için her iki girişin de lojik-0 olması gerekir.

A	B	Q
0	0	0
0	1	1
1	0	1
1	1	1



ŞEKİL 18: VEYA kapısının doğruluk tablosu ve simgesi.

VEYA kapısı programlamada istenilen bitlerin değerini 1 yapmak amacıyla kullanılır. Örneğin B'11001100' sayısının 0. ve 1. bitlerini 1 yapmak, diğer bitlerini ise değerleri değişmeden korumak istiyoruz. Bu durumda sayıyı B'00000011' sayısı ile VEYA işlemine tabi tutarız.

```
   11001100
(+) 00000011
------------
   11001111
```

ÖZEL VEYA (EXOR) Kapısı

ÖZEL VEYA kapısı karşılaştırma işlemi yapar. Yani girişler birbirine eşit olduğunda lojik-0, girişler birbirinden farklı olduğunda ise lojik-1 bilgisi üretir.

A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

ŞEKİL 19: ÖZEL VEYA kapısının doğruluk tablosu ve simgesi.

ÖZEL VEYA kapısı programlamada bilgilerin karşılaştırılması amacıyla kullanılır. Örneğin **SAYAC1** ve **SAYAC2** isimli iki register içerisindeki bilgilerin aynı olup olmadığını tespit etmek istiyoruz. Bu durumda bu iki register arasında ÖZEL VEYA işlemi yaparız.

```
      SAYAC1   ------>      11011101
(EXOR) SAYAC2  ------>  (EXOR) 11011101
------------------------  ------------
      SONUÇ    ------>      00000000
```

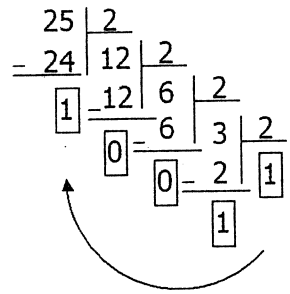
Sonucun 0 çıkması register'lar içerisindeki bilgilerin eşit olduğu anlamına gelir. Program içerisinde sonucun 0 olup olmadığını kontrol ettirerek sayıların eşit olup olmadığını kontrol edebiliriz.

Sayı Sistemlerinin Birbirine Çevrilmesi

Decimal Sayının Binary Sayıya Çevrimi

Decimal sayıyı binary sayıya çevirmek için sayı sürekli olarak 2'ye bölünür ve elde edilen kalanlar işaretlenir. Daha sonra kalanlar tersten yazılarak binary değer elde edilir.

D(25) = B (?)



Decimal 25 sayısının binary karşılığı 11001 bilgisidir. Bu işlemi kısa yoldan şu şekilde de yapabiliriz. 8 bitlik bir binary sayıyı düşünelim. Bu sayının bitlerinin basamak değerleri aşağıdaki gibidir:

128	64	32	16	8	4	2	1

Buna göre bir decimal sayıyı binary sayıya çevirmek istediğimizde toplamları o sayıyı veren bitlere 1, diğer bitlere ise 0 yazmamız gerekir. Örneğimizdeki 25 sayısı yukarıdaki basamak değerlerinden 16+8+1 şeklinde elde edilebileceğinden bu basamaklara 1, diğerlerine 0 yazdığımızda elde edeceğimiz sayı 00011001 şeklinde olacaktır.

Decimal 135 sayısını binary sayıya kısa yoldan çevirelim. 135 = 128 + 4 +2 + 1 olduğundan binary karşılığı 10000111 şeklinde elde edilir.

Binary Sayının Decimal Sayıya Çevrimi

Binary sayıyı decimal sayıya çevirmek için sayının 1 olan bitlerinin basamak değerleri toplanır. Toplam decimal sayıyı verir. Örneğin B'01101001' sayısını decimal sayıya çevirelim.

128	64	32	16	8	4	2	1
0	1	1	0	1	0	0	1

⟹ 64 + 32 + 8 + 1 = 105

Aynı şekilde B'00110101' sayısının decimal karşılığı aşağıdaki şekilde bulunur.

0	0	1	1	0	1	0	1

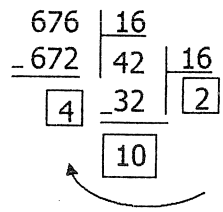
⟹ 32 + 16 + 4 + 1 = 53

Decimal Sayının Hexadecimal Sayıya Çevrimi

Hexadecimal sayılar 0-15 arasındaki 16 adet rakamdan meydana gelir. 9'dan 15'e kadar olan rakamlar A, B, C,D, E ve F harfleri ile ifade edilir.

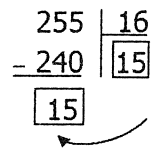
Decimal sayıyı hexadecimal sayıya çevirmek için sayı sürekli olarak 16'ya bölünür ve elde edilen kalanlar işaretlenir. Daha sonra kalanlar tersten yazılarak hexadecimal değer elde edilir.

D(676) = H (?)



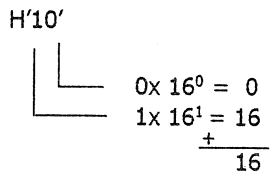
Elde edilen kalanlardan ikincisi 10'dur. Ancak hexadecimal sayılarda 10 yerine A harfi kullanıldığından sonuç H'2A4' olur.

Aynı şekilde D'255' sayısının hexadecimal karşılığı aşağıdaki işlem sonucuna göre H'FF' olur.



Hexadecimal Sayının Decimal Sayıya Çevrimi

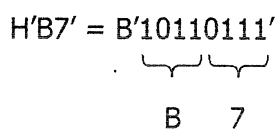
Hexadecimal sayının decimal sayıya çevrilmesi için her basamaktaki sayı ile o basamağın değeri çarpılarak elde edilen sonuçlar toplanır. Basamak değerleri 16^0, 16^1, 16^2... şeklinde sıralanır. Örneğin H'10' sayısını decimal sayıya çevirelim.

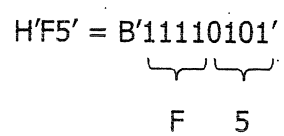


Yani H'10' sayısı D'16' sayısına eşittir.

Hexadecimal Sayının Binary Sayıya Çevrimi

Hexadecimal sayılar binary sayılara çevirilirken her bir rakamın binary karşılığı dört bit ile yazılır. Elde edilen sayı binary karşılığı verir.





Binary Sayının Hexadecimal Sayıya Çevrimi

Binary sayı hexadecimal sayıya çevrilirken sayı en sağdan itibaren dörderli gruplara ayrılır ve her bir grubun hexadecimal rakam değeri yazılarak sonuç elde edilir.

B'00011010' = H'1A'

1 A

B'10000110' = H'86'

8 6

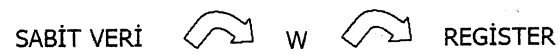
Bölüm 4

Portlara Veri Gönderme

Register'lara veri gönderme işlemi **W** register'ı aracılığıyla yapılır. Portlar da birer register olduğuna göre tüm veri aktarımlarında **W** register'ını kullanacağız.

Sabit Veriyi Register'a Yüklemek

Bir sabit sayıyı herhangi bir register'a yüklemek için **MOVLW** ve **MOVWF** komutları birlikte kullanılır. Sayı önce **MOVLW** komutu ile **W** register'ına yüklenir, daha sonra **MOVWF** komutu ile **W**'daki bilgi hedef register'a atılır.

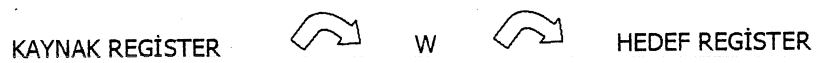



```
MOVLW  k                ; k sabit sayısını W'ya yükle

MOVWF  f                ; W'daki sayıyı f register'ına yükle
```

```
MOVLW  B'11110000'      ; B'11110000' sayısını W'ya yükle
MOVWF  PORTB            ; W'daki sayıyı PORTB'ye yükle

MOVLW  D'25'            ; D'25' bilgisini
MOVWF  SAYAC            ; SAYAC'a yükle
```

Bir Register'daki Bilgiyi Başka Bir Register'a Yüklemek

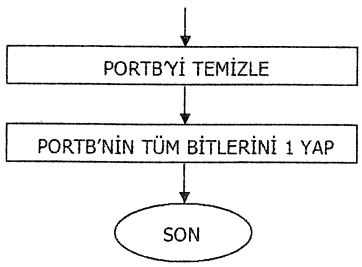
Bir register'daki bilgiyi başka bir register'a yüklemek için **MOVF** ve **MOVWF** komutları birlikte kullanılır. Sayı önce **MOVF** komutu ile kaynak register'dan **W** register'ına yüklenir, daha sonra **MOVWF** komutu ile **W**'daki bilgi hedef register'a atılır.




```
MOVF   f, d             ; Register'daki bilgiyi W'ya yükle

MOVWF  f                ; W'daki sayıyı f register'ına yükle
```

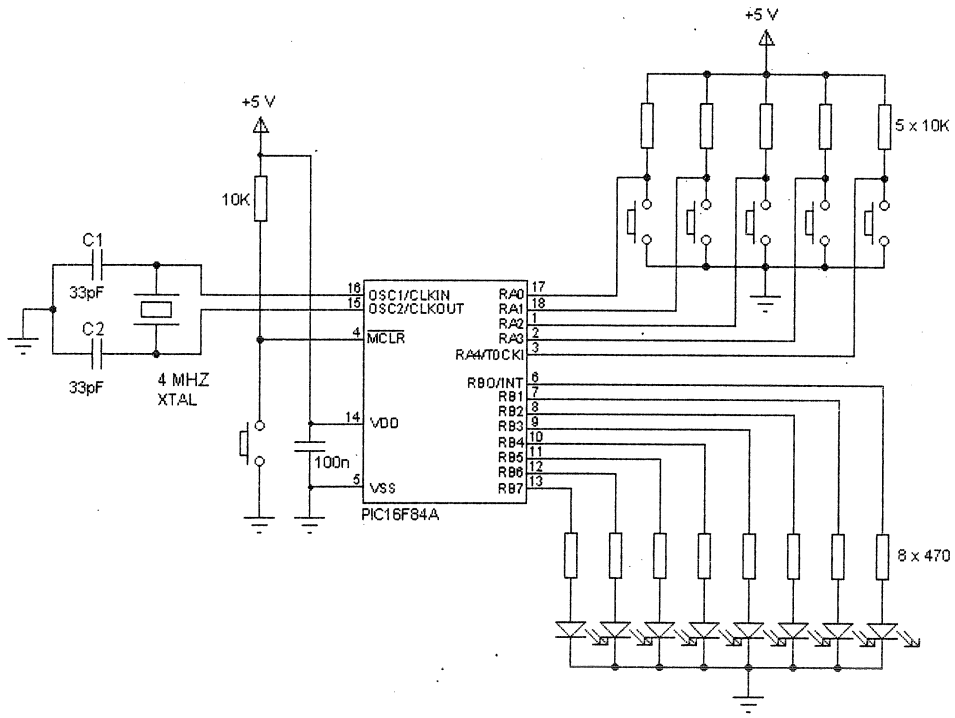
```
MOVF   SAYAC,W          ; SAYAC register'ındaki bilgiyi W'ya yükle
MOVWF  PORTB            ; W'daki sayıyı PORTB'ye yükle

MOVF   PORTA,W          ; PORTA'daki bilgiyi
MOVWF  PORTB            ; PORTB'ye yükle
```

PROGRAM 2: PORTB'ye Bağlı Olan Tüm Ledleri Yakan Program




```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF PORTB              ; PORTB'yi temizle
    MOVLW H'FF'             ; PORTB'ye H'FF' bilgisini yükle
    MOVWF PORTB             ; (Yani tüm ledleri yak)
DONGU
    GOTO DONGU              ; Programı durdur
    END
```

PORTB'ye bağlı olan tüm ledleri yakabilmek için 8 bitine de lojik-1 göndermemiz gerekir. Yani **PORTB**'ye B'11111111'(H'FF') sabit sayısı gönderilmelidir. **PORTB** bir register, H'FF' de bir sabit sayı olduğuna göre kullanmamız gereken komut **MOVLW**'dir.



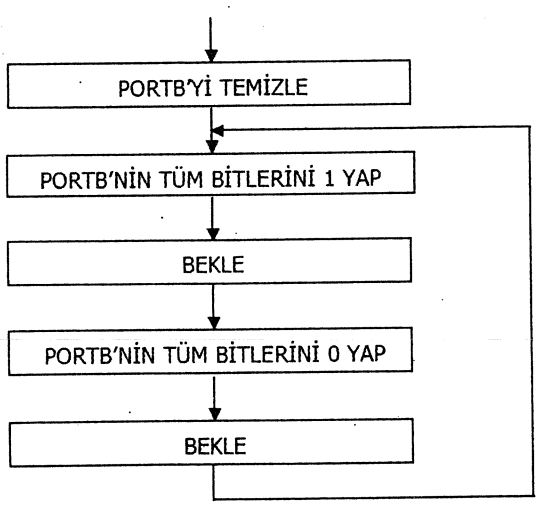
ŞEKİL 20: Buton ve ledli uygulamalarda kullanılabilecek basit bir devre.

PROGRAM 3: PORTB'ye Bağlı Ledleri • ○ • ○ • ○ Şeklinde Yakan Program

```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF PORTB              ; PORTB'yi temizle
    MOVLW H'AA'             ; PORTB'ye H'AA' bilgisini yükle
    MOVWF PORTB             ; (Yani ledleri istenilen şekilde yak)
DONGU
    GOTO DONGU              ; Programı durdur
    END
```

Bu programda portta yanmasını istediğimiz ledlere 1, sönmesini istediğimiz ledlere 0 bilgisini göndermemiz gerekir. Bu durumda göndereceğimiz bilgi B'10101010' yani H'AA' olmalıdır. Yine bir sabit sayıyı **PORTB** register'ına atmak sözkonusu olduğundan **MOVLW** komutu kullanılmalıdır.

PROGRAM 4: PORTB'ye Bağlı Ledleri Sürekli Yakıp Söndüren Program (Flaşör Programı)




```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
SAYAC1 EQU H'0C'            ; H'0C' adresinde SAYAC1,
SAYAC2 EQU H'0D'            ; H'0D' adresinde SAYAC2 isimli bir register oluştur
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF PORTB              ; PORTB'yi temizle
DONGU
    MOVLW H'FF'             ; PORTB'ye H'FF' bilgisini yükle
    MOVWF PORTB             ; (Yani tüm ledleri yak)
```

```
    CALL GECIKME          ; Bir süre bekle (Ledler yanık kalsın)
    CLRF PORTB            ; Tüm ledleri söndür
    CALL GECIKME          ; Bir süre bekle (Ledler sönük kalsın)
    GOTO DONGU            ; Ledleri tekrar yakmak için DONGU etiketine dallan
GECIKME
    MOVLW H'FF'
    MOVWF SAYAC1
DONGU1
    MOVLW H'FF'
    MOVWF SAYAC2          ; GECIKME alt programının açıklaması
DONGU2                    ; Sayıcılar bölümünde verilmiştir. Bu alt program ana
    DECFSZ SAYAC2,F       ; programı bir süre kaldığı yerde bekletir.
    GOTO DONGU2
    DECFSZ SAYAC1,F
    GOTO DONGU1
    RETURN
    END
```

Ledleri flash yaptırabilmek için yak-söndür-yak-söndür... işleminin sürekli tekrar edilmesi gerekir. Ancak PIC çok hızlı çalışan bir eleman olduğundan (4 MHZ osilatör frekansında bir komutun işlenmesi 1μs yani saniyenin milyonda biri kadar zaman alır) yakıp söndürme işlemini peş peşe yaptırırsak gözümüz ledlerin yanıp söndüğünü algılayamaz. Bu yüzden mutlaka bir gecikme programı yazmamız gerekir. Bu programın çalışma mantığı sayıcılar konusunda anlatılacaktır. **SAYAC1** ve **SAYAC2** register'ları da gecikme alt programında yer aldıklarından bunlarla ilgili açıklama da burada verilmemiştir.

Bazı programlarda yapılan işlemin sürekli olarak tekrarlanması gerekir. Örneğin flaşör programında ledlerin yanıp sönme işlemi sürekli olarak tekrarlanmaktadır. Böyle durumlarda döngü düzenlenir. Yapılacak işlem tekrarlanan kısmın başına bir etiket yazmak, sonra da **GOTO** komutu ile sürekli o etikete dallanmaktır.

```
DONGU
    ...
    ...
    GOTO DONGU
```

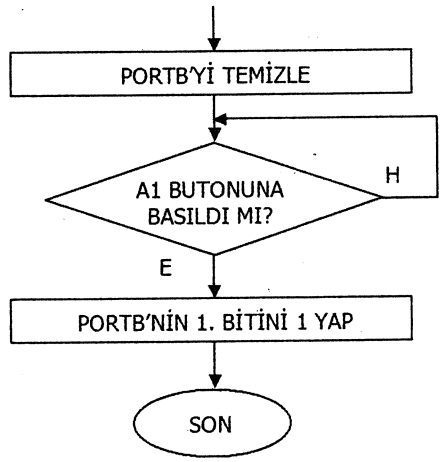
Program **GOTO DONGU** komutunu her gördüğünde **DONGU** etiketine dallanacak ve o satırdan itibaren olan komutları işleyecektir. Yani **GOTO** komutu program sayıcının konumunu değiştirir.

Bölüm 5

Buton Kontrolü

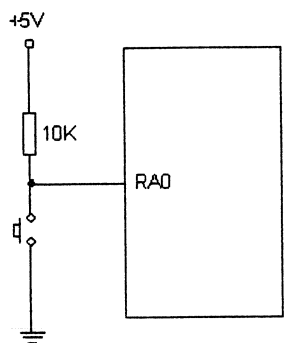
Tek Butonlu Uygulamalar

PROGRAM 5: PORTA'nın 1. Bitine Bağlı Butona Basınca PORTB'nin 1. Bitine Bağlı Ledi Yakan Program




```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF PORTB              ; PORTB'yi temizle
BASLA
    BTFSC PORTA,1           ; RA1 butonuna basıldı mı?
    GOTO BASLA              ; Hayır, yeniden test etmek için BASLA etiketine dallan
    BSF PORTB,1             ; Evet, RB1'e bağlı ledi yak
DONGU
    GOTO DONGU              ; Programı durdur
    END
```

Bu programı anlamak için önce buton bağlantısının nasıl yapıldığını incelemek yerinde olur. Butonlar genel olarak 0 aktif çalışacak şekilde aşağıdaki gibi bağlanır.



ŞEKİL 21: PIC16F84'e buton bağlantısı.

Butona basılmadığı zaman lojik-1 seviyesinde olan RA0 pini, butona basılınca şase seviyesine, yani 0'a iner. Demek ki butona basıldı mı diye test etmek için o pinin lojik-0 olup olmadığını kontrol etmemiz gerekiyor.

Bir bitin lojik seviyesi **BTFSC** ve **BTFSS** komutları ile test edilir.

```
BTFSC f, b          ; Register içerisindeki b. biti test et. 0 ise bir komut atla. 1
                    ; ise hemen altındaki satırdan çalışmaya devam et.
```

```
BTFSC PORTA, 4      ; PORTA'nın 4. bitini test et. 0 ise bir komut atla. 1 ise hemen
                    ; altındaki satırdan çalışmaya devam et.
```

```
BTFSS f, b          ; Register içerisindeki b. biti test et. 1 ise bir komut atla. 0
                    ; ise hemen altındaki satırdan çalışmaya devam et.
```

```
BTFSS PORTA, 0      ; PORTA'nın 0. bitini test et. 1 ise bir komut atla. 0 ise hemen
                    ; altındaki satırdan çalışmaya devam et.

TEST
   BTFSC PORTA,0    ; PORTA'nın 0. biti 0 mı?
   GOTO TEST        ; Hayır, TEST etiketine dallanarak yeniden kontrol et.
   CLRF PORTB       ; Evet, PORTB'yi temizle.
```

PROGRAM 6: Turn-off Devresine Ait Program (A0 Butonuna Basılınca B0 Ledini Yakacak, Bir Süre Sonra Ledi Kendiliğinden Söndürecek)

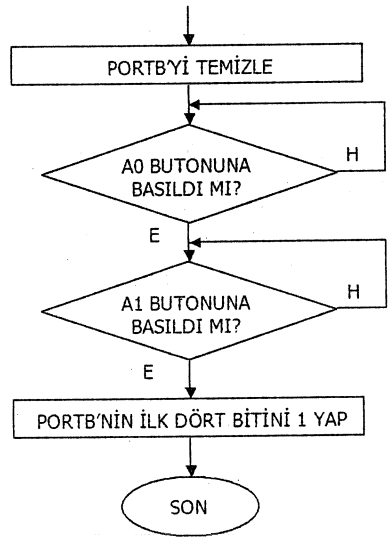
```
   LIST P=16F84          ; Kullanılacak PIC'i tanıt
   INCLUDE "P16F84.INC"  ; INCLUDE dosyasını yükle
SAYAC1 EQU H'0C'         ; H'0C' adresinde SAYAC1,
SAYAC2 EQU H'0D'         ; H'0D' adresinde SAYAC2 isimli bir register oluştur
   BSF STATUS,5          ; Bank-1'e geç
   CLRF TRISB            ; PORTB'yi çıkış olarak yönlendir
   BCF STATUS,5          ; Bank-0'a geri dön
   CLRF PORTB            ; PORTB'yi temizle
BAŞLA
   BTFSC PORTA,0         ; RA0 butonuna basıldı mı?
   GOTO BASLA            ; Hayır, yeniden test etmek için BASLA etiketine
                         ; dallan
   BSF PORTB,0           ; Evet, RB0'a bağlı ledi yak
   CALL GECIKME          ; Bir süre bekle ( Ledler yanık kalsın)
   CALL GECIKME          ;  "  "  "  "
   CALL GECIKME          ;  "  "  "  "
   BCF PORTB,0           ; RB0'a bağlı ledi söndür
   GOTO BASLA            ; Yeniden butonu test etmek için BASLA etiketine
                         ; dallan
GECIKME
   MOVLW H'FF'
   MOVWF SAYAC1
DONGU1
   MOVLW H'FF'
   MOVWF SAYAC2
DONGU2
   DECFSZ SAYAC2,F
```

```
   GOTO DONGU2
   DECFSZ SAYAC1,F
   GOTO DONGU1
   RETURN
   END
```

Bu programda yapılan işlem sürekli olarak butona basılıp basılmadığını kontrol etmektir. Butona basıldığı anda led yakılır. Bir süre gecikme sağlanarak ledlerin bir müddet yanık kalması sağlanır. Daha sonra led söndürülür ve program tekrar butonu test etmek için **BASLA** etiketine dallandırılır.

Çok Butonlu Uygulamalar

PROGRAM 7: PORTA'nın 0. ve 1. Bitlerine Bağlı Butonlara Sırasıyla Basıldığında PORTB'ye Bağlı Ledlerin İlk Dördünü Yakan Program

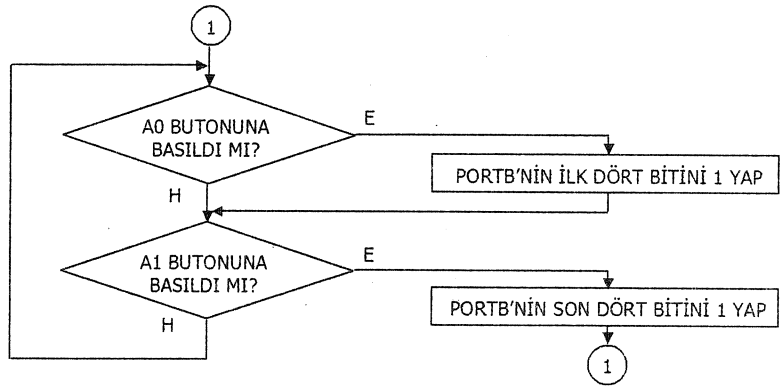



```
   LIST P=16F84            ; Kullanılacak PIC'i tanıt
   INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
   BSF STATUS,5            ; Bank-1'e geç
   CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
   BCF STATUS,5            ; Bank-0'a geri dön
   CLRF PORTB              ; PORTB'yi temizle
BUTON1
   BTFSC PORTA,0           ; RA0 butonuna basıldı mı?
   GOTO BUTON1             ; Hayır, yeniden test etmek için BUTON1 etiketine
                           ; dallan
BUTON2
   BTFSC PORTA,1           ; Evet, RA1 butonuna basıldı mı?
   GOTO BUTON2             ; Hayır, yeniden test etmek için BUTON2 etiketine
                           ; dallan
   MOVLW H'0F'             ; Evet, PORTB'nin ilk dört bitine
   MOVWF PORTB             ; bağlı ledleri yak
DONGU
```

```
   GOTO DONGU              ; Programı durdur
   END
```

Bu programda butonlara sırasıyla basılması gerektiğinden önce birinci buton kontrol edilir. Bu butona basılmadığı sürece ikinci butonu test etmeye gerek yoktur. Birinci butona basıldığında bu kez ikinci butona basılma durumu kontrol edilir. Butona basılmadığı sürece ledlerin yanması istenmediğinden bu buton sürekli olarak kontrol edilmelidir. İkinci butona basıldığı anda ledler yakılır ve program sona erdirilir. PORTB'nin ilk dört bitine bağlı ledlerin yakılması istendiğinden göndereceğimiz bilgi B'00001111' yani H'0F' olur.

PROGRAM 8: PORTA'nın 0. Bitine Bağlı Butona Basınca PORTB'deki Ledlerin İlk Dördünü, 1. Bitine Bağlı Butona Basınca Son Dördünü Yakan Program

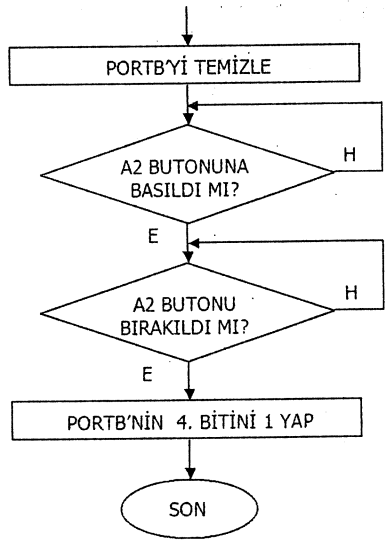



```
   LIST P=16F84            ; Kullanılacak PIC'i tanıt
   INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
   BSF STATUS,5            ; Bank-1'e geç
   CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
   BCF STATUS,5            ; Bank-0'a geri dön
   CLRF PORTB              ; PORTB'yi temizle
BUTON1
   BTFSC PORTA,0           ; RA0 butonuna basıldı mı?
   GOTO BUTON2             ; Hayır, BUTON2'ye basıldı mı kontrol et
   MOVLW  H'0F'            ; Evet, PORTB'nin ilk dört bitine
   MOVWF PORTB             ; bağlı ledleri yak
BUTON2
   BTFSC PORTA,1           ; RA1 butonuna basıldı mı?
   GOTO BUTON1             ; Hayır, BUTON1'e basıldı mı kontrol et
   MOVLW  H'F0'            ; Evet, PORTB'nin son dört bitine
   MOVWF PORTB             ; bağlı ledleri yak
   GOTO BUTON1             ; BUTON1'e basıldı mı kontrol et
   END
```

Bu programda olduğu gibi eğer birden fazla buton varsa ve her biri farklı bir fonksiyonu gerçekleştiriyorsa tüm butonların sürekli olarak taranması gerekir. Tarama işlemi çok hızlı yapıldığı için basılan buton hemen algılanır ve istenilen işlem yerine getirilir. Bir buton taranırken diğerine basılır da PIC bunu algılayamaz gibi bir düşünce akla gelmemelidir.

Birinci butona basılıp basılmadığı kontrol edilir. Eğer basılmışsa ona ait fonksiyon yani ledlerin ilk dördünü yakma işlevi yerine getirilir. Basılmamışsa ikinci buton kontrol edilir. Bu butona basılmışsa son dört led yakılır. Basılmamışsa tekrar birinci buton test edilir. Yani hiçbir butona basılmadığı sürece program sürekli olarak önce birinci sonra da ikinci butonu tarayacaktır. Herhangi bir butona basıldığı anda da ona ait işlevi yerine getirecektir.

PROGRAM 9: PORTA'nın 2. Bitine Bağlı Butona Basılıp Bırakıldığında PORTB'nin 4. Bitine Bağlı Ledi Yakan Program




```
   LIST P=16F84            ; Kullanılacak PIC'i tanıt
   INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
   BSF STATUS,5            ; Bank-1'e geç
   CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
   BCF STATUS,5            ; Bank-0'a geri dön
   CLRF PORTB              ; PORTB'yi temizle
BASLA
   BTFSC PORTA,2           ; RA2 butonuna basıldı mı?
   GOTO BASLA              ; Hayır, yeniden test etmek için BASLA etiketine dallan
BIRAK
   BTFSS PORTA,2           ; Evet, RA2 butonu bırakıldı mı?
   GOTO BIRAK              ; Hayır, yeniden test etmek için BIRAK etiketine dallan
   BSF PORTB,4             ; Evet, RB4'e bağlı ledi yak.
DONGU
   GOTO DONGU              ; Programı durdur.
   END
```

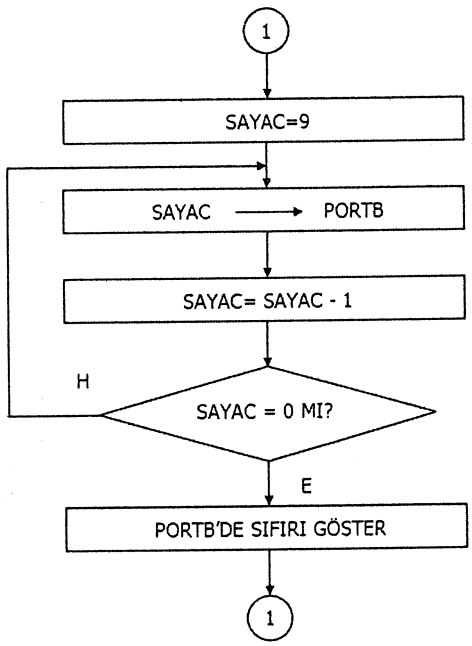
Basılan buton 0 değerini aldığı için basma olayı **BTFSC** ile kontrol ettirilmiştir. Buton bırakıldığı anda RA2 pini yeniden lojik-1 seviyesine döneceğinden bırakma olayı **BTFSS** komutu ile test edilmiştir. Devreyi kurduğunuz zaman butona basıp basılı tutarsanız ledin yanmadığını, ancak butondan parmağınızı çektiğiniz zaman ledin yandığını göreceksiniz.

Bölüm 6

Sayıcılar

Geri Sayıcılar

PROGRAM 10: PORTB'ye Bağlı Ledler Üzerinde Binary 9'dan 0'a Kadar Azaltarak Ekrani Sayıdıran Program




```
   LIST P=16F84            ; Kullanılacak PIC'i tanıt
   INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
SAYAC1 EQU H'0C'           ; H'0C' adresinde SAYAC1,
SAYAC2 EQU H'0D'           ; H'0D' adresinde SAYAC2,
SAYAC EQU H'0E'            ; H'0E' adresinde SAYAC isimli bir register oluştur
   BSF STATUS,5            ; Bank-1'e geç
   CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
   BCF STATUS,5            ; Bank-0'a geri dön
   CLRF PORTB              ; PORTB'yi temizle
YUKLE
   MOVLW D'9'
   MOVWF SAYAC             ; SAYAC'a başlangıç değerini yükle
BASLA
   MOVF SAYAC,W            ; SAYAC'taki bilgiyi
   MOVWF PORTB             ; PORTB'ye yükle
   CALL GECIKME            ; Sayının görülebilmesi için bir süre bekle
   DECFSZ SAYAC,F          ; SAYAC'ı 1 azalt, 0 oldu mu?
   GOTO BASLA              ; Hayır,sayının azaltılması için BASLA etiketine dallan
   CLRF PORTB              ; Evet,  0 bilgisini görmek için PORTB'yi temizle
   CALL GECIKME            ; 0'ın görülebilmesi için bir süre bekle
   GOTO YUKLE              ; Başlangıç değerini yeniden yüklemek için YUKLE'ye
dallan
```

```
GECIKME
   MOVLW H'FF'
   MOVWF SAYAC1
DONGU1
   MOVLW H'FF'
   MOVWF SAYAC2
DONGU2
   DECFSZ SAYAC2,F
   GOTO DONGU2
   DECFSZ SAYAC1,F
   GOTO DONGU1
   RETURN
   END
```

PIC Assembly'de azaltma işlemi yapan iki komut vardır: **DECF** ve **DECFSZ**

```
DECF f, d       ; File register içerisindeki sayıyı 1 azalt. Sonucu hedef (d) W ise
                ; W'ya, f ise aynı register içerisine yükle.
```

```
DECF SAYAC,F    ; SAYAC register'ı içerisindeki sayıyı 1 azalt. Sonucu yine SAYAC'a
                ; yükle.
```

```
DECFSZ f, d     ; File register içerisindeki sayıyı 1 azalt. Sonucu hedefe yükle.
                ; Sonuç 0 ise 1 komut atla. Değilse bir alt satırdan programa devam
                ; et.
```

```
1- DECFSZ SAYAC,F     ; SAYAC register'ı içerisindeki sayıyı 1 azaltıp yine SAYAC'a
2- GOTO DONGU         ; yükle. Sonuç 0 değilse ise 2 numaralı komut satırını,
3- BSF PORTB,2        ; 0 ise 3 numaralı komut satırını işle.
```

O halde geri sayıcı uygulamalarında bu iki komuttan faydalanabiliriz. Sayma işlemlerinde, sayılan sayıları tutmak için ayrı bir register oluşturulur. Bu register içerisindeki sayı değiştikçe çıkış portuna aktarılır. Programımızda **SAYAC** adında bir register oluşturalım. PIC16F84'ün özel amaçlı register'larının isim ve adresleri sabit olduğu için **INCLUDE** komutu kullanarak programımıza dahil etmiştik. **SAYAC** ise bizim oluşturduğumuz bir register olduğu için bu register'ı bir adres atayarak tanıtmamız gerekir. Bu işlemi de bildiğiniz gibi **EQU** komutu ile yapıyoruz.

```
SAYAC EQU H'0C'
```

NOT **Programcının kullanımına ayrılmış olan RAM bellek adreslerinin H'0C'- H'4F' adresleri arasında olduğunu unutmayalım. Bu adres aralığı dışında bir adres belirtemeyiz.**

Bu programda yapılan işlem **SAYAC** register'ı içerisine başlangıç sayısını yüklemek, sonra bu sayıyı tek tek eksilterek çıkış portuna aktarmaktır.

Önce **SAYAC**'a 9 yüklenir. **SAYAC**'taki bu bilgi **PORTB**'ye aktarılır. Çıkışa her bir sayı atıldıktan sonra bir gecikme alt programı çağrılarak sayıların görülebilmesi sağlanır. Daha sonra azaltma komutu ile **SAYAC**'ın içeriği 1 eksiltilir. Sonuç 0 olmadığı için bir alt satır yani **GOTO BASLA** satırı işlenir ve program bu etikete dallanarak **SAYAC**'taki 8 sayısı çıkış portuna aktarılır. Bu işlem

SAYAC'ın içeriği 0 olana kadar devam eder. **SAYAC** sıfırlanınca program **CLRF PORTB** satırına atlar ve **PORTB** temizlenerek sıfır sayısının görülmesi sağlanır. Sayma işleminin sürekli olması istendiği için buradan sonra program yeniden yükleme kısmına dallandırılmıştır. Yani, 0'dan sonra tekrar 9 görülecektir.

Gecikme Alt Programı

Tek Döngülü Gecikme

```
GECIKME
   MOVLW H'FF'
   MOVWF SAYAC1
DONGU
   DECFSZ SAYAC1,F
   GOTO DONGU
   RETURN
```

Programların işleyişi sırasında kısa süreli duraklamalar meydana getirmek için gecikme alt programları kullanılır. Örneğin yukarıdaki sayıcı programında gecikme programı yazmazsak, sayılar çıkışa aktarılırken PIC'in çok hızlı çalışmasından dolayı (μs'ler bazında) hiç bir şey göremeyiz. Bu yüzden her bir sayının porta atılması işleminden sonra gecikme alt programı çağrılmıştır.

Yukarıda tek döngülü gecikme alt programı verilmiştir. Burada önce **SAYAC1** register'ına hexadecimal FF (yani 255) sayısı yüklenmiş, sonra **DECFSZ** komutu ile birer birer azaltılarak sayı 0 olana kadar programın döngüde kalması sağlanmıştır. **SAYAC1** sıfırlanınca bir satır atlayıp **RETURN** komutunu görecek, dolayısıyla alt programdan çıkıp ana programda kaldığı yere dönecektir.

Bu program ile yaklaşık 765μs'lik bir gecikme sağlanır. Bu çok kısa bir süre olduğundan uygulamalarımızda iç içe çift döngülü gecikme alt programı kullanacağız.

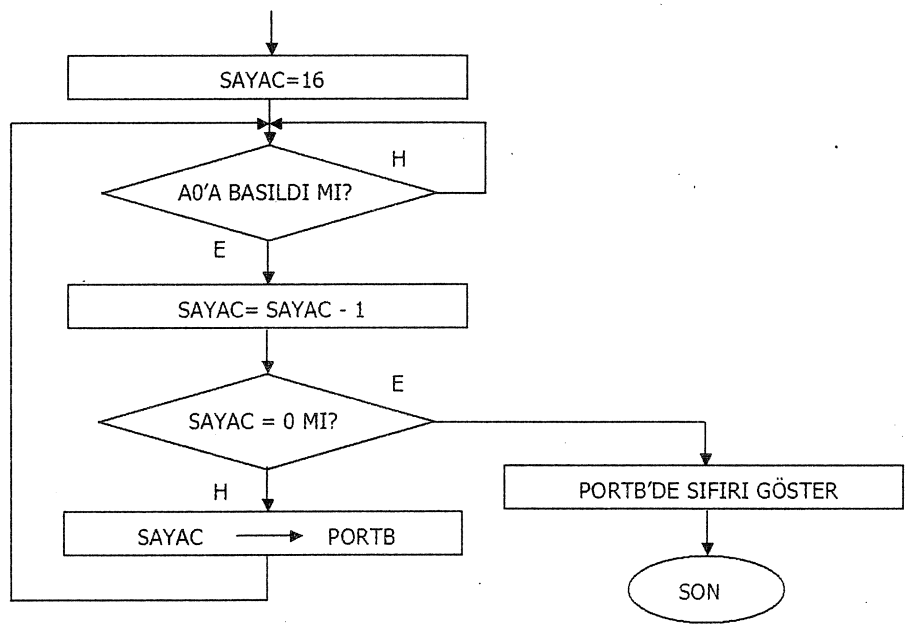
Çift Döngülü Gecikme

```
GECIKME
   MOVLW H'FF'
   MOVWF SAYAC1
DONGU1
   MOVLW H'FF'
   MOVWF SAYAC2
DONGU2
   DECFSZ SAYAC2,F
   GOTO DONGU2
   DECFSZ SAYAC1,F
   GOTO DONGU1
   RETURN
   END
```

Bu programda iki sayac kullanılmış, bu sayaçların her ikisine de 255 sayısı yüklenmiştir. Program önce **SAYAC2** register'ının içeriği sıfırlanana kadar sürekli **DONGU2**'ye dallanmış (255 komut saykılı), sonra **SAYAC1** register'ından bir eksiltilerek **DONGU1** etiketine dallanarak **SAYAC2**'ye tekrar 255 sayısı yüklenmiştir. **SAYAC1**'in içeriği sıfırlanana kadar, yani yaklaşık 255x255 komut saykılı boyunca bu işlem tekrarlanır.

Bu program ile yaklaşık 197 ms'lik bir gecikme sağlanır. Bu sürenin de yeterli gelmediği durumlarda iç içe üç döngülü gecikme alt programı yazabilir veya birkaç tane **CALL GECIKME** komut satırını alt alta kullanabilirsiniz.

PROGRAM 11: PORTA'nın 0. Bitine Bağlı Butona Her Basışta PORTB'ye Bağlı Ledler Üzerinde 15'den 0'a Kadar Sayıp 0'da Duran Program

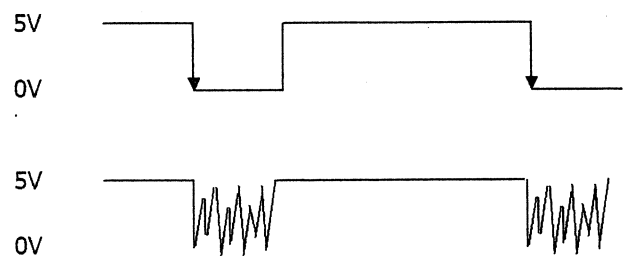



```
   LIST P=16F84            ; Kullanılacak PIC'i tanıt
   INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
SAYAC1 EQU H'0C'           ; H'0C' adresinde SAYAC1,
SAYAC2 EQU H'0D'           ; H'0D' adresinde SAYAC2,
SAYAC EQU H'0E'            ; H'0E' adresinde SAYAC isimli bir register oluştur
   BSF STATUS,5            ; Bank-1'e geç
   CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
   BCF STATUS,5            ; Bank-0'a geri dön
   CLRF PORTB              ; PORTB'yi temizle
YUKLE
   MOVLW D'16'
   MOVWF SAYAC             ; SAYAC'a başlangıç değerini yükle
TEST
   BTFSC PORTA,0           ; RA0 butonuna basıldı mı?
   GOTO TEST               ; Hayır, yeniden test etmek için TEST etiketine dallan
```

```
    CALL GECIKME          ; Evet, buton arkını söndürmek için bir süre bekle
    DECFSZ SAYAC,F        ; SAYAC'ı 1 azalt, 0 oldu mu?
    GOTO GOSTER           ; Hayır, SAYAC'ın içeriğini görüntülemek için
                          ; GOSTER'e dallan
    CLRF PORTB            ; Evet, 0 bilgisini göstermek için PORTB'yi temizle
DONGU
    GOTO DONGU            ; Sonsuz döngü ile programı durdur

GOSTER
    MOVF SAYAC,W          ; SAYAC'taki bilgiyi
    MOVWF PORTB           ; PORTB'ye yükle
    CALL GECIKME          ; Sayının görülebilmesi için bir süre bekle
    CALL GECIKME
    GOTO TEST             ; Yeniden butona basılıp basılmadığını kontrol etmek
                          ; için TEST etiketine dallan
GECIKME
    MOVLW H'FF'
    MOVWF SAYAC1

DONGU1
    MOVLW H'FF'
    MOVWF SAYAC2
DONGU2
    DECFSZ SAYAC2,F
    GOTO DONGU2
    DECFSZ SAYAC1,F
    GOTO DONGU1
    RETURN
    END
```

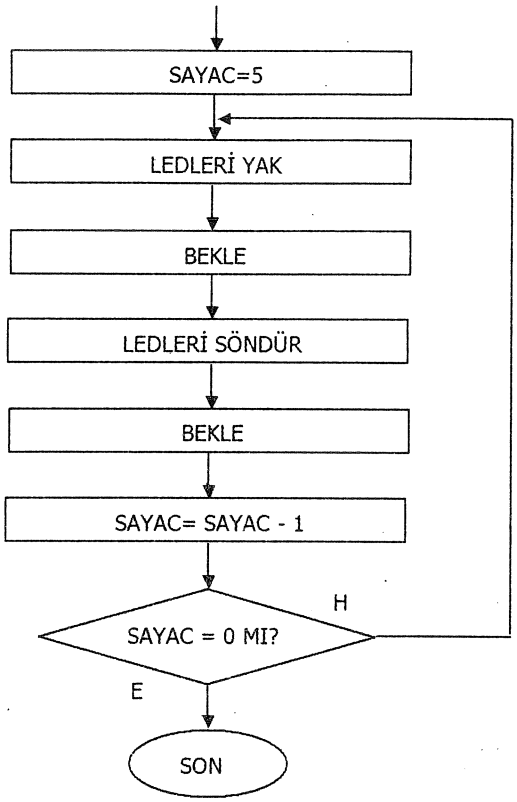
Bu programın diğerinden farkı sayma işleminin bir buton kontrol edilerek yapılmasıdır. Önce bir **SAYAC** register'ına başlangıç değeri yüklenir. Butona her basışta bu register'ın içeriği 1 azaltılarak çıkış portuna aktarılır. **SAYAC**'ın içeriği 0 olduğunda ise port temizlenerek program bir sonsuz döngü ile sona erdirilir.

Bu programda butona basıldığı tespit edildiği anda gecikme alt programı çağrılmış ve bir süre beklenmiştir. Bunun sebebi buton arkını söndürmektir. Butona bastığımız zaman 1 seviyesinden 0 seviyesine aniden inemez. Butonun bağlı bulunduğu pinde dalgalanmalar olur. Mikrodenetleyici tarama işlemini çok hızlı yaptığı için butona bir kez bastığımız halde birkaç kez basılmış gibi algılayıp ona göre işlem yapabilir. Bu da programın düzgün çalışmamasına sebep olur. Bu programda olduğu gibi aynı butona peş peşe basılıyorsa mutlaka buton arkının söndürülmesi, yani bir gecikme programı yazılması gerekir.



Yukarıdaki ilk şekil ideal butona basılıp bırakılma durumunu, ikinci şekil ise gerçekteki durumu ifade etmektedir. Görüldüğü gibi biz elimizi butondan çekene kadar defalarca kez 1'den 0'a düşüş meydana gelir. Eğer programda buton arkını söndürmezsek butona bastıkça sayıların tek tek değil, ikişer üçer azaldığını görürüz.

PROGRAM 12 : PORTB'ye Bağlı Ledleri 5 Kez Yakıp Söndüren Program




```
   LIST P=16F84            ; Kullanılacak PIC'i tanıt
   INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
SAYAC1 EQU H'0C'           ; H'0C' adresinde SAYAC1,
SAYAC2 EQU H'0D'           ; H'0D' adresinde SAYAC2,
SAYAC EQU H'0E'            ; H'0E' adresinde SAYAC isimli bir register oluştur
   BSF STATUS,5            ; Bank-1'e geç
   CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
   BCF STATUS,5            ; Bank-0'a geri dön
   CLRF PORTB              ; PORTB'yi temizle
   MOVLW D'5'
   MOVWF SAYAC             ; SAYAC'a tekrar sayısını yükle
BASLA
   MOVLW H'FF'             ; PORTB'ye bağlı
   MOVWF PORTB             ; tüm ledleri yak
   CALL GECIKME            ; Bir süre ledler yanık kalsın
```

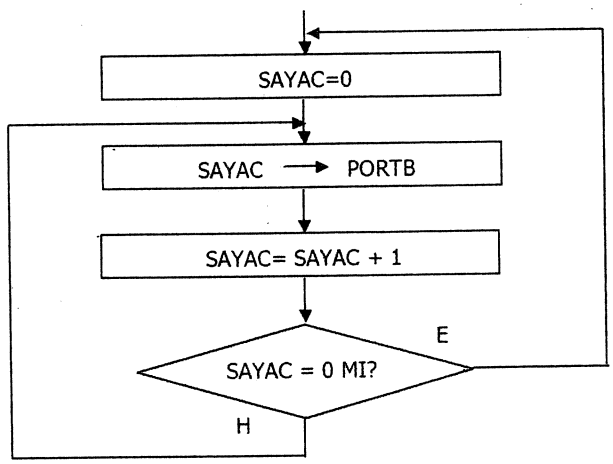
```
    CLRF PORTB              ; Tüm ledleri söndür
    CALL GECIKME            ; Bir süre ledler sönük kalsın
    DECFSZ SAYAC,F          ; SAYAC'ı 1 azalt, 0 oldu mu?
    GOTO BASLA              ; Hayır, yeniden ledleri yakmak için BASLA etiketine
                            ; dallan
DONGU
    GOTO DONGU              ; Evet,  sonsuz döngü ile programı durdur
GECIKME
    MOVLW H'FF'
    MOVWF SAYAC1
DONGU1
    MOVLW H'FF'
    MOVWF SAYAC2
DONGU2
    DECFSZ SAYAC2,F
    GOTO DONGU2
    DECFSZ SAYAC1,F
    GOTO DONGU1
    RETURN
    END
```

Bir işlemin belli bir sayıda tekrar edilmesi isteniyorsa yine sayıcılardaki mantık kullanılır. Bir register içerisine tekrar sayısı yüklenir. İstenen iş yapıldıkça bu sayı 1 azaltılır. Register'ın içeriği 0 olduğu zaman işlem istenilen sayıda tekrarlanmış demektir. Bu durumda program durdurulur.

Bu programda ledlerin 5 kez yanıp sönmesi istendiğinden **SAYAC**'a 5 yüklenmiş, daha sonra ledler bir kez yakıp söndürülmüştür. Flash işleminin 4 kez daha yapılması gerektiğinden **SAYAC**'taki tekrar sayısı 1 eksiltilir, daha sonra yeniden flash kısmına dallandırılır. **SAYAC**=0 olduğunda işlem 5 kez tekrarlanmış olacağından bir sonsuz döngü ile program durdurulur.

İleri Sayıcılar

PROGRAM 13: PORTB'ye Bağlı Ledler Üzerinde 0-255 Arasında Sürekli Sayan Program



```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
SAYAC1 EQU H'0C'            ; H'0C' adresinde SAYAC1,
SAYAC2 EQU H'0D'            ; H'0D' adresinde SAYAC2,
SAYAC EQU H'0E'             ; H'0E' adresinde SAYAC isimli bir register oluştur
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF PORTB              ; PORTB'yi temizle
YUKLE
    CLRF SAYAC              ; SAYAC'a başlangıç değeri olarak 0 yükle
GOSTER
    MOVF SAYAC,W            ; SAYAC'taki bilgiyi
    MOVWF PORTB             ; PORTB'ye yükle
    CALL GECIKME            ; Sayının görülebilmesi için bir süre bekle
    INCFSZ SAYAC,F          ; SAYAC'ı 1 artır. 0 oldu mu?
    GOTO GOSTER             ; Hayır, SAYAC'ın içeriğini görüntülemek için
                            ; GOSTER'e dallan
    GOTO YUKLE              ; Evet, yeniden 0'dan saymaya başlamak için yükleme
                            ; kısmına dallan
GECIKME
    MOVLW H'FF'
    MOVWF SAYAC1
DONGU1
    MOVLW H'FF'
    MOVWF SAYAC2
DONGU2
    DECFSZ SAYAC2,F
    GOTO DONGU2
    DECFSZ SAYAC1,F
    GOTO DONGU1
    RETURN
    END
```

PIC Assembly'de artırma işlemi yapan iki komut vardır: **INCF** ve **INCFSZ**

```
INCF f, d               ; File register içerisindeki sayıyı 1 artır. Sonucu hedef (d)
                        ; W ise W'ya, f ise aynı register'a yükle.
```

```
INCF SAYAC,F            ; SAYAC register'ı içerisindeki sayıyı 1 artır. Sonucu yine
                        ; SAYAC' a yükle.
```

```
INCFSZ f, d             ; File register içerisindeki sayıyı 1 artır. Sonucu hedefe
                        ; yükle. Sonuç 0 ise bir komut atla. Değilse bir alt satırdan
                        ; programa devam et.
```

```
1- INCFSZ SAYAC,F       ;SAYAC register'ındaki sayıyı 1 artırıp yine SAYAC 'a yükle.
2- GOTO DONGU           ; Sonuç 0 değilse 2 numaralı satırı,
3- BSF PORTB,2          ; 0 ise 3 numaralı satırı işle.
```

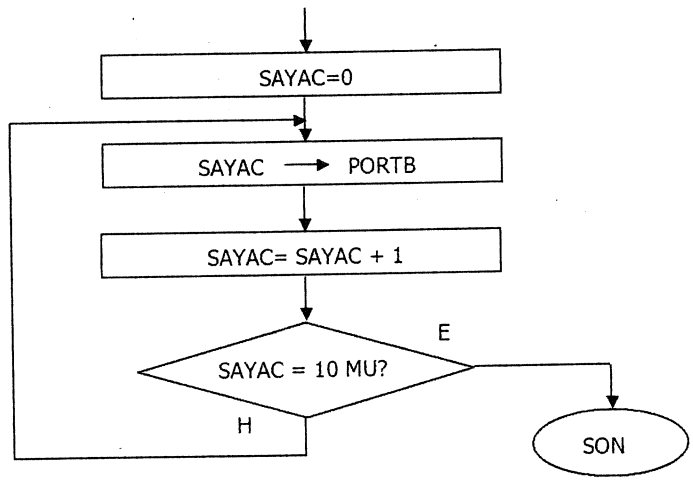
Bir ileri sayma işleminde sonuç ne zaman 0 olur? Bir register 8 bit olduğuna göre bu bitlerin hepsi 1 olduktan sonra register'ın içeriği 0 olur. Yani;

FF + 1 = 100 11111111 + 1 = 1 00000000 (8 bit)

Demek ki **INCFSZ** komutunu kullanabilmemiz için sayıcının H'FF'e yani 255'e kadar sayması gerekiyor. Daha küçük bir değere, örneğin 10'a kadar sayan bir sayıcı programı yazmak istiyorsak bu komutu kullanamayız.

Yukarıdaki programda sayma işlemi 0'dan başladığı için **SAYAC**'a başlangıç değeri olarak 0 yüklenmiştir. Artırma işlemi **INCFSZ** komutu kullanılarak yapılabilir, çünkü sayma işlemi 255'e kadar devam etmektedir. **SAYAC**'taki sayı arttıkça çıkış portuna aktarılıp bir müddet gecikme sağlanarak sayıların görülebilmesi sağlanmıştır. Sayıcının değeri 255 olduktan sonra tekrar 0'a dönmesi istendiğinden **SAYAC** 0 olduğunda program yükleme kısmına dallandırılmıştır.

PROGRAM 14: PORTB'ye Bağlı Ledler Üzerinde 0'dan 9'a Kadar Sayıp 9'da Duran Program




```
   LIST P=16F84            ; Kullanılacak PIC'i tanıt
   INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
SAYAC1 EQU H'0C'           ; H'0C' adresinde SAYAC1,
SAYAC2 EQU H'0D'           ; H'0D' adresinde SAYAC2,
SAYAC EQU H'0E'            ; H'0E' adresinde SAYAC isimli bir register oluştur
   BSF STATUS,5            ; Bank-1'e geç
   CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
   BCF STATUS,5            ; Bank-0'a geri dön
   CLRF PORTB              ; PORTB'yi temizle
   CLRF SAYAC              ; SAYAC'a başlangıç değeri olarak 0 yükle
TEKRAR
   MOVF SAYAC,W            ; SAYAC'taki bilgiyi
   MOVWF PORTB             ; PORTB'ye yükle
   CALL GECIKME            ; Sayının görülebilmesi için bir süre bekle
   INCF SAYAC,F            ; SAYAC'ı 1 artır.
```

```
    MOVLW D'10'             ; Karşılaştırma değeri olarak W'ya 10 yükle
    SUBWF SAYAC,W           ; SAYAC'ın içerisindeki sayıdan 10 çıkar
    BTFSS STATUS,2          ; Sonuç 0 mı? (Yani sayı 10 olmuş mu?)
    GOTO TEKRAR             ; Hayır, SAYAC'taki bilgiyi göstermek için TEKRAR
                            ; etiketine dallan
DONGU
    GOTO DONGU              ; Evet, sonsuz döngü ile programı durdur
GECIKME
    MOVLW H'FF'
    MOVWF SAYAC1
DONGU1
    MOVLW H'FF'
    MOVWF SAYAC2
DONGU2
    DECFSZ SAYAC2,F
    GOTO DONGU2
    DECFSZ SAYAC1,F
    GOTO DONGU1
    RETURN
    END
```

Sayma işleminin 255'ten geçtiği durumlarda **INCFSZ** komutu kullanarak sayıcının içeriğinin 255 olup olmadığını test edebiliyorduk. Daha küçük bir değere kadar saydırılmak isteniyorsa yine bir **SAYAC** register'ının içeriği teker teker artırılır, ancak her artırma işleminden sonra sayının istenilen değere ulaşıp ulaşmadığı test edilir. Örneğin yukarıdaki programda sayıcının 9'a kadar sayması istendiğinden **SAYAC**'ın içeriği artırıldıkça 9 olup olmadığı sürekli olarak test edilmektedir.

İleri sayıcılarda istenilen değere ulaşılıp ulaşılmadığını anlamak için **SUBWF** komutundan faydalanabiliriz.

```
SUBWF f, d              ; File register içerisindeki sayıdan W içerisindeki sayıyı
                        ; çıkart. Sonucu hedef (d) W ise W'ya , f ise aynı
                        ; register'a yükle.
```

```
SUBWF   SAYAC, F        ; SAYAC'tan W'yu çıkar, sonucu SAYAC'a yükle
SUBWF   TUT,W           ; TUT'tan W'yu çıkar, sonucu W'ya yükle.
```

Bu komutta çıkarılan sayı **W** içerisinde bulunduğundan çıkarma işlemi yapmadan hemen önce çıkarılmasını istediğimiz sayıyı **W**'ya yükleriz.

Yukarıdaki programda **SAYAC** içerisindeki sayı 10 olduğunda, **W**'ya da 10 yüklenmiş olduğundan sonuç 10-10 = 0 olacaktır. O halde **SAYAC**'ın içindeki sayının 10 olup olmadığını anlamak için çıkarma işleminin sonucunun 0 olup olmadığını kontrol edebiliriz. Bu işlemi Zero Flag'i kontrol ederek yapıyoruz.

Zero Flag (Sıfır Bayrağı)

STATUS register'ının 2. biti zero flag (sıfır bayrağı) olarak isimlendirilir. Yapılan herhangi bir işlemin sonucu 0 olduysa zero flag **Z**=1 olur, aksi halde **Z**=0'dır.

	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
STATUS	IRP	RP1	RP0	TO	PD	Z	DC	C

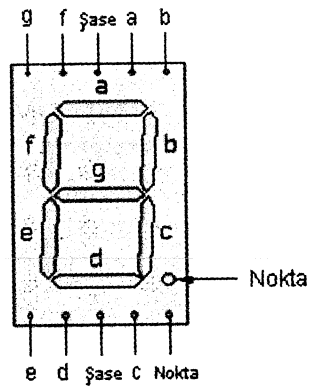
Zero Flag

Program 14'te **W**'ya ulaşılması istenen değer, yani 10 yüklenir. Sayıcının 9'a kadar sayması istendiği halde 10 yüklenmesinin sebebi, sayının 10 olması durumunda porta atılmak yerine programın son kaldığı sayıda yani 9'da durdurulmasıdır.

SAYAC içerisindeki sayının örneğin 6 olduğunu düşünelim. Bu durumda 6 – 10 = -4 işlemi yapılacak, sonuç 0 olmadığından Zero Flag = 0 olacak, bu yüzden program **TEKRAR** etiketine dallanacaktır. **SAYAC** = 10 olduğunda ise 10 – 10 = 0 olacağından Zero Flag=1 olacak, bu durumda program sonsuz döngüye girerek portta en son 9 değeri kalacaktır.

7-Segment Display

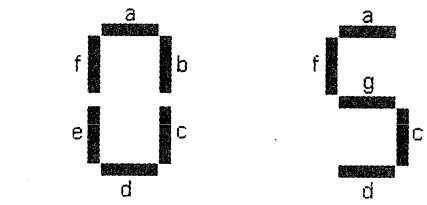
7-segment display yedi adet çubuk şeklinde ledin bir rakam oluşturacak şekilde bir araya gelmesinden oluşan bir elemandır. 7-segment display'ler ortak anotlu ve ortak katotlu olmak üzere ikiye ayrılır. Bir ortak anotlu display'de tüm segmentlerin – yani tüm ledlerin – anotları birleştirilmiştir ve bu uca +5V uygulanır. Bu durumda katotuna şase gelen segment yanar. Ortak katotlu display'de ise tüm segmentlerin katotları birleştirilmiştir ve bu uç şaseye bağlanır. Yanmasını istediğimiz segmente +5V yani lojik-1 vermemiz gerekir.



ŞEKİL 22: Seven-segment display.

Dijital elektronik uygulamalarında 7-segment display kullanırken bir kod çözücü entegreye ihtiyaç duyarız (Ortak katotlu display için 7448, ortak anotlu display için 7447). Bu kod çözücünün görevi girişine binary kodda gelen sayıları display'in anlayacağı dile çevirmektir. Örneğin ortak katotlu display'de 0 rakamını görmek için abcdef segmentlerine 1, g segmentine 0 vermemiz gerekir.

Yani binary 0000 sayısını 3F'e çevirebilirsek display'de 0 yandığını görebiliriz. Bu çevirme işlemini kod çözücü entegre kullanmadan programlama yoluyla gerçekleştirmemiz mümkündür. Tablo 8'de 0-15 arasındaki sayılar için 7-segment display'e göndermemiz gereken kodlar listelenmiştir.



Nokta g f e d c b a

0 0 1 1 1 1 1 1

3 F

TABLO 8: İstenilen Rakamın Görülebilmesi İçin 7-Segment Display'e Gönderilmesi Gereken Kodlar

Decimal Sayı	Display'e gönderilmesi gereken kod (binary)	Display'e gönderilmesi gereken kod (hex)	7-segmentte görülecek sayı
0	B'00111111'	H'3F'	0
1	B'00000110'	H'06'	1
2	B'01011011'	H'5B'	2
3	B'01001111'	H'4F'	3
4	B'01100110'	H'66'	4
5	B'01101101'	H'6D'	5
6	B'01111101'	H'7D'	6
7	B'00000111'	H'07'	7
8	B'01111111'	H'7F'	8
9	B'01101111'	H'6F'	9
10	B'01110111'	H'77'	A
11	B'01111100'	H'7C'	b
12	B'00111001'	H'39'	C
13	B'01011110'	H'5E'	d
14	B'01111001'	H'79'	E
15	B'01110001'	H'71'	F

Bu örnekte olduğu gibi belli değerlere karşılık gelen sabitleri çağırma işlemi çevrim tablosu kullanarak yapılır. Aşağıda ortak katotlu 7-segment display için kullanılan çevrim tablosu görülmektedir.

```
CEVRIM_TABLOSU

   ADDWF PCL,F
   RETLW H'3F'        ; 0
   RETLW H'06'        ; 1
   RETLW H'5B'        ; 2
```

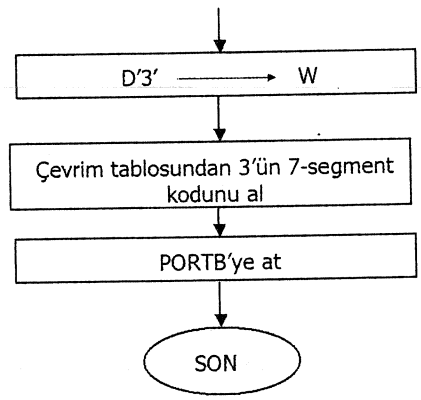
```
RETLW H'4F'      ; 3
RETLW H'66'      ; 4
RETLW H'6D'      ; 5
RETLW H'7D'      ; 6
RETLW H'07'      ; 7
RETLW H'7F'      ; 8
RETLW H'6F'      ; 9
RETLW H'77'      ; A
RETLW H'7C'      ; B
RETLW H'39'      ; C
RETLW H'5E'      ; D
RETLW H'79'      ; E
RETLW H'71'      ; F
```

Çevrim tablolarında yapılan işlem **W** register'ına yüklenen sayıya göre program sayıcının (PCL) konumunu değiştirmektir. Örneğin **W**'ya 0 değeri yüklenip çevrim tablosu çağrıldığında **ADDWF PCL,F** komutu program sayıcının o andaki değeri ile **W**'ya yüklediğimiz değeri yani 0'ı toplar, bu durumda 0. değeri yani H'3F' sayısını **W**'ya yeni değer olarak yükler. Daha sonra ana programda çağrıldığı yere geri döner. Her bir satırda **RETLW** yani "sabit değerle birlikte geri dön" komutu bulunduğundan diğer alt programlarda olduğu gibi **RETURN** komutuna ihtiyaç duyulmamaktadır.

Aynı şekilde örneğin **W**'ya 5 yükleyerek çevrim tablosunu çağırırsak 5'in display kodu olan H'6D', 9 yükleyerek çağırırsak 9'un kodu olan H'6F' sayısı **W**'ya yüklenerek ana programa geri dönülecektir.

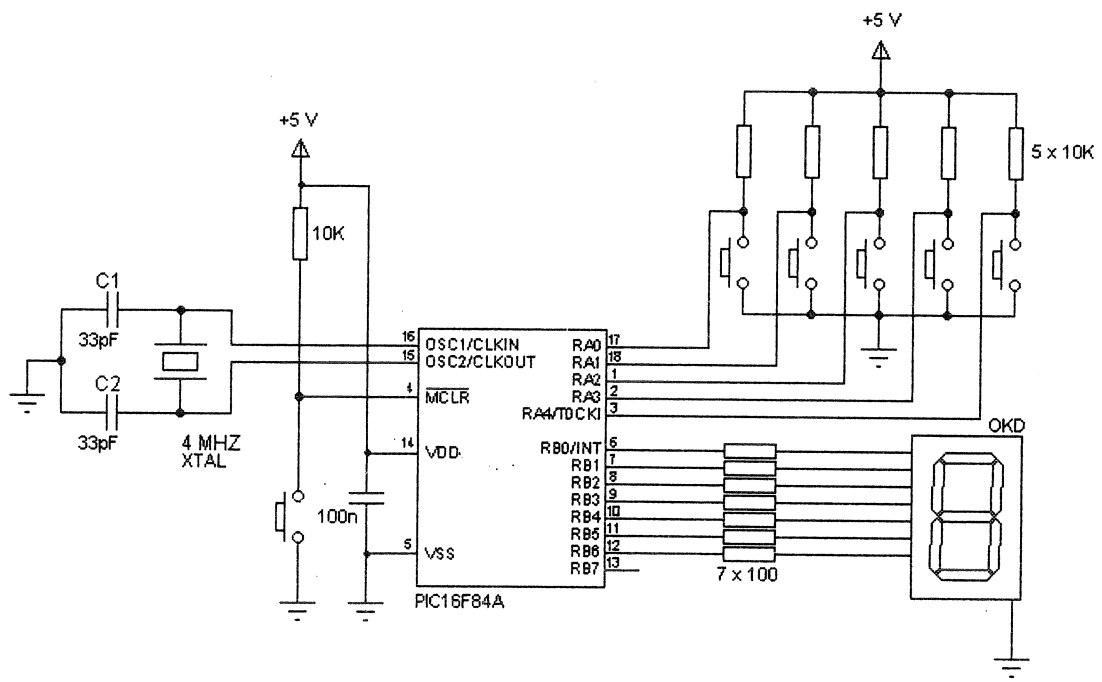
```
RETLW   k        ; Sabit sayıyı W'ya yükleyerek ana programda kaldığın yere geri
                 ; dön.
```

PROGRAM 15: Ortak Katotlu Display'de 3 Rakamını Yakan Program



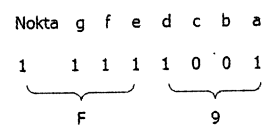

```
LIST P=16F84            ; Kullanılacak PIC'i tanıt
INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
BSF STATUS,5            ; Bank-1'e geç
CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
BCF STATUS,5            ; Bank-0'a geri dön
CLRF PORTB              ; PORTB'yi temizle
```

```
    MOVLW D'3'              ; Display'de görüntülenmek istenen  3 rakamını W'ya
                            ; yükle
    CALL CEVRIM             ; CEVRIM tablosundan 3'ün karşılığı olan kodu al
    MOVWF PORTB             ; PORTB'ye yükle
DONGU
    GOTO DONGU              ; Programı durdur
CEVRIM
    ADDWF PCL,F
    RETLW H'3F'
    RETLW H'06'
    RETLW H'5B'
    RETLW H'4F'
    RETLW H'66'
    RETLW H'6D'
    RETLW H'7D'
    RETLW H'07'
    RETLW H'7F'
    RETLW H'6F'
    RETLW H'77'
    RETLW H'7C'
    RETLW H'39'
    RETLW H'5E'
    RETLW H'79'
    RETLW H'71'
    END
```



ŞEKİL 23: Seven-segment display bağlantı şeması.

Çevrim tablosunda verdiğimiz değerler ortak katotlu display'i çalıştıracak şekilde, yani 1 aktif olarak yazılmıştır. Ortak anotlu display kullanmak isterseniz kodları değiştirmeniz gerekir. Çünkü display'in ortak ucu +5V'a bağlıdır ve yanmasını istediğimiz segmentlere 0 göndermemiz gerekir. Örneğin 1 rakamını yakmak için yazılacak kod;



şeklinde olmalıdır.

7448 kod çözücü kullanıldığında 9'dan sonraki rakamlar display'de simgeler halinde görülür. Ancak programlama yoluyla display'de yanacak segmentleri programcı belirlediğinden H'0A' sayısı için display'de A, H'0B' sayısın için display'de b, vs. yazılması sağlanmıştır

PROGRAM 16: PORTB'ye Bağlı 7-Segment Display Üzerinde 0-15 Arasında Sürekli Sayan Program

```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
SAYAC1 EQU H'0C'            ; H'0C' adresinde SAYAC1,
SAYAC2 EQU H'0D'            ; H'0D' adresinde SAYAC2,
SAYAC EQU H'0E'             ; H'0E' adresinde SAYAC isimli bir register oluştur
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF SAYAC              ; SAYAC'ı sıfırla
    CLRF PORTB              ; PORTB'yi temizle
DONGU
    MOVF SAYAC,W            ; SAYAC'taki sayıyı W'ya yükle
    ANDLW B'00001111'       ; Sayının 15'i geçmemesi için maskeleme işlemi yap
    CALL CEVRIM             ; CEVRIM tablosundan sayının karşılığı olan kodu al
    MOVWF PORTB             ; PORTB'ye yükle
    CALL GECIKME            ; Sayının görülebilmesi için bir süre bekle
    INCF SAYAC,F            ; SAYAC'taki sayıyı 1 artır
    GOTO DONGU              ; SAYAC'taki yeni sayıyı görmek için DONGU'ye
                            ; dallan
CEVRIM
    ADDWF PCL,F
    RETLW H'3F'
    RETLW H'06'
    RETLW H'5B'
    RETLW H'4F'
    RETLW H'66'
    RETLW H'6D'
    RETLW H'7D'
    RETLW H'07'
    RETLW H'7F'
    RETLW H'6F'
    RETLW H'77'
```

```
    RETLW H'7C'
    RETLW H'39'
    RETLW H'5E'
    RETLW H'79'
    RETLW H'71'
GECIKME
    MOVLW H'FF'
    MOVWF SAYAC1
DONGU1
    MOVLW H'FF'
    MOVWF SAYAC2
DONGU2
    DECFSZ SAYAC2,F
    GOTO DONGU2
    DECFSZ SAYAC1,F
    GOTO DONGU1
    RETURN
    END
```

Programın daha önce yazdığımız ileri sayıcı programından tek farkı çıkış portuna binary sayı yerine o sayıya karşılık gelen kodun aktarılıyor olmasıdır. Yani **SAYAC**'taki sayıyı W'ya attıktan sonra doğrudan porta aktarmak yerine önce çevrim tablosu çağırıp buradan alınan sayıyı porta aktarıyoruz.

```
MOVF SAYAC,W
MOVWF PORTB
```

⇩

Binary sayıyı aktarır

```
MOVF SAYAC,W
CALL CEVRIM
MOVWF PORTB
```

⇩

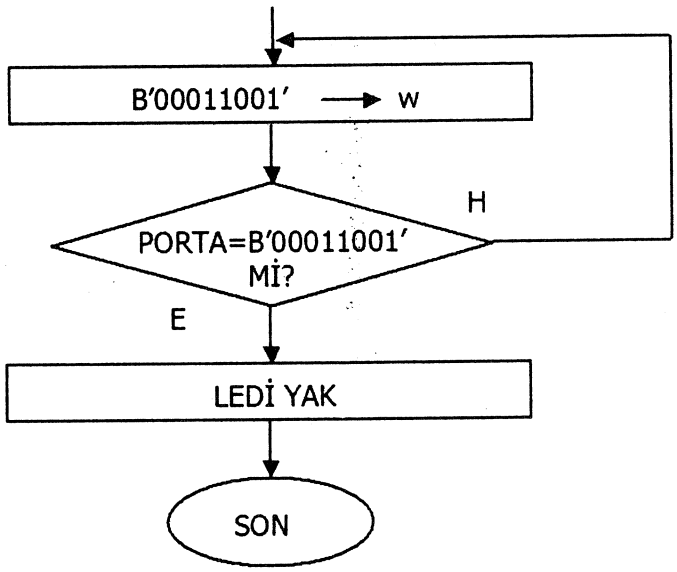
Display kodunu aktarır

Bu programda kullanılan maskeleme işlemi Step Motorlar bölümünde anlatılmıştır. Burada kısaca bahsetmek gerekirse maskeleme işleminin amacı istenilen bitleri sıfırlamak, istenilenleri ise değerini değiştirmeden sabit tutmaktır. AND işlemi aritmetik çarpma işlemidir. **ANDLW** komutu yanına yazılan sabit sayı ile **W**'daki sayı arasında AND işlemi yapar, sonucu yine **W**'ya yükler. Programımızda B'00001111' sayısı ile **W**'daki sayı arasında AND işlemi yapılmış, üst dört bit daima 0 ile çarpılacağından değerleri 0'da sabit tutulmuş, alt dört bit ise 1 ile çarpıldığından kendi değerleri ne ise korunmuştur. Eğer **W**'daki sayı 15'i yani B'00001111'i geçerse, örneğin B'00010000' olursa B'00001111' x B'00010000' = B'00000000' olur, yani sayının 15'den 16'ya değil, 0'a geçmesi sağlanır.

Bölüm 7

Portlardan Veri Girişi

PROGRAM 17: PORTA'dan B '00011001' Bilgisi Girildiği Zaman PORTB'nin 0. Bitine Bağlı Ledi Yakan Program




```
   LIST P=16F84            ; Kullanılacak PIC'i tanıt
   INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
SAYAC1 EQU H'0C'           ; H'0C' adresinde SAYAC1,
SAYAC2 EQU H'0D'           ; H'0D' adresinde SAYAC2 isimli bir register oluştur
   BSF STATUS,5            ; Bank-1'e geç
   CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
   BCF STATUS,5            ; Bank-0'a geri dön
   CLRF PORTB              ; PORTB'yi temizle
TEST
   MOVLW B'00011001'       ; Karşılaştırılacak bilgiyi W'ya yükle
   XORWF PORTA,W           ; PORTA'dan girilen bilgi ile bu bilgiyi karşılaştır
   BTFSS STATUS,2          ; Zero Flag=1 mi? (Sayılar eşit mi?)
   GOTO TEST               ; Hayır, yeniden test etmek için TEST'e dallan
   BSF PORTB,0             ; Evet, RB0 bitine bağlı ledi yak
DONGU
   GOTO DONGU              ; Programı durdur
   END
```

Bazı elektronik uygulamalarında mikrodenetleyicinin girişine gelen bilgiye göre çıkış vermesi gerekebilir. Bu giriş bilgisi başka bir sistemin çıkışı olabileceği gibi doğrudan buton ve anahtarlardan da alınabilir.

Girişten gelen bilginin bizim istediğimiz bilgi olup olmadığını karşılaştırma yaparak anlayabiliriz. Yukarıdaki programda karşılaştırma işlemi için **XORWF** komutu kullanılmıştır.

Bu komutun açıklamasına geçmeden önce XOR yani ÖZEL VEYA kapısının çalışmasını hatırlayalım.

A	B	Q=A⊕B
0	0	0
0	1	1
1	0	1
1	1	0

Görüldüğü gibi ÖZEL VEYA kapısı, girişlerine gelen bilgiler aynı ise 0, farklı ise 1 bilgisi üretmektedir. Demek ki birbirine eşit iki veriyi ÖZEL VEYA işlemine tabi tutarsak elde edeceğimiz sonuç 0 olur.

Şimdi XORWF komutunu inceleyelim.

```
XORWF f, d          ; File register içerisindeki bilgi ile W içerisindeki bilgi
                    ; arasında ÖZEL VEYA işlemi yap. Sonucu hedef W ise W'ya, f
                    ; ise file register'a yükle.
```

```
XORWF PORTA,W       ; PORTA'dan gelen bilgi ile W içerisindeki bilgi arasında XOR
                    ; işlemi yap. Sonucu W'ya yükle.
```

Bu programda yapmak istediğimiz, **PORTA**'dan gelen bilginin 11001 olup olmadığını tespit etmektir. O halde 11001 bilgisini **W**'ya yükleyip giriş bilgisi ile karşılaştırmamız gerekir. Eğer **PORTA**'dan gelen bilgi 11001 ise sayılar eşit olduğundan sonuç 0 olacak, bu durumda **Z** (zero flag) = 1 olacaktır. Biz de zero flag'in 1 olma durumunu test ederek çıkıştaki ledi yakacağız.

```
  0001 1001
⊕ 0001 1001   ⟶   Z = 1
  0000 0000
```

NOT **PORTA'nın sadece 5 biti kullanılabildiğinden son üç bit daima 0'dır.**

Aynı programı **XORLW** komutu ile de yapabiliriz.

```
XORLW k             ; W'daki sayı ile k sabit sayısı arasında ÖZEL VEYA işlemi yap.
                    ; Sonucu W'ya yükle.
```

Programın test kısmını bu komutla yeniden yazalım.

```
TEST
   MOVF PORTA,W          ; PORTA'daki bilgiyi W'ya yükle
   XORLW B'00011001'     ; W'daki bilgi ile B'00011001' bilgisini XOR işlemi
                         ; ile karşılaştır
   BTFSS STATUS,2        ; Sayılar birbirine eşit mi? (Zero Flag = 1 mi? )
   GOTO TEST             ; Hayır, yeniden test etmek için TEST'e dallan
   BSF PORTB,0           ; Evet, PORTB'nin 0. bitine bağlı ledi yak
```

PROGRAM 18: PORTA'nın 0, 1, 2 ve 3 ve 4 Numaralı Uçlarına Bağlı Olan Butonlardan Girilen Sayının Değerini PORTB'ye Bağlı Ledlerde Gösteren Program

```
   LIST P=16F84          ; Kullanılacak PIC'i tanıt
   INCLUDE "P16F84.INC"  ; INCLUDE dosyasını yükle
SAYAC1 EQU H'0C'         ; H'0C' adresinde SAYAC1,
SAYAC2 EQU H'0D'         ; H'0D' adresinde SAYAC2 isimli bir register oluştur
   BSF STATUS,5          ; Bank-1'e geç
   CLRF TRISB            ; PORTB'yi çıkış olarak yönlendir
BCF STATUS,5             ; Bank-0'a geri dön
   CLRF PORTB            ; PORTB'yi temizle
OKU
   MOVF PORTA,W          ; PORTA'dan girilen sayıyı W'ya yükle
   XORLW B'00011111'     ; Sayının gerçek değerini bulmak için ilk 5 biti tersle
   MOVWF PORTB           ; Sayıyı PORTB'de göster
   GOTO OKU              ; Yeniden PORTA'yı okumak için OKU'ya dallan
   END
```

Bu programı anlayabilmek için önce butonlardan sayı girişinin nasıl yapıldığına bakalım. Aşağıda butonlardan sayıların girilişine birkaç örnek verilmiştir.

RA4	RA3	RA2	RA1	RA0	
2^4	2^3	2^2	2^1	2^0	
-	B	B	-	-	= 12
B	-	-	-	B	= 17
-	-	B	B	-	= 6

B = Basılı

Örneğin, RA4 ve RA0 butonlarına aynı anda basıldığında $2^4+2^0 = 17$ sayısı girilmiş olacaktır. Görüldüğü gibi basılan butonların değerinin 1, basılmayanların ise 0 olması gerekmektedir. Ancak bizim kullandığımız bağlantıya göre basılan buton 0 bilgisi üretmekte, basılmayanlar ise lojik-1 seviyesinde kalmaktadır. O halde butonlardan, yani **PORTA**'dan gelen bilginin terslenmesi gerekir (Son üç bitin değişmeden 0 olarak kalması gerekir, bu yüzden terslenmemelidir). Tersleme işlemi **XORLW** komutu ile yapılabilir.

0 ile **XOR**'lanan bitler aynı kalırken 1 ile **XOR**'lanan bitler terslenir.

Programda **PORTA**'daki bilgi **W**'ya atılmış, daha sonra 00011111 bilgisi ile **XOR**'lanarak RA0-RA4 arasındaki bitler terslenmiştir. Elde edilen bilgi **PORTB**'de gösterildikten sonra program **PORTA**'dan yeni sayı girişi için **OKU** etiketine dallandırılmıştır.

Örneğin **PORTA**'dan 17 sayısını girmek için RA4 ve RA0'a aynı anda bastığımızda **PORTA**'dan girilen bilgi 00001110 olur. Bu sayıyı B'00011111' bilgisi ile **XOR**'ladığımızda;

```
   0000 1110
⊕  0001 1111
-------------
   0001 0001
```

bilgisi, yani 17 sayısı elde edilir. Bu bilgi çıkış portuna atılarak ledler üzerinde gösterilmesi sağlanır.

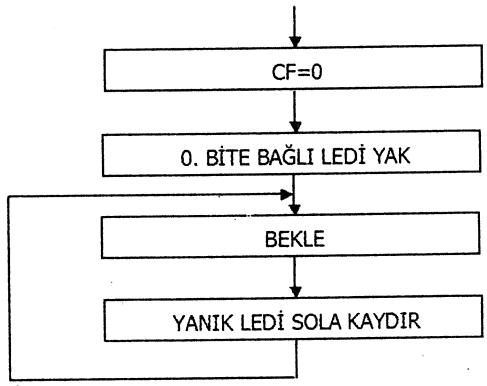
Aynı program **XORWF** komutu ile şu şekilde yapılabilir.

```
OKU
   MOVLW B'00011111'       ; W'ya B'00011111' sayısını yükle
   XORWF PORTA,W           ; PORTA'dan girilen sayının ilk 5 bitini tersle
   MOVWF PORTB             ; Sayıyı PORTB'de göster
   GOTO OKU                ; OKU'ya dallan
```

Bölüm 8

Bit Kaydırma

PROGRAM 19: PORTB'ye Bağlı Ledler Üzerinde Bir Ledin Yanışını 0. Bitten 7. Bite doğru Sürekli Kaydıran Program




```
    LIST P=16F84          ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"  ; INCLUDE dosyasını yükle
SAYAC1 EQU H'0C'          ; H'0C' adresinde SAYAC1,
SAYAC2 EQU H'0D'          ; H'0D' adresinde SAYAC2 isimli bir register oluştur
    BSF STATUS,5          ; Bank-1'e geç
    CLRF TRISB            ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5          ; Bank-0'a geri dön
    BCF STATUS,0          ; Carry Flag'i sıfırla
    CLRF PORTB            ; PORTB'yi temizle
    MOVLW H'01'           ; PORTB'nin 0. bitine bağlı
    MOVWF PORTB           ; ledi yak
BASLA
    CALL GECIKME          ; Yanan ledin görülebilmesi için bir süre bekle
    RLF PORTB,F           ; PORTB'deki bilgiyi sola kaydır
    GOTO BASLA            ; BASLA etiketine dallan
GECIKME
    MOVLW H'FF'
    MOVWF SAYAC1
DONGU1
    MOVLW H'FF'
    MOVWF SAYAC2
DONGU2
    DECFSZ SAYAC2,F
    GOTO DONGU2
    DECFSZ SAYAC1,F
    GOTO DONGU1
    RETURN
    END
```

PIC Assembly'de iki adet kaydırma komutu vardır: **RLF** ve **RRF**

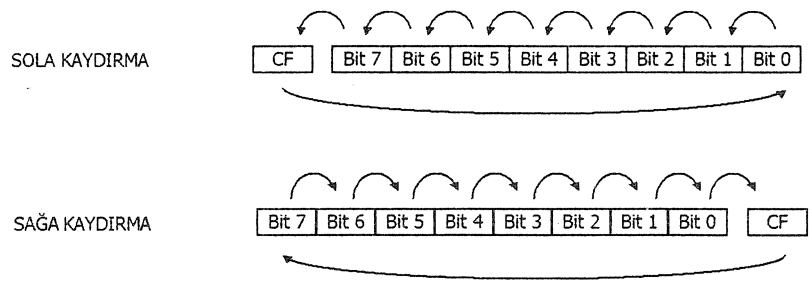
```
RLF f, d        ; File register içerisindeki sayıyı bir pozisyon sola kaydır.
                ; Sonucu hedef W ise W'ya, f ise file register'a yükle.
```

```
RLF SAYAC, F    ; SAYAC register'ı içerisindeki sayıyı bir pozisyon sola kaydır.
                ; Sonucu yine SAYAC register'ına yükle
```

```
RRF f, d        ; File register içerisindeki sayıyı bir pozisyon sağa kaydır.
                ; Sonucu hedef W ise W'ya, f ise file register'a yükle.
```

```
RRF SAYAC, F    ; SAYAC register'ı içerisindeki sayıyı bir pozisyon sağa kaydır.
                ; Sonucu yine SAYAC register'ına yükle
```

Sola ve sağa kaydırma işlemlerini şematik olarak inceleyelim.



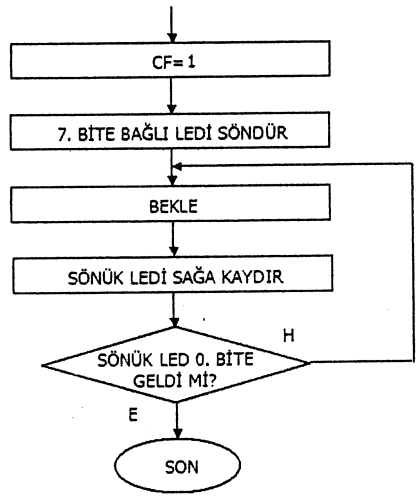
Görüldüğü gibi kaydırma işlemleri Carry Flag üzerinden yapılmaktadır. Örneğin sağa kaydırma işleminde tüm bitler bir sağ pozisyona yerleştiğinde, en sağdaki (0.) bit boşta kalacağı için, yani kayabileceği bir pozisyon kalmadığı için carry flag içerisine yüklenir. Carry flag içerisindeki sayı ise register'ın 7. bitine yüklenir.

Sola kaydırma işleminde ise 7. bit carry flag'e, carry flag içerisinde daha önceden bulunan sayı ise register'ın 0. bitine yüklenir.

Demek ki kaydırma işlemine başlamadan önce carry flag içerisinde bulunan sayı önemlidir. Yukarıdaki programda tek bir yanık led olması istendiğine göre carry flag içerisine önceden 0 bilgisi yüklenmesi gerekir. Aksi halde carry flag'den 1 bilgisi gelebilir, bu durumda uygulamada bir değil, iki yanık ledin kaydığı gözlenir.

Programda önce **PORTB**'ye 00000001 bilgisi gönderilerek 0. bite bağlı led yakılmış, daha sonra **RLF** komutu ile sola kaydırma işlemi gerçekleştirilmiştir. Kayma işleminin görülebilmesi için yanık ledin pozisyonunun her değişiminde gecikme alt programı çağrılmıştır. Sürekli **BASLA** etiketine dallanıldığından kayma işlemi de sürekli olmaktadır.

PROGRAM 20: PORTB'ye Bağlı Yanık Ledler Üzerinde Bir Sönük Ledi 7. Bitten 0. Bite Doğru Kaydıran, Sönük Led 0. Bite Geldiğinde Duran Program

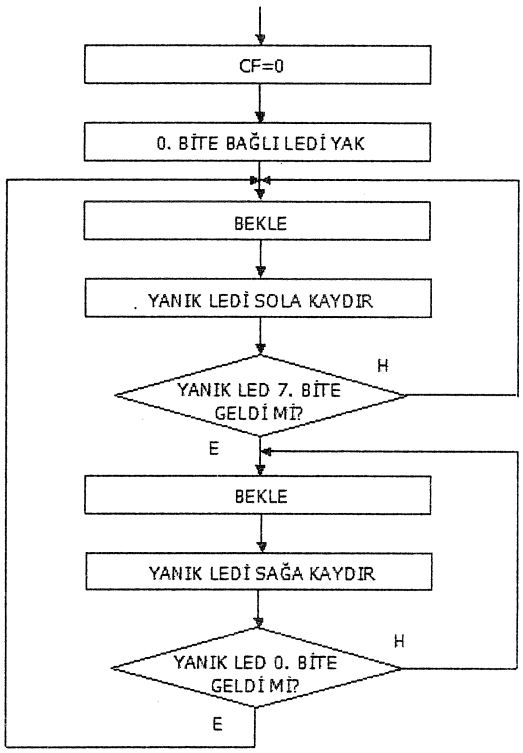



```
   LIST P=16F84            ; Kullanılacak PIC'i tanıt
   INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
SAYAC1 EQU H'0C'           ; H'0C' adresinde SAYAC1,
SAYAC2 EQU H'0D'           ; H'0D' adresinde SAYAC2 isimli bir register oluştur
   BSF STATUS,5            ; Bank-1'e geç
   CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
   BCF STATUS,5            ; Bank-0'a geri dön
   BSF STATUS,0            ; Carry Flag'i 1 yap
   CLRF PORTB              ; PORTB'yi temizle
   MOVLW B '01111111'      ; PORTB'nin 7. bitine bağlı ledi söndür,
                           ; diğerlerini yak
   MOVWF PORTB
BASLA
   CALL GECIKME            ; Sönük ledin görülebilmesi için bir süre bekle
   RRF PORTB,F             ; PORTB'deki bilgiyi sağa kaydır
   BTFSC PORTB,0           ; Sönük led 0. bite yani en sağa geldi mi?
   GOTO BASLA              ; Hayır, bilgiyi yeniden kaydırmak için BASLA'ya
                           ; dallan
DONGU
   GOTO DONGU              ; Evet, programı durdur
GECIKME
   MOVLW H'FF'
   MOVWF SAYAC1
DONGU1
   MOVLW H'FF'
   MOVWF SAYAC2
DONGU2
   DECFSZ SAYAC2,F
   GOTO DONGU2
   DECFSZ SAYAC1,F
```

```
GOTO DONGU1
RETURN
END
```

Bu programın Program 19'dan farkı kayma işleminin bir kez yaptırılmasıdır. Önce porttaki tüm ledleri yakıp sadece 7. bite bağlı ledi söndürmek için B'01111111' bilgisi **PORTB**'ye yüklenir. Bir süre gecikme sağlanarak sönük led görüldükten sonra porttaki bilgi bir pozisyon sağa kaydırılır, yani sönük led bir sağa geçer.Bu noktada sönük ledin en sağa gelip gelmediğini test etmemiz gerekir. 0. bit 0 olmuşsa, sönük led en sağa gelmiş demektir. Bu durumda program sonsuz döngüye sokularak kayma işlemi durdurulur, olmamışsa sönük led en sağa gelene kadar bitlerin pozisyonu kaydırılmaya devam edilir.

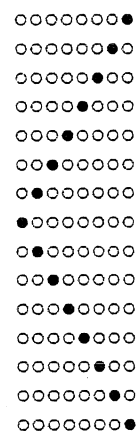
PROGRAM 21: PORTB'ye Bağlı Ledler Üzerinde Bir Yanık Ledi Sürekli Olarak Sağa-Sola Kaydıran Program (Kara Şimşek)




```
   LIST P=16F84            ; Kullanılacak PIC'i tanıt
   INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
SAYAC1 EQU H'0C'           ; H'0C' adresinde SAYAC1,
SAYAC2 EQU H'0D'           ; H'0D' adresinde SAYAC2 isimli bir register oluştur
   BSF STATUS,5            ; Bank-1'e geç
   CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
   BCF STATUS,5            ; Bank-0'a geri dön
   BCF STATUS,0            ; Carry Flag'i sıfırla
```

```
    CLRF PORTB              ; PORTB'yi temizle
    MOVLW H'01'             ; PORTB'nin 0. bitine bağlı ledi yak
    MOVWF PORTB
SOL
    CALL GECIKME            ; Yanık ledin görülebilmesi için bir süre bekle
    RLF PORTB,F             ; PORTB'deki bilgiyi sola kaydır
    BTFSS PORTB,7           ; Yanan led en sola geldi mi?
    GOTO SOL                ; Hayır, yeniden sola kaydırmak için SOL'a dallan
SAG
    CALL GECIKME            ; Evet, yanık ledin görülebilmesi için bir süre bekle
    RRF PORTB,F             ; PORTB'deki bilgiyi sağa kaydır
    BTFSS PORTB,0           ; Yanık led 0. bite yani en sağa geldi mi?
    GOTO SAG                ; Hayır, yeniden sağa kaydırmak için SAG'a dallan
    GOTO SOL                ; Evet, sola kaydırmak için SOL'a dallan
GECIKME
    MOVLW H'FF'
    MOVWF SAYAC1
DONGU1
    MOVLW H'FF'
    MOVWF SAYAC2
DONGU2
    DECFSZ SAYAC2,F
    GOTO DONGU2
    DECFSZ SAYAC1,F
    GOTO DONGU1
    RETURN
    END
```

Programda önce 0. bite bağlı olan led yakılmış, bir süre bekledikten sonra porttaki bilgi sola kaydırılmıştır. 7. bit 1 olana kadar sola kaydırma işlemi devam etmiş, bundan sonra bilgi sağa kaydırılmaya başlamıştır. Yanık ledin en sağa gelip gelmediği yine portun 0. biti kontrol edilerek tespit edilmiştir. Program döngüye sokularak kayma işleminin sürekli olması sağlanmıştır. Ledlerin aşağıdaki şekilde sürekli bir yanık ledi kaydırdığı gözlenecektir.



Bölüm 9

Motor Kontrolü

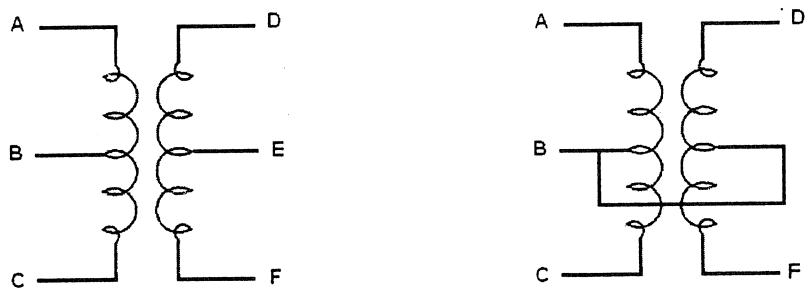
Step Motorlar

Step motor, uçlarına lojik sinyaller uygulandığında dönme hareketi gerçekleştiren motordur. Lojik sinyallerle çalışabilmesi mikroişlemcili/mikrodenetleyicili sistemler tarafından kolaylıkla kontrol edilmesine imkan verir. Örneğin bilgisayarların disket sürücülerinde, yazıcılarda ve motor hareketinin hassas bir biçimde denetlenebilmesini gerektiren diğer uygulamalarda step motorlar kullanılır.

Step motor, bobinlerine uygulanan darbenin her değişiminde bir adım atar. Adım (step) motor ismini de buradan alır. Her bir adımda motorun döneceği açı derecesi motorun üzerinde yazar. Piyasada 1.8 ve 7.5 derecelik step motorlar bulunmaktadır.

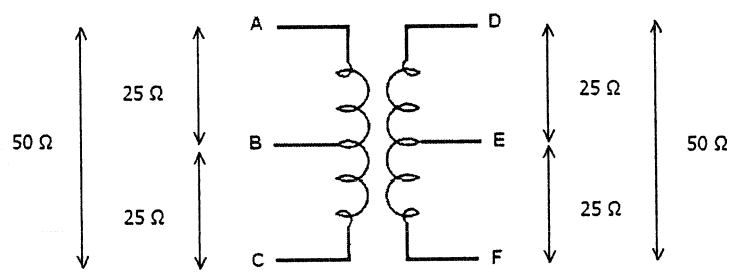
Step Motor Uçlarının Bulunması

Step motorlar 5 veya 6 uçlu olabilir. 6 uçlu bir step motorda 4 uç bobin ucu, 2 uç ise ortak uçtur. 5 uçlu step motorlarda ise iki ortak uç birleştirilerek tek bir ortak uç elde edilmiştir, kalan dört uç ise bobin uçlarıdır.



ŞEKİL 24: 5 ve 6 uçlu step motorlar.

6 uçlu bir step motorun kabloları üçerli iki gruba ayrılmış durumdadır. Bu üçlü gruplardan biri alınır. AVOmetre ohm kademesine getirilir. Kablolardan biri diğer ikisi ile aynı direnç değerini gösterecektir. Bu ortak uçtur. Kalan ikisi ise kendi aralarında ortak uçla gösterdiklerinin iki katı direnç gösterecektir. Bunlar da bobin uçlarıdır. İkinci üçlü gruptaki direnç değerleri birincisi ile aynı çıkar.



ŞEKİL 25: 6 uçlu step motor uçlarında ölçülen direnç değerleri.

Ölçüm sonuçlarına göre B ve E uçları ortak uç, A, C, D ve F uçları ise bobin uçlarıdır.

5 uçlu bir step motorun uçlarını bulmak için yine AVOmetre ohm kademesine alınır. Uçlardan biri diğer dört uçla aynı direnç değerini gösterecektir. Bu ortak uçtur. Kalan uçlar ise bobin uçlarıdır.

Uygulanacak Darbe Sırasının Tespiti

Step motorun bir yöne doğru dönebilmesi için ortak uca +12V uygulanır. Bobin uçlarına ise sırasıyla şase yani lojik-0 verilmesi gerekir. Bobin uçlarına hangi sırayla şase verileceği nasıl tespit edilebilir? Bunun tek yolu, doğru kombinasyonu bulana kadar denemektir. Uçları yanlış bağlarsanız motor dönmek yerine olduğu yerde titreyecektir. Şaseyi kablolara doğru sıralama ile değdirdiğinizde ise motor hep aynı yöne doğru adımlar atmaya başlayacaktır. Bazen motor üç kabloya şase verdiğinizde aynı yöne doğru döner, 4.'de ise hiçbir yöne dönmeyip olduğu yerde kalır. Motorun aynı yerde durduğunu fark etmeyip adım attığını düşünür ve bu şekilde bağlarsanız devreniz düzgün çalışmayacaktır.

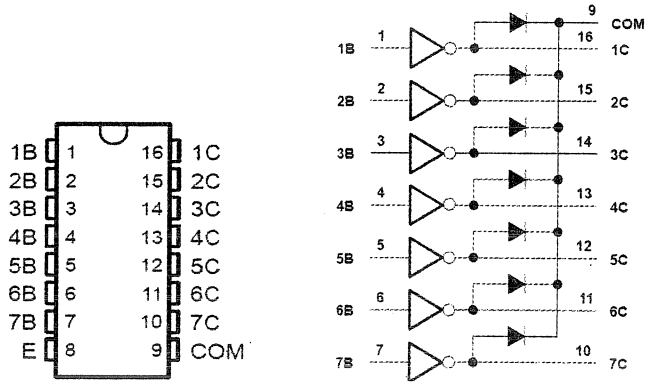
Step Motor Sürücü Devreleri

Step motorlar da diğer motorlar gibi PIC entegresine doğrudan bağlanmaz. Çünkü motorun çekeceği akım PIC16F84'ün verebileceği 20 mA'den çok daha fazladır. Bu yüzden PIC çıkışı ile motor arasına mutlaka bir sürücü devresi bağlanır.

Sürücü devresi olarak transistör/diyot elemanları veya bu elemanları içinde barındıran ve kullanımı daha basit olan entegreler kullanılabilir. Bu kitaptaki uygulamalarda ULN2003A sürücü entegresi kullanılacaktır.

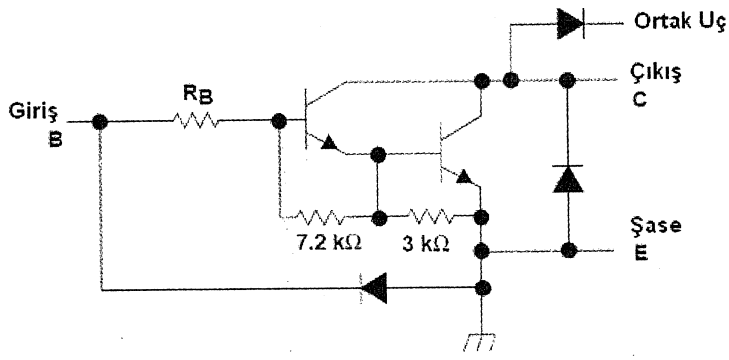
ULN2003A Entegresi

ULN2003A, içerisinde 7 adet NPN darlington transistör barındıran 16 bacaklı bir sürücü entegresidir. 1B, 2B,...,7B giriş uçları, 1C, 2C, ...7C ise bu girişlere karşılık gelen çıkış uçlarıdır. Kullanacağımız step motorda 4 adet bobin ucu bulunmaktadır. Her bir bobin ucuna uygulanacak akımın yükseltilmesi gerektiğinden entegrenin 4 giriş/çıkışı kullanılacaktır. Bu uçlar 1B-1C, 2B-2C, 3B-3C, 4B-4C çiftleri olabileceği gibi istediğiniz herhangi dört çift de olabilir.



ŞEKİL 26: ULN2003A'nın pin yapısı ve lojik diyagramı.

Entegrenin iç yapısında bulunan diyotlar sayesinde endüktif yüklerin transistörlere zarar vermeden çalıştırılması mümkün olur. Bu sayede uygulamalarımızda ULN2003A'yı step motora bağladığımızda harici diyot kullanmamıza gerek kalmayacaktır.

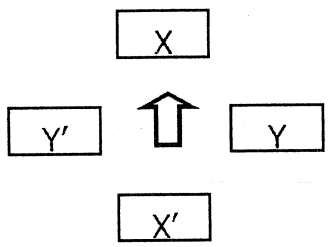


ŞEKİL 27: ULN2003A'nın iç yapısı.

Step Motorların Sürülmesi

Bir step motor 1 fazlı, 2 faz-tam adımlı ve 2 faz-yarım adımlı olmak üzere üç farklı şekilde sürülebilir. Her bir sürüş şekli için motora uygulanacak lojik darbelerin nasıl olması gerektiğini aşağıdaki şekilden faydalanarak yazabiliriz.

Ortadaki ibrenin motorun dönen kısmı yani rotor olduğunu düşünelim. X X' Y Y' bobin uçlarına uygulanacak lojik sinyallere göre meydana gelecek manyetik alan rotorun dönmesini sağlayacaktır. Hangi uçlara lojik-1 verilirse ibre o yöne dönmeye çalışacaktır.



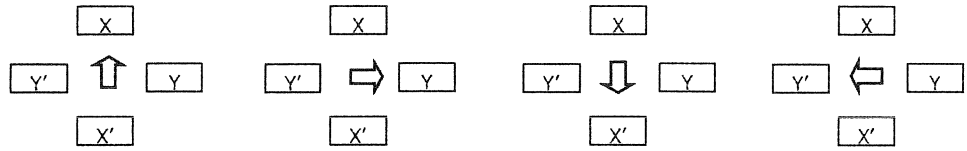
ŞEKİL 28: Bobinler ve rotor.

Step Motorun 1 Fazlı Sürülmesi

TABLO 9: 1 Fazlı Sürüm İçin Uygulanacak Lojik Sinyaller

ADIM	X	Y	X'	Y'
1.	1	0	0	0
2.	0	1	0	0
3.	0	0	1	0
4.	0	0	0	1

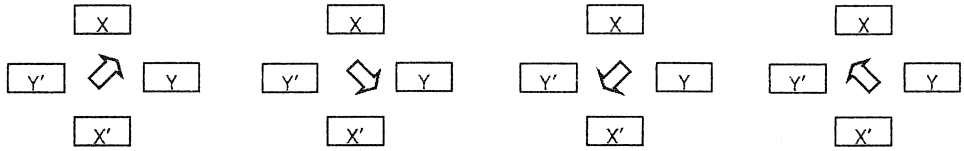
1 fazlı sürümde her defasında bir uca lojik-1 uygulanır. Dört adım tamamlandıktan sonra yeniden başa dönülür ve X=1 yapılır.



ŞEKİL 29: 1 fazlı sürümde rotor hareketi.

Step Motorun 2 Faz - Tam Adımlı Sürülmesi

2 fazlı sürümde uçların aynı anda ikisine birden lojik-1 uygulanır. Step motorun tam adımlı dönmesi demek, her bir adım atışında üzerinde yazan açı derecesi kadar döneceği anlamına gelir. Tam adım sürme metodu ile motordan daha fazla tork elde edilebilir. Tork, motorun döndürme kuvvetidir. Yani tam adım sürümle daha güçlü bir motor kullanmış olursunuz.



ŞEKİL 30: 2 faz tam adımlı sürümde rotor hareketi.

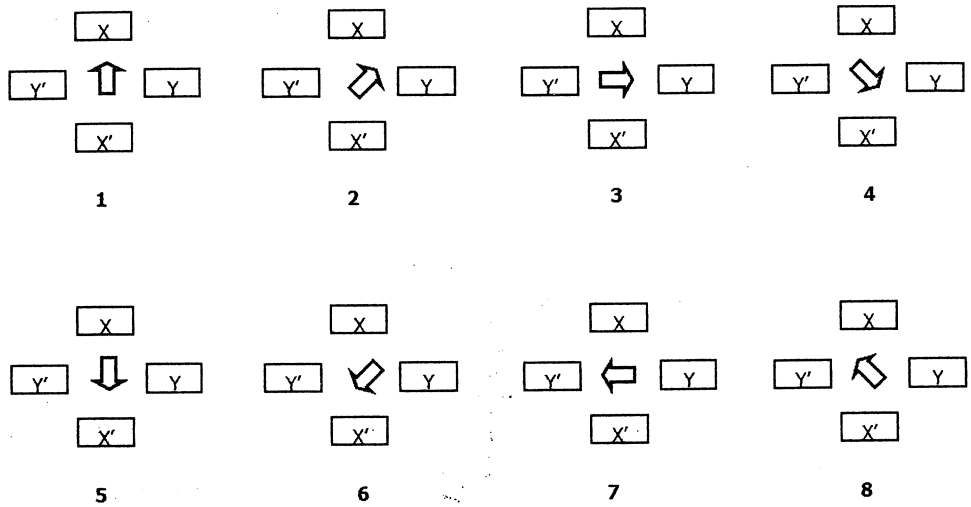
TABLO 10: 2 Faz - Tam Adımlı Sürüm İçin Uygulanacak Lojik Sinyaller

ADIM	X	Y	X'	Y'
1.	1	1	0	0
2.	0	1	1	0
3.	0	0	1	1
4.	1	0	0	1

Step Motorun 2 Faz - Yarım Adımlı Sürülmesi

TABLO 11: 2 Faz - Yarım Adımlı Sürüm İçin Uygulanacak Lojik Sinyaller

ADIM	X	Y	X'	Y'
1.	1	0	0	0
2.	1	1	0	0
3.	0	1	0	0
4.	0	1	1	0
5.	0	0	1	0
6.	0	0	1	1
7.	0	0	0	1
8.	1	0	0	1



ŞEKİL 31: 2 faz yarım adımlı sürümde rotor hareketi.

Step motorun yarım adım dönmesi demek, her bir adım atışında üzerinde yazan açı derecesinin yarısı kadar ilerleyeceği anlamına gelir. Örneğin 7.5 derecelik bir step motor kullanıyorsak motoru 3,75 derecelik adımlarla ilerletebiliriz. Bu da bize daha hassas hareket kontrolü sağlar. Ancak motorun torku yani döndürme kuvveti oldukça düşer.

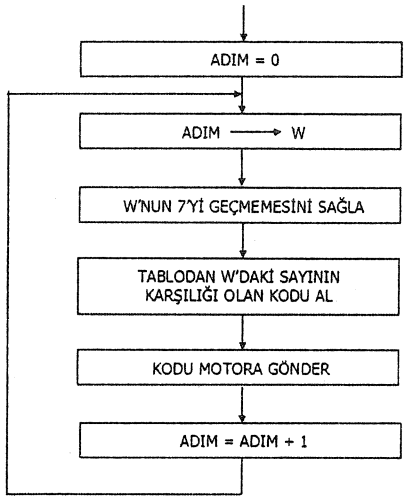
NOT **Yukarıdaki şekil motora uygulanması gereken palslerin nasıl tespit edildiğini anlatmak için çizilmiş temsili bir şekildir. Motora örneğin yarım adım için 8 darbe gönderildiğinde tam bir tur atıyormuş izlenimi verebilir. Aslında durum farklıdır. Yarım adımlı sürüm için her bir adım 3,75 derece olduğundan sekiz adım tamamlandığında motorun döneceği açı miktarı 3,75 x 8 = 30 derece olacaktır.**

Tablo 11'deki kodlar step motor uygulamalarımızda kullanacağımız çevrim tablosudur. **ADIMTBL** başlığı altında bu kodları aşağıdaki şekilde yazabiliriz.

```
ADIMTBL
   ADDWF PCL,F
   RETLW B'1000'
   RETLW B'1100'
   RETLW B'0100'
   RETLW B'0110'
   RETLW B'0010'
   RETLW B'0011'
   RETLW B'0001'
   RETLW B'1001'
```

Hatırlayacağınız gibi çevrim tabloları sabit değerleri çağırma işlemlerinde kullanılır. Çevrim tablosu çağırmadan önce **W** içerisine istenilen değer yüklenir. **ADDWF PCL,F** komut satırı ile program sayıcının değeri **W**'daki sayı kadar ilerletilir. Örneğin **W**'da 0 varsa program sayıcının içeriği 0 ile toplanacak, yani bir sonraki işlenecek satır hemen altındaki satır olan **RETLW B'1000'** satırı olacaktır. Eğer **W**'ya 1 yükleyip tabloyu çağırırsak bu kez bir alt satır işlenecektir. **RETLW** komutunun yanındaki sabit sayı **W**'ya yeni değer olarak yüklenir ve ana programa geri dönülür.

PROGRAM 22: Bir Step Motoru Sürekli Sağa Doğru Döndüren Program




```
   LIST P=16F84            ; Kullanılacak PIC'i tanıt
   INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
SAYAC1 EQU H'0C'           ; H'0C' adresinde SAYAC1,
SAYAC2 EQU H'0D'           ; H'0D' adresinde SAYAC2,
ADIM EQU H'0E'             ; H'0E' adresinde ADIM isimli bir register oluştur
   BSF STATUS,5            ; Bank-1'e geç
   CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
   BCF STATUS,5            ; Bank-0'a geri dön
   CLRF PORTB              ; PORTB'yi temizle
   CLRF ADIM               ; ADIM register'ına 0 yükle
BASLA
   MOVF ADIM,W             ; ADIM'daki bilgiyi W'ya yükle
   ANDLW B'00000111'       ; Bilgiyi 0-7 arasında tut
   CALL ADIMTBL            ; ADIMTBL'den motora gönderilecek kodu al
   MOVWF PORTB             ; PORTB'ye yükle
   CALL GECIKME            ; Sargıların enerjilenmesi için bir süre bekle
   INCF ADIM,F             ; ADIM'ı 1 artır.
   GOTO BASLA              ; Bir adım daha atmak için BASLA'ya dallan
ADIMTBL
   ADDWF PCL,F
   RETLW B'0001'
   RETLW B'1001'
   RETLW B'1000'
   RETLW B'1100'
   RETLW B'0100'
   RETLW B'0110'
   RETLW B'0010'
   RETLW B'0011'
GECIKME
```

```
    MOVLW H'FF'
    MOVWF SAYAC1
DONGU1
    MOVLW H'FF'
    MOVWF SAYAC2
DONGU2
    DECFSZ SAYAC2,F
    GOTO DONGU2
    DECFSZ SAYAC1,F
    GOTO DONGU1
    RETURN
    END
```

Bir step motoru bir yöne doğru sürekli döndürebilmek için uçlarına uygulanması gereken palsleri yukarıda tespit ettik. Eğer bu palsleri motora sırası ile sürekli uygularsak motor da istediğimiz yöne doğru sürekli dönecektir.

7-segment display kullanımı anlatılırken sayılara karşılık gelen sabit sayıları bir tablo haline getirmiş ve sabitleri tablodan **CALL** komutu ile çağırarak programımızda kullanmıştık. Step motoru döndürmek için gerekli olan kodlar da sabit olduğuna göre yine çevrim tablosundan faydalanabiliriz.

Bu kitaptaki uygulamalarda step motor 2 faz-yarım adımlı sürüleceğinden çevrim tablosuna 0-7 arasındaki toplam sekiz adet kodun yazılması gerekir. Programda yapılan işlem, 0-7 arasında sürekli sayan bir sayıcı tasarlamak, sayılar yerine her sayıya karşılık gelen kodu çevrim tablosundan alarak sürekli olarak çıkış portuna göndermektir.

Ana programda öncelikle **ADIM** register'ına 0 bilgisi yüklenir. Çevrim tablosu çağırabilmek için **ADIM** register'ındaki bilgiyi önce **W**'ya yüklememiz gerektiğini biliyoruz. **W**'ya yükleme işleminden hemen sonra **ANDLW** komutu ile maskeleme yapılmıştır. Buradaki amaç sayılan sayıları 0-7 arasında tutabilmektir. Çünkü çevrim tablosunda en son 7. kod vardır. **W**'ya 8 sayısını yükleyip çevrim tablosunu çağırırsak 8. kod olmadığı için program çalışmayacaktır.

Maskeleme İşlemi

```
ANDLW k          ; W register'ındaki sayı ile k sabit sayısı arasında VE işlemi
                 ; yapar, sonucu W'ya yükler.
```

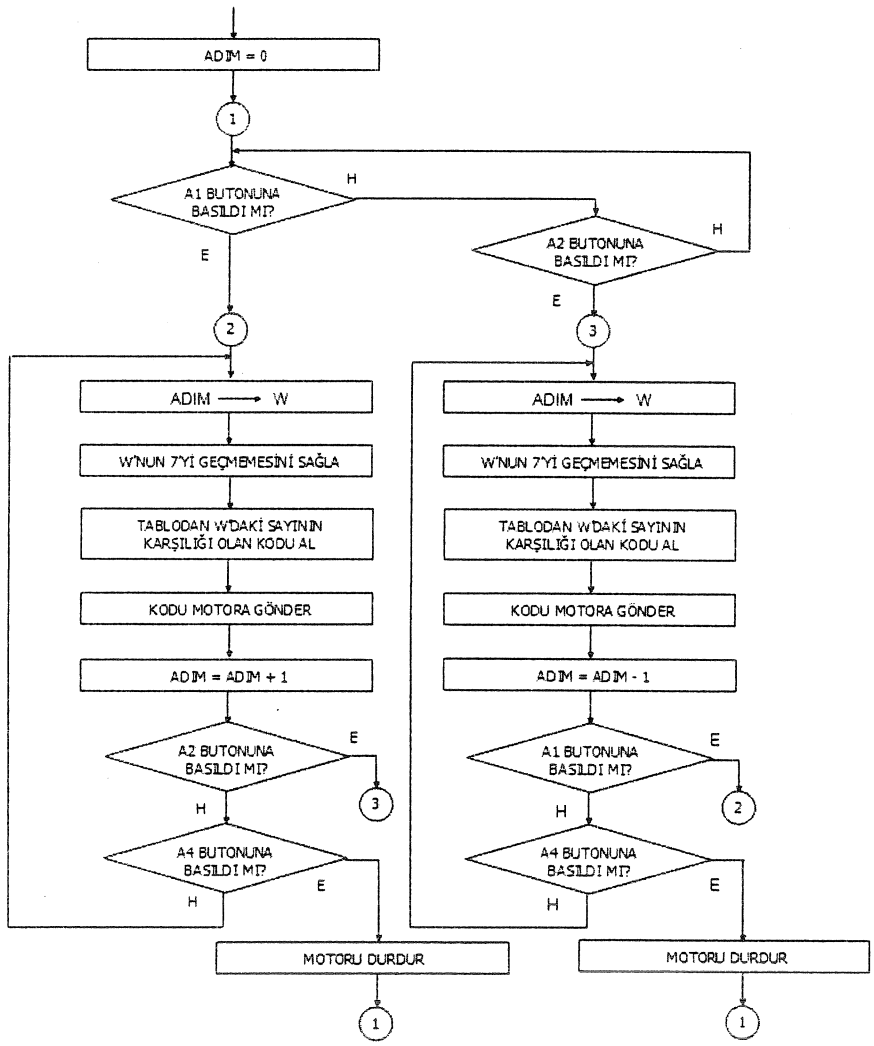
Bildiğiniz gibi lojik AND işlemi aritmetik çarpma işlemidir. Buna göre 0 ile AND'lenen bir bit 0 olurken 1 ile AND'lenen bitin değeri değişmeden kalır. Programımızda ilk üç bit haricindeki bitlerin hep 0 olmasını istediğimiz için B'00000111' bilgisi ile AND'leme işlemi gerçekleştirilmiştir. **W**'nun içeriği 8 olduğunda AND'leme sonucu 0 bilgisi elde edilir, böylece 7'den tekrar 0'a dönülmesi sağlanmış olur.

```
W  ------>      0000 1000
k  ------>    x 0000 0111
              ----------
              0000 0000
```

Çevrim tablosundan alınan kod motorun bağlı olduğu porta atılır. Step motora her bir kodun gönderilmesinden sonra mutlaka bir gecikme alt programı çağrılmalıdır. Aksi halde PIC'in çalışması çok hızlı olduğundan bobinler enerjilenemeden motor uçlarına uygulanan kod değişecek, bu da motorun çalışmamasına sebep olacaktır. Buraya kadar motor bir adım atmış olur. Bir sonraki adımı atabilmesi için **ADIM** register'ının içeriği 1 artırılır. Yeni bilginin motora gönderilmesi için program **BASLA** etiketine dallandırılır.

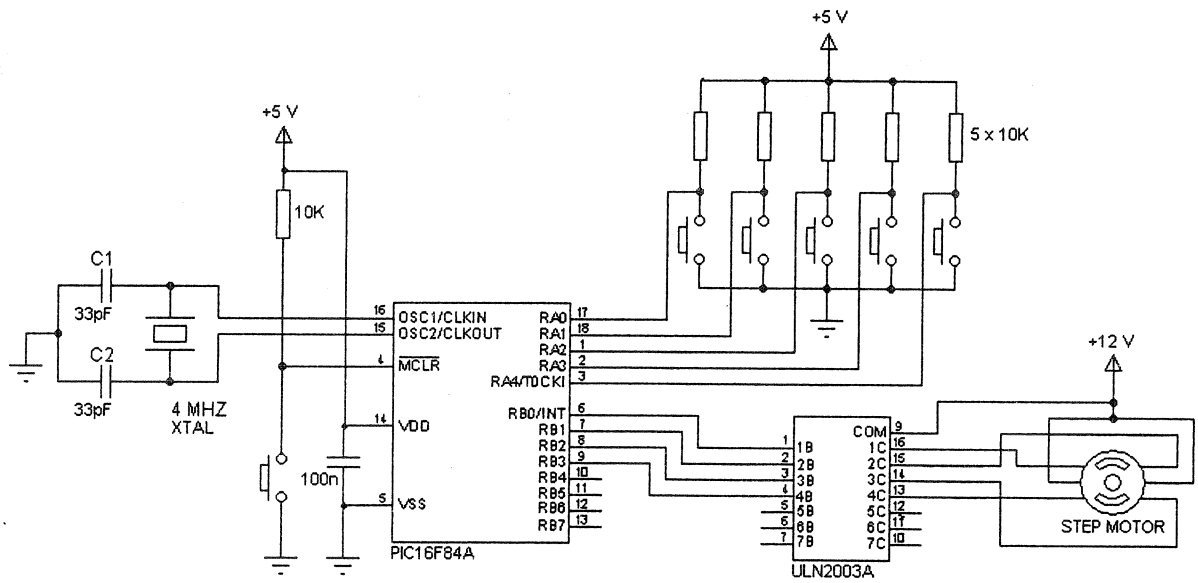
Step motor kontrol uygulamalarında Şekil 32'deki devre şeması kullanılabilir. ULN2003A entegresinin 8 numaralı ucu şaseye bağlanmalıdır. Step motorun ortak uçlarını +12V'a bağladıktan sonra daha önceden tespit ettiğiniz bobin uçlarını sırasıyla 1C, 2C, 3C, 4C uçlarına bağlamanız gerekir.

PROGRAM 23: A1 Butonuna Basınca Step Motoru Sürekli Sağa, A2 Butonuna Basınca Sürekli Sola Döndüren, Bu Arada A4'e Basılırsa Motoru Olduğu Yerde Durduran Program



```
    LIST P=16F84            ; Kullanılacak pic'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
SAYAC1 EQU H'0C'            ; H'0C' adresinde SAYAC1,
SAYAC2 EQU H'0D'            ; H'0D' adresinde SAYAC2,
ADIM EQU H'0E'              ; H'0E' adresinde ADIM isimli bir register oluştur
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF PORTB              ; PORTB'yi temizle
    CLRF ADIM               ; ADIM register'ına 0 yükle
A1
    BTFSC PORTA,1           ; A1 butonuna basıldı mı?
    GOTO A2                 ; Hayır, A2'yi test et.
    GOTO SAG                ; Evet, motoru sağa döndürmek için SAG'a dallan
A2
    BTFSC PORTA,2           ; A2 butonuna basıldı mı?
    GOTO A1                 ; Hayır, A1'i test et.
    GOTO SOL                ; Evet, motoru sola döndürmek için SOL'a dallan
SAG
    MOVF ADIM,W             ; ADIM'daki bilgiyi W'ya yükle
    ANDLW B'00000111'       ; Bilgiyi 0-7 arasında tut
    CALL ADIMTBL            ; ADIMTBL'den motora gönderilecek kodu al
    MOVWF PORTB             ; PORTB'ye yükle
    CALL GECIKME            ; Sargıların enerjilenmesi için bir süre bekle
    INCF ADIM,F             ; ADIM'ı 1 artır
    BTFSS PORTA,2           ; RA2'ye basıldı mı?
    GOTO SOL                ; Evet, SOL etiketine dallan
    BTFSS PORTA,4           ; Hayır, RA4'e basıldı mı?
    GOTO DUR                ; Evet, DUR etiketine dallan
    GOTO SAG                ; Hayır, sağa dönmeye devam etmek için SAG'a dallan
SOL
    MOVF ADIM,W             ; ADIM'daki bilgiyi W'ya yükle
    ANDLW B'00000111'       ; Bilgiyi 0-7 arasında tut
    CALL ADIMTBL            ; ADIMTBL'den motora gönderilecek kodu al
    MOVWF PORTB             ; PORTB'ye yükle
    CALL GECIKME            ; Sargıların enerjilenmesi için bir süre bekle
    DECF ADIM,F             ; ADIM'ı 1 azalt
    BTFSS PORTA,1           ; RA1'e basıldı mı?
    GOTO SAG                ; Evet, SAG etiketine dallan
    BTFSS PORTA,4           ; Hayır, RA4'e basıldı mı?
    GOTO DUR                ; Evet, DUR etiketine dallan
    GOTO SOL                ; Hayır, sola dönmeye devam etmek için SOL'a dallan
DUR
    CLRF PORTB              ; Motoru durdur
    GOTO A1                 ; Yeniden butonlara basılıp basılmadığını kontrol et
ADIMTBL
    ADDWF PCL,F
    RETLW B'0001'
    RETLW B'1001'
    RETLW B'1000'
    RETLW B'1100'
    RETLW B'0100'
    RETLW B'0110'
    RETLW B'0010'
    RETLW B'0011'
```

```
GECIKME
   MOVLW H'FF'
   MOVWF SAYAC1
DONGU1
   MOVLW H'FF'
   MOVWF SAYAC2
DONGU2
   DECFSZ SAYAC2,F
   GOTO DONGU2
   DECFSZ SAYAC1,F
   GOTO DONGU1
   RETURN
   END
```



ŞEKİL 32: Step motor kontrol devresi.

Step motoru sola doğru döndürebilmek için çevrim tablosundaki kodların tersten, yani 7. koddan 0. koda doğru porta gönderilmesi gerekir. Bunun sebebini anlayabilmek için yine Şekil 28'deki çizimden yararlanabiliriz. Motoru saat yönünde çevirmek için X'den sonra Y ucunun enerjilendirilmesi gerekiyordu. Demek ki ters yönde çevrilmesi için X'den sonra Y' ucunun enerjilendirilmesi gerekmektedir. Kodları ters yönde alabilmek için **INCF** yerine **DECF** komutunu kullanıyoruz.

Programda sürekli olarak A1 ve A2 butonları kontrol edilmektedir. İlk aşamada A4 butonunu kontrol etmeye gerek yoktur, çünkü zaten motor durduğu için durdurma butonuna basılıp basılmaması önemli değildir. Eğer kullanıcı A1 butonuna basarsa program **SAG** etiketine dallanır. Burada step motora sağa doğru bir adım attırıldıktan sonra (**MOVF ADIM,W** ile **CALL GECIKME** komutları arasında motor 1 adım atmaktadır.) RA2'ye basılıp basılmadığı test edilir. Basılmışsa **SOL** etiketine dallanılarak motorun sola döndürülmesini sağlayan komutlar işlenir. RA2'ya basılmamışsa bu kez RA4'e basılıp basılmadığı kontrol edilir. Basılmışsa **DUR** etiketine dallanılarak motor durdurulur. Basılmamışsa motorun sağa dönmeye devam etmesi için program **SAG** etiketine dallandırılır.

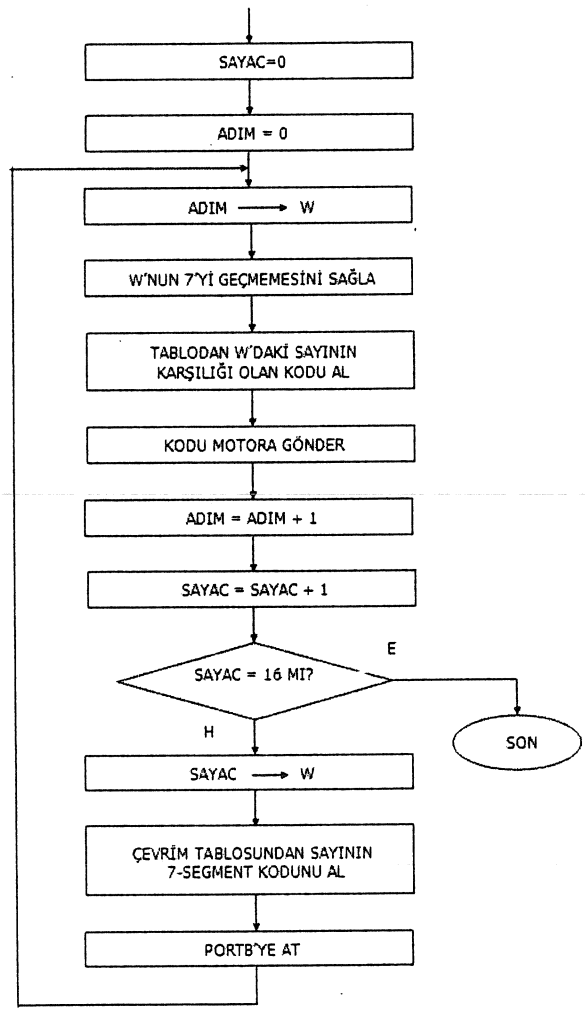
Eğer kullanıcı programın başında RA2'ye basarsa bu kez SOL etiketine dallanılır ve motor sola döndürülür. Her bir adımdan sonra motorun durdurulmak veya sağa çevrilmek istenip istenmediğini anlamak için RA1 ve RA4 kontrol edilir. Bu butonlara basılmamışsa motor sürekli olarak sola döner. Herhangi birine basılmışsa ona ait işlev yerine getirilir.

Step motoru durdurmak için porta sıfır bilgisi gönderilir. Motor durduktan sonra tekrar döndürülebilmesi için program yeniden buton kontrol kısmına dallandırılır.

Step Motoru Belirlenen Adım Sayısı Kadar Döndürme

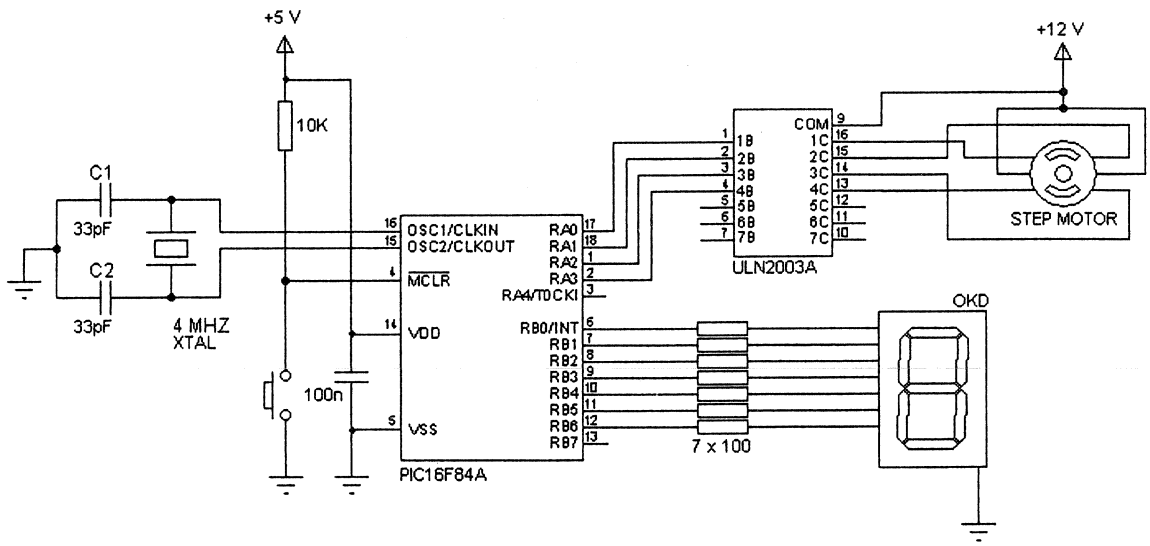
Step motorun en önemli avantajının istediğimiz pozisyona kadar ilerleyip aniden durabilme özelliği olduğunu biliyoruz. Step motoru önceden belirlenen adım sayısı kadar ilerletebilmek için yine sayıcılardaki mantık kullanılır.

PROGRAM 24: PORTA'ya Bağlı Step Motorun Sağa Doğru Her Adım Atışında PORTB'ye Bağlı Display'deki Sayıyı 1 Artıran, Adım Sayısı 15 Olunca Motoru Durduran Program (Display'de 15 kalacak)



```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
SAYAC1 EQU H'0C'            ; H'0C' adresinde SAYAC1,
SAYAC2 EQU H'0D'            ; H'0D' adresinde SAYAC2,
ADIM EQU H'0E'              ; H'0E' adresinde ADIM,
SAYAC EQU H'0F'             ; H'0F' adresinde SAYAC isimli bir register oluştur
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISA              ; PORTA'yı çıkış olarak yönlendir
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF PORTB              ; PORTB'yi temizle
    CLRF PORTA              ; PORTA'yı temizle
    CLRF SAYAC              ; SAYAC'ı sıfırla
    CLRF ADIM               ; ADIM register'ına 0 yükle
BASLA
    MOVF ADIM,W             ; ADIM'daki bilgiyi W'ya yükle
    ANDLW B'00000111'       ; Bilgiyi 0-7 arasında tut
    CALL CEVRIM             ; CEVRIM'den motora gönderilecek kodu al
    MOVWF PORTA             ; PORTA'ya yükle
    CALL GECIKME            ; Sargıların enerjilenmesi için bir süre bekle
    INCF ADIM,F             ; ADIM'ı 1 artır
    INCF SAYAC,F            ; SAYAC'ı 1 artır
    MOVLW D'16'             ; W'ya 16 yükle
    SUBWF SAYAC,W           ; SAYAC'taki sayıdan 16 çıkar
    BTFSC STATUS,2          ; Zero Flag=1mi? (Sayı 16 oldu mu?)
DONGU
    GOTO DONGU              ; Evet, motoru durdur
    MOVF SAYAC,W            ; Hayır, SAYAC'taki bilgiyi W'ya yükle
    CALL CEVRIMTBL          ; CEVRIMTBL'den display'e gönderilecek kodu al
    MOVWF PORTB             ; PORTB'ye yükle
    CALL GECIKME            ; Sayının görülebilmesi için bir süre bekle
    GOTO BASLA              ; Adımı ve display'deki sayıyı artırmak için BASLA'ya
                            ; dallan
CEVRIM
    ADDWF PCL,F
    RETLW B'0001'
    RETLW B'1001'
    RETLW B'1000'
    RETLW B'1100'
    RETLW B'0100'
    RETLW B'0110'
    RETLW B'0010'
    RETLW B'0011'
CEVRIMTBL
    ADDWF PCL,F
    RETLW H'3F'         ; 0
    RETLW H'06'         ; 1
    RETLW H'5B'         ; 2
    RETLW H'4F'         ; 3
    RETLW H'66'         ; 4
    RETLW H'6D'         ; 5
    RETLW H'7D'         ; 6
    RETLW H'07'         ; 7
```

```
    RETLW H'7F'          ; 8
    RETLW H'6F'          ; 9
    RETLW H'77'          ; A
    RETLW H'7C'          ; B
    RETLW H'39'          ; C
    RETLW H'5E'          ; D
    RETLW H'79'          ; E
    RETLW H'71'          ; F
GECIKME
    MOVLW H'FF'
    MOVWF SAYAC1
DONGU1
    MOVLW H'FF'
    MOVWF SAYAC2
DONGU2
    DECFSZ SAYAC2,F
    GOTO DONGU2
    DECFSZ SAYAC1,F
    GOTO DONGU1
    RETURN
    END
```



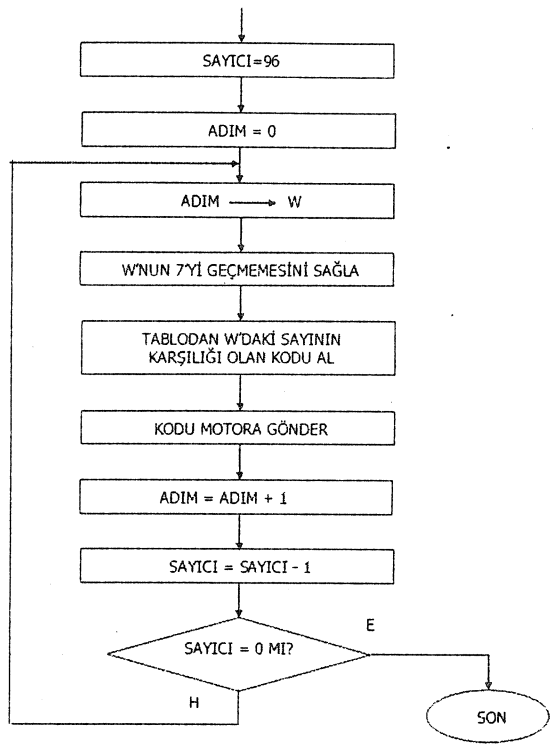
ŞEKİL 33: Step motor ve display bağlantısı.

Yukarıdaki programda motor her bir adım attıktan sonra **SAYAC** register'ının içeriği 1 artırılır. Sayı 16'ya ulaşmamışsa **SAYAC**'taki sayının 7 segment display kodu çevrim tablosundan alınarak display'e gönderilir. Sonra 1 adım daha atması için **BASLA** etiketine dallandırılır. Sayı 16 olmuşsa 15 adım tamamlanmış olduğundan program sonsuz döngü ile sona erdirilir. Yani motor olduğu yerde durur, display'de de son rakam 15 kalır.

NOT **Display'de 0-15 arasındaki rakamlar görüntülenebilir. Ancak 9'dan sonraki rakamların görüntüsü Tablo 8'deki gibi olur.**

Step Motoru Belirlenen Tur Sayısı Kadar Döndürme

PROGRAM 25: Step Motoru Bir Tam Tur (360°) Döndürüp Durduran Program




```
   LIST P=16F84            ; Kullanılacak PIC'i tanıt
   INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
SAYAC1 EQU H'0C'           ; H'0C' adresinde SAYAC1,
SAYAC2 EQU H'0D'           ; H'0D' adresinde SAYAC2
ADIM EQU H'0E'             ; H'0E' adresinde ADIM
SAYICI EQU H'0F'           ; H'0F' adresinde SAYICI isimli bir register oluştur
   CLRF PORTB              ; PORTB'yi temizle
   BSF STATUS,5            ; Bank-1'e geç
   CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
   BCF STATUS,5            ; Bank-0'a geri dön
   CLRF ADIM               ; ADIM'a 0 yükle
   MOVLW D'96'             ; SAYICI register'ına
   MOVWF SAYICI            ; 96 yükle
BASLA
   MOVF ADIM,W             ; ADIM'daki bilgiyi W'ya yükle
   ANDLW B'00000111'       ; Bilgiyi 0-7 arasında tut
   CALL ADIMTBL            ; ADIMTBL'den motora gönderilecek kodu al
   MOVWF PORTB             ; PORTB'ye yükle
   CALL GECIKME            ; Sargıların enerjilenmesi için bir süre bekle
   INCF ADIM,F             ; ADIM'ı 1 artır
```

```
    DECFSZ SAYICI,F          ; SAYICI'yı yani atılan adım sayısını 1 azalt. 0 mı?
    GOTO BASLA               ; Hayır, 96 adımı tamamlamak için BASLA'ya dallan
DONGU
    GOTO DONGU               ; Evet, motoru durdur
ADIMTBL
    ADDWF PCL,F
    RETLW B'0001'
    RETLW B'1001'
    RETLW B'1000'
    RETLW B'1100'
    RETLW B'0100'
    RETLW B'0110'
    RETLW B'0010'
    RETLW B'0011'
GECIKME
    MOVLW H'FF'
    MOVWF SAYAC1
DONGU1
    MOVLW H'FF'
    MOVWF SAYAC2
DONGU2
    DECFSZ SAYAC2,F
    GOTO DONGU2
    DECFSZ SAYAC1,F
    GOTO DONGU1
    RETURN
    END
```

Step motorun 1 tam tur dönebilmesi için kaç adım atması gerektiğini hesaplayalım. Kullandığımız step motorun açısı 7°30'dur (7 derece 30 dakika). 60 dakika = 1 derece olduğundan bu açı 7,5 derecedir. Motoru döndürmek için yarım adım sürüm metodu kullandığımızdan motor her adım attığında 7,5°/2= 3,75° dönecektir. Demek ki motorun 360° dönmesi için 360°/3,75°=96 adım atması gerekir.

Programda yapılan işlem bir **SAYAC** içerisine 96 sayısını yüklemek ve motorun her adım atışında bu sayıyı 1 azaltmaktır. **SAYAC** sıfırlandığı zaman 96 adım yani 1 tam tur tamamlanmış olacağından sonsuz döngü ile motor kaldığı yerde durdurulur.

Bu program Proteus ISIS'te denenirken Step motorun özelliklerine girerek Step Angle = 7.5 yapmanız, yani motorun bir adımda atacağı adımın derecesini 7.5 derece olarak değiştirmeniz gerekir.

PROGRAM 26: Bir Step Motoru Önce 5 Tur Sağa, Hemen Ardından 2 Tur Sola Döndürüp Durduran Program

```
    LIST P=16F84                 ; Kullanılacak pic'i tanıt
    INCLUDE "P16F84.INC"         ; INCLUDE dosyasını yükle
    CBLOCK H'0C'                 ; H'0C' adresinde SAYAC1, H'0D' adresinde SAYAC2
    SAYAC1,SAYAC2,SAYICI,TUR,ADIM ; H'0E' adresinde SAYICI, H'0F' adresinde TUR
    ENDC                         ; ve H'10' adresinde ADIM isimli bir
                                 ; register oluştur
    CLRF PORTB                   ; PORTB'yi temizle
```

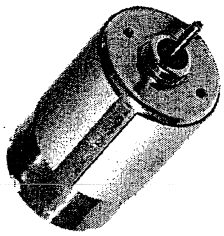
```
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF ADIM               ; ADIM'a 0 yükle
    MOVLW D'5'              ; TUR registerına
    MOVWF TUR               ; 5 bilgisini yükle
YUKLE
    MOVLW D'96'             ; SAYICI registerına
    MOVWF SAYICI            ; 96 bilgisini yükle
SAG
    MOVF ADIM,W             ; ADIM'daki bilgiyi W'ya yükle
    ANDLW B'00000111'       ; Bilgiyi 0-7 arasında tut
    CALL ADIMTBL            ; ADIMTBL'den motora gönderilecek kodu al
    MOVWF PORTB             ; PORTB'ye yükle
    CALL GECIKME            ; Sargıların enerjilenmesi için bir süre bekle
    INCF ADIM,F             ; ADIM'ı 1 artır
    DECFSZ SAYICI,F         ; SAYICI'yı (atılan adım sayısını) 1 azalt. 0 oldu mu?
    GOTO SAG                ; Hayır, 96 adımı tamamlamak için SAG'a dallan
    DECFSZ TUR,F            ; Evet, TUR sayısını 1 azalt. 0 oldu mu?
    GOTO YUKLE              ; Hayır, YUKLE'ye dallan
    MOVLW D'2'              ; Evet, TUR registerına yeni tur sayısı olan
    MOVWF TUR               ; 2 bilgisini yükle
YUKLE2
    MOVLW D'96'             ; SAYICI registerına
    MOVWF SAYICI            ; 96 bilgisini yükle
SOL
    MOVF ADIM,W             ; ADIM'daki bilgiyi W'ya yükle
    ANDLW B'00000111'       ; Bilgiyi 0-7 arasında tut
    CALL ADIMTBL            ; ADIMTBL'den motora gönderilecek kodu al
    MOVWF PORTB             ; PORTB'ye yükle
    CALL GECIKME            ; Sargıların enerjilenmesi için bir süre bekle
    DECF ADIM,F             ; ADIM'ı 1 azalt
    DECFSZ SAYICI,F         ; SAYICI'yı (atılan adım sayısını) 1 azalt. 0 oldu mu?
    GOTO SOL                ; Hayır, 96 adımı tamamlamak için SOL'a dallan
    DECFSZ TUR,F            ; Evet, TUR sayısını 1 azalt. 0 oldu mu?
    GOTO YUKLE2             ; Hayır, YUKLE2'ye dallan
DONGU
    GOTO DONGU              ; Evet, motoru durdur
ADIMTBL
    ADDWF PCL,F
    RETLW B'0001'
    RETLW B'1001'
    RETLW B'1000'
    RETLW B'1100'
    RETLW B'0100'
    RETLW B'0110'
    RETLW B'0010'
    RETLW B'0011'
GECIKME
    MOVLW H'55'
    MOVWF SAYAC1
DONGU1
    MOVLW H'FF'
```

```
    MOVWF SAYAC2
DONGU2
    DECFSZ SAYAC2,F
    GOTO DONGU2
    DECFSZ SAYAC1,F
    GOTO DONGU1
    RETURN
    END
```

Programda tur sayısını tutmak için **TUR** register'ı, adım sayısını tutmak için de **SAYICI** register'ı tanımlanmıştır. Önce motorun 5 tur sağa adım atması için **TUR** register'ına 5, **SAYICI**'ya da bir tur için gerekli olan 96 sayısı yüklenir. Motorun her bir adım atışında **SAYICI** 1 azaltılır. **SAYICI** sıfırlandığında 1 tur tamamlanmış olacağından **TUR** register'ı 1 eksiltilir. Yeni bir tur için program yeniden yükleme kısmına dallandırılır. Tur sayısı sıfırlandığında 5 tur tamamlanmış olacağından bu kez sola dönme işlemini gerçekleştirmek için yeni tur sayısı olan 2, **TUR** register'ı içerisine yüklenir. Aynı işlemler burada da tekrarlanır.

DC Motorlar

DC motorlar uçlarına DC gerilim uygulandığında dönme hareketi gerçekleştiren motorlardır. Uygulanan gerilimin yönü değiştirilirse motor ters yönde döner. Gerilimin ortalama değeri değiştirilerek motorun hız kontrolü yapılabilir. Örneğin 5V'luk bir DC motora 1V uygulasak da motor döner, ancak hızı çok yavaş olur. Gerilim artırıldıkça motorun hızının da gittikçe arttığı görülür.



ŞEKİL 34: DC motor.

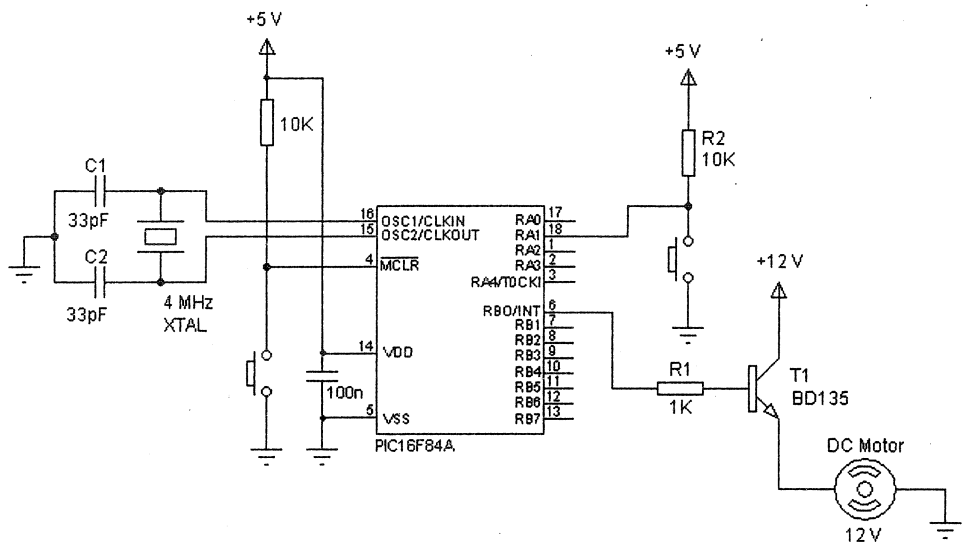
DC motorlar PIC entegresine doğrudan bağlanmaz. Çünkü motorun çekeceği akım fazladır ve bu durumda PIC'i yakar. Bu yüzden motorun bir transistör veya entegre ile sürülmesi gerekir. Bu kitapta transistorlü sürücü devreleri kullanılmıştır.

PROGRAM 27: Bir DC Motoru Bir Yöne Doğru Sürekli Döndüren, Bu Arada RA1 Butonuna Basılırsa Motoru Durduran Program

```
LIST P=16F84            ; Kullanılacak PIC'i tanıt
INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
BSF STATUS,5            ; Bank-1'e geç
CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
BCF STATUS,5            ; Bank-0'a geri dön
```

```
   CLRF PORTB              ; PORTB'yi temizle
DONDUR
   BSF PORTB,0             ; Motoru döndür
   BTFSC PORTA,1           ; RA1'e basıldı mı?
   GOTO DONDUR             ; Hayır, motoru döndürmeye devam et
   BCF PORTB,0             ; Evet, motoru durdur
DONGU
   GOTO DONGU              ; Motorun sürekli durmasını sağla
   END
```

Bu programda enerji verildiği anda motorun dönmesi istendiğinden motorun bağlı bulunduğu uca doğrudan lojik-1 gönderilir. Transistör iletime geçer ve motor üzerinden akım geçerek dönme hareketi sağlanır. Bu arada RA1 butonu kontrol edilir. Butona basılmamışsa motorun dönmeye devam etmesi için **DONDUR** etiketine dallanılır. Basılmışsa RB0 ucuna lojik-0 bilgisi gönderilir, transistör kesime gider ve motor durur.



ŞEKİL 35: DC motor kontrol devresi.

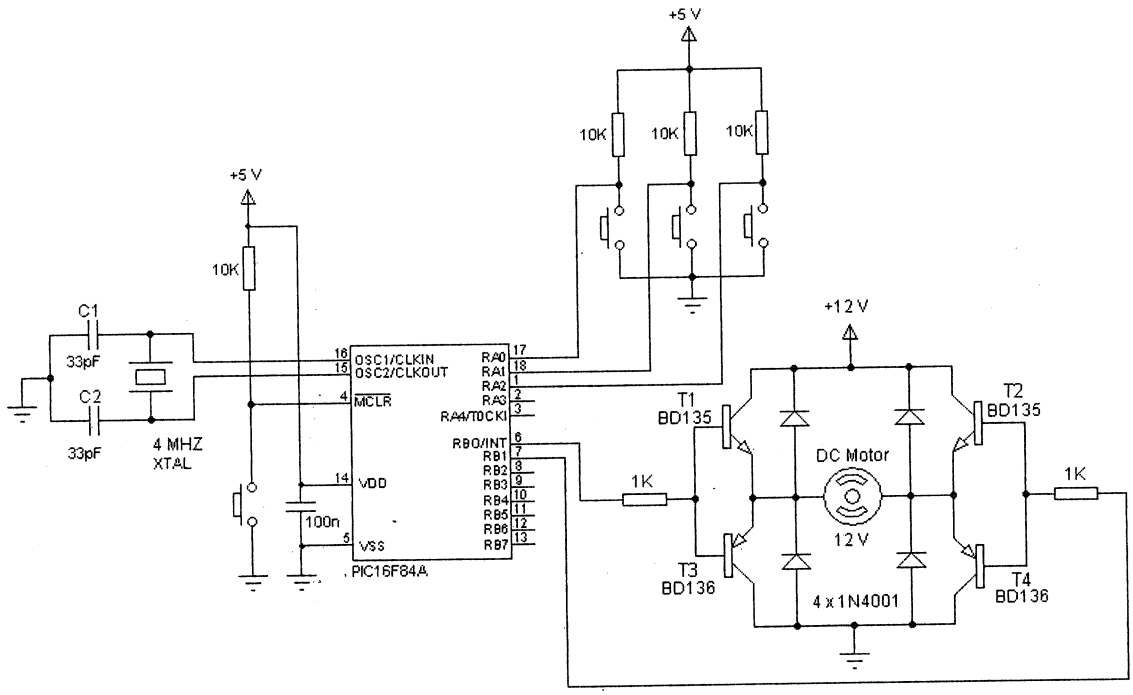
DC Motor Yön Kontrolü

DC motorun transistörlü yön kontrolünün yapılabilmesi için şekildeki bağlantının yapılması gerekir. T1 ve T2 transistörlerinin NPN, T3 ve T4'ün ise PNP transistor olduğuna dikkat ediniz.

Şekle göre RB0=1 ve RB1=0 yapılırsa T1 ve T4 iletime geçer, T2 ve T3 kesime gider. Bu durumda akım T1, motor ve T4 transistörü üzerinden geçerek şaseye akar. Akımın geçiş yönü saat yönünde olduğu için motor sağa doğru döner.

RB0=0 ve RB1=1 yapılırsa bu kez T2 ve T3 iletime geçer, T1 ve T4 kesime gider. Bu durumda akım T2, motor ve T3 transistörü üzerinden şaseye akar. Akımın geçiş yönü bir öncekinin tersi olduğundan motor sola doğru döner.

Motoru olduğu yerde frenlemek için RB0=1 ve RB1=1, motoru boşa almak için de RB0=0 ve RB1=0 yapılmalıdır.



ŞEKİL 36: DC motor yön kontrol devresi.

Devredeki diyotların kullanılma amacı motor bobininde meydana gelen zıt E.M.K'dan transistörlerin zarar görmesini önlemektir.

PROGRAM 28: A0 Butonuna Basılınca DC Motoru Sağa, A1 Butonuna Basılınca Sola Döndüren, A2 Butonuna Basınca Motoru Durduran Program

```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF PORTB              ; PORTB'yi temizle
SAG
    BTFSC PORTA,0           ; RA0'a basıldı mı?
    GOTO  SOL               ; Hayır, SOL etiketine dallan
    MOVLW B'00000001'       ; Evet, motoru sağa döndürmek için RB0'a 1
    MOVWF PORTB             ; RB1'e 0 gönder
SOL
    BTFSC PORTA,1           ; RA1'e basıldı mı?
    GOTO  DUR               ; Hayır, DUR etiketine dallan
    MOVLW B'00000010'       ; Evet, motoru sola döndürmek için RB0'a 0
    MOVWF PORTB             ; RB1'e 1 gönder
DUR
    BTFSC PORTA,2           ; RA2'ye basıldı mı?
    GOTO  SAG               ; Hayır, SAG etiketine dallan
    CLRF  PORTB             ; Evet, motoru durdur
    GOTO  SAG               ; Buton testi için yeniden SAG etiketine dallan
    END
```

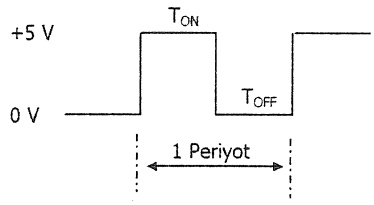
Programda ilk olarak butonların sürekli olarak kontrol edilmesi gerekir. Herhangi bir butona basılmadığı sürece motor durmaktadır. A0 butonuna basıldığında RB0=1 RB1=0 yapılarak T1 ve T4 iletime geçirilir, T2 ve T3 ise kesime götürülür. Bu durumda motor sağa doğru döner. Hemen alt satırda RA1 kontrol edilmektedir. Eğer bu butona basılmışsa bu kez RB0=0 RB1=1 yapılarak T2 ve T3 iletime geçirilir. Motor sola doğru döner. RA2'ye basılması durumunda ise porta 0 bilgisi gönderilerek motor durdurulur.

DC Motor Hız Kontrolü

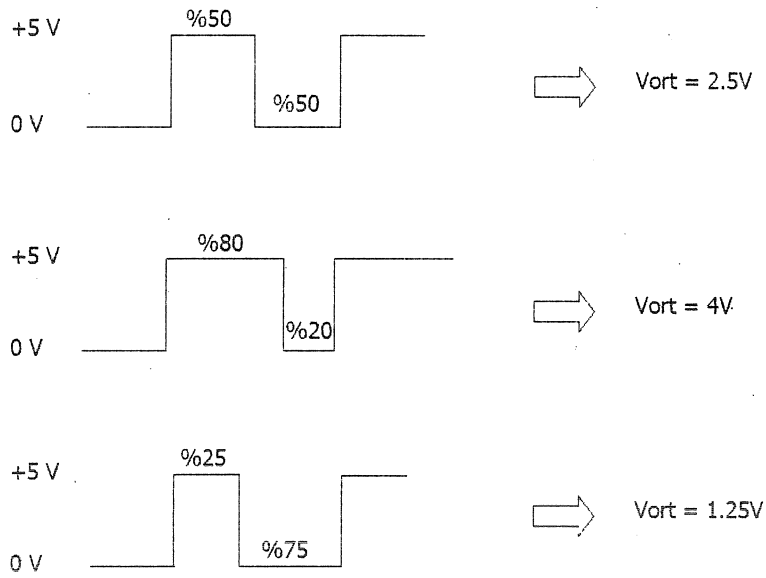
DC motorların hızı motor uçlarına uygulanan gerilimin değiştirilmesi ile kontrol edilir. Gerilim azaldıkça motor yavaşlar, gerilim arttıkça motor hızlanır. Görüldüğü gibi DC motorun hızını ayarlayabilmek için uçlarına uygulanan gerilimin analog olarak değişmesi gereklidir. O halde dijital olarak çıkış veren (sadece +5V ve 0V) PIC mikrodenetleyici ile DC motor hızını nasıl kontrol edebiliriz? PWM metodu ile.

PWM Metodu

Şekilde 0-5 V arasında değişen bir dijital sinyal görülmektedir. Bir periyot içerisinde sinyalin +5V'ta kalma süresine duty cycle (iş çevrimi – T_{ON}) adı verilir. 0'da kalma süresi ise T_{OFF} olarak isimlendirilir.



T_{ON} ve T_{OFF} sürelerinin ortalaması sinyalin ortalama değerini verir. T_{ON} süresi arttıkça sinyalin ortalama değeri artar, T_{ON} süresi azaldıkça ortalama değeri azalır.



Görüldüğü gibi dijital sinyaldeki + ve – darbelerin genişliğini değiştirerek sinyalin ortalama değerini değiştirebiliriz. Bu işlem PWM (Pulse Width Modulation – Darbe Genişlik Modülasyonu) olarak isimlendirilir. PWM sayesinde dijital sistemlere bağlı elemanlara analog sinyaller uygulayabiliriz. Böylece bir DC motorun uçlarına uygulanan gerilimi değiştirerek hızını kontrol edebileceğimiz gibi bir ledin parlaklığını kontrol etmemiz veya bir hoparlörden farklı ses sinyalleri elde etmemiz mümkündür.

PROGRAM 29: PWM Metodu ile DC Motorun Hızını Kontrol Eden Program (Devrede AKTIF register'ı içerisine atılan sayı değiştirildikçe dönme hızının da değiştiği gözlenecektir. Örneğin AKTIF=250 için çok hızlı, AKTIF= 30 için çok yavaş döner.)

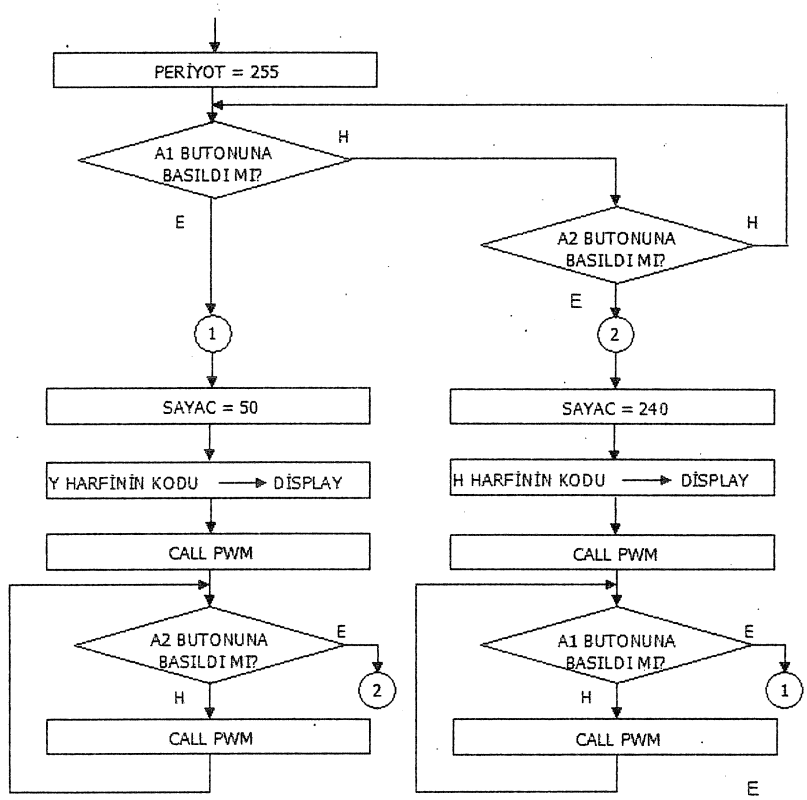
```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
PERIYOT EQU H'0C'           ; H'0C' adresinde PERIYOT,
AKTIF EQU H'0D'             ; H'0D' adresinde AKTIF,
PASIF EQU H'0E'             ; H'0E' adresinde PASIF,
SAYAC EQU H'0F'             ; H'0F' adresinde SAYAC isimli bir register oluştur
    CLRF PORTB              ; PORTB'yi temizle
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
YUKLE
    MOVLW D'255'
    MOVWF PERIYOT           ; PERIYOT'a D'255' yükle
PWM
    MOVLW D'30'
    MOVWF AKTIF             ; AKTIF'e D'30' yükle
    SUBWF PERIYOT,W         ; PERIYOT – 30 = D'225'
    MOVWF PASIF             ; bilgisini PASIF'e yükle
    BSF PORTB,0             ; Motora 1 bilgisi gönder (TON)
PWM0
    CALL GECIKME            ; Bir süre bekle
    DECFSZ AKTIF,F          ; AKTIF'teki bilgiyi 1 azalt, 0 oldu mu?
    GOTO PWM0               ; Hayır, PWM0'a dallan
    BCF PORTB,0             ; Evet, motora 0 bilgisi gönder (TOFF)
PWM1
    CALL GECIKME            ; Bir süre bekle
    DECFSZ PASIF,F          ; PASIF'teki bilgiyi 1 azalt, 0 oldu mu?
    GOTO PWM1               ; Hayır, PWM1'e dallan
    GOTO PWM                ; Evet, PWM'e dallan
GECIKME
    MOVLW D'100'
    MOVWF SAYAC
DONGU
    DECFSZ SAYAC,F
    GOTO DONGU
    RETURN
    END
```

Yapılan iş **PORTB**'nin DC motora giden ucunda bir kare dalga sinyal oluşturmaktır. Bu uca lojik-1 gönderip bir süre bekleyerek T_{ON} süresini, 0 gönderip bir süre bekleyerek de T_{OFF} süresini

elde ediyoruz. Programda T_{ON} süresini tutmak için **AKTIF**, T_{OFF} süresini tutmak için de **PASIF** adında birer register oluşturulmuş, bu register'lara istenen motor hızına göre değerler yüklenmiştir. **AKTIF** içerisindeki ilk değer 30 olduğu için program çalıştırıldığında motorun çok yavaş döndüğü gözlenir. Bu değer örneğin 240 olarak değiştirilirse motor çok hızlı dönecektir.

NOT **Bir register içerisine yüklenecek maksimum değer H'FF'=D'255'tir.**

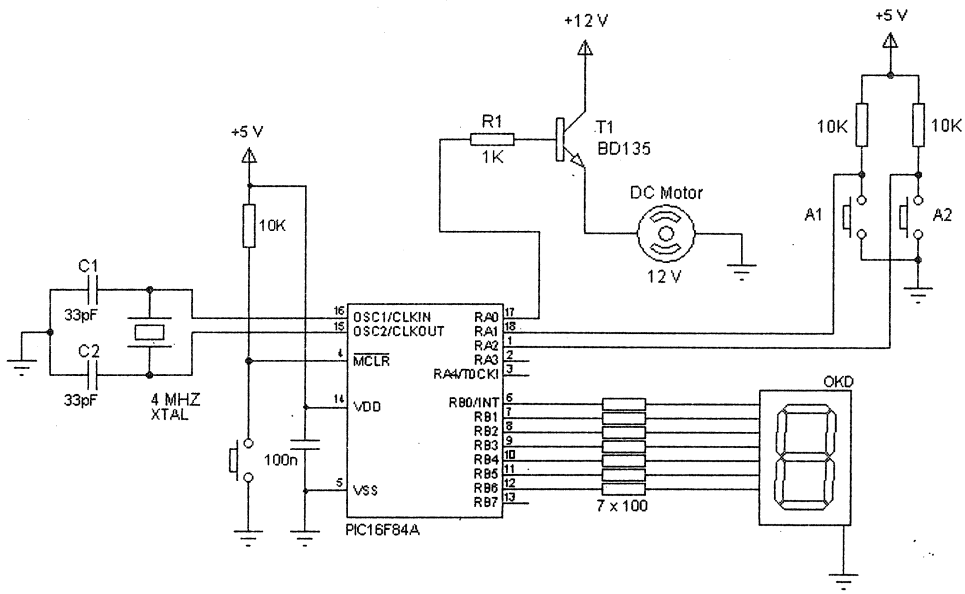
PROGRAM 30: Bir DC Motoru A1 Butonuna Basınca Yavaş, A2 Butonuna Basınca İse Hızlı Döndüren, Motor Hızlı Dönerken 7-segment Display'de H Harfini, Yavaş Dönerken İse Y Harfini Gösteren Program




```
LIST P=16F84            ; Kullanılacak PIC'i tanıt
INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
CBLOCK H'0C'            ; H'0C' adresinde PERIYOT, H'0D' adresinde AKTIF
PERIYOT,AKTIF,PASIF,SAYAC,SAYICI ; H'0E' adresinde PASIF, H'0F'
                        ; adresinde SAYAC ve
ENDC                    ; H'10' adresinde SAYICI isimli bir register oluştur
BSF STATUS,5            ; Bank-1'e geç
CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
MOVLW H'FE'             ; PORTA'nın tüm pinlerini giriş, RA0'ı çıkış
MOVWF TRISA             ; olarak yönlendir
BCF STATUS,5            ; Bank-0'a geri dön
CLRF PORTB              ; PORTB'yi temizle
```

```
   CLRF PORTA              ; PORTA'yı temizle
YUKLE
   MOVLW D'255'
   MOVWF PERIYOT           ; PERIYOT'a D'255' yükle
RA1
   BTFSC PORTA,1           ; RA1'e basıldı mı?
   GOTO RA2                ; Hayır, RA2'yi test et
YAVAS
   MOVLW D'50'             ; Evet, SAYAC'a 50 yükle
   MOVWF SAYAC
   MOVLW H'6E'
   MOVWF PORTB             ; Display'de Y harfini yakacak olan H'6E'yi gönder
   CALL PWM                ; PWM alt programını çağır
TEST
   BTFSS PORTA,2           ; RA2'ye basıldı mı?
   GOTO HIZLI              ; Evet, HIZLI etiketine dallan
   CALL PWM                ; Hayır, PWM alt programını çağır
   GOTO TEST               ; RA2'yi yeniden test et
RA2
   BTFSC PORTA,2           ; RA'ye basıldı mı?
   GOTO RA1                ; Hayır, RA1'i test et
HIZLI

   MOVLW D'240'
   MOVWF SAYAC             ; Evet, SAYAC'a 240 yükle
   MOVLW H'76'
   MOVWF PORTB             ; Display'de H harfini yakacak olan H'76'yı gönder
   CALL PWM                ; PWM alt programını çağır
TEST2
   BTFSS PORTA,1           ; RA1'e basıldı mı?
   GOTO YAVAS              ; Evet, YAVAS etiketine dallan
   CALL PWM                ; Hayır, PWM alt programını çağır
   GOTO TEST2              ; RA1'i yeniden test et
PWM
   MOVF SAYAC,W            ; SAYAC'taki sayıyı
   MOVWF AKTIF             ; AKTIF'e yükle
   SUBWF PERIYOT,W         ; PERIYOT'tan AKTIF'ı çıkar
   MOVWF PASIF             ; Kalanı PASIF'e yükle
   BSF PORTA,0             ; Motora 1 gönder
PWM0
   CALL GECIKME            ; Bir süre bekle
   DECFSZ AKTIF,F          ; AKTIF içerisindeki sayıyı 1 azalt. 0 oldu mu?
   GOTO PWM0               ; Hayır, PWM0'a dallan
   BCF PORTA,0             ; Evet, motora 0 gönder
PWM1
   CALL GECIKME            ; Bir süre bekle
   DECFSZ PASIF,F          ; PASIF içerisindeki sayıyı 1 azalt. 0 oldu mu?
   GOTO PWM1               ; Hayır, PWM1'e dallan
   RETURN                  ; Evet, ana programda kaldığın yere geri dön
GECIKME
   MOVLW D'100'
   MOVWF SAYICI
DONGU
```

```
DECFSZ SAYICI,F
GOTO DONGU
RETURN
END
```

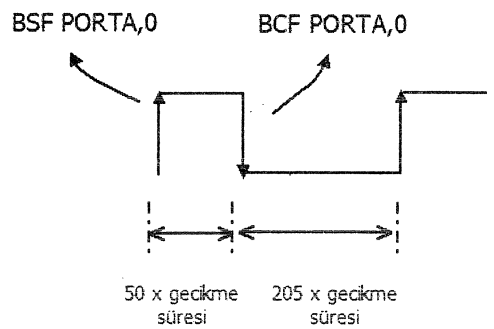


ŞEKİL 37: DC motor ve display bağlantısı.

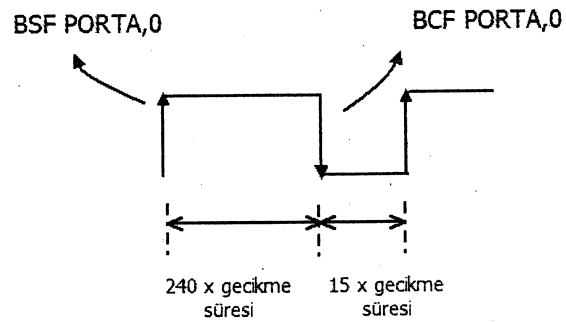
Bu programda farklı butonlara basıldığında motorun farklı hızlarda dönmesi istendiğinden **AKTIF** ve **PASIF** register'larına sabit değerler yüklenmez. Bir **SAYAC** register'ı kullanılarak istenen hıza göre bu register'dan **AKTIF**'e yükleme yapılır. Bir periyottan **AKTIF** değeri çıkarılarak **PASIF** değeri elde edilir.

RA1 butonuna basıldığında motorun yavaş dönmesi istendiğinden **SAYAC**'a 50 yüklenir. Display'de Y harfini yakacak olan H'6E' kodu **PORTB**'ye atılır ve **PWM** alt programı çağrılır.

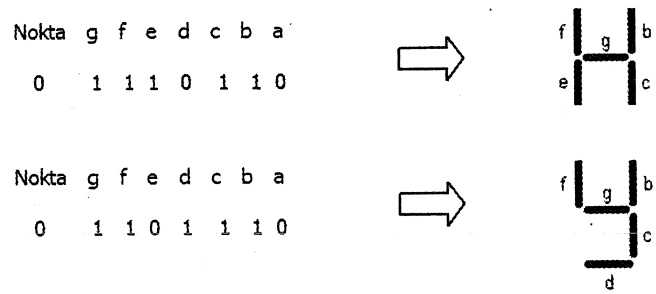
PWM alt programında **AKTIF**'e **SAYAC**'taki 50 bilgisi **PASIF**'e de 255-50=205 bilgisi yüklenir ve motora lojik-1 bilgisi gönderilir. Bir süre gecikme (50 x gecikme) sağlandıktan sonra port 0 seviyesine çekilir ve bu durumda da bir süre (205 x gecikme) gecikme sağlandıktan sonra ana programa geri dönülür. Yani alt programda bir periyotluk sinyal üretilmiş olur.



Ana programda RA2 butonuna basılıp basılmadığı kontrol edilir. Butona basılmışsa motorun hızlı dönmesi istendiğinden program **HIZLI** etiketine dallanır. Basılmamışsa yeniden **PWM** alt programı çağrılarak bir periyotluk sinyal daha üretilir. Yani her bir periyotun sonunda RA2 butonu test edilmiş olur. RA2'ye basıldığında program **HIZLI** etiketine dallanarak **SAYAC** register'ına 240 yükler. Display'e de H simgesini yakacak H'76' kodu yüklenir. Daha sonra program yine **PWM** alt programına dallanır. Burada bu kez **SAYAC**'taki 240 bilgisi **AKTIF** register'ına, 255-240=15 bilgisi de **PASIF** register'ına yüklenir ve yine bir periyotluk sinyal üretilir. Bu kez RA1'e basılıp basılmadığı kontrol edilir ve program bu şekilde devam eder.



Programda display'de H ve Y harflerini yakacak kodu tespit etmek için hangi segmentlere lojik-1 vermemiz gerektiğini aşağıdaki şekilde belirleriz.



Bölüm 10

Aritmetik İşlemler

Toplama İşlemi

8 Bitlik Toplama

PIC Assembly dilinde toplama işlemi yapan iki komut vardır: **ADDWF** ve **ADDLW**

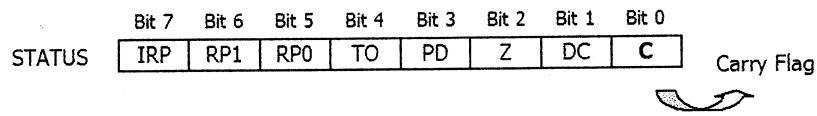
```
ADDWF f, d          ; File register'daki sayı ile W'daki sayıyı topla. Sonucu hedef
                    ; W ise W'ya, f ise aynı register içerisine yükle.
```

```
ADDWF SAYAC,F       ; SAYAC'taki sayı ile W'daki sayıyı topla. Sonucu SAYAC'a yükle.
```

```
ADDLW k             ; W'daki sayı ile k sabit sayısını topla. Sonucu W'ya yükle.
```

```
ADDLW B'11110000' ; W'daki sayı ile B'11110000' sayısını topla. Sonucu W'ya yükle.
```

STATUS register'ın 0. biti elde bayrağıdır (Carry Flag). Carry flag toplama ve çıkarma işlemlerinde, işlem sonucuna göre 0 veya 1 olur, böylece sonucun durumu hakkında bize bilgi verir.



8 bitlik iki sayı toplandığında elde edilecek sonuç 8 biti aşabilir, yani elde meydana gelebilir. Bu durumda Carry Flag =1 olur. Elde yoksa, yani sonuç 8 bit ise C=0'dır.

```
    1011 0111
(+) 0010 1000
-------------
    1101 1111
    --------
      8 bit
```

Elde yok (C=0)

```
    1111 0111
(+) 0111 0000
-------------
  1 0110 0111
    --------
      8 bit
```

Elde var (C=1)

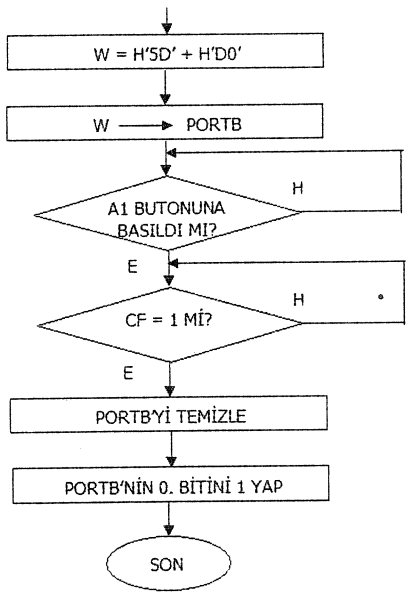
PROGRAM 31: H'12' Sayısı ile H'AD' Sayısını Toplayan ve Sonucu PORTB'deki Ledlerde Gösteren Program

```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF PORTB              ; PORTB'yi temizle
    MOVLW H'12'             ; W'ya H'12' yükle
    ADDLW H'AD'             ; W'daki H'12' ile H'AD' yi topla, sonucu W'ya yükle
    MOVWF PORTB             ; Sonucu PORTB'de göster
DONGU
```

```
GOTO DONGU              ; Programı durdur
END
```

Programda iki sabit sayı toplandığı için **ADDLW** komutu kullanılabilir. Sayılardan biri **W** register'ına yüklenir. Daha sonra diğer sayı ile toplanır. İşlem sonucu **W**'da kaldığı için doğrudan çıkış portuna atılır.

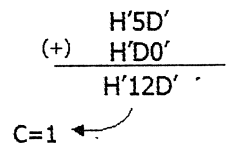
PROGRAM 32: H'5D' Sayısı ile H'D0' Sayısını Toplayıp Sonucu PORTB'de Gösteren, Bu Arada RA1 Butonuna Basılırsa Elde Olup Olmadığını Gösteren Program (Elde Varsa RB0'a Bağlı Led Yanacak, Elde Yoksa İşlem Sonucu Görüntülenmeye Devam Edecek)




```
   LIST P=16F84           ; Kullanılacak PIC'i tanıt
   INCLUDE "P16F84.INC"   ; INCLUDE dosyasını yükle
   BSF STATUS,5           ; Bank-1'e geç
   CLRF TRISB             ; PORTB'yi çıkış olarak yönlendir
   BCF STATUS,5           ; Bank-0'a geri dön
   CLRF PORTB             ; PORTB'yi temizle
   MOVLW H'5D'            ; W'ya H'5D' yükle
   ADDLW H'D0'            ; W'daki H'5D' ile H'D0' ı topla, sonucu W'ya yükle
   MOVWF PORTB            ; Sonucu PORTB'de göster
BUTON
   BTFSC PORTA,1          ; RA1 butonuna basıldı mı?
   GOTO BUTON             ; Hayır, yeniden test etmek için BUTON'a dallan
TEST
   BTFSS STATUS,0         ; Evet, Carry Flag=1 mi? (Elde var mı?)
   GOTO TEST              ; Hayır, yeniden test etmek için TEST etiketine dallan
   CLRF PORTB             ; Evet, PORTB'deki işlem sonucunu sil
   BSF PORTB,0            ; RB0'a bağlı ledi yak
DONGU
```

```
    GOTO DONGU              ; Programı durdur
    END
```

Programda H'5D' ile H'D0' sayısı toplanır, sonuç **PORTB**'de gösterilir. Bu arada RA1 butonu test edilir. Butona basılmışsa Carry Flag'in durumuna bakılır. C=1 ise elde olduğundan RB0'a bağlı led yakılır. C=0 ise işlem sonucu portta kalır. H'5D'+ H'D0' işleminin sonucunda elde meydana geleceği için butona basıldığında ledin yandığı görülür.



PROGRAM 33: H'26' Sayısı ile PORTA'dan Girilecek Sayıyı Toplayan Program (Butonlar RA0, RA1, RA2, RA3 ve RA4'e bağlı olacak. A portu 5 bit olduğu için 5. 6. ve 7. bitler daima 0'dır.)

```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF PORTB              ; PORTB'yi temizle
OKU
    MOVLW B'00011111'       ; B'00011111' bilgisini W'ya yükle
    XORWF PORTA,W           ; PORTA'dan girilen bilgiyi bu bilgi ile XOR'la
    ADDLW H'26'             ; Elde edilen sayı ile H'26'yı topla
    MOVWF PORTB             ; Sonucu PORTB'de göster
    GOTO OKU                ; Yeni sayı girişi için OKU'ya dallan
    END
```

Program 18'de portlardan girilen sayının değerinin nasıl tespit edileceği açıklanmıştı. Burada da aynı şekilde, porttaki butonlardan girilen sayı, butonlar sıfır aktif çalıştığı için son üç bit hariç terslenmiş ve elde edilen sonuç **W**'ya yüklenmiştir. Daha sonra diğer sayı ile toplanarak sonuç **PORTB**'de gösterilmiştir.

Programda hiçbir butona basılmadığında 0 sayısı girilmiş olacağından H'26' + 0 = H'26' sayısı görülecektir. Örneğin RA0, RA1 ve RA2 butonlarına basılarak D'7' sayısı girilirse portta görülen bilgi H'26' + H'07'=H'2D' olur.

16 Bitlik Toplama

PIC16F84'ün veri belleğindeki register'ların uzunluğu 8 bit olduğu için 16 bitlik toplama işlemini yapabilmek için sayıların 8 bitlik iki gruba ayrılarak toplanması gerekir. 16 bitlik toplama işlemini anlayabilmek için toplama işlemi yaparken izlediğimiz yolu inceleyelim.

AH	AL
+ BH	BL
AH + BH	AL + BL

⟹

38	29
+ 41	10
79	39

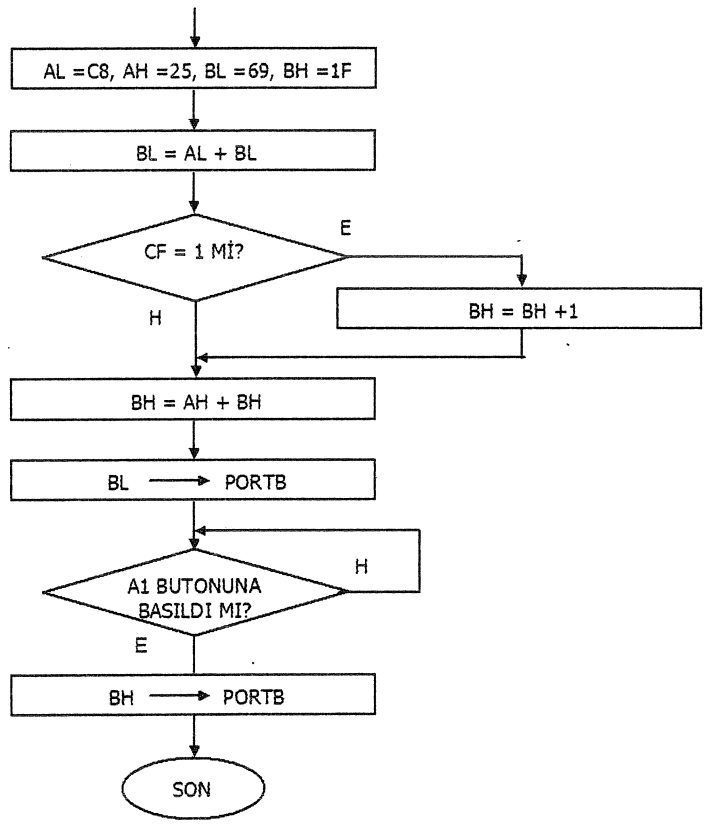
1	
AH	AL
+ BH	BL
AH + BH + 1	AL + BL

⟹

1	
12	F4
+ 83	34
96	28

Görüldüğü gibi, alt byteların toplamı 8 biti geçmezse üst byte'ların toplamı direkt yazılır, geçerse üst bytelardan birine 1 eklenerek diğer üst byte ile toplanır. Şimdi programı buna göre yazalım.

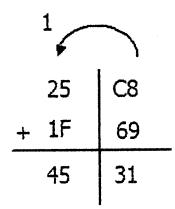
PROGRAM 34: H'25C8' ve H'1F69' Sayılarını Toplayarak Sonucun Alt Byte'ını PORTB'de Gösteren, A1 Butonuna Basıldığında İse Üst Byte'ını Gösteren Program




```
LIST P=16F84            ; Kullanılacak PIC'i tanıt
INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
CBLOCK H'0C'            ; H'0C' adresinde AL, H'0D' adresinde AH,
AL, AH, BL, BH          ; H'0E' adresinde BL, H'0F' adresinde BH
```

```
    ENDC                    ; isimli bir register oluştur
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF PORTB              ; PORTB'yi temizle
    MOVLW H'C8'             ; A sayısının alt byte'ı olan H'C8' sayısını
    MOVWF AL                ; AL register'ına yükle
    MOVLW H'25'             ; A sayısının üst byte'ı olan H'25' sayısını
    MOVWF AH                ; AH register'ına yükle
    MOVLW H'69'             ; B sayısının alt byte'ı olan H'69' sayısını
    MOVWF BL                ; BL register'ına yükle
    MOVLW H'1F'             ; B sayısının üst byte'ı olan H'1F' sayısını
    MOVWF BH                ; BH register'ına yükle
TOPLA
    MOVF AL,W               ; AL'yi W'ya yükle
    ADDWF BL,F              ; AL ile BL'yi topla, sonucu BL'ye yükle
    BTFSC STATUS,0          ; Carry Flag = 0 mı? (Elde yok mu?)
    INCF BH,F               ; Elde var, B sayısına 1 ekle
    MOVF AH,W               ; Elde yok, B'ye 1 eklemeden AH'yi W'ya yükle
    ADDWF BH,F              ; AH ile BH'yi topla, sonucu BH'ye yükle
ALT_BYTE
    MOVF BL,W               ; BL'yi (yani toplamın alt byte'ını)
    MOVWF PORTB             ; PORTB'de göster
A1
    BTFSC PORTA,1           ; A1'e basıldı mı?
    GOTO A1                 ; Hayır, yeniden test et
UST_BYTE
    MOVF BH,W               ; Evet, BH'yi (yani toplamın üst byte'ını)
    MOVWF PORTB             ; PORTB'de göster
DONGU
    GOTO DONGU              ; Programı durdur
    END
```

H'25C8' sayısı **A**, H'1F69' ise **B** sayısı olarak isimlendirilmiştir. Bu sayıların alt byte'ları **AL** ve **BL**, üst byte'ları ise **AH** ve **BH**'dir. Yapacağımız şey öncelikle sayıları bu register'lar içerisine yüklemektir. Daha sonra alt byte'lar toplanır. Eğer alt byte'ların toplamı 8 biti aşmışsa elde meydana gelir ve C=1 olur. Bu durumda üst byte'lar toplanmadan önce herhangi birine 1 eklememiz gerekir. Eğer taşma yoksa üst byte'lar direkt toplanır. Çıkış portuna alt byte'ların toplamı yüklenir. A1 butonuna basılmışsa bu kez üst byte'ların toplamı gösterilmeye başlar.



Program çalıştırıldığında portta H'31' bilgisi görülecek, A1 butonuna bastığımızda ise H'45' bilgisi gösterilmeye başlayacaktır.

Çıkarma İşlemi

8 Bitlik Çıkarma

PIC Assembly dilinde çıkarma işlemi yapan iki komut vardır; **SUBWF** ve **SUBLW**

```
SUBWF f, d      ; File register içerisindeki sayıdan W'daki sayıyı çıkar.
                ; Sonucu hedef W ise W'ya, f ise aynı register içerisine yükle.
```

```
SUBWF SAYAC, F    ; SAYAC'taki sayıdan W'daki sayıyı çıkar. Sonucu SAYAC'a yükle.
```

```
SUBLW k           ; k sabit sayısından W'daki sayıyı çıkar. Sonucu W'ya yükle.
```

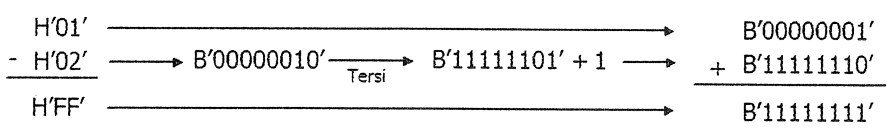
```
SUBLW B'11110000' ; B'11110000' sayısından W'daki sayıyı çıkar. Sonucu W'ya yükle.
```

8 bitlik çıkarma işleminde elde edilen sonuç pozitif veya sıfırsa Carry Flag =1, negatifse Carry Flag =0 olur.

H'FF'	H'FF'	H'11'
- H'FF'	- H'11'	- H'FF'
H'00'	H'EE'	H'12'
CF = 1	CF = 1	CF = 0
ZF = 1	ZF = 0	ZF = 0

NOT **Toplama işleminde Carry Flag elde oluşma durumunu gösterirken çıkarma işleminde ödünç alma durumunu göstermediğine dikkat ediniz.**

Mikrodenetleyici çıkarma işlemini doğrudan yapmaz. Çıkarılacak olan sayının tersini (komplementini) alıp 1 ekler. Daha sonra çıkarılan sayı ile toplayarak sonucu bulur. Örneğin;



Carry Flag'in sonuç negatif olduğunda 0, pozitif olduğunda ise 1 olması mikrodenetleyicinin çıkarma işlemini bu şekilde yapmasından kaynaklanmaktadır.

Yukarıdaki örneklere dikkat edilirse çıkarma işleminin sonucu negatif olduğunda sonuç istediğimiz gibi görülememektedir. Örneğin H'01'-H'02' işleminin sonucu -1olduğundan ledler üzerinde 1 bilgisini görmek isteriz. Ancak ledler üzerinde H'FF' bilgisi okunacaktır. Bu sakıncayı ortadan kaldırmak için programda işlem sonucunun tersini alıp 1 eklememiz gerekir. Sonuç pozitifse bu işlemin yapılmasına gerek yoktur.

Bir register içerisindeki sayının tersini (komplementini) almak için **COMF** komutu kullanılır.

```
COMF f, d       ; File register içerisindeki sayının tersini al. Sonucu hedef
                ; W ise W'ya, f ise aynı register içerisine yükle.
```

```
COMF SAYAC,F      ; SAYAC'taki sayıyı tersle. Sonucu yine SAYAC'a yükle.
```

Bir sayının komplementini almak demek o sayı içerisindeki 1'leri 0, 0'ları ise 1 yapmak demektir. Örneğin B '11001100' sayısının komplementi B '00110011' sayısıdır.

PROGRAM 35: H'F2' Sayısından H'2A' Sayısını Çıkarıp Sonucu PORTB'de Gösteren Program

```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF PORTB              ; PORTB'yi temizle
    MOVLW H'2A'             ; H'F2' sayısını W'ya yükle
    SUBLW H'F2'             ; H'F2'den H'2A' sayısını çıkar
    MOVWF PORTB             ; Sonucu PORTB'de göster
DONGU
    GOTO DONGU              ; Programı durdur
    END
```

Programda önce çıkarılacak olan sayı H'2A' **W**'ya yüklenir. Daha sonra **SUBLW** komutu ile H'F2' den çıkarılır. Sonuç **W** register'ına yükleneceğinden doğrudan **PORTB**'ye atılır.

PROGRAM 36: H'2A' Sayısından H'F2' Sayısını Çıkarıp Sonucu PORTB'de Gösteren Program

```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF PORTB              ; PORTB'yi temizle
    MOVLW H'F2'             ; H'2A' sayısını W'ya yükle
    SUBLW H'2A'             ; H'2A' dan H'F2' yi çıkar
    MOVWF PORTB             ; Sonucu PORTB'ye yükle
    COMF PORTB,F            ; Sonucu tersle
    INCF PORTB,F            ; 1 ekle ve sonucu PORTB'de göster
DONGU
    GOTO DONGU              ; Programı durdur
    END
```

Programda H'F2' sayısı **W** register'ına yüklenir, daha sonra H'2A'dan H'F2' çıkarılarak sonuç **PORTB**'ye atılır. Bu bilgi atıldıktan sonra program durdurulmadığı veya başka bir yere dallandırılmadığı için **PORTB**'de görülmeyecektir. Bulunan sayı önce terslenir, sonra da 1 eklenerek doğru sonuç elde edilir. Sonuç bilgisi **PORTB**'ye yüklenip program durdurulduğundan H'C8' bilgisi ledler üzerinde görüntülenecektir.

Bu işlem sonucunda negatif bir sayı çıktığından Carry Flag = 0 olacaktır.

PROGRAM 37: H'FF' Sayısından PORTA'ya Bağlı Olan Butonlardan Girilecek Sayıyı Çıkaran ve Sonucu PORTB'de Gösteren Program

```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
```

```
SAYAC EQU H'0C'          ; H'0C' adresinde SAYAC isimli bir register oluştur
   BSF STATUS,5          ; Bank-1'e geç
   CLRF TRISB            ; PORTB'yi çıkış olarak yönlendir
   BCF STATUS,5          ; Bank-0'a geri dön
   CLRF PORTB            ; PORTB'yi temizle
   MOVLW H'FF'
   MOVWF SAYAC           ; SAYAC'a H'FF' yükle
OKU
   MOVLW B'00011111'     ; B'00011111' bilgisini W'ya yükle
   XORWF PORTA,W         ; PORTA'dan girilen bilgiyi bu bilgi ile XOR'la
   SUBWF SAYAC,W         ; H'FF'den PORTA'dan girilen sayıyı çıkar
   MOVWF PORTB           ; Sonucu PORTB'de göster
   GOTO OKU              ; Yeni sayı girişi için OKU'ya dallan
   END
```

Programda H'FF' bilgisi **SAYAC** register'ı içerisine yüklenir. Daha sonra **PORTA**'dan girilen sayının değeri tespit edilir. H'FF'den **PORTA**'dan girilen sayı çıkarılır ve sonuç **PORTB**'de görüntülenir. **PORTA**'dan yeni sayı girildiğinde işlem yapılabilmesi için program **OKU** etiketine dallanır.

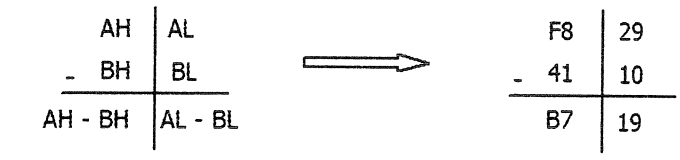
Programı çalıştırıp hiçbir butona basmadığımızda **PORTA**'nın değeri B'00011111' olacaktır. ÖZEL VEYA işlemi sonucunda;

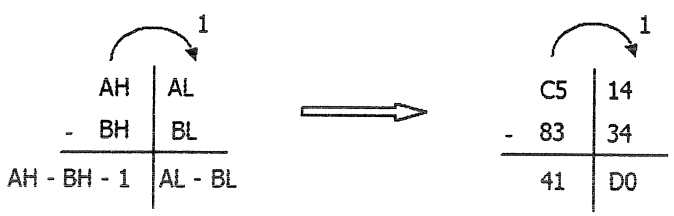
```
        B'00011111'
(XOR)   B'00011111'
-----------------
        B'00000000'
```

0 bilgisi **W**'ya yeni değer olarak yüklenir. **SAYAC**'taki FF sayısından bu sayı çıkarıldığında H'FF' – H'00' = H'FF' bilgisi elde edileceğinden **PORTB**'ye bağlı tüm ledlerin yandığı görülecektir.

16 Bitlik Çıkarma

16 bitlik çıkarma işlemi de toplama işleminde olduğu gibi sayıları 8 bitlik iki parçaya bölerek gerçekleştirilir. Bu işlemi anlayabilmek için çıkarma işlemi yaparken izlediğimiz yolu inceleyelim.





Görüldüğü gibi eğer alt byteların farkı pozitifse üst byteların farkı alınarak sonuç yazılır, negatifse **AH**'den 1 çıkarılır, daha sonra fark alınarak sonuç yazılır. Şimdi programı buna göre yazalım.

PROGRAM 38: H'F569' Sayısından H'23E1' Sayısını Çıkararak Sonucun Alt Byte'ını PORTB'de Gösteren, A1 Butonuna Basıldığında İse Üst Byte'ını Gösteren Program

```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
    CBLOCK H'0C'            ; H'0C' adresinde AL, H'0D' adresinde AH,
    AL, BL, AH, BH          ; H'0E' adresinde BL, H'0F' adresinde BH
    ENDC                    ; isimli bir register oluştur
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF PORTB              ; PORTB'yi temizle
BASLA
    MOVLW H'69'             ; A sayısının alt byte'ı olan H'69' sayısını
    MOVWF AL                ; AL register'ına yükle
    MOVLW H'F5'             ; A sayısının üst byte'ı olan H'F5' sayısını
    MOVWF AH                ; AH register'ına yükle
    MOVLW H'E1'             ; B sayısının alt byte'ı olan H'E1' sayısını
    MOVWF BL                ; BL register'ına yükle
    MOVLW H'23'             ; B sayısının üst byte'ı olan H'23' sayısını
    MOVWF BH                ; BH register'ına yükle
ÇIKAR
    MOVF BL,W               ; BL'yi W'ya yükle
    SUBWF AL,F              ; AL'den BL'yi çıkar, sonucu AL'ye yükle
    BTFSS STATUS,0          ; Carry Flag = 1 mi? (Sonuç pozitif mi?)
    DECF AH,F               ; Hayır, AH'yi 1 eksilt
    MOVF BH,W               ; Evet, AH'yi eksiltmeden BH'yi W'ya yükle
    SUBWF AH,F              ; AH'den BH'yi çıkar, sonucu AH'ye yükle
ALT_BYTE
    MOVF AL,W               ; AL'yi (yani toplamın alt byte'ını)
    MOVWF PORTB             ; PORTB'de göster
A1
    BTFSC PORTA,1           ; A1'e basıldı mı?
    GOTO A1                 ; Hayır, yeniden test et
UST_BYTE
    MOVF AH,W               ; Evet,  AH'yi (yani toplamın üst byte'ını)
    MOVWF PORTB             ; PORTB'de göster
DONGU
    GOTO DONGU              ; Programı durdur
    END
```

Programda 16 bitlik toplama işleminde olduğu gibi **A** ve **B** sayılarının alt ve üst byte'ları birer register içerisine yüklenir. Daha sonra **AL** sayısından **BL** sayısı çıkarılır. Eğer **AL**, **BL**'den küçükse **AH**'den borç alınması gerekir. Bu yüzden **AH** 1 eksiltilir. **AH – BH** işlemi buna göre yapılır. Çıkış portunda alt byte'ların farkı gösterilir. Bu arada A1 butonuna basılırsa üst byte'ların farkı gösterilmeye başlayacaktır.

Çarpma İşlemi

PIC16F84'te çarpma ve bölme işlemi yapan komut yoktur. Çarpma işlemi toplamanın, bölme işlemi ise çıkarmanın belirli sayıda tekrarlanması ile yapılır. Aşağıda çarpma işleminin yapılışına örnek bir program verilmiştir.

PROGRAM 39: 7 x 14 İşlemini Yaparak Sonucu PORTB'ye Bağlı Ledlerde Gösteren Program

```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
SONUC EQU H'0C'             ; H'0C' adresinde SONUC,
CARPAN1 EQU H'0D'           ; H'0D' adresinde CARPAN1 isimli bir register oluştur
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF PORTB              ; PORTB'yi temizle
    CLRF SONUC              ; SONUC'a sıfır yükle
    MOVLW D'7'
    MOVWF CARPAN1           ; CARPAN1'e D'7' bilgisini yükle
TOPLA
    MOVLW D'14'             ; Çarpanlardan diğerine 14 yükle
    ADDWF SONUC,F           ; SONUC'un içindeki sayı ile 14'ü topla
    DECFSZ CARPAN1,F        ; CARPAN1'i 1 eksilt, 0 oldu mu?
    GOTO TOPLA              ; Hayır, TOPLA'ya dallan
    MOVF SONUC,W            ; Evet, elde edilen sonucu
    MOVWF PORTB             ; PORTB'ye bağlı ledlerde göster
DONGU
    GOTO DONGU              ; Programı durdur
    END
```

7x14 işlemi aslında 7 tane 14 sayısının toplamasıdır. Programımızı da bu mantığa göre yazıyoruz. Önce çarpanlardan herhangi biri **CARPAN1** register'ına, diğer çarpan da **W**'ya yüklenir. Programda **CARPAN1**'e 7, **W**'ya ise 14 sayısı yüklenmiştir. Toplama işlemi sonucu **SONUC**'ta 0 bilgisi olduğundan 0+14=14 bilgisi **SONUC**'a yeni değer olarak yüklenir. Geriye toplanması gereken 6 adet 14 bilgisi kaldığından **CARPAN1**'in içeriğinin 1 azaltılması gerekir. **CARPAN1**'in içeriği sıfırlanana kadar toplama işlemi devam eder. Sıfır olduğunda çarpma işlemi tamamlanmıştır ve sonuc **PORTB**'ye yüklenerek ledler üzerinde D'98' yani H'62' bilgisi gösterilir.

```
 98 | 16
-96 | 6
  2
```

Bildiğiniz gibi decimal sayıların hexadecimal sayılara çevrilişi yukarıdaki örnekte olduğu gibidir. Decimal sayı sürekli 16'ya bölünür ve elde edilen kalanlar işaretlenir. Daha sonra bu kalan sayılar sondan başa doğru yazılarak hexadecimal değer elde edilir.

Bölüm 11

Kesmeler

Kesme (Interrupt) Nedir?

Kesme, mikrodenetleyicinin işlemekte olduğu program kodlarını bırakıp başka bir işievi yerine getirmesine neden olan, yazılım veya donanımsal olaydır. Kesme işlemini anlayabilmek için günlük hayatımızdan bir örnek verelim. Örneğin, DVD'de bir film seyrediyorsunuz ve telefon çaldı. Telefonun cevaplanması filmden daha öncelikli bir olay olduğu için filmi durdurur, telefon görüşmenizi yapar, sonra filme kaldığı yerden devam edersiniz. İşte kesme de böyle bir şeydir. Ana program çalışırken, kesme kaynaklarından herhangi biri bir kesmeye sebep olursa program normal çalışmasını durdurur, kesme alt programına dallanarak buradaki komutları işler, kesme alt programı bitince ana programda kaldığı yere geri döner.

Kesmenin diğer alt program çağırma işleminden ne farkı var diye düşünebilirsiniz. Programda alt programın ne zaman işleneceğine siz karar verirsiniz, istediğiniz noktada **CALL** komutunu yazar ve alt programı çalıştırırsınız. Halbuki kesme alt programının ne zaman işleneceğine donanımsal veya yazılımda sizin müdahale edemeyeceğiniz bir değişiklik karar verir. Örneğin RB0 harici kesmesi kullanılıyorsa, programınız normal işleyişini sürdürürken RB0 ucundan alınan kesme sinyali kesme alt programının işlenmesine sebep olacaktır. Aksi halde sizin kesme alt programına erişmeniz mümkün değildir.

Peki kesme alt programı nereye yazılır? Bir kesme meydana geldiğinde program önce kaldığı yerin adresini **STACK** register'ına kaydeder. Daha sonra otomatik olarak H'04' adresine atlar. H'04' adresi kesme vektörünün sabit adresidir. Bu adreste bulunan **GOTO** komutu ile program kesme alt programına dallanır ve buradaki komutları çalıştırmaya başlar. Kesme alt programları **RETFIE** komutu ile sonlandırılır. Program **RETFIE** komutunu görünce **STACK** register'ından kaldığı yerin adresini alır ve ana programda kaldığı yere geri döner. Bundan sonra program normal işleyişini sürdürür. Programda istenilen sayıda kesme sinyali alınabilir. Bir defaya mahsus bir olay değildir.

Kesme Kullanılan Programların Yapısı

Kesme kullanılan uygulamalarda program formatı aşağıdaki şekilde olur.

```
    ORG H'00'
    GOTO BASLA
    ORG H'04'
    GOTO KESME_ALT_PROG
BASLA
    .....
    .....
    .....
KESME_ALT_ROG
    .....
    .....
    RETFIE
    END
```

Buradaki **ORG** komutu ORIGIN kelimesinin kısaltılmışı olup adres bildirir. Program çalışmaya başladığında H'00' adresinde olacağından **BASLA** etiketine dallanarak normal çalışmasını yerine getirir. Bir kesme oluştuğunda program sayıcı ORG H'04' adresine konumlanacağından bu-

rada bulunan **GOTO** komutunu işleyerek **KESME_ALT_PROG** programına dallanır ve kesme alt programına ait komutları işler. **RETFIE** komutu ile alt programdan çıkılıp ana programa geri dönülür.

Kesme Kaynakları ve INTCON Register'ı

PIC16F84 dört farklı kaynaktan kesme alabilir:

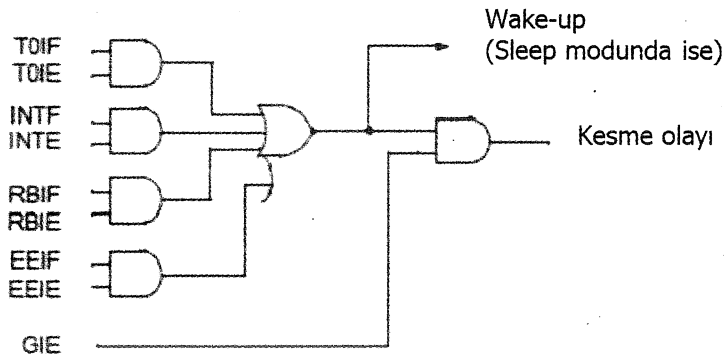
- RB0/INT pini harici kesmesi
- PORTB değişim kesmesi (RB4~RB7)
- TMR0 taşma kesmesi
- EEPROM'a yazma işlemi tamamlandığında oluşan kesme

Kesme işlemlerini kontrol etmek için **INTCON** register'ı kullanılır. Bu register'da kesmeleri etkinleştiren bitler ile kesme meydana gelip gelmediğini gösteren flagler bulunur. Ayrıca bütün kesme işlemlerini iptal eden veya geçerli hale getiren GIE biti mevcuttur. GIE aktif değilse hiçbir kesme sinyalini alamayız.

Bir kesme meydana geldiğinde GIE biti otomatik olarak sıfırlanır. Kesme alt programı işlenip ana programa geri dönülürken ise yeniden otomatik olarak 1 yapılır. Bu şekilde kesme alt programı işlenirken yeni oluşacak bir kesmenin program akışını bozması engellenmiş olur.

INTE, RBIE, EEIE ve T0IE bitleri kesmeleri etkinleştiren bitlerdir. Hangi kesmeyi kullanacaksak programımızda onu etkinleştiren biti 1 yaparak o kesmeye izin vermemiz gerekir.

Bir kesme meydana geldiği zaman ona ait olan flag 1 olur ve böylece program kesme oluştuğunu anlayarak kesme alt programına dallanır. Kesme alt programına girildiğinde GIE otomatik olarak sıfırlanır demiştik. Kesmelere ait flagler ise otomatik olarak sıfırlanmaz. Programcı tarafından kesme alt programı içerisinde sıfırlanmaları gerekir. Aksi halde kesme alt programından ana programa dönüldüğünde işlemci tekrar flagleri kontrol eder, flagin 1 olduğunu görüp kesme oluşmadığı halde yeniden kesme alt programına dallanır. Bu da programın sürekli olarak kesme alt programına dallanarak ana programdaki komutları işleyememesi anlamına gelir.



ŞEKİL 38: Kesme olayının mantığı.

Aynı program içerisinde birden fazla kaynaktan kesme sinyali almak istiyorsak kesme alt programına dallandığımızda flagleri kontrol ettirebiliriz. Örneğin **INT** kesmesi ile **PORTB** değişim kesmesini aynı program içinde kullanacağımızı varsayalım. **INT** kesmesi geldiğinde bir işlem, **PORTB** değişim kesmesi geldiğinde ise başka bir işlem yaptırmak istediğimizi düşünelim.

Programımız hangi kesme gelirse gelsin H'04' adresine dallanacak, buradan kesme alt programının adresine dallanarak buradaki komutları işleyecektir. Farklı bir adrese alt program yazma şansımız olmadığına göre, kesme alt programı içerisine girildiğinde **BTFSS** komutu ile kesmelere ait flagleri kontrol ettiririz. Böylece gelen kesmenin türüne göre istediğimiz komutları çalıştırabiliriz.

```
KESME
   BTFSS INTCON,INTF
   GOTO HIGH_INT
   GOTO RB0_INT
HIGH_INT
   BCF INTCON,RBIF
   ...
   ...
   RETFIE
RB0_INT
   BCF INTCON,INTF
   ...
   ...
   RETFIE
```

Tablo 12'de **INTCON** register'ı ve içerisindeki flaglerin görevleri verilmiştir.

Harici bir kesme meydana geldiğinde üç-dört komut saykılı kadar bir gecikme olur. Çok hassas uygulamalarda bu gecikme dikkate alınmalıdır.

Kesme Türleri

RB0/INT Pini Harici Kesmesi (INT Kesmesi)

RB0/INT pini harici kesmesi kenar tetiklemelidir; yani bu uçtaki sinyalin 1'den 0'a veya 0'dan 1'e geçişi kesmeye sebep olur. Kesmenin yükselen kenarda mı düşen kenarda mı gerçekleşeceğini programcı belirler. Bunun için **OPTION_REG** register'ındaki **INTEDG** biti kullanılır.

```
INTEDG = 1  Yükselen kenar tetiklemeli
INTEDG = 0  Düşen kenar tetiklemeli
```

INT kesmesinin kullanılabilmesi için **INTCON** register'ındaki **INTE** biti 1 yapılarak kesmeye izin verilmelidir. **INTE** = 0 yapılırsa bu kesme engellenmiş olur. INT kesmesi meydana geldiği anda yine **INTCON** register'ındaki **INTF** biti 1 olur. Programda kesme alt programı içerisinde **INTF** = 0 yapılmalıdır.

Şimdi RB0/INT kesmesini kullanmak için gerekli aşamaları özetleyelim.

- RB0/INT ucunu giriş olarak yönlendir.
- **OPTION_REG** register'ındaki **INTEDG** biti ile düşen veya yükselen kenar tetiklemesini seç.
- **INTE** = 1 yaparak INT kesmesine izin ver.
- **GIE** = 1 yaparak tüm kesmeleri geçerli yap.
- Kesme meydana gelip program kesme alt programına dallandığında **INTF** = 0 yap.

TABLO 12: INTCON Register

Bit 7							Bit 0
GIE	EEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF

Bit 7 : GIE : Tüm kesmeleri etkinleştirme biti

1 : Tüm kesmelere izin verilir
0 : Hiçbir kesmeye izin verilmez

Bit 6 : EEIE : EEPROM kesmesi etkinleştirme biti

1 : EEPROM kesmesine izin verilir
0 : EEPROM kesmesine izin verilmez

Bit 5 : TOIE : TMR0 kesmesi etkinleştirme biti

1 : TMR0 kesmesine izin verilir
0 : TMR0 kesmesine izin verilmez

Bit 4 : INTE : RB0/INT kesmesi etkinleştirme biti

1 : RB0/INT kesmesine izin verilir
0 : RB0/INT kesmesine izin verilmez

Bit 3 : RBIE : PORTB değişim kesmesi etkinleştirme biti

1 : PORTB değişim kesmesine izin verilir
0 : PORTB değişim kesmesine izin verilmez

Bit 2 : TOIF : TMR0 kesme bayrağı

1 : TMR0 kesmesi meydana geldi
0 : TMR0 kesmesi meydana gelmedi

Bit 1 : INTF : RB0/INT kesme bayrağı

1 : RB0/INT kesmesi meydana geldi
0 : RB0/INT kesmesi meydana gelmedi

Bit 0 : RBIF : PORTB değişim kesmesi bayrağı

1 : PORTB değişim kesmesi meydana geldi
0 : PORTB değişim kesmesi meydana gelmedi

PROGRAM 40: RB0/INT Ucuna Bağlı Butona Basıldığında PORTB'nin Diğer Uçlarına Bağlı Yanık Ledleri Bir Süre Söndüren Program

NOT **Aslında bu programda kesme kullanmaya gerek yoktur. Sadece RB0/INT kesmesinin kullanımının anlaşılabilmesi amacıyla verilmiş örnek bir programdır.**

```
LIST P=16F84            ; Kullanılacak PIC'i tanıt
INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
CBLOCK H'0C'            ; H'0C' adresinde SAYAC,
SAYAC,SAYAC1            ; H'0D' adresinde SAYAC1,
ENDC                    ; isimli bir register oluştur
ORG H'00'               ; Ana program H'00' adresindeki
```

```
    GOTO BASLA              ; BASLA etiketinden başlıyor
    ORG H'04'               ; Kesme alt programı H'04' adresindeki
    GOTO KESME              ; KESME etiketinden başlıyor
BASLA
    BSF STATUS,5            ; Bank-1'e geç
    MOVLW B'00000001'       ; PORTB'nin 0. bitini giriş, diğer
    MOVWF TRISB             ; bitlerini çıkış olarak yönlendir
    BCF OPTION_REG,6        ; RB0/INT ucunu düşen kenar tetiklemeli seç
    BCF STATUS,5            ; Bank-0'a geri dön
    BSF INTCON, INTE        ; INT kesmesine izin ver
    BSF INTCON, GIE         ; Tüm kesmelere izin ver
    CLRF PORTB              ; PORTB'yi temizle
YUKLE
    MOVLW H'FF'             ; PORTB'ye bağlı
    MOVWF PORTB             ; tüm ledleri yak
    GOTO YUKLE              ; Ledleri sürekli yanık tutmak için YUKLE'ye dallan
KESME
    BCF INTCON,INTF         ; INT kesmesine ait flag'i sıfırla
    CLRF PORTB              ; PORTB'ye bağlı tüm ledleri söndür
    CALL GECIKME            ; Ledlerin söndüğünün görülebilmesi için bir süre
                            ; bekle
    RETFIE                  ; Ana programda kaldığın yere geri dön
GECIKME
    MOVLW H'FF'
    MOVWF SAYAC
DONGU1
    MOVLW H'FF'
    MOVWF SAYAC1
DONGU2
    DECFSZ SAYAC1,F
    GOTO DONGU2
    DECFSZ SAYAC,F
    GOTO DONGU1
    RETURN
    END
```

Ana programda **PORTB**'ye bağlı olan tüm ledler yakılmıştır. Bir kesme sinyali gelmediği sürece ledler sürekli olarak yanacaktır. RB0 ucuna bağlı butona basıldığında kesme meydana gelir. Program **KESME** alt programına dallanır. Alt programda **INTF** bayrağı sıfırlandıktan sonra ledleri söndürmek için port temizlenir. Ledlerin söndüğünün görülebilmesi için gecikme alt programı çağrılarak bir süre beklenir. Daha sonra ana programa geri dönülür. Dolayısıyla ledler tekrar yanar.

Kesme alt programı içerisinden **GECIKME** alt programının çağrıldığına dikkat edin. Demek ki bir alt program içerisinden başka bir alt program çağrılması mümkündür. Peki bu şekilde iç içe kaç alt program çağrılabilir? Bildiğiniz gibi, alt programa dallanıldığında ana programda kalınan yerin adresi **STACK** register içerisinde tutulur. PIC16F84 sekiz seviye derinliğinde **STACK** register'a sahiptir. O halde iç içe 8 kez alt program çağırabiliriz. 9. alt programı çağırmak istediğimizde ne olur? **STACK** register FIFO (First In First Out) mantığı ile çalışır. Yani 9. programı çağırmak istediğimizde 1. programın adresi atılarak 9. için yer açılır. Bu durumda 1. adres kaybolduğundan programın çalışması bozulur. Program yazarken bu durum göz önünde bulundurulmalıdır.

PORTB Değişim Kesmesi

PORTB'nin 4. ~ 7. (RB4 ~ RB7) bitlerinde bir değişim meydana gelmesi **PORTB** değişim kesmesine neden olur. Bu kesmeyi aktif hale getirmek için **INTCON** register'ının **RBIE** bitinin 1 yapılması gerekir. **PORTB** değişim kesmesi oluştuğunda **RBIF** = 1 olur ve program kesme alt programına dallanır. Program **RBIF** bayrağını otomatik sıfırlayamadığı için alt program içerisinde **RBIF** = 0 yapılmalıdır.

PORTB değişim kesmesi kullanmak için gerekli aşamalar aşağıdaki şekilde özetlenebilir:

- **PORTB**'nin 4., 5., 6. ve 7. bitlerini giriş olarak yönlendir.
- **RBIE** = 1 yaparak **PORTB** değişim kesmesine izin ver.
- **GIE** = 1 yaparak tüm kesmeleri geçerli yap.
- Kesme meydana gelip program kesme alt programına dallandığında **RBIF** = 0 yap.

PROGRAM 41: RB4 ~ RB7 Uçlarına Bağlı Butonlardan Bir veya Bir Kaçına Basıldığında PORTA'nın İlk İki Bitine Bağlı Ledleri Bir Süre Boyunca Yakan Program

```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
    CBLOCK H'0C'            ; H'0C' adresinde SAYAC,
    SAYAC,SAYAC1            ; H'0D' adresinde SAYAC1,
    ENDC                    ; isimli bir register oluştur
    ORG H'00'               ; Ana program H'00' adresindeki
    GOTO BASLA              ; BASLA etiketinden başlıyor
    ORG H'04'               ; Kesme alt programı H'04' adresindeki
    GOTO KESME              ; KESME etiketinden başlıyor
BASLA
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISA              ; PORTA'yı çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    BSF INTCON, RBIE        ; PORTB değişim kesmesine izin ver
    BSF INTCON, GIE         ; Tüm kesmelere izin ver
YUKLE
    CLRF PORTA              ; PORTA'ya bağlı ledleri söndür
    GOTO YUKLE              ; Ledleri sürekli sönük tutmak için YUKLE'ye dallan
KESME
    BCF INTCON,RBIF         ; PORTB değişim kesmesine ait flag'i sıfırla
    MOVLW H'03'             ; PORTA'nın ilk iki bitine bağlı
    MOVWF PORTA             ; ledleri yak
    CALL GECIKME            ; Ledlerin yandığının görülebilmesi için bir süre bekle
    RETFIE                  ; Ana programda kaldığın yere geri dön
GECIKME
    MOVLW H'FF'
    MOVWF SAYAC
DONGU1
    MOVLW H'FF'
    MOVWF SAYAC1
DONGU2
    DECFSZ SAYAC1,F
    GOTO DONGU2
    DECFSZ SAYAC,F
```

```
    GOTO DONGU1
    RETURN
    END
```

EEPROM Kesmesi

EEPROM'a yazma işlemi tamamlandığı zaman EEPROM kesmesi meydana gelir ve **INTCON** register'ındaki **EEIF** bayrağı 1 olur. Bu kesmeyi kullanabilmemiz için INTCON register'ındaki **EEIE** bitinin 1 yapılarak kesmeye izin verilmesi gerekir. Program kesme alt programına dallandığında **EEIF** = 0 yapılarak yeni kesmelere hazırlanılması sağlanır.

EEPROM kesmesini kullanmak için gerekli aşamalar aşağıdaki şekilde özetlenebilir:

- **EEIE** = 1 yaparak EEPROM kesmesine izin ver.
- **GIE** = 1 yaparak tüm kesmeleri geçerli yap.
- Kesme meydana gelip program kesme alt programına dallandığında **EEIF** = 0 yap.

TMR0 Kesmesi

TMR0 sayıcısının FF'den 00'a geçmesi **TMR0** kesmesine sebep olur ve **INTCON** register'ındaki **TOIF** bayrağı 1 olur. Bu kesmeyi kullanabilmemiz için **TOIE** = 1 yapılarak kesmeye izin verilmesi gerekir. Program kesme alt programına dallandığında **TOIF** = 0 yapılarak programın yeni kesmelere hazırlanması sağlanır.

TMR0 kesmesini kullanmak için gerekli aşamalar aşağıdaki şekilde özetlenebilir:

- **TOIE** = 1 yaparak **TMR0** kesmesine izin ver.
- **GIE** = 1 yaparak tüm kesmeleri geçerli yap.
- Kesme meydana gelip program kesme alt programına dallandığında **TOIF** = 0 yap.

Kesme Sırasındaki Bilgileri Saklamak

Bir kesme meydana geldiğinde **STACK** register'ına sadece geri dönüş adresi yani PC (program sayıcı)'nın değeri kaydedilir. Ancak programcı kesme sırasında **W**, **STATUS** gibi bazı register'ların içerisinde bulunan bilgileri de saklamak isteyebilir. Kaydetme işlemi program içerisinde yapılır. Aşağıda **W** ve **STATUS** register'larının içeriğinin saklanması ile ilgili örnek program verilmiştir. Yapılan işlemin basamakları şu şekildedir:

a. **W**'nun içeriğini **W_TEMP** register'ına kaydet.
b. **STATUS**'un içeriğini **STATUS_TEMP** register'ına kaydet.
c. Kesme alt programını işle.
d. **STATUS_TEMP**'in içeriğini **STATUS**'a kaydet.
e. **W_TEMP**'in içeriğini **W**'ya kaydet.

```
PUSH
    MOVWF W_TEMP
    SWAPF STATUS,W
    MOVWF STATUS_TEMP
KESME_ALT_PROG
    ...
```

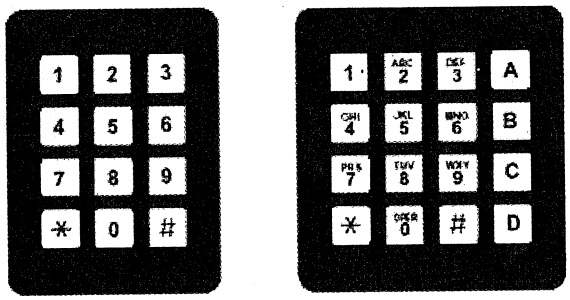
```
    ...
    ...
POP
    SWAPF STATUS_TEMP,W
    MOVWF STATUS
    SWAPF W_TEMP,F
    SWAPF W_TEMP,W
```

Programlarda **SWAPF** komutunun kullanılmasının sebebi bu komut işlendiğinde **STATUS** register'ının içeriğini değiştirmiyor olmasıdır.

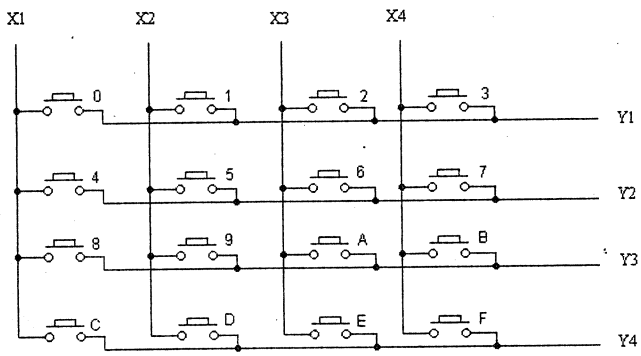
Bölüm 12

Tuş Takımı Uygulamaları

Tuş takımı (klavye) elektronik uygulamaların pek çoğunda sıklıkla kullanılan bir elemandır. Piyasada 4x4 (4 satır – 4 sütun) ve 4x3 (4 satır – 3 sütun)'lük tuş takımları bulunmaktadır. Ancak kendiniz de buton kullanarak istediğiniz sayıda tuşa sahip olan bir klavye yapabilirsiniz.



ŞEKİL 39: 4x3 ve 4x4 tuş takımları.



ŞEKİL 40: 4x4 tuş takımınındaki satır ve sütunlar.

Yukarıdaki şekil 4x4 tuş takımının iç yapısını temsil etmektedir. Buna göre her bir tuşu bir buton kabul edebiliriz. Butonların birer ucu birleştirilerek satırlar, diğer uçları birleştirilerek ise sütunlar oluşturulmuştur. X1, X2, X3 ve X4 sütunlar, Y1, Y2, Y3 ve Y4 ise satırlar olarak adlandırılır. Satır ve sütunların PIC'e bağlantı şeması Şekil 41'de gösterilmiştir. Buradaki tuş takımı 4x3'lük olduğundan sütun sayısı üçtür.

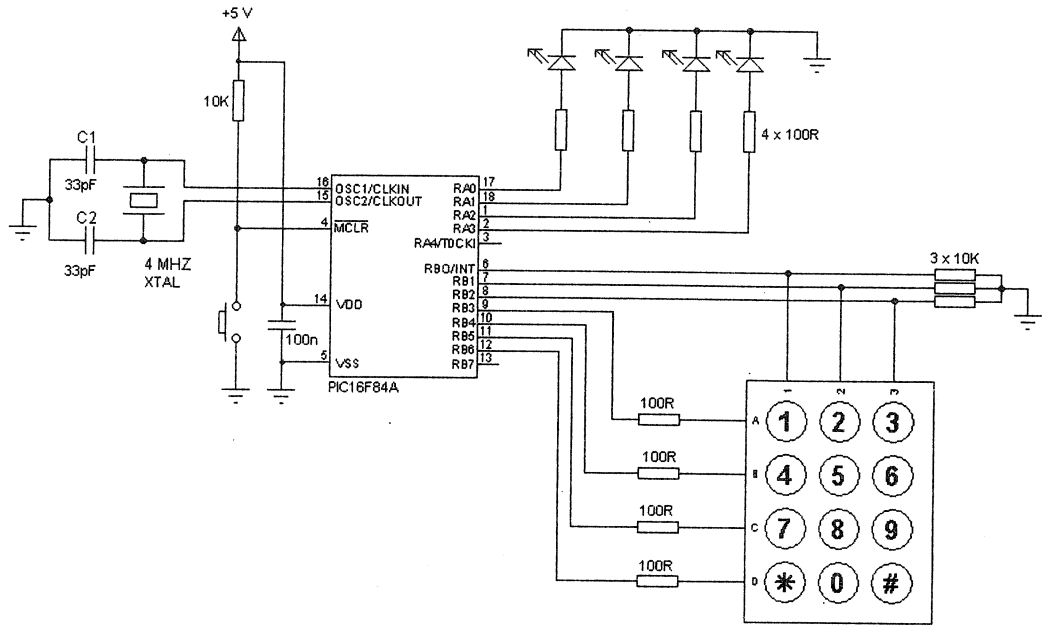
Bir tuş takımının satır ve sütunlarını tespit etmek için AVOmetre direnç kademesine alınır. Herhangi bir tuşa bastığımız zaman çok düşük direnç gösteren uçlar o tuşa ait satır ve sütunlardır. Örneğin 2 tuşuna bastığımız zaman X3-Y1 arasında çok düşük direnç değeri okumamız gerekir.

PIC16F84 ile tuş takımı kullanmak için iki ayrı metottan faydalanılabilir: Tarama metodu ve klavye entegresi kullanma

Tarama Metodu

Tarama metodunda mikrodenetleyici klavyedeki tüm tuşları sürekli olarak tarayarak bir tuşa basılıp basılmadığını kontrol eder. Herhangi bir tuşa basılmışsa ona ait işlev yerine getirilir. Basılmamışsa tuşlar sürekli olarak taranmaya devam eder.

Tarama metodu ile tuş takımı kullanmak için Şekil 41'deki devreyi kullanacağız.



ŞEKİL 41: Tuş takımı uygulama devresi

Yapılacak işlem oldukça basittir: Önce 1. satıra, yani RB3 ucuna lojik-1 bilgisi gönderilir (RB4, RB5 ve RB6 lojik-0 olarak kalır) Sonra bu satırın hizasındaki 1, 2 ve 3 numaralı tuşlar; yani RB0, RB1 ve RB2'nin 1 olup olmadığı kontrol edilir. Örneğin, 2 tuşuna basılmış olsun. Bu durumda RB3 ucundan gönderilen +5V'luk sinyal 2 tuşu üzerinden devresini tamamlayarak R2 direnci üzerinde +5V gerilim oluşturacak, böylece RB1 ucunda lojik-1 bilgisi oluşacaktır. RB1 ucunda lojik-1 olması 2 tuşuna basıldığı anlamına gelir. Aynı şekilde eğer RB0=1 olursa 1 tuşuna, RB2=1 olursa 3 tuşuna basıldığını anlarız.

1, 2 ve 3 tuşlarından hiçbirine basılmamışsa bu kez 2. satıra yani RB4 ucuna lojik-1 gönderilir. Yine RB0, RB1 ve RB2 uçları taranarak bu satırda bir tuşa basılıp basılmadığı test edilir. Aynı işlem 3. ve 4. satırlar için de tekrarlanır. Dört satırın taraması bittikten sonra yeniden 1. satıra dönülür. Herhangi bir tuşa basılana kadar bu işlem sürekli olarak tekrar eder.

Aşağıda basılan tuşun değerinin tespit edilmesine dair birkaç örnek verilmiştir.

RB6	RB5	RB4	RB3	RB2	RB1	RB0	Basılan Tuş
1	0	0	0	0	1	0	0
0	0	0	1	1	0	0	3
0	1	0	0	0	0	1	7
1	0	0	0	0	0	1	*

PROGRAM 42: 4x3 Tuş Takımından Girilen Sayıyı PORTA'ya Bağlı Ledler Üzerinde Gösteren Program

```
LIST P=16F84            ; Kullanılacak PIC'i tanıt
INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
BSF STATUS,5            ; Bank-1'e geç
CLRF TRISA              ; PORTA'yı çıkış olarak yönlendir
```

```
   MOVLW B'00000111'      ; PORTB'nin ilk 3 bitini giriş, diğer bitlerini çıkış
   MOVWF TRISB            ; olarak yönlendir
   BCF STATUS,5           ; Bank-0'a geri dön
   CLRF PORTA             ; PORTA'yı temizle
   CLRF PORTB             ; PORTB'yi temizle
BASLA
BIR
   MOVLW B'00001000'      ; Birinci satıra lojik-1 gönder
   MOVWF PORTB
   BTFSS PORTB,0          ; 1. sütun 1 mi? ( 1'e basıldı mı?)
   GOTO IKI               ; Hayır, 2'ye basıldı mı kontrol etmek için IKI'ye
                          ; dallan
   MOVLW H'01'
   MOVWF PORTA            ; Evet, PORTA'da 1 bilgisini göster
   GOTO BASLA             ; Tuşları yeniden taramak için BASLA'ya dallan
IKI
   BTFSS PORTB,1          ; 2. sütun 1 mi? ( 2'ye basıldı mı?)
   GOTO UC                ; Hayır, 3'e basıldı mı kontrol etmek için UC'e dallan
   MOVLW H'02'
   MOVWF PORTA            ; Evet, PORTA'da 2 bilgisini göster
   GOTO BASLA             ; Tuşları yeniden taramak için BASLA'ya dallan
UC
   BTFSS PORTB,2          ; 3. sütun 1 mi? ( 3'e basıldı mı?)
   GOTO DORT              ; Hayır, 4'e basıldı mı kontrol etmek için DORT'e
                          ; dallan
   MOVLW H'03'
   MOVWF PORTA            ; Evet, PORTA'da 3 bilgisini göster
   GOTO BASLA             ; Tuşları yeniden taramak için BASLA'ya dallan
DORT
   MOVLW B'00010000'      ; İkinci satıra lojik-1 gönder
   MOVWF PORTB
   BTFSS PORTB,0          ; 1. sütun 1 mi? ( 4'e basıldı mı?)
   GOTO BES               ; Hayır, 5'e basıldı mı kontrol etmek için BES'e dallan
   MOVLW H'04'
   MOVWF PORTA            ; Evet, PORTA'da 4 bilgisini göster
   GOTO BASLA             ; Tuşları yeniden taramak için BASLA'ya dallan
BES
   BTFSS PORTB,1          ; 2. sütun 1 mi? ( 5'e basıldı mı?)
   GOTO ALTI              ; Hayır, 6'ya basıldı mı kontrol etmek için ALTI'ya
                          ; dallan
   MOVLW H'05'
   MOVWF PORTA            ; Evet, PORTA'da 5 bilgisini göster
   GOTO BASLA             ; Tuşları yeniden taramak için BASLA'ya dallan
ALTI
   BTFSS PORTB,2          ; 3. sütun 1 mi? ( 6'ya basıldı mı?)
   GOTO YEDI              ; Hayır, 7'ye basıldı mı kontrol etmek için YEDI'ye
                          ; dallan
   MOVLW H'06'
   MOVWF PORTA            ; Evet, PORTA'da 6 bilgisini göster
   GOTO BASLA             ; Tuşları yeniden taramak için BASLA'ya dallan
YEDI
   MOVLW B'00100000'      ; Üçüncü satıra lojik-1 gönder
   MOVWF PORTB
```

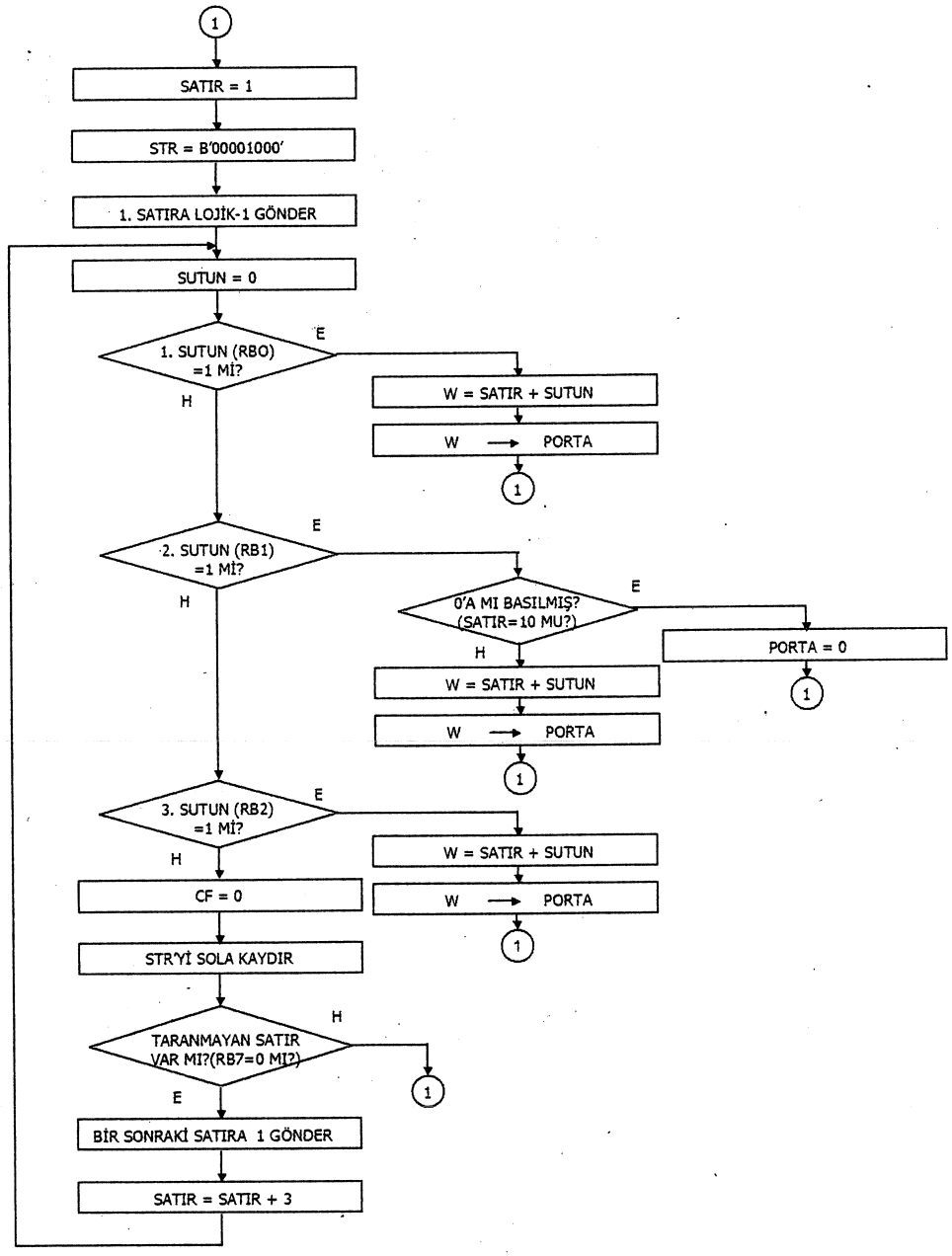
```
    BTFSS PORTB,0          ; 1. sütun 1 mi? ( 7'ye basıldı mı?)
    GOTO SEKIZ             ; Hayır, 8'e basıldı mı kontrol etmek için SEKIZ'e
                           ; dallan
    MOVLW H'07'
    MOVWF PORTA            ; Evet, PORTA'da 7 bilgisini göster
    GOTO BASLA             ; Tuşları yeniden taramak için BASLA'ya dallan
SEKIZ
    BTFSS PORTB,1          ; 2. sütun 1 mi? (8'e basıldı mı?)
    GOTO DOKUZ             ; Hayır, 9'a basıldı mı kontrol etmek için DOKUZ'a
                           ; dallan
    MOVLW H'08'
    MOVWF PORTA            ; Evet, PORTA'da 8 bilgisini göster
    GOTO BASLA             ; Tuşları yeniden taramak için BASLA'ya dallan
DOKUZ
    BTFSS PORTB,2          ; 3. sütun 1 mi? ( 9'a basıldı mı?)
    GOTO YILDIZ            ; Hayır, *'a basıldı mı kontrol etmek için YILDIZ'a
                           ; dallan
    MOVLW H'09'
    MOVWF PORTA            ; Evet, PORTA'da 9 bilgisini göster
    GOTO BASLA             ; Tuşları yeniden taramak için BASLA'ya dallan
YILDIZ
    MOVLW B'01000000'      ; Dördüncü satıra lojik-1 gönder
    MOVWF PORTB
    BTFSS PORTB,0          ; 1. sütun 1 mi? ( YILDIZ'a basıldı mı?)
    GOTO SIFIR             ; Hayır, 0'a basıldı mı kontrol etmek için SIFIR'a
                           ; dallan
    MOVLW H'0A'
    MOVWF PORTA            ; Evet, PORTA'da 10 bilgisini göster
    GOTO BASLA             ; Tuşları yeniden taramak için BASLA'ya dallan
SIFIR
    BTFSS PORTB,1          ; 2. sütun 1 mi? (0'a basıldı mı?)
    GOTO DIYEZ             ; Hayır, #'e basıldı mı kontrol etmek için DIYEZ'e
                           ; dallan
    MOVLW H'00'
    MOVWF PORTA            ; Evet, PORTA'da 0 bilgisini göster
    GOTO BASLA             ; Tuşları yeniden taramak için BASLA'ya dallan
DIYEZ
    BTFSS PORTB,2          ; 3. sütun 1 mi? ( #'e basıldı mı?)
    GOTO BIR               ; Hayır, 1'e basıldı mı kontrol etmek için BIR'e dallan
    MOVLW H'0B'
    MOVWF PORTA            ; Evet, PORTA'da 11 bilgisini göster
    GOTO BASLA             ; Tuşları yeniden taramak için BASLA'ya dallan
END
```

Bu programda önce klavyenin 1. satırına lojik-1 (RB3=1), diğer satırlarına lojik-0 verilir. Bu durumda RB0=1 olursa 1 tuşuna basılmamıştır, bu yüzden 2 tuşuna basılıp basılmadığını anlamak için RB1 ucu kontrol edilir. 2'ye da basılmamışsa 3 tuşu kontrol edilir. Bu tuşa da basılmamışsa 1. satırın tamamı taranmış olacağı için 2. satırdaki tuşların taraması işlemine geçilir. 2. satır için RB4=1 yapılır. Bu satırdaki 4, 5 ve 6 numaralı tuşlar taranmaya başlar. İşlem bu şekilde sürekli devam eder.

Sonuç olarak yapılan işlem satırlara sırasıyla 1 göndermek, her bir satır için üç sütunun 1 olup olmadığını kontrol etmekten ibarettir.

Aynı program tüm tuşlar için teker teker kontrol edilmeden daha kısa bir biçimde aşağıdaki şekilde yapılabilir.

PROGRAM 43: 4x3 Tuş Takımından Girilen Sayıyı PORTA'ya Bağlı Ledler Üzerinde Gösteren Program (Kısa Yoldan Yapılışı)



```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
    CBLOCK H'0C'            ; H'0C' adresinde SATIR, H'0D' adresinde SUTUN,
    SATIR, SUTUN, TUS, STR  ; H'0E' adresinde TUS, H'0F' adresinde STR
    ENDC                    ; isimli bir register oluştur
    BSF STATUS,5            ; Bank-1'e geç
    MOVLW H'07'             ; PORTB'nin ilk üç bitini giriş,
    MOVWF TRISB             ; diğer bitlerini çıkış olarak yönlendir
    CLRF TRISA              ; PORTA'yı çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF PORTA              ; PORTA'yı temizle
    CLRF PORTB              ; PORTB'yi temizle
TUS_TARA
    MOVLW D '1'
    MOVWF SATIR             ; SATIR=1
    MOVLW B '00001000'
    MOVWF PORTB             ; 1. satıra lojik-1 gönder
    MOVWF STR               ; STR'ye B'00001000' bilgisini yükle
SUTUN_OKU
    CLRF SUTUN              ; SUTUN=0
    BTFSS PORTB,0           ; 1. sütun 1 mi?
    GOTO IKI                ; Hayır, 2. sütunu kontrol etmek için IKI'ye dallan
    GOTO RAKAM              ; Evet, basılan tuşu bulmak için RAKAM'a dallan
IKI
    INCF SUTUN,F            ; SUTUN=1
    BTFSS PORTB,1           ; 2. sütun 1 mi?
    GOTO UC                 ; Hayır, 3. sütunu kontrol etmek için UC'e dallan
    MOVLW D '10'            ; Evet, W'ya 10 yükle
    SUBWF SATIR,W           ; SATIR'dan 10 çıkart
    BTFSS STATUS,Z          ; Zero flag=1 mi? (Satır =10 mu?)
    GOTO RAKAM              ; Hayır, basılan tuşu bulmak için RAKAM'a dallan
    CLRF PORTA              ; Evet, 0 bilgisini göster
    GOTO TUS_TARA           ; Tuşları yeniden taramak için TUS_TARA'ya dallan
UC
    INCF SUTUN,F            ; SUTUN=2
    BTFSS PORTB,2           ; 3. sütun 1 mi?
    GOTO DIGER_SATIR        ;Hayır, bir sonraki satıra geçmek için
                            ; DIGER_SATIR'a dallan
    GOTO RAKAM              ; Evet, basılan tuşu bulmak için RAKAM'a dallan
DIGER_SATIR
    BCF STATUS,C            ; Carry Flag'i sıfırla
    RLF STR,F               ; STR'yi bir pozisyon sola kaydır
    BTFSC STR,7             ; RB7 = 0 mı? (Son satıra ulaşılmamış mı?)
    GOTO TUS_TARA           ; Hayır, tüm satırlar taranmış, başa dön
    MOVF STR,W              ; Evet, taranmayan satır var, STR'deki bilgiyi
    MOVWF PORTB             ; PORTB'ye yükle (Bir sonraki satıra lojik-1 gönder)
    MOVLW H'03'
    ADDWF SATIR,F           ; SATIR'ı 3 artır
    GOTO SUTUN_OKU          ; Yeniden sütunları kontrol et
RAKAM
    MOVF SATIR,W            ; SATIR'daki sayı ile
    ADDWF SUTUN,W           ; SUTUN'daki sayıyı topla
    MOVWF PORTA             ; PORTA'da sonucu göster
```

```
GOTO TUS_TARA          ; Tuşları yeniden tara
END
```

Devrede 4x3'lük tuş takımı kullanıldığından ilk tuşun değeri 1'dir. Bu yüzden programda **SATIR** register'ına başlangıç değeri olarak 1 yüklenir. Önce 1. satır taranacağı için RB3=1 yapılır. Ayrıca **STR** register'ına B '00001000' bilgisi yüklenir. **SUTUN** register'ı sıfırlanır. Bu durumda RB0=1 ise 1 tuşuna basıldığı anlamına gelir. Tuşun değerini bulmak için

```
TUS = SATIR + SUTUN
TUS = 1 + 0 = 1
```

işlemi yapılır.

1'e basılmamışsa **SUTUN** register'ının değeri 1 artırılarak (**SUTUN** =1) 2'ye basılıp basılmadığını kontrol etmek için RB1 test edilir. RB1=1 ise basılan tuşun değeri

```
TUS= 1 + 1 =2
```

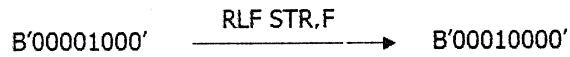
olarak bulunur.

Üçüncü sütunu test etmek için **SUTUN** yine 1 artırılarak 2 yapılır. RB2 =1 ise 3'e basılmış olacağından;

```
TUS = 1 + 2 =3
```

olarak bulunur.

Birinci satırda herhangi bir tuşa basılmamışsa program **DIGER_SATIR** etiketine dallanır. **STR** içerisindeki bilgi bir pozisyon sola kaydırılarak bu kez RB4=1 yapılır.



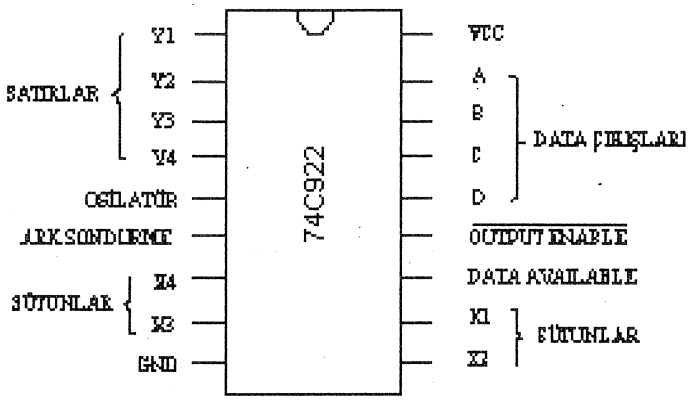
Yeniden **SUTUN_OKU** etiketine dallanarak ikinci satıra ait olan üç sütun yani 4, 5 ve 6 rakamları taranır. İkinci satırın ilk tuşu 4 olduğundan **SATIR** bilgisinin değerinin 3 artırılması gerekir. Bu yüzden halihazırda 1 olan **SATIR** register'ı 3 ile toplanarak 4 bilgisi elde edilir.

Herbir satırdan diğerine geçişte **STR** register'ı bir pozisyon sola kaydırılarak sırasıyla RB3, RB4, RB5 ve RB6 uçları 1 yapılır. 1 bilgisi RB7 ucuna geldiğinde 4 satırın taranması bitmiş olacağından program yeniden başa döndürülür.

Tuş takımı üzerinde 4. satır 2. sütunda 0 tuşu bulunmaktadır. Program tuş bilgisini **SATIR** + **SUTUN** mantığıyla hesaplandığından 0 tuşunun değerini 10 + 1=11 olarak bulacaktır. Bu sakıncayı ortadan kaldırmak için ikinci sütun taranırken bulunulan satır 4. satır ise (yani **SATIR** =10 ise) **PORTA**'ya toplam bilgisi yerine 0 bilgisi gönderilmelidir. Bu amaçla 2. sütunun taraması esnasında **SATIR** register'ının 10 olup olmadığı, bir çıkarma işlemi yapıp sonucun 0 olup olmadığını zero flag yoluyla kontrol edilmek suretiyle tespit edilmiştir. Zero Flag =1 ise **PORTA** temizlenerek ledler üzerinde 0 bilgisi gösterilmiş, daha sonra yeniden tuşları tarama kısmına dallandırılmıştır.

Klavye Entegresi Kullanma

Tarama metodu kullanıldığında mikrodenetleyici bir tuşa basılıp basılmadığını anlamak için tuşları sürekli olarak taramak zorundadır. Halbuki hiçbir tuşa basılmamışsa programın sürekli olarak tarama işlemi ile ilgilenmesi hassas uygulamalarda zaman kaybına sebep olur. Bu sebeple tuş tarama işlemini kendisi gerçekleştiren özel entegreler kullanılır. Bu kitaptaki uygulamalarda 74C922 4x4 tuş takımı kod çözücü entegresi kullanılacaktır.



ŞEKİL 42: 74C922 klavye kod çözücü entegresi.

74C922 entegresi CMOS yapılı bir entegredir. Çalışma aralığı 3-15 V arasındadır. Herhangi bir tuşa basıldığında bu tuşun binary karşılığı çıkış uçlarında görülür. Örneğin, 5 tuşuna basıldığında DCBA = 0101 olur. Butona basıldığı zaman meydana gelen arkı söndürmek için bir ark söndürme (keybounce mask) ucu mevcuttur. Bu uca bağlanacak bir kondansatör sayesinde istenilen gecikme süresi elde edilir. Osilatör ucuna harici bir osilatör devresi veya tek bir kondansatör bağlanabilir. Entegrenin sıfır aktif çalışan bir Output Enable (Çıkışa İzin Verme) ucu mevcuttur. Bu uç 0 yapılırsa basılan tuşun değeri çıkış uçlarında görülür, 1 yapılırsa bilgi oluşmasına rağmen çıkış uçlarında görülmez. Data Available (Bilgi Mevcut) çıkışı ise bir tuşa basıldığı zaman 1, basılmadığı zaman ise 0 bilgisi üreterek tuş takımında herhangi bir tuşa basılıp basılmadığını anlamamızı sağlar.

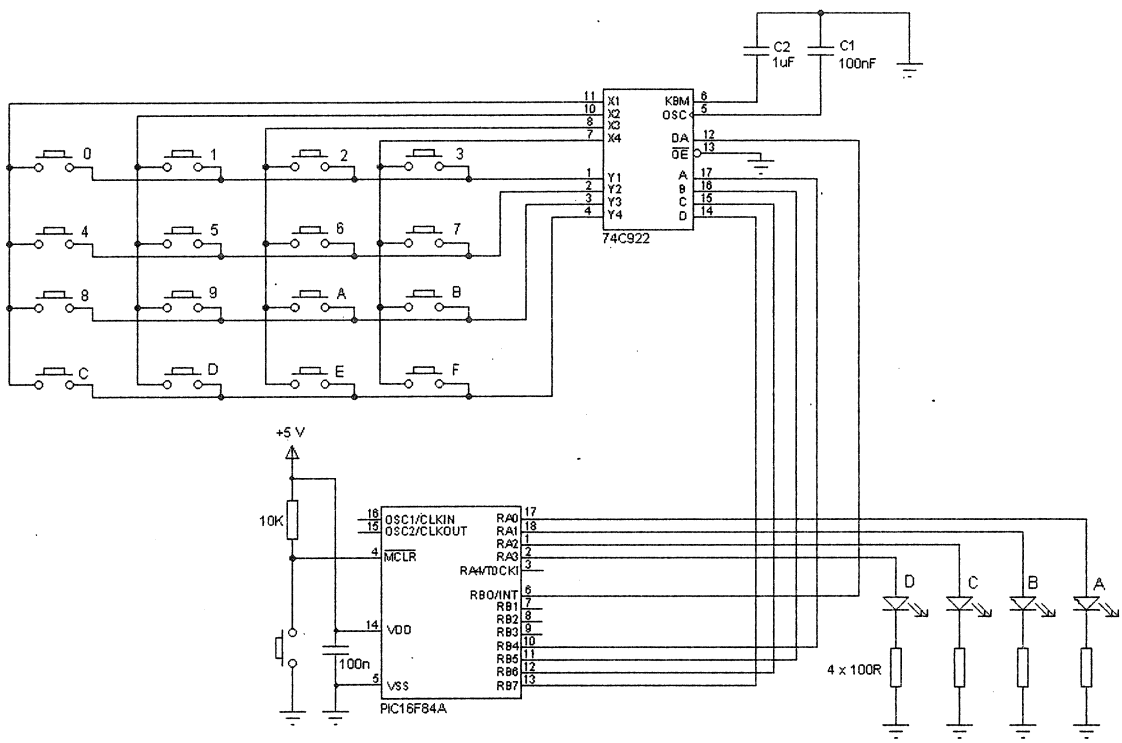
PROGRAM 44: 4x4 Klavyeden Girilen Sayıyı Ledler Üzerinde Binary Olarak Gösteren Program (74C922 klavye entegresi ve RB0 interrupt kullanılarak)

```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
    ORG H'00'
    GOTO BASLA              ; Başlangıç adresi H'00'
    ORG H'04'
    GOTO KLAVYE             ; Kesme vektörünün adresi H'04'
BASLA
    CALL KURMA              ; KURMA alt programını çağır
TEKRAR
    NOP                     ; İşlem yapmadan bekle
    GOTO TEKRAR
```

```
KURMA
   BSF STATUS,5            ; Bank-1'e geç
   CLRF TRISA              ; PORTA'yı çıkış olarak yönlendir
   MOVLW B'11000000'       ; RB0 pull-up dirençleri iptal
   MOVWF OPTION_REG        ; Yükselen kenar tetiklemesi seçilir
   MOVLW B'10010000'       ; GIE=1   INTE=1
   MOVWF INTCON
   BCF STATUS,5            ; Bank-0'a geri dön
   CLRF PORTA              ; PORTA'yı temizle
   RETURN                  ; Ana programda kaldığın yere geri dön
KLAVYE
   BCF INTCON,INTF         ; INTF=0
   SWAPF PORTB,W           ; PORTB'nin alt dört biti ile üst dört bitini değiştir
   ANDLW H'OF'             ; Üst dört biti sıfırla
   MOVWF PORTA             ; Sayıyı PORTA'da göster
   RETFIE                  ; Ana programda kaldığın yere geri dön
   END
```

Kesme kullanılan programlarda daha önceden de belirttiğimiz gibi programın başında ana programın adresi ile kesme alt programının adresinin yazılması gerekir. Ana program H'00' adresinde kesme vektörü ise H'04' adresindedir. Daha sonra kurma işlemleri yapılır. **INT** kesmesi kullanıldığında RB0 ucundan gelen sinyalin yükselen kenarda mı, düşen kenarda mı kesmeye neden olacağı belirtilmelidir. Bu işlem **OPTION_REG** register'ının **INTEDG** ucu kullanılarak yapılır. 74C922 entegresinin **DA** ucu, tuş takımından herhangi bir tuşa basıldığında 1 olur. Bu durumda **RB0** ucu lojik-0'dan lojik-1'e geçeceğinden **INTEDG**=1 yapılarak yükselen kenar tetiklemesi seçilir. **INTE**=1 yapılarak **INT** kesmesi, **GIE**=1 yapılarak ise tüm kesmeler aktif hale getirilir. Çıkış portu temizlenerek ana programa geri dönülür.

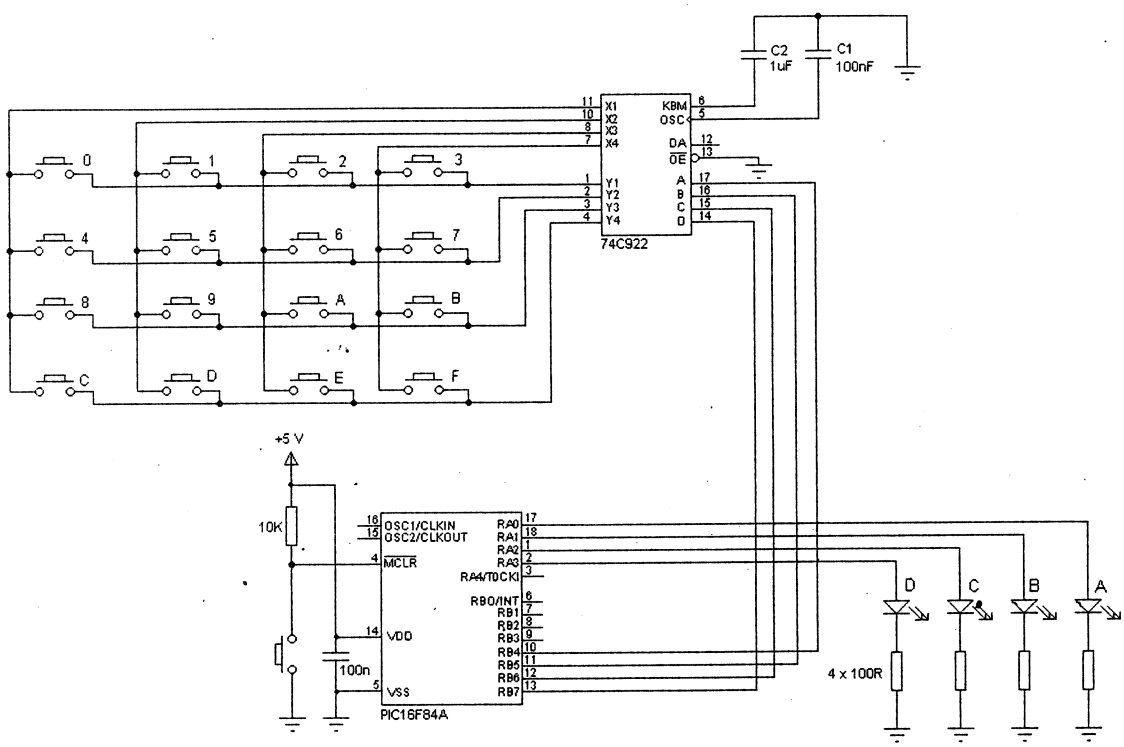
Programda yapılması istenen şey herhangi bir tuşa basıldığında o tuşun binary değerini **PORTA**'da göstermektir. Yani tuş takımından bir tuşa basılmadığı sürece program hiçbirşey yapmayacaktır. Bu yüzden ana programımız **NOP** komutundan ibarettir. Bir döngü ile sürekli olarak **NOP** (No Operation – İşlem yok) komutu işlenmektedir. Tuş takımından bir tuşa basıldığı zaman **DA**=1 olur, **RB0** ucu lojik-0 durumundan lojik-1 durumuna geçeceğinden kesme olayı meydana gelir ve program **KLAVYE** alt pogramına dallanır. Kesme alt programına girildiği anda ilk yapılması gereken şey o kesmeye ait flag'i 0 yapmaktır. Çünkü flagler otomatik olarak 1 olur, ancak otomatik olarak 0 olmaz. Bu da kesme alt programından çıkıldığında kesmeyi meydana getiren olay ortadan kalkmış olmasına rağmen **INTF**=1 olduğundan yeni bir kesme meydana gelmiş gibi işlem yapılmasına; yani sürekli olarak kesme alt programına dallanılarak ana programdaki komutların işlenememesine sebep olur. O halde Klavye alt programındaki ilk işimiz **INTF**=0 yapmaktır. Daha sonra **PORTB**'nin alt dört biti ile üst dört bitinin yerleri değiştirilmiştir. Şekil 43'e dikkat ederseniz 74C922 çıkışlarının **PORTB**'nin üst dört bitine bağlı olduğunu görürsünüz. Bu bir zorunluluktur çünkü **INT** kesmesi kullanıldığından **RB0** ucu doludur. Örneğin 5 tuşuna basılmış olsun. Bu durumda **PORTB**'deki bilgi 0101xxxx olur. Bu bilgi doğrudan **PORTA**'ya atılsaydı ledler üzerinde xxxx bilgisini görürdük. Bu yüzden üst dört bitteki 5 bilgisini alt dört bite taşımamız gerekir. **SWAPF** komutu işlendikten sonra xxxx0101 bilgisi elde edilir. X yani bilinmeyen (0 veya 1) kısımlarını sıfırlamak için lojik VE işlemi yapılır. Böylece elde edilen 00000101 bilgisi **PORTA**'ya atıldığında ledler üzerinde 5 bilgisinin yandığı görülür.



ŞEKİL 43: 74C922 ile tuş takımı devresi (RB0 kesmesi için).

PROGRAM 45: 4x4 Klavyeden Girilen Sayıyı Ledler Üzerinde Binary Olarak Gösteren Program (74C922 Klavye Entegresi – RBHigh Interrupt Kullanılarak)

```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
    ORG H'00'
    GOTO BASLA              ; Başlangıç adresi H'00'
    ORG H'04'
    GOTO KLAVYE             ; Kesme vektörünün adresi H'04'
BASLA
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISA              ; PORTA'yı çıkış olarak yönlendir
    MOVLW B'10001000'       ; GIE=1  RBIE=1
    MOVWF INTCON
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF PORTA              ; PORTA'yı temizle
TEKRAR
    NOP
    GOTO TEKRAR             ; İşlem yapmadan bekle
KLAVYE
    BCF INTCON,RBIF         ; RBIF=0
    SWAPF PORTB,W           ; PORTB'nin alt dört biti ile üst dört bitini değiştir
    ANDLW H'0F'             ; Üst dört biti sıfırla
    MOVWF PORTA             ; Sayıyı PORTA'da göster
    RETFIE                  ; Ana programda kaldığın yere geri dön
    END
```



ŞEKİL 44: 74C922 ile tuş takımı devresi (RBHigh kesmesi için).

Bu programda yine önce ana program ile kesme vektörünün adresleri **ORG** direktifi ile tanımlanmış, daha sonra **INTCON** register'ında gereken ayarlar yapılmıştır. **PORTB** değişim kesmesi kullandığımız için bu kez **RBIE**=1 yapmamız gerekir. Tüm kesme türlerinde **GIE**=1 yapmamız gerektiğini zaten biliyoruz. Ana programımız yine bir **NOP** komutundan oluşmaktadır.

Tuş takımından herhangi bir tuşa basıldığında 74C922 çıkışlarında o tuşun binary değeri meydana gelecek, bu da **PORTB**'nin üst dört bitinde bir değişiklik olmasına sebep olacaktır. Bu değişiklik kesme olayı olarak algılanacak ve program **KLAVYE** alt programına dallanacaktır. Burada yine ilk olarak **RBIF**=0 yapılacak, daha sonra tuş bilgisini elde etmek için Program 44'deki işlem basamakları aynen izlenecektir.

Bölüm 13

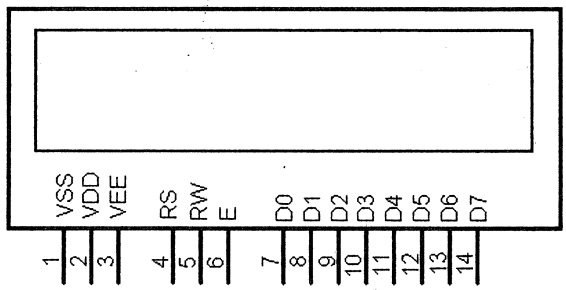
LCD Uygulamaları

LCD'nin Yapısı

LCD yani Likit Kristal Display günümüzde oldukça yaygın olarak kullanılan bir display türüdür. Daha önce incelediğimiz 7-segment display ile sadece rakamlar görüntülenebildiğinden harflerin veya sembollerin görüntülenmesi gereken uygulamalarda yetersiz kalmaktadır. Bu yüzden LCD'lerin nasıl kullanıldığı konusunda bilgi sahibi olmanız çalışmalarınızda oldukça faydalı olacaktır.

Karakter tabanlı dot matrix LCD ve grafik LCD olmak üzere iki çeşit LCD vardır. Bu kitapta kullanımı daha basit olduğundan dot matrix LCD incelenmiştir. Dot matrix LCD'ler mikroişlemcili sistemlerle kullanıldıklarında sürekli tarama işlemi yapılması gerektiğinden bir sürücü entegreye ihtiyaç duyarlar. Piyasada sıkça kullanılan Hitachi firmasına ait LCD display'ler HD44780 sürücü entegresi ile bir arada kullanıma sunulmaktadır. Örnek uygulamalarımızda bu LCD'yi kullanacağız.

Aşağıda bir LCD'nin pin yapısı gösterilmiştir.



ŞEKİL 45: LCD pin yapısı.

Görüldüğü gibi LCD pinleri 3 ana gruba ayrılmış durumdadır. İlk grup enerji gereksiniminin sağlanması ile ilgili olan VSS, VEE ve VCC uçlarını kapsar. VSS şase ucu olup toprağa, VCC ise besleme ucu olup +5V'a bağlanır. VEE ucu ise LCD'nin arka plan aydınlatmasında kullanılan kontrast ucudur. Bu uca bir potansiyometre bağlanarak LCD'nin parlaklığı ayarlanabilir.

İkinci grup LCD'nin kontrol uçlarını kapsar. RS ucu LCD'ye veri veya komut göndereceğimizi belirttiğimiz uçtur. Eğer LCD'ye veri göndermek istiyorsak RS=1, komut göndermek istiyorsak RS=0 yapmamız gerekir. R/W okuma/yazma ucudur. LCD'ye yazmak için R/W=0, LCD'den okumak için ise R/W=1 yapmamız gerekir. E ucu enable yani etkinleştirme ucudur. LCD'ye her bir komutu gönderdikten sonra E ucu lojik-1 durumundan lojik-0 durumuna geçirilerek tetiklenir. Tetikleme geldiği anda diğer pinlerin durumuna bakarak LCD ne yapması isteniyorsa o işlevi yerine getirir.

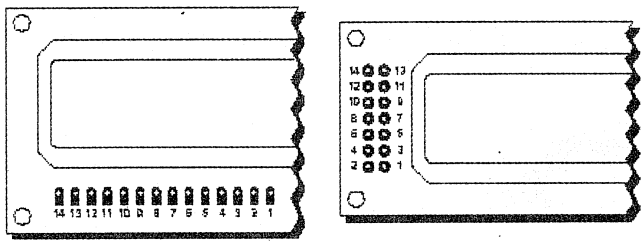
D0~D7 arasındaki uçlar data uçlarıdır. Veri alışverişi bu uçlar üzerinden gerçekleştirilir.

TABLO 13: 80'den Az Karakterli LCD Displaylerin Ayak Yapısı

Pin No.	Pin Adı	Seviye	Giriş/Çıkış	Fonksiyon
1	VSS	-	-	GND
2	VCC	-	-	+5V
3	VEE	-	-	Kontrast ayarı

TABLO 13: 80'den Az Karakterli LCD Displaylerin Ayak Yapısı

Pin No.	Pin Adı	Seviye	Giriş/Çıkış	Fonksiyon
4	RS	0/1	Giriş	0 = Komut girişi 1 = Veri girişi
5	R/W	0/1	Giriş	0 = LCD'ye yaz 1 = LCD'den oku
6	E	1,-1< 0	Giriş	Enable (Etkinleştirme)
7	DB0	0/1	Giriş/Çıkış	Veri yolu - 0 LSB
8	DB1	0/1	Giriş/Çıkış	Veri yolu - 1
9	DB2	0/1	Giriş/Çıkış	Veri yolu - 2
10	DB3	0/1	Giriş/Çıkış	Veri yolu - 3
11	DB4	0/1	Giriş/Çıkış	Veri yolu - 4
12	DB5	0/1	Giriş/Çıkış	Veri yolu - 5
13	DB6	0/1	Giriş/Çıkış	Veri yolu - 6
14	DB7	0/1	Giriş/Çıkış	Veri yolu - 7 MSB



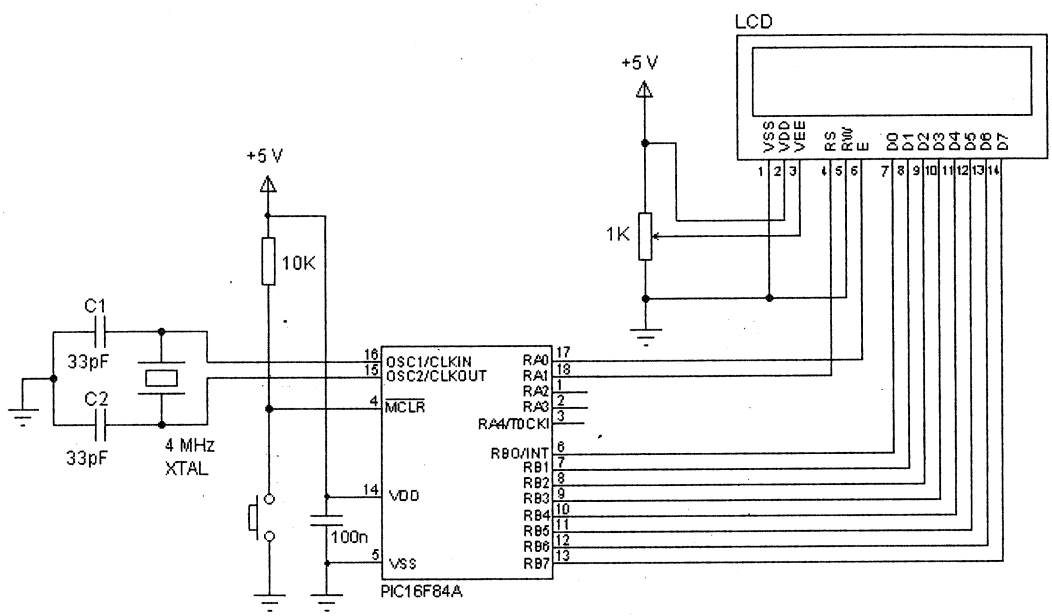
ŞEKİL 46: LCD pinlerinin dizilişi.

LCD'nin pin sıralaması genellikle yukarıdaki şekilde olur ve pinlerin numaraları LCD üzerinde yazar.

Şekil 47'de PIC16F84 ile LCD display arasındaki bağlantı gösterilmiştir. **PORTB**'nin sekiz pini data uçlarına bağlanmıştır (RB0~RB7 – D0~D7). R/W ucu doğrudan şaseye bağlanmıştır; çünkü sadece LCD'ye yazma işlemi yapılacaktır. RS ve E uçları A portuna bağlıdır. VEE ucuna 1KΩ'luk bir potansiyometre bağlanarak LCD'nin parlaklık ayarının yapılması sağlanmıştır. Besleme ve şase bağlantıları da yapıldıktan sonra yapılacak tek şey gereken programı PIC'e yükleyip çalıştırmaktır.

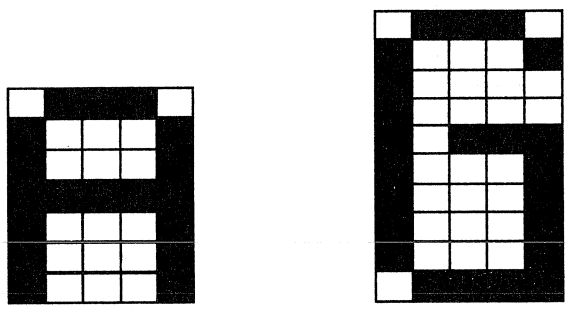
LCD Komutları

LCD'nin çalıştırılabilmesi için kullanılması gereken komutlar Tablo 13'te verilmiştir. Tabloda RS, R/W ve data uçlarına uyguladığımız lojik sinyallere göre LCD'nin hangi fonksiyonu yerine getireceği açıklanmıştır. Herhangi bir kodun dikkate alınabilmesi için mutlaka kod gönderildikten hemen sonra E ucu tetiklenerek etkinleştirme işleminin yapılması gerekir.



ŞEKİL 47: LCD kontrol devresi.

5x7 karakter fontu 5 yatay ve 7 dikey noktadan bir karakter meydana getirir. 5x10 fontta ise 5 yatay ve 10 dikey nokta bulunur. Aşağıda her iki fonttan karakterlerin elde edilişine birer örnek verilmiştir.



ŞEKİL 48: 5X7 ve 5x10 displayde karakterlerin elde edilişi.

Ekranı Temizle

RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	0	0	0	0	0	0	0	1

Bu komut tüm DDRAM adreslerine boşluk kodu olan H'20' bilgisini yükleyerek ekranı temizler. Adres sayıcıyı DDRAM'in 0. adresine konumlandırır ve display'i -eğer kaydırılmışsa- orjinal konumuna döndürür. Kursör display'in en soluna alınır (eğer iki satır kullanılıyorsa 1. satırın en soluna). Ayrıca I/D=1 yaparak artırma moduna geçilir. Giriş modunun S değeri değişmez.

TABLO 13: LCD Komut Listesi

	Kod										
Komut	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	Tanım
Ekranı temizle (Clear Display)	0	0	0	0	0	0	0	0	0	1	Ekranı temizler ve kursörü ekranın başına (adres 0) getirir.
Kursörü başa getir (Return Home)	0	0	0	0	0	0	0	0	1	*	Kursörü ekranın başına (adres 0) getirir. Kaydırılan display'i orijinal pozisyonuna döndürür. DDRAM içeriği değişmeden kalır.
Giriş modu (Entry Mode Set)	0	0	0	0	0	0	0	1	I/D	S	Kursörün hareket yönünü (I/D) ayarlar. Display'in kayma durumunu (S) belirler.
Ekran On/Off Kontrolü (Display On/Off)	0	0	0	0	0	0	1	D	C	B	Display'i açar/kapatır (D). Kursörü açar/kapatır (C). Kursörün yanıp sönme durumunu (blink) belirler (B).
Kursör/Display Kaydırma (Cursor and Display Shift)	0	0	0	0	0	1	S/C	R/L	*	*	Kursör hareketini ve display'in kayma durumunu (S/C)ayarlar. Kayma yönünü belirler (R/L). DDRAM içeriği değişmeden kalır.
Fonksiyon seti (Function Set)	0	0	0	0	1	DL	N	F	*	*	Veri uzunluğunu (DL), satır sayısını (N) ve karakter fontunu (F)ayarlar.
CGRAM adres ayarı (Set CGRAM Address)	0	0	0	1	CGRAM Adresi						CGRAM Adresini ayarlar. Bu ayarlamadan sonra CGRAM verileri gönderilip alınabilir.
DDRAM adres ayarı (Set DDRAM Address)	0	0	1	DDRAM Adresi							DDRAM Adresini ayarlar. Bu ayarlamadan sonra DDRAM verileri gönderilip alınabilir.
Busy flag'i ve adres sayıcıyı oku (Read Busy Flag and Address)	0	1	BF	CGRAM/DDRAM Adresi							Dahili işlemlerin devam ettiğini gösteren Busy Flag'i (BF),CGRAM veya DDRAM adres sayıcının içeriğini okur.
CGRAM veya DDRAM'e yaz (Write Data CGRAM or DDRAM)	1	0	Yazılacak bilgi								CGRAM veya DDRAM'e yazar.
CGRAM veya DDRAM'den oku (Read Data From CGRAM and DDRAM)	1	1	Okunacak bilgi								CGRAM veya DDRAM'den okur.

DDRAM : Display Data RAM (Kursör pozisyonunu gösterir)
CGRAM : Character Generator RAM (Karakter üretici RAM)
* = Don't Care (Farketmez – 0 veya 1)

Kursörü Başa Getir

RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	0	0	0	0	0	0	1	*

Bu komut adres sayıcıyı DDRAM'in 0. adresine konumlandırır ve display'i -eğer kaydırılmışsa- orjinal konumuna döndürür. DDRAM içeriği değişmez. Kursör display'in en soluna alınır (eğer iki satır kullanılıyorsa 1. satırın en soluna).

TABLO 14: Bit Değerlerine Göre LCD'nin Durumu

BİT	DURUM	
I/D	0 = Kursör pozisyonunu azaltır	1 = Kursör pozisyonunu artırır
S	0 = Display kaymaz	1 = Display kayar
D	0 = Display kapalı	1 = Display açık
C	0 = Kursör kapalı	1 = Kursör açık
B	0 = Kursör yanıp sönmez	1 = Kursör yanıp söner
S/C	0 = Kursörü taşı	1 = Displayi kaydır
R/L	0 = Sola kaydır	1 = Sağa kaydır
DL	0 = 4-bit bağlantı	1 = 8-bit bağlantı
N	0 = 1 satır	1 = 2 satır
F	0 = 5x7 karakter fontu	1 = 5x10 karakter fontu
BF	0 = Komut kabul edebilir (İşlem bitti)	1 = LCD meşgul

I/D : Increment/ Decrement Cursor Position
S : Display Shift
D : Display On/Off
C : Cursor On/Off
B : Cursor Blink On/Off
S/C : Shift/Cursor
R/L : Shift Right/Left
DL : Data Lenght
N : Number Of Lines
F : Font
BF : Busy Flag

Giriş Modu

RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	0	0	0	0	0	1	I/D	S

I/D=1 yapıldığında DDRAM'e yazıp okuma işlemlerinde adresin 1 artırılacağı, I/D=0 yapıldığında ise 1 azaltılacağı anlamına gelir. Yani örneğin artırma modunda her bir karakter yazıldıktan sonra kursör sağa, azaltma modunda ise sola geçecektir.

S=1 yapıldığında display'i sağa veya sola doğru kaydırır (I/D=1 ise sola, I/D=0 ise sağa). Böylece sanki kursör duruyor, display hareket ediyormuş gibi görünür.

Ekran On/Off Kontrolü

RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	0	0	0	0	1	D	C	B

D=1 yapıldığında ekranı açar, D=0 yapıldığında kapatır. Display kapatıldığında veriler DDRAM'de kalır, D=1 yapıldığı anda ekranda görüntülenir.

C=1 yapıldığında kursör görüntülenir, C=0 yapıldığında görüntülenmez. 5x7 karakter fontu seçilmişse kursör 8. satırda, 5x10 karakter fontu seçilirse 11. satırda görüntülenir.

B=1 yapıldığında kursör karakteri yanıp söner.

Kursör/Display Kaydırma

RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	0	0	0	1	S/C	R/L	*	*

S/C	R/L	
0	0	; Kursör pozisyonunu sola kaydırır
0	1	; Kursör pozisyonunu sağa kaydırır
1	0	; Displayi sola kaydırır
1	1	; Displayi sağa kaydırır

Fonksiyon Seti

RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	0	0	1	DL	N	F	*	*

DL=1 yapıldığında bilgiler 8 bit halinde gönderilir. Yani PIC ile LCD arasında D0-D7 arasında 8 hat vardır.

DL=0 yapıldığında bilgiler 4 bit halinde gönderilir. (DB4~DB7)

4 bitlik bağlantı seçildiğinde bilgiler 2 kerede gönderilir.

N satır sayısını seçmek için, F ise karakter fontunu seçmek için kullanılır. Bunların 0 ve 1 durumlarına göre display'in durumu yukarıdaki tabloda verilmiştir. Font 5x10 dot seçildiğinde 2. satır görüntülenemez.

CGRAM Adres Ayarı

RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	0	1	A	A	A	A	A	A

CGRAM adresini ayarlar. Bu ayarlamadan sonra CGRAM bilgisi gönderilir.

DDRAM Adres Ayarı

RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	1	A	A	A	A	A	A	A

DDRAM adresini, yani bilginin ekranda konumlanacağı yeri ayarlar. Bu ayarlamadan sonra DDRAM bilgisi gönderilir.

N=0 yani 1 satırlık kullanım seçildiğinde adres bilgisi AAAAAAA 00h-4Fh arasındadır.N=1 yani 2 satırlık kullanım seçildiğinde adres bilgisi AAAAAAA 1. satır için 00h-27h, 2. satır için 40h-67h arasındadır.

0	1	2	3	4	5	6	7	8	9	10	11	.	.	29	30	31	.	.	39	Karakter pozisyonu (Dec)
00	01	02	03	04	05	06	07	08	09	0A	0B	.	.	1D	1E	1F	.	.	27	1. Satır DDRAM Adresi (Hex)
40	41	42	43	44	45	46	47	48	49	4A	4B	.	.	5D	5E	5F	.	.	67	2. Satır DDRAM Adresi (Hex)

TABLO 15: 2 Satır LCD Ekranlar İçin DDRAM Adresleri

Display boyutu	Karakter pozisyonu	DDRAM adresleri
2*16	00 15	0x00..0x0F + 0x40..0x4F
2*20	00 19	0x00..0x13 + 0x40..0x53
2*24	00 23	0x00..0x17 + 0x40..0x57
2*32	00 31	0x00..0x1F + 0x40..0x5F
2*40	00 39	0x00..0x27 + 0x40..0x67

NOT **2*16 iki adet 16 karakterlik satırı olan LCD, 2*40 iki adet 40 karakterlik satırı olan LCD anlamına gelir.**

Tablolar incelenirse 2*16'lık bir LCD için 1. satırın 00h-0Fh adresleri arasında, 2. satırın ise 40h-4Fh adresleri arasında olduğu görülür. LCD'ye enerji verildiğinde imleç 1. satırın başına yani 00h adresine yerleşir. Bu durumda örneğin LCD'nin 1. satır 5. sütunundan itibaren MERHABA yazmak istediğimizde başlangıç adresi olarak 04h adresini vermeliyiz. Aynı şekilde 2. satırın başından itibaren yazmaya başlamak istiyorsak adresimiz 40h olmalıdır. Ancak bu adresleri verirken, yukarıda adres bilgisinin AAAAAAA şeklinde 7 bit ile ifade edildiğine dikkat ediniz. Yani örneğin 40h adresi verileceğinde 01000000 şeklinde değil 1000000 şeklinde yazıyoruz. Bu durumda yukarıdaki formata göre yazacağımız komut;

```
RS R/W DB7 DB6 DB5 DB4 DB3 DB2 DB1 DB0
=== === === === === === === === === ===
 0   0   1   1   0   0   0   0   0   0
         \_____________________________/
                      H'C0'
```

şeklinde olur.

Tüm LCD modüllerinde yazılabilir 80 display konumu vardır. Eğer display daha küçük boyutlarda ise örneğin 1*16 LCD kullanıyorsak display'de sadece ilk 16 konum görüntülenebilir. Kalan 64 karakter görünmeyen konumlarda yer alır. Display kaydırıldığında diğer konumların da görülmesi sağlanabilir.

Meşgul Bayrağını ve Adresi Okuma

RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	1	BF	A	A	A	A	A	A	A

Bu komut meşgul bayrağını (BF) okur. BF=1 ise bir önceki komutun hala işlenmekte olduğunu gösterir. BF=0 olana kadar yeni bir komut kabul edilmez.

Aynı zamanda adres sayıcının da değeri okunur. Adres sayıcı hem CGRAM hem de DDRAM için kullanılır.

CGRAM veya DDRAM'e Yazma

RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
1	0	D	D	D	D	D	D	D	D

DDRAM veya CGRAM'e 8 bitlik veri yazar. Hangi RAM'e yazılacağı daha önce yapılan ayarlama ile belirlenir. Yazma işleminden sonra seçilen giriş moduna göre adres otomatik olarak 1 artırılır veya 1 azaltılır.

CGRAM veya DDRAM'den Okuma

RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
1	1	D	D	D	D	D	D	D	D

DDRAM veya CGRAM'den 8 bitlik veri okur. Hangi RAM'den okunacağı daha önce yapılan ayarlama ile belirlenir.

Tabloya göre bazı LCD işlemleri için izlememiz gereken basamaklara birkaç örnek verelim.

- Ekranı temizleyip kursörü ekranın başına getirmek için;

  1. RS=0 yaparak komut girileceğini LCD'ye bildir.
  2. R/W=0 yaparak mikrodenetleyiciden LCD'ye yazma işlemi yapılacağını bildir.
  3. PORTB'ye H'01' bilgisini yükle.

RS	R/W	D7	D6	D5	D4	D3	D3	D1	D0
0	0	0	0	0	0	0	0	0	1

H'01'

  4. E'yi tetikle.

- Ekrana A harfi yazmak için;

1. RS=1 yaparak veri girileceğini LCD'ye bildir.
2. R/W=0 yaparak mikrodenetleyiciden LCD'ye yazma işlemi yapılacağını bildir.
3. PORTB'ye A'nın kodu olan H'41' bilgisini yükle.

RS	R/W	D7	D6	D5	D4	D3	D3	D1	D0
1	0	0	1	0	0	0	0	0	1

H'41'

4. E'yi tetikle

Tablo 16'da LCD'de görüntülenebilecek karakterlerin kod tablosu verilmiştir. Kodlar yazılırken dikey olanlar üst byte'ı, yatay olanlar ise alt byte'ı oluşturacak şekilde yazılmalıdır. Örneğin "e" karakterinin kodu B'01100101' yani H'65', $ karakterinin kodu ise B'00100100' yani H'24' şeklindedir.

TABLO 16: LCD Karakter Tablosu

Char. code	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
xxxx0000		0	@	P	`	p		ー	タ	ミ	α	p
xxxx0001		!	1	A	Q	a	q	。	ア	チ	ム	ä	q
xxxx0010		"	2	B	R	b	r	「	イ	ツ	メ	ß	θ
xxxx0011		#	3	C	S	c	s	」	ウ	テ	モ	ε	∞
xxxx0100		$	4	D	T	d	t	、	エ	ト	ヤ	μ	Ω
xxxx0101		%	5	E	U	e	u	・	オ	ナ	ユ	σ	ü
xxxx0110		&	6	F	V	f	v	ヲ	カ	ニ	ヨ	ρ	Σ
xxxx0111		'	7	G	W	g	w	ァ	キ	ヌ	ラ	q	π
xxxx1000		(	8	H	X	h	x	ィ	ク	ネ	リ	ɾ	x̄
xxxx1001		)	9	I	Y	i	y	ゥ	ケ	ノ	ル	⁻¹	ч
xxxx1010		*	:	J	Z	j	z	ェ	コ	ハ	レ	i	千
xxxx1011		+	;	K	[	k	{	ォ	サ	ヒ	ロ	×	万
xxxx1100		,	<	L	¥	l	|	ャ	シ	フ	ワ	¢	円
xxxx1101		-	=	M	]	m	}	ュ	ス	ヘ	ン	£	÷
xxxx1110		.	>	N	^	n	→	ョ	セ	ホ	゛	ñ	
xxxx1111		/	?	O	_	o	←	ッ	ソ	マ	°	ö	■

LCD'nin Kullanıma Hazırlanması

LCD'ye veri gönderme işlemine geçmeden önce aşağıdaki işlemlerin yapılması gerekir.

1. Ekranı temizle.
2. Fonksiyon seti ayarlarını yap.

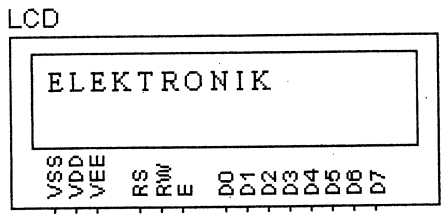
- DL: LCD ile PIC arasında kaç bitlik bağlantı yapılacağını belirle
- N: Ekranda kaç satır görüleceğini belirle
- F: Karakter fontunu belirle

3. Display ON/OFF ayarlarını yap
   - D: Display'i aç/kapat
   - C: Kursörü aç/kapat
   - B: Kursörün yanıp sönme veya sabit kalma durumunu belirle
4. Giriş modu ayarlarını yap
   - I/D: Kursörün hareket yönünü belirle
   - S: Display'in kayma durumunu belirle

PROGRAM 46: 8 Bit ile LCD'nin İlk Satırına ELEKTRONİK Yazan Program (1. YOL)

```
   LIST P=16F84            ; Kullanılacak PIC'i tanıt
   INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
SAYAC1 EQU H'0C'           ; H'0C' adresinde SAYAC1,
SAYAC2 EQU H'0D'           ; H'0D' adresinde SAYAC2 isimli bir register oluştur
   CLRF PORTB              ; PORTB'yi temizle
   BSF STATUS,5            ; Bank-1'e geç
   CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
   CLRF TRISA              ; PORTA'yı çıkış olarak yönlendir
   BCF STATUS,5            ; Bank-0'a geri dön
BASLA
   MOVLW H'01'             ; Ekranı temizler ve kursörü
   CALL KOMUTYAZ           ; ekranın başına (adres 0) getirir
   MOVLW H'30'             ; 8 bitlik bağlantı, 1 satır ve
   CALL KOMUTYAZ           ; 5x7 karakter fontu seçilir
   MOVLW H'0C'             ; Ekranı açar, kursörü kapatır ve
   CALL KOMUTYAZ           ; kursör yanıp sönmez
   CALL SATIRYAZ           ; Ekrana yazdırma kısmına dallan
DEVAM
   GOTO DEVAM              ; Bilgiyi ekranda sürekli göstermek için sonsuz
                           ; döngüye gir
KOMUTYAZ
   BCF PORTA,1             ; RS=0 yaparak
   MOVWF PORTB             ; komut girileceğini bildir
   BSF PORTA,0             ; E=1
   CALL GECIKME            ;
   BCF PORTA,0             ; E= 0
   RETURN
VERIYAZ
   BSF PORTA,1             ; RS=1 yaparak
   MOVWF PORTB             ; veri girileceğini bildir
   BSF PORTA,0             ; E=1
   CALL GECIKME            ;
   BCF PORTA,0             ; E= 0
   RETURN
SATIRYAZ
```

```
    MOVLW 'E'
    CALL VERIYAZ            ; E
    MOVLW 'L'
    CALL VERIYAZ            ; L
    MOVLW 'E'
    CALL VERIYAZ            ; E
    MOVLW 'K'
    CALL VERIYAZ            ; K
    MOVLW 'T'
    CALL VERIYAZ            ; T
    MOVLW 'R'
    CALL VERIYAZ            ; R
    MOVLW 'O'
    CALL VERIYAZ            ; O
    MOVLW 'N'
    CALL VERIYAZ            ; N
    MOVLW 'I'
    CALL VERIYAZ            ; I
    MOVLW 'K'
    CALL VERIYAZ            ; K
    RETURN
GECIKME
    MOVLW H'05'
    MOVWF SAYAC1
D1
    MOVLW H'FF'
    MOVWF SAYAC2
D2
    DECFSZ SAYAC2,F
    GOTO D2
    DECFSZ SAYAC1,F
    GOTO D1
    RETURN
    END
```



Program çalıştırıldığında yukarıdaki şekilde ekranın 1. satırında ilk adresten itibaren ELEKTRONIK yazısı görülecektir. Bu yazı bilgisi veri, diğer ayarlamalar ise komuttur. Programda veri gönderme işleminden önce LCD ayarlarının yapılması gerekir. Bu amaçla portları giriş-çıkış olarak yönlendirme işleminden hemen sonra LCD ile PIC arasında kaç bitlik bağlantı yapılacağı, satır sayısı, kursörün ekranda görünüp görünmemesi, kursörün yanıp sönmesi, karakter fontu, vs. ayarlamalar yapılmıştır. Bunlar birer komut olduğu için her bir kodun **W**'ya yüklenmesinden sonra **CALL KOMUTYAZ** satırı işlenir ve **KOMUTYAZ** alt programında RS=0 yapıla-

rak gönderilecek bilginin komut olduğu LCD'ye bildirilir. LCD'ye gönderilen her bir koddan sonra E ucu tetiklenir (düşen kenar tetiklemeli olduğu için önce 1 sonra 0 yapılarak tetikleme sağlanır) ve kodun işleme alınması sağlanır.

H'01' kodu Tablo 13'den görüldüğü gibi zaten ekranı temizleyip kursörü başa almak için kullanılan sabit bir koddur. Diğer iki kod H'30' ve H'0C'nin elde edilişini inceleyelim.

DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
0	0	1	DL	N	F	*	*	
0	0	1	1	0	0	0	0	= H'30'

DL=1 8 bitlik bağlantı yapılacak

N=0 1 satır olacak

F=0 5x7 karakter fontu kullanılacak

NOT *** işareti yerine 0 yazılabildiği gibi 1 yazılması da mümkündür. Yani H'33' kodu da aynı işlevi yerine getirecektir.**

DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
0	0	0	0	1	D	C	B	
0	0	0	0	1	1	0	0	= H'0C'

D=1 Ekranı aç

C=0 Kursör görünmesin

B=0 Kursör yanıp sönmesin

Programda H'0C' yerine H'0E' yazılırsa ekranda ELEKTRONIK_ bilgisi görülür, yani kursör karakterin yanında görüntülenir. H'0F' yazılırsa kursör karakteri sürekli yanıp söner.

Gerekli ayarlamalar yapıldıktan sonra verinin yani E-L-E-K-T-R-O-N-I-K karakterlerinin ekrana yazılabilmesi için **SATIRYAZ** alt programı çağrılır. Burada her bir karakter **W** register'ına yüklendikten sonra **VERIYAZ** alt programı çağrılarak RS=0 yapılarak kodun bir veri bilgisi olduğu LCD'ye bildirilir. **W**'daki karakter bilgisi **PORTB**'ye yüklenerek D0-D7 uçları üzerinden LCD'ye gönderilir. Yine E ucu tetiklenerek işlem etkinleştirilir.

Programda R/W ucu ile ilgili herhangi bir yapılmamasının sebebi LCD'den okuma işleminin olmamasıdır. R/W=1 yapıldığında LCD'den okuma, R/W =0 yapıldığında ise LCD'ye yazma işlemi yapıldığını söylemiştik. Biz sadece yazma işlemi yapacağımız için R/W ucunu doğrudan şaseye bağlıyoruz. LCD program uygulamalarımızda Şekil 47'deki devreyi kullanabiliriz.

PROGRAM 47: 8 Bit ile LCD'nin İlk Satırına ELEKTRONIK Yazan Program (2. YOL)

```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
    CBLOCK H'0C'            ; H'0C' adresinde SAYAC1,
    SAYAC1,SAYAC2,SAYICI    ; H'0D' adresinde SAYAC2 ve
    ENDC                    ; H'0E' adresinde SAYICI isimli bir register oluştur
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    CLRF TRISA              ; PORTA'yı çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF SAYICI             ; SAYICI'yı sıfırla
    CLRF PORTB              ; PORTB'yi temizle
    CLRF PORTA              ; PORTA'yı temizle
BASLA
    MOVLW H'01'             ; Ekranı temizler ve kursörü
    CALL KOMUTYAZ           ; ekranın başına (adres 0) getirir
    MOVLW H'30'             ; 8 bitlik bağlantı, 1 satır ve
    CALL KOMUTYAZ           ; 5x7 karakter fontu seçilir
    MOVLW H'0C'             ; Ekranı açar, kursörü kapatır ve
    CALL KOMUTYAZ           ; kursör yanıp sönmez
    CALL SATIRYAZ           ; Ekrana yazdırma kısmına dallan
DEVAM
    GOTO DEVAM              ; Bilgiyi ekranda sürekli göstermek için sonsuz
                            ; döngüye gir
KOMUTYAZ
    BCF PORTA,1             ; RS=0 yaparak
    MOVWF PORTB             ; komut girileceğini bildir
    BSF PORTA,0             ; E=1 ──┐
    CALL GECIKME            ;       │
    BCF PORTA,0             ; E= 0  └──
    RETURN
SATIRYAZ
    CALL GECIKME
    BSF PORTA,1             ; RS=1 yaparak veri girileceğini bildir.
    MOVF SAYICI,W           ; SAYICI'daki bilgiyi W'ya yükle
    CALL MESAJ              ; MESAJ alt programını çağır
    IORLW H'00'             ; W'ya yüklenen karakteri 0 ile OR'la
    BTFSC STATUS,Z          ; Sonuç 0 mı? (Son karakter gelmiş mi?)
    GOTO BITIR              ; Evet, BITIR etiketine dallan
    MOVWF PORTB             ; Hayır, karakteri LCD'ye gönder
    INCF SAYICI,F           ; Sonraki karakteri almak için SAYICI'yı 1 artır
    BSF PORTA,0             ; E=1 ──┐
    CALL GECIKME            ;       │
    BCF PORTA,0             ; E= 0  └──
    GOTO SATIRYAZ           ; Karakteri yazdırmak için SATIRYAZ'a dallan
BITIR
    GOTO BITIR              ; Bilgiyi ekranda sürekli göstermek için sonsuz
                            ; döngüye gir
MESAJ
    ADDWF PCL,F
    RETLW 'E'
    RETLW 'L'
```

```
    RETLW 'E'
    RETLW "K"
    RETLW "T"
    RETLW "R"
    RETLW "O"
    RETLW "N"
    RETLW "I"
    RETLW "K"
    RETLW H'00'              ; Son karakter olarak 0 tanımlandı
GECIKME
    MOVLW H'05'
    MOVWF SAYAC1
D1
    MOVLW H'FF'
    MOVWF SAYAC2
D2
    DECFSZ SAYAC2,F
    GOTO D2
    DECFSZ SAYAC1,F
    GOTO D1
    RETURN
    END
```

Bu programın Program 46'dan farkı LCD'ye yazdırılacak karakterlerin bir çevrim tablosu kullanılarak çağırılmasıdır. İlk olarak yapmamız gereken yine LCD ayarlarını yapmaktır. Bu sebeple önce ekran temizlenir, kursör ekranın başına alınır, tek satır kullanılacağı, 8 bitlik bağlantı yapılacağı ve karakterlerin 5x7 fontunda görüntülenmek istediği belirtilir. Sonra ekran açılır ve karakterlerin yazdırılması işlemine geçilir. Bu programda **SAYICI** adında bir register oluşturulmuş ve içerisinde yazdırılacak karakterlerin sıra numarası tutulmuştur. **SAYICI** önce sıfırlanmış ve çevrim tablosu çağrılarak 0. değer olan 'E' harfinin kodu **W**'ya yeni değer olarak yüklenmiştir. Burada **W**'daki değer ile H'00' bilgisi arasında lojik VEYA işlemi yapılmıştır. 'E' harfinin kodu Tablo 16'dan görüleceği gibi 01000101'dir. Bu bilgi ile 00000000 arasında VEYA işlemi yapalım.

$$\begin{array}{r} 0100\ 0101 \\ +\ 0000\ 0000 \\ \hline 0100\ 0101 \end{array}$$

Görüldüğü gibi sonuç E'nin kodu olan 01000101 bilgisini değiştirmeden 0 olup olmadığını kontrol etmiştir. Sonuç 0 olmadığı için Zero Flag=0'dır. Bu yüzden program bir satır atlar ve **W**'daki kodu **PORTB**'ye yükler, yani LCD'ye gönderir. Enable ucu tetiklenerek karakterin LCD'de görüntülenmesi sağlanır.

Daha sonra bir sonraki 'L' karakterini almak için **SAYICI**'nın değeri 1 yapılmalıdır. Bu amaçla **INCF** komutu ile artırma işlemi yapılır. Yeniden **SATIRYAZ** etiketine dallanılarak bu kez 'L' harfinin kodu çevrim tablosundan alınarak gösterilir. Bu işlem son karakter olarak tanımlanan 0 gelene kadar devam eder. Çevrim tablosu çağrılıp H'00' bilgisi **W**'ya yüklendiğinde lojik VEYA işlemi sonucunun nasıl olacağına bakalım.

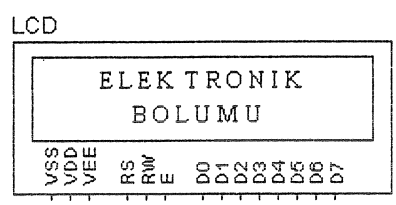
```
  0000 0000
+ 0000 0000
-----------
  0000 0000
```

Görüldüğü gibi VEYA işleminin sonucu 0 olacak, bu durumda Zero Flag=1 olacağından program **BITIR** etiketine dallanacak ve sonsuz döngüye girerek ekranda sürekli ELEKTRONIK yazması sağlanacaktır.

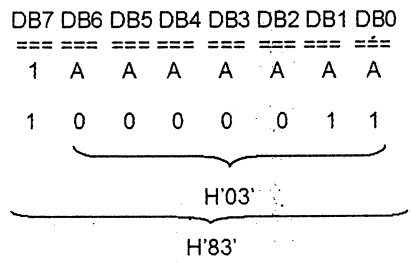
PROGRAM 48: 8 Bit ile LCD'nin İlk Satırının Ortasına ELEKTRONIK İkinci Satırın Ortasına BOLUMU Yazan Program

```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
SAYAC1 EQU H'0C'            ; H'0C' adresinde SAYAC1,
SAYAC2 EQU H'0D'            ; H'0D' adresinde SAYAC2 isimli bir register oluştur
    CLRF PORTB              ; PORTB'yi temizle
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    CLRF TRISA              ; PORTA'yı çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
BASLA
    MOVLW H'01'             ; Ekranı temizler ve kursörü
    CALL KOMUTYAZ           ; ekranın başına (adres 0) getirir
    MOVLW H'38'             ; 8 bitlik bağlantı, 2 satır ve
    CALL KOMUTYAZ           ; 5x7 karakter fontu seçilir
    MOVLW H'0C'             ; Ekranı açar, kursörü kapatır ve
    CALL KOMUTYAZ           ; kursör yanıp sönmez
    MOVLW H'83'             ; Kursörü 1. satırın 4. sütununa
    CALL KOMUTYAZ           ; konumlandır
    CALL SATIR_1            ; Ekrana 1. satırı yazdırma kısmına dallan
    CALL GECIKME
    CALL SATIR_2            ; Ekrana 2. satırı yazdırma kısmına dallan
DEVAM
    GOTO DEVAM              ; Bilgiyi ekranda sürekli göstermek için sonsuz
                            ; döngüye gir
KOMUTYAZ
    BCF PORTA,1             ; RS=0 yaparak
    MOVWF PORTB             ; komut girileceğini bildir
    BSF PORTA,0             ; E=1 ──┐
    CALL GECIKME            ;       │
    BCF PORTA,0             ; E= 0  └──
    RETURN
VERIYAZ
    BSF PORTA,1             ; RS=1 yaparak
    MOVWF PORTB             ; veri girileceğini bildir
    BSF PORTA,0             ; E=1 ──┐
    CALL GECIKME            ;       │
    BCF PORTA,0             ; E= 0  └──
    RETURN
SATIR_1
    MOVLW 'E'               ; E
    CALL VERIYAZ
    MOVLW 'L'               ; L
```

```
    CALL VERIYAZ
    MOVLW 'E'               ; E
    CALL VERIYAZ
    MOVLW 'K'               ; K
    CALL VERIYAZ
    MOVLW 'T'               ; T
    CALL VERIYAZ
    MOVLW 'R'               ; R
    CALL VERIYAZ
    MOVLW 'O'               ; O
    CALL VERIYAZ
    MOVLW 'N'               ; N
    CALL VERIYAZ
    MOVLW 'I'               ; I
    CALL VERIYAZ
    MOVLW 'K'               ; K
    CALL VERIYAZ
    RETURN
SATIR_2
    MOVLW H'C5'             ; Kursörü 2. satırın 6. sütununa
    CALL KOMUTYAZ           ; konumlandır
    MOVLW 'B'
    CALL VERIYAZ            ; B
    MOVLW 'O'
    CALL VERIYAZ            ; O
    MOVLW 'L'
    CALL VERIYAZ            ; L
    MOVLW 'U'
    CALL VERIYAZ            ; U
    MOVLW 'M'
    CALL VERIYAZ            ; M
    MOVLW 'U'
    CALL VERIYAZ            ; U
    RETURN
GECIKME
    MOVLW H'05'
    MOVWF SAYAC1
D1
    MOVLW H'FF'
    MOVWF SAYAC2
D2
    DECFSZ SAYAC2,F
    GOTO D2
    DECFSZ SAYAC1,F
    GOTO D1
    RETURN
    END
```



Bu programın ilk programlardan farkı ekranda 2 satır görüntülenmesi, ayrıca karakterlerin satırın en başından değil istenilen farklı bir yerinden itibaren yazdırılmasıdır. 2 satır kullanımı seçmek için Fonksiyon Seti ayarlarındaki N=1 yapılmış, böylece Program 46'daki H'30' kodu H'38' olarak değişmiştir. Elektronik yazısının 1. satırın 4. sütunundan başlaması için kursör H'03' adresine konumlandırılmıştır. H'03' adresinin seçilmesi DDRAM Adres Ayarı konusunda da anlatıldığı gibi;



şeklinde olur. Bu yüzden H'03' yazılmaz, H'83' yazılır.

Aynı şekilde 2. satırın 6. sütununun adresi H'45' olduğundan yazılması gereken kod H'C5' olur. 1. satıra yazdırma işlemi sonunda 2. satıra geçerken H'C5' komutu icra edilmelidir.

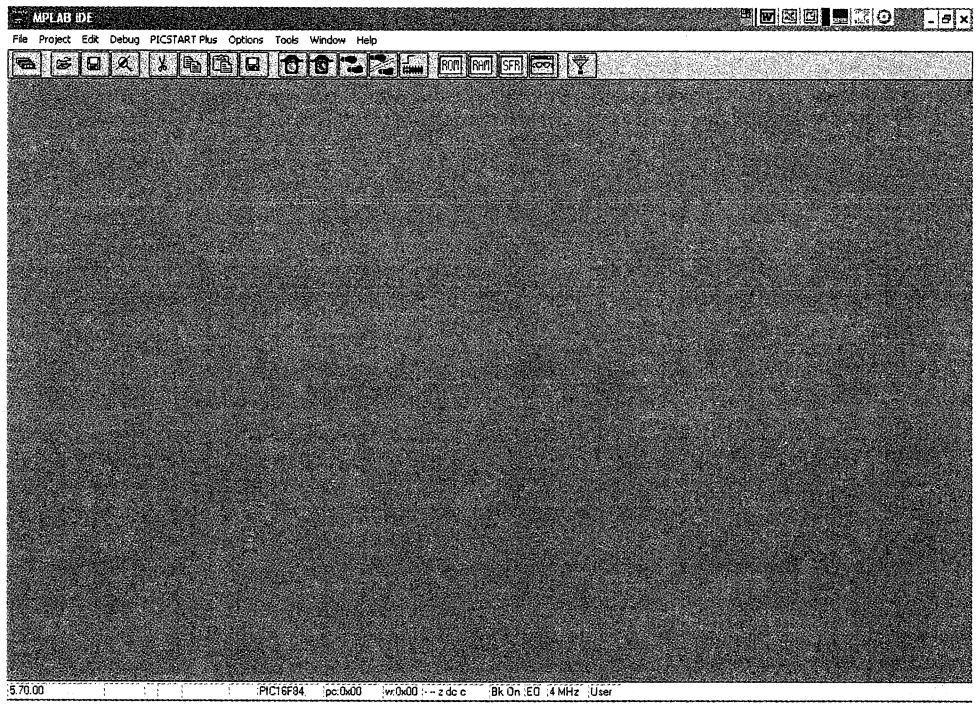
Bölüm 14

MPLAB Programı

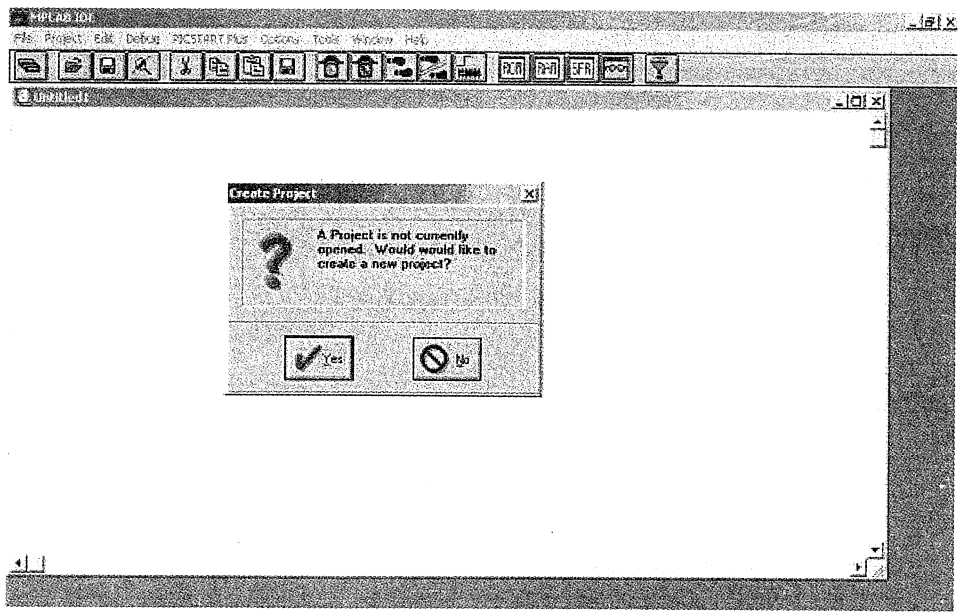
MPLAB programı assembly dilinde yazdığımız programları derleyen yani hexadecimal kodlara çeviren bir programdır. Derleyici (assembler) programları sayesinde makine dilinden daha üst seviyeli dillerle program yazmamız mümkün olur.

ŞEKİL 49: Assembler (derleyici) programlarının işlevi.

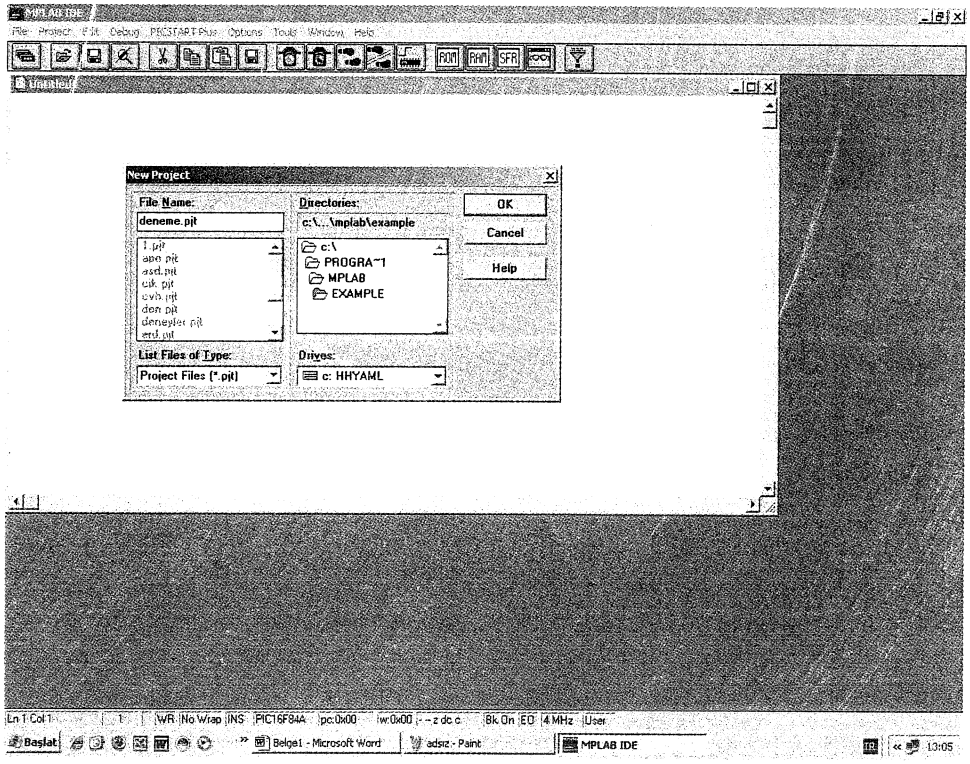
Bu kitapta MPLAB programının bilgisayarınıza kurulumu anlatılmayacaktır. Programı Internet'teki pek çok siteden ücretsiz olarak indirebilirsiniz. Bilgisayarınıza kurulum işlemini tamamladıktan sonra programı açtığınızda aşağıdaki ekran görüntüsüyle karşılacaksınız.



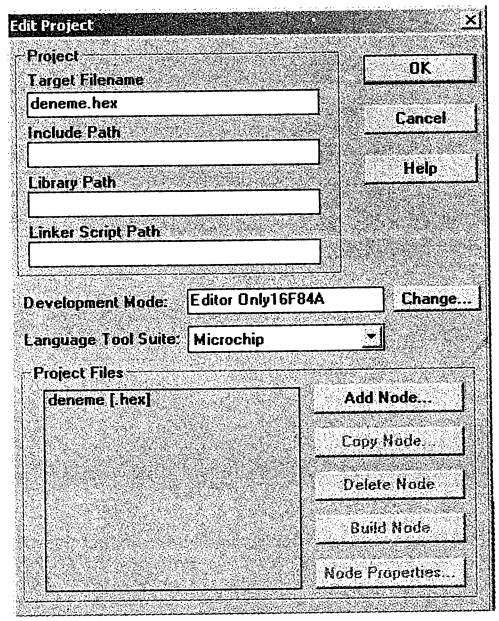
Önce yeni bir proje dosyası oluşturmamız gerekir. Bunun için File'dan New seçeneğini tıklayınız. Yeni bir proje oluşturmak isteyip istemediğinizi sorgulayan bir pencere ekrana gelecektir.



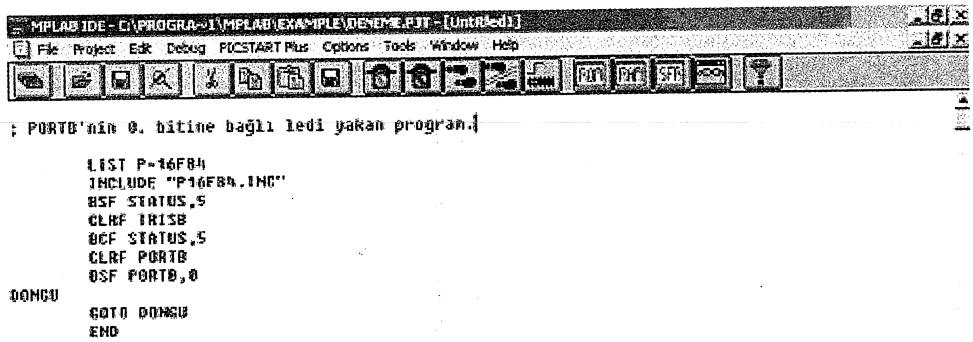
Yes seçeneğini tıkladığınızda yeni projeyi nerede oluşturmak istediğinizi belirteceğiniz bir pencere gelir. Burada **C:/Program Files/MPLAB/Example** seçilmiş, yani **Example** klasörüne kaydedilmek istediği belirtilmiştir. **MPLAB** klasörü içerisinde yeni bir klasör oluşturup programlarınızı burada saklamanız da mümkündür. Projemizin ismini deneme olarak belirleyip File Name kısmında **deneme.pjt** olarak yazdık



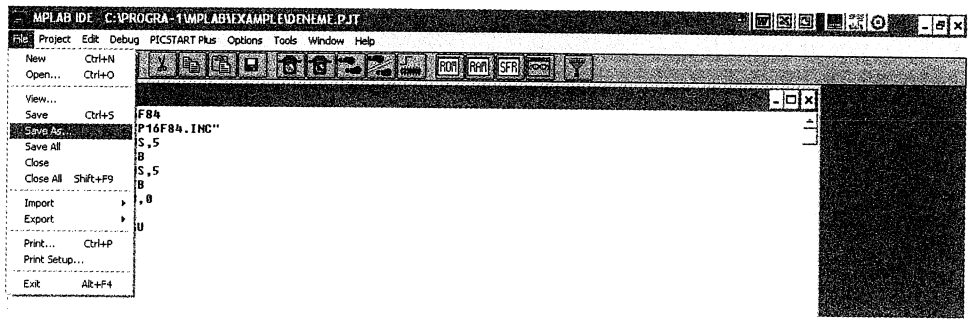
OK dediğimiz zaman aşağıdaki Edit Project penceresi ekrana gelecektir. OK diyerek bu pencereyi bu aşamada doğrudan geçiyoruz.

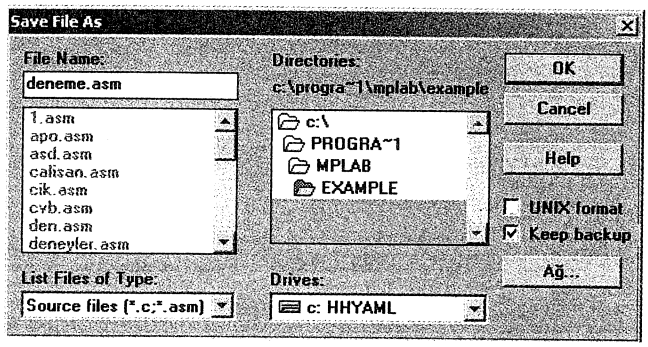


Ekranda boş bir sayfa göreceksiniz. Bu text editöründe programımızı aşağıdaki şekilde yazıyoruz. Programı yazarken açıklamaların başına ; işareti koymak, etiketleri 1. sütuna komutları 2. sütuna yazmak gibi kuralları yerine getirdiğimizden emin olalım.

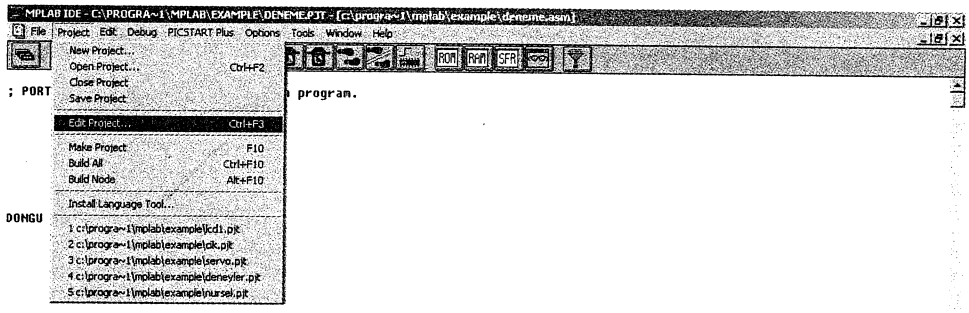


Programımızı yazdıktan sonra sıra kaydetme işlemine gelir. File'dan Save As seçeneğini tıkladığımızda aşağıdaki ekran gelecektir.

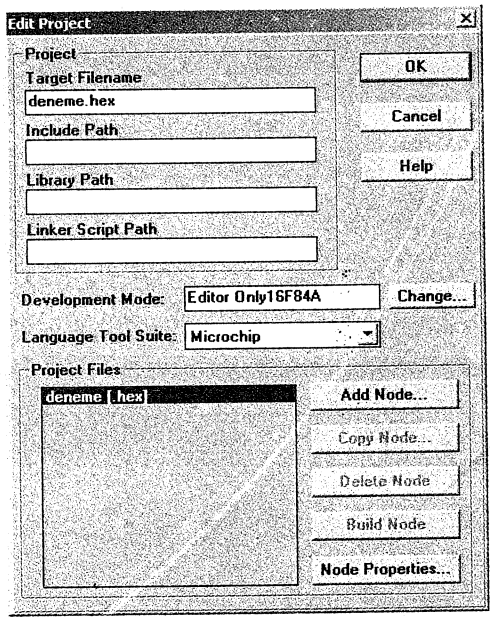




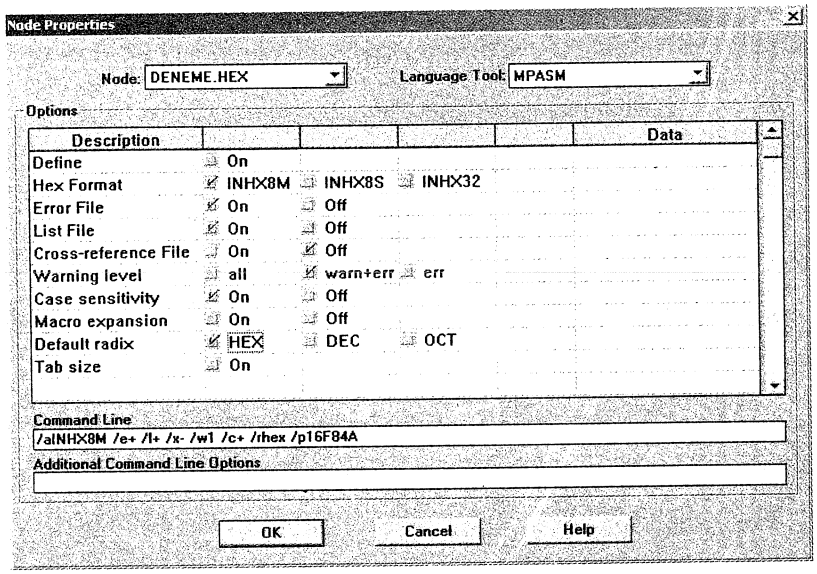
Burada Directories kısmında biraz önce seçtiğimiz konumu yani **C:/Program Files/MPLAB/Example**'ı tıklayıp File Name kısmına **deneme.asm** yazarak ve OK diyerek dosyamızı kaydediyoruz. Dosyanın uzantısının **.asm** olarak kaydettiğimize dikkat edin.



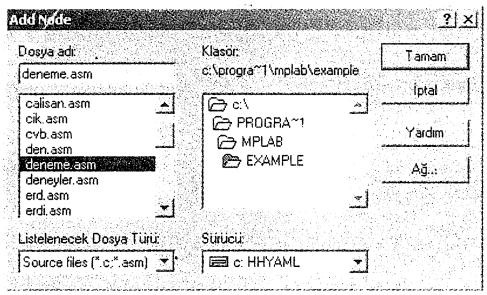
Dosyamızı kaydettikten sonra Project'den Edit Project seçeneğini tıkladığımızda aşağıdaki pencere ekrana gelir.



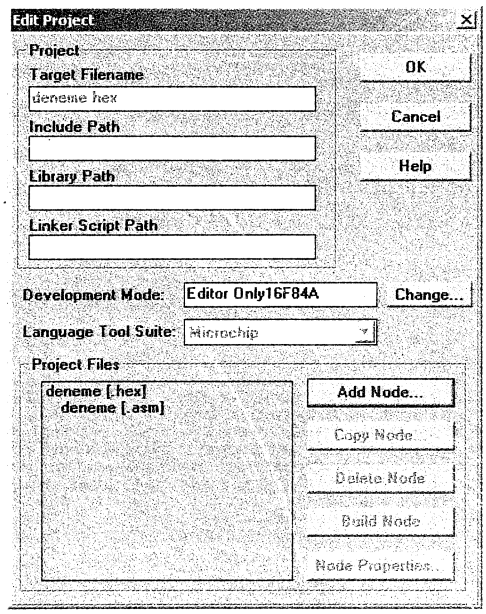
Project Files kısmından deneme[.hex] dosyasını üzerine bir kez tıklayarak seçtikten sonra sağ taraftaki Node Properties sekmesini tıkladığımızdaki aşağıdaki pencere ekrana gelir.



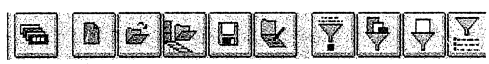
Gerekli işaretlemeleri yukarıdaki şekilde yaptıktan sonra OK diyerek bu pencereden çıkıyoruz. Ekrana yeniden Edit Project penceresi gelecektir. Bu kez yine sağ taraftaki Add Node sekmesini tıklıyoruz. Yeni açılan pencerede Dosya Adı kısmından kendi dosyamızı yani **deneme.asm**'yi seçip OK düğmesini tıklıyoruz.



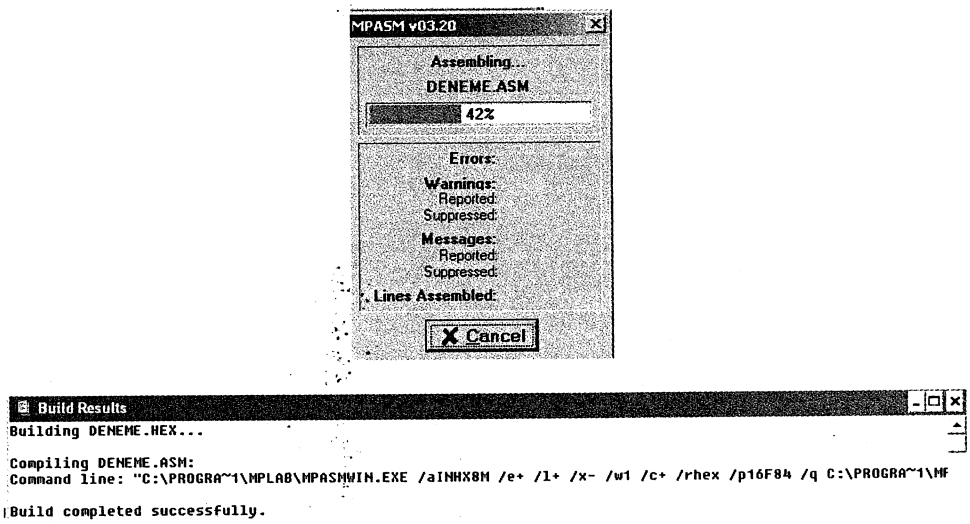
Bu onaylama işleminden sonra yeniden ekranda beliren Edit Project penceresinde Project Files kısmında **deneme.hex** ve **deneme.asm** olmak üzere iki dosya bulunduğunu göreceksiniz. OK tuşuna basarak bu pencereden çıkıp ana ekranımıza dönüyoruz.



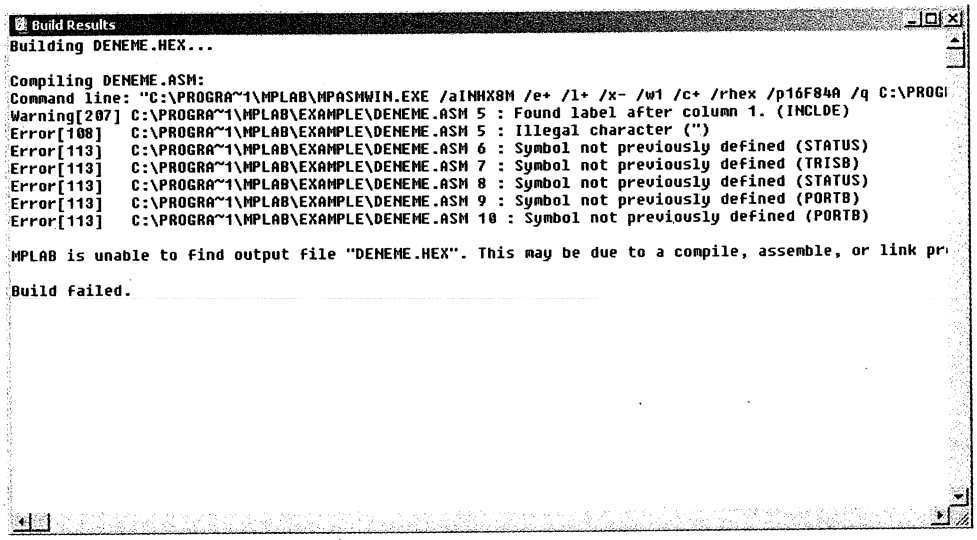
Ana ekranda Menü araç çubuğunun hemen altındaki simgesine birkaç kez basın. Her basışınızda simgenin sağ tarafındaki butonların değiştiğini göreceksiniz. Programın derlenmesi ile ilgili işlemleri kapsayan araç çubuğu aşağıdaki araç çubuğudur.



Bu araç çubuğu üzerinde sağdan üçüncü butona basıldığında yazdığımız program derlenir. Ekranda derleme işleminin yapıldığına dair bir pencere belirir. Eğer programda hata yoksa programın yüzde kaçının derlendiğini gösteren kısım yeşil renk alır. Daha sonra programın başarı ile derlendiğini belirten Build Completed Successfully mesajı görülür.



Eğer programımızda bir hata varsa yüzdeyi gösteren kısım kırmızı renk alır ve daha sonra Build Results yani derleme sonuçları penceresi açılarak derleme işleminin yapılamadığını belirten Build Failed mesajı görülür. Bu pencerede programdaki hatalı satırlar ve hata kaynağı da görülmektedir. Bu şekilde hatalı satırları düzeltmeniz kolaylaşır.



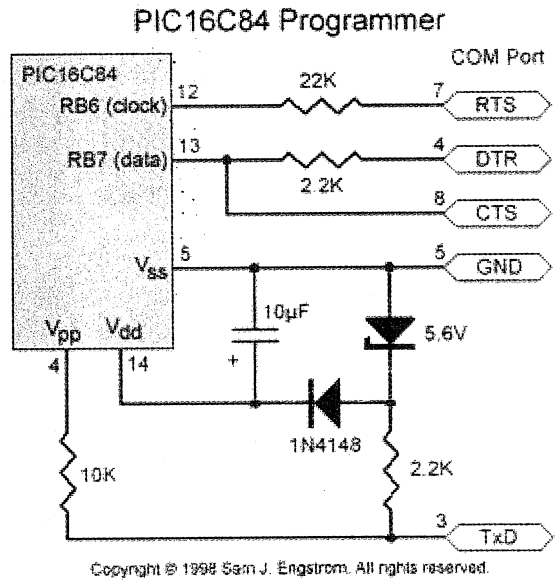
MPLAB programı programlardaki yazım veya kullanım hatalarını tespit eder, mantık hatalarını tespit edemez. Örneğin yazdığımız programda **INCLUDE** komutu **INCLDE** şeklinde yazılarak bir yazım yanlışı oluşturulmuş ve yukarıdaki hata mesajı elde edilmiştir. Ancak örneğin, bir step motor programında motora gönderilecek kodlar yanlış yazılmış olsun. Bu durumda MPLAB programı derler ve başarı ile derlendiği mesajını verir. Programı entegreye yükleyip çalıştırdığınızda ise kodlar yanlış olduğundan motor dönmeyecektir.

Bölüm 15

ICPROG Programı

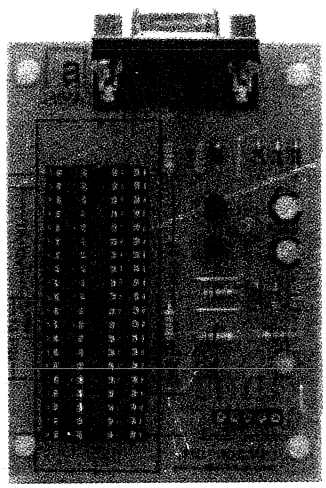
MPLAB programı ile Assembly dilinde yazdığımız kodları mikrodenetleyicinin anlayacağı hexadecimal kodlara çevirdik. Artık programımız entegreye yüklenmeye hazır bir hale gelmiştir. Derlenmiş programı PIC'e yüklemek için ICPROG programını kullanacağız. Yükleme işlemi için ayrıca bir programlama kartına ihtiyacımız var.

PIC16F84 entegresini aşağıdaki programlama devresini kullanarak bilgisayarınızın seri portu üzerinden programlayabilirsiniz.

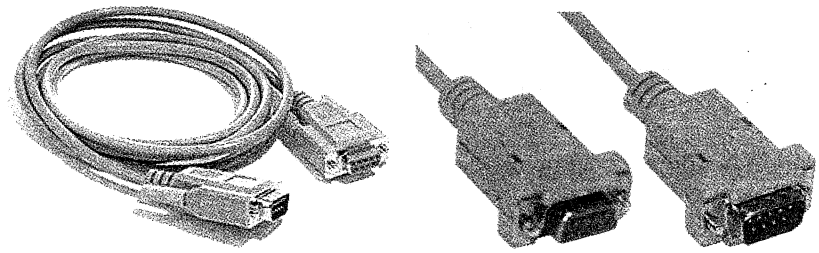


ŞEKİL 50: PIC programlama kartı şeması.

Devre yapımıyla uğraşmak istemezseniz piyasada pekçok hazır programlama kartı bulunmaktadır. Bu kartlarla seri port veya USB portu üzerinden PIC entegresine istediğiniz programı yükleyebilirsiniz.



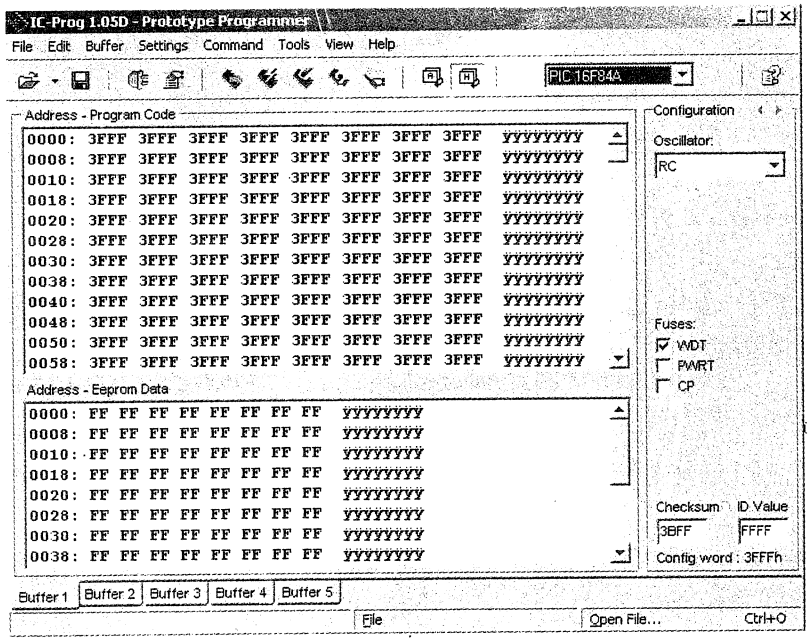
ŞEKİL 51: Altaş Elektronik firmasına ait PIC PROG V4 programlama kartı.



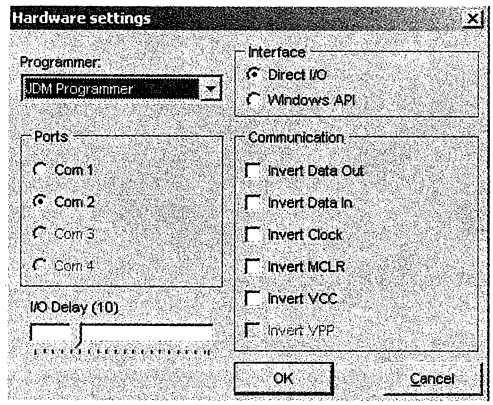
ŞEKİL 52: RS232 seri port kablosu.

Yukarıdaki gibi bir karta sahip olduktan sonra yapmanız gereken tek şey bir RS232 kablosu alıp bir ucunu bilgisayarınızın seri portuna diğer ucunu ise elinizdeki karta takmaktan ibarettir. Programlama kartı üzerindeki soket pek çok PIC'i programlayabilecek şekilde seçilmiştir. PIC16F84 entegremizi kart üzerine doğru bir biçimde yerleştiriyoruz. Artık yükleme işlemi için gereken donanım hazırdır.

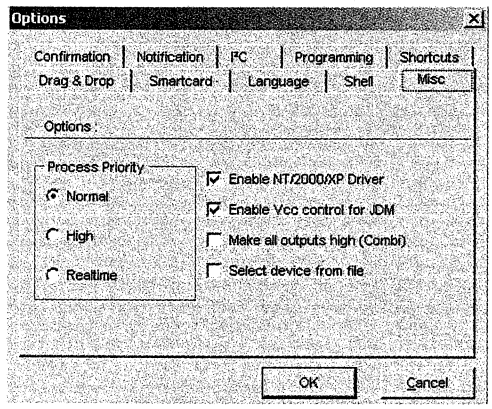
ICPROG'u bilgisayarınızı kurduktan sonra programı açtığınızda aşağıdaki ekran görüntüsüyle karşılaşacaksınız. Hiçbir program açılmadığı zaman ekranda 3FFF kodlarını göreceksiniz.



ICPROG üzerinde önce birkaç donanım ayarı yapmanız gerekir. Bunun için Menü Çubuğu üzerindeki Ayarlar'dan Donanım'ı tıklatın. Donanım Ayarları penceresi açılacaktır. Burada Programlayıcı kısmından JDM Programmer'ı seçin. Daha sonra Ports kısmında programlama kartını bağladığınız portu işaretleyin.

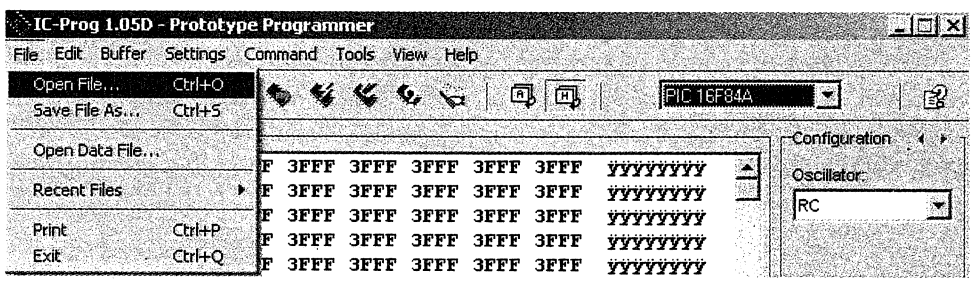


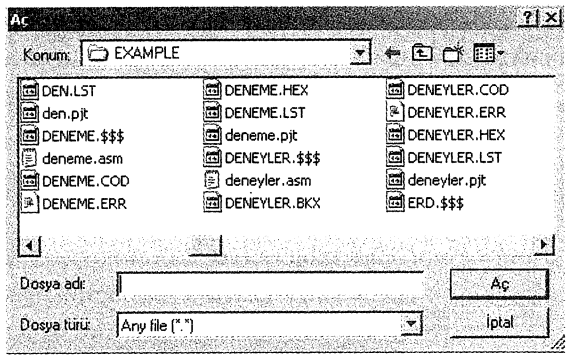
Eğer bilgisayarınızda Windows XP işletim sistemi kurulu ise programı kurduktan sonra açmaya çalıştığınızda Privileged Instruction hata penceresi ekrana gelecek ve programlama yapmanıza izin verilmeyecektir. Bunu ortadan kaldırmak için yine Ayarlar'dan Seçenekler'i tıklayın ve açılan pencerede Misc (Karışık) sekmesini tıklayın.



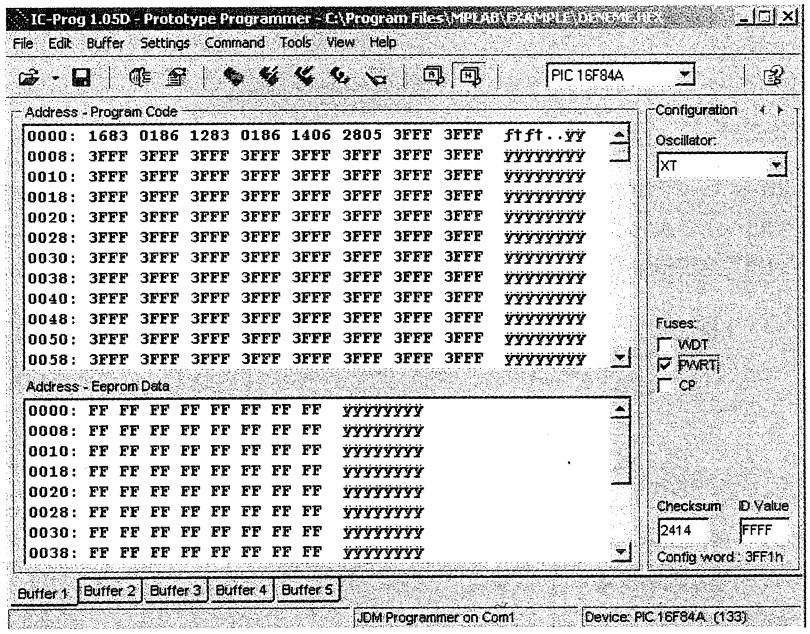
Enable NT/2000/XP Driver seçeneğini aktif hale getirin. OK tuşuna bastığınızda artık Windows XP işletim sistemi ile programı kullanmanız mümkün olacaktır.

Menü araç çubuğu üzerindeki File seçeneğinden Open File'ı veya hemen altındaki simgesini tıklayınız. Aç penceresi ekrana gelecektir.





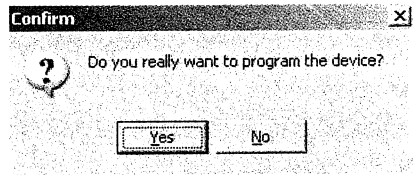
Bu pencerede Konum kısmında dosyamızın bulunduğu yeri belirtiyoruz. Örnek olarak kullandığımız *deneme* dosyası **Example** klasörü içerisinde bulunduğundan bu kısımda **Example** klasörünü seçiyoruz. Aşağıda **Example** klasörü içerisinde bulunan tüm dosyalar listelenecektir. Bu dosyalardan yazdığımız programa ait hex dosyasını yani **deneme.hex**'i seçip Aç butonuna basıyoruz.



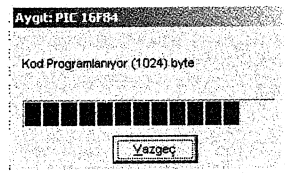
Ekranda Adress-Program Code kısmında deneme.hex dosyasına ait hexadecimal kodlar görülecektir. Kullanacağımız PIC'i PIC16F84A şeklinde seçtikten sonra kullanacağımız osilatör tipini belirtmemiz gerekir. Bu kitaptaki uygulamalarda kristal osilatör kullanıldığı için XT seçeneğini tıklıyoruz. Fuses kısmında üç ayrı seçenek bulunmaktadır. WDT (Watchdog Timer), PWRT (Power-up Timer) ve CP (Code Protect). Programda WDT kullanılıyorsa WDT seçeneği işaretli olmalıdır. PWRT özelliği, PIC'e enerji verildikten sonra 72 msn'lik bir bekleme süresi sağlar. Bu süre VDD yani besleme geriliminin kararlı bir seviyeye çıkmasını beklemek içindir. CP yani kod koruması ise PIC entegresine yüklediğimiz programın başkaları tarafından okuna-

mamasını sağlayan bir özelliktir. Eğer CP seçeneği işaretlenirse entegre okutulduğu zaman program kodları gözükmez. Böylece yazdığınız programı korumuş olursunuz.

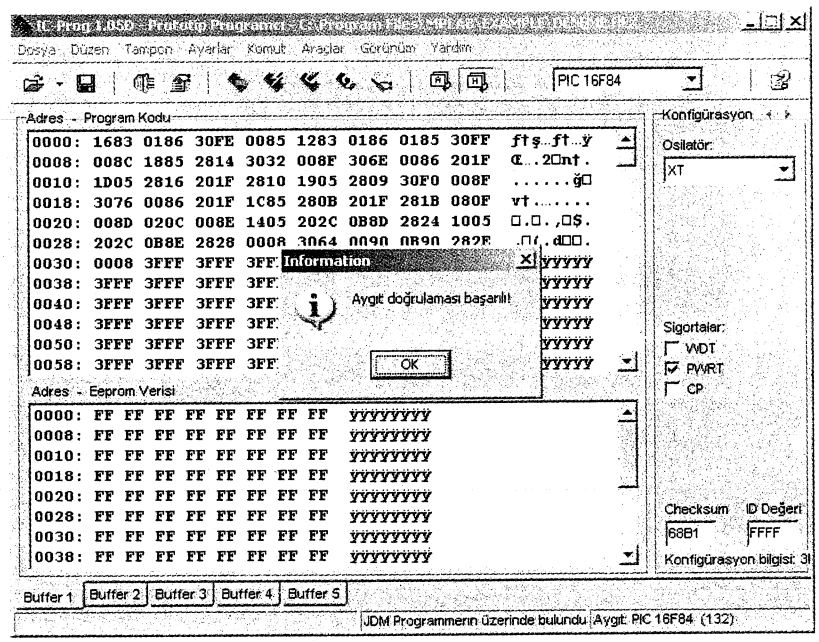
Programlama kartınızı seri port veya USB üzerinden bilgisayarınıza bağladığınızdan emin olduktan sonra artık kart üzerine yerleştirdiğiniz PIC'e program kodlarını gönderebilirsiniz. Bunun için simgesine yani Program All seçeneğine tıklamanız yeterlidir. Bu butona tıkladığınızda "Gerçekten aygıtın programlanmasını istiyor musunuz?" şeklinde bir onay penceresi gelecektir. Yes butonuna tıklayarak onay verildiğinde programı entegreye yüklemeye başlayacaktır.

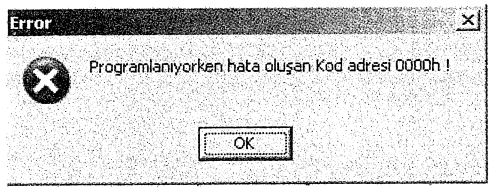


Programın yüklenmesi esnasında aşağıdaki ekran mesajı görülür.



Program entegreye başarı ile yüklenirse ekranda "Aygıt Doğrulaması Başarılı" mesajı görülür. Eğer yükleme sırasında bir problem meydana gelirse, örneğin kullandığınız PIC bozuksa ekranda hata (error) mesajı görülecektir.





Programlanmış bir PIC içerisindeki kodları okumak için (Read All) simgesine, programı silmek içinse (Erase All) simgesine tıklamanız yeterlidir. (Verify All) simgesi ise tampon bellekteki program ile PIC içerisindeki programı karşılaştırır.

Bölüm 16

PROTEUS Programı

PROTEUS ISIS programı bir elektronik çizim,animasyon, simülasyon ve baskı devre (PCB) çizim programıdır. Herhangi bir deneme kartı ile yazdığımız programları gerçek elektronik elemanlar üzerinde deneme imkanı buluruz. Ancak programlamaya yeni başlayanlar için bu işlem biraz yorucu olmaktadır. Aynı anda hem programlama kartı, hem de deneme kartı olarak kullanılabilen kartlar vardır. Bunlar işimizi biraz kolaylaştırabilir. Ancak bu iki kart ayrı ayrı ise her defasında PIC'i deneme kartından söküp programlama kartına takmak, sonra tekrar deneme kartına geri takmak sıkıcı olmaktadır. Ayrıca entegrenin bacakları da zarar görebilmektedir. Bu yüzden programlama becerinizi geliştirmek için programlarınızı gerçek ortam yerine sanal ortamda denemek oldukça faydalı olabilir. İşte Proteus programı bu amaç için kullanılabilecek mükemmel bir araçtır.

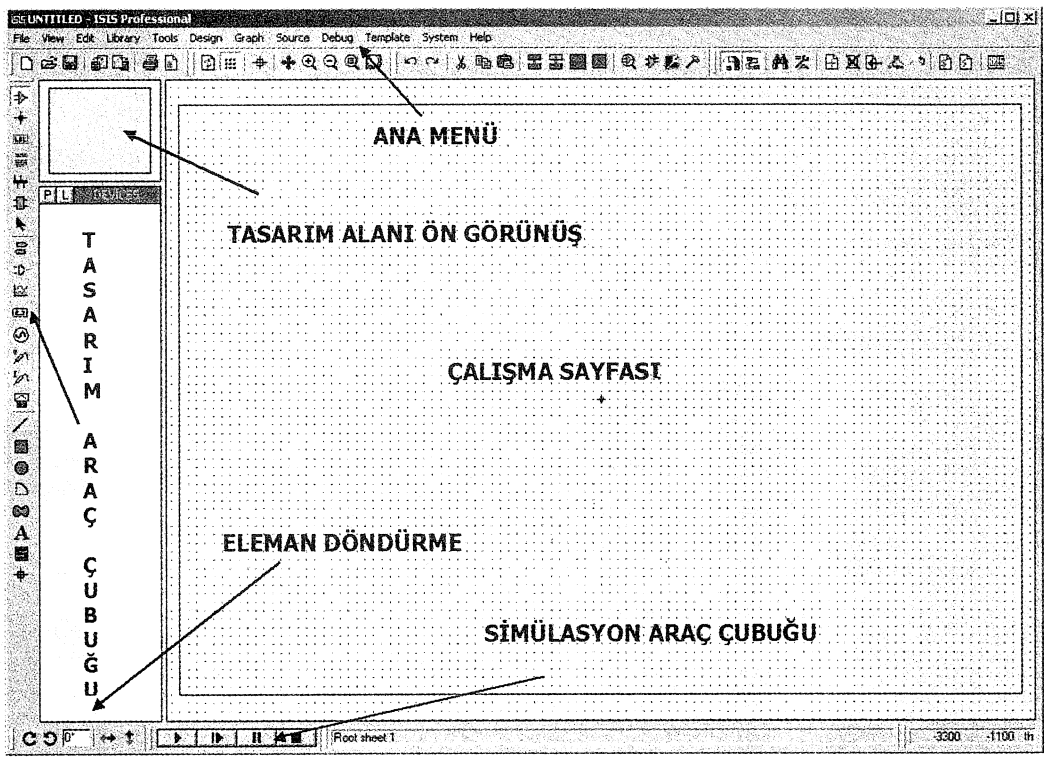
Proteus programının demo versiyonunu **www.labcenter.co.uk** adresinden indirebilirsiniz. Bu siteye girdikten sonra Downloads seçeneğini tıklayın ve ekrana gelen alt seçeneklerden Proteus 6 Demo seçeneğini seçin. **PRODEMO.EXE**'yi tıkladığınızda programı bilgisayarınıza diğer Windows programlarının kurulum aşamalarından farksız bir biçimde yükleyebilirsiniz.

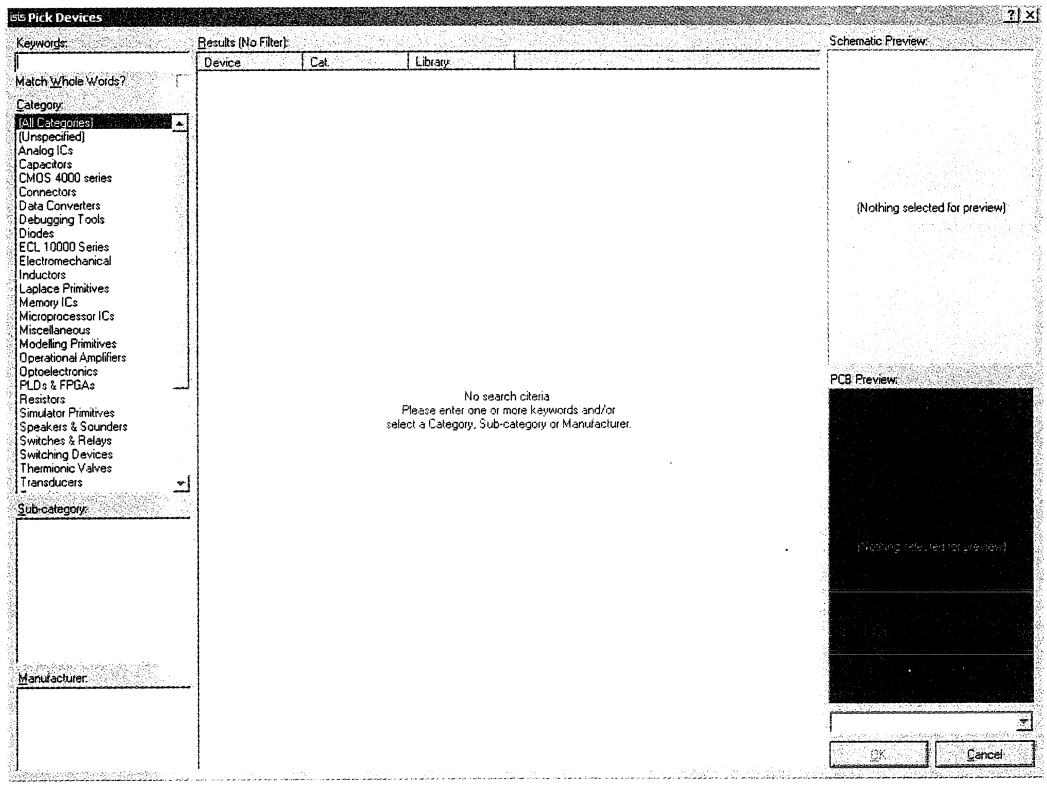
Proteus programında devremizi çizip programımızı yükledikten sonra sadece Play tuşuna basarak programımızın nasıl çalıştığını görebiliriz. Eğer bir problem varsa MPLAB'da gerekli düzeltmeleri yaptığımızda yapılan değişiklik otomatik olarak sanal entegremize yüklenecek ve yapmamız gereken tek şey yeniden Play tuşuna basmak olacaktır. Şimdi bu programda devrelerimizi nasıl simüle edebileceğimizi görelim.

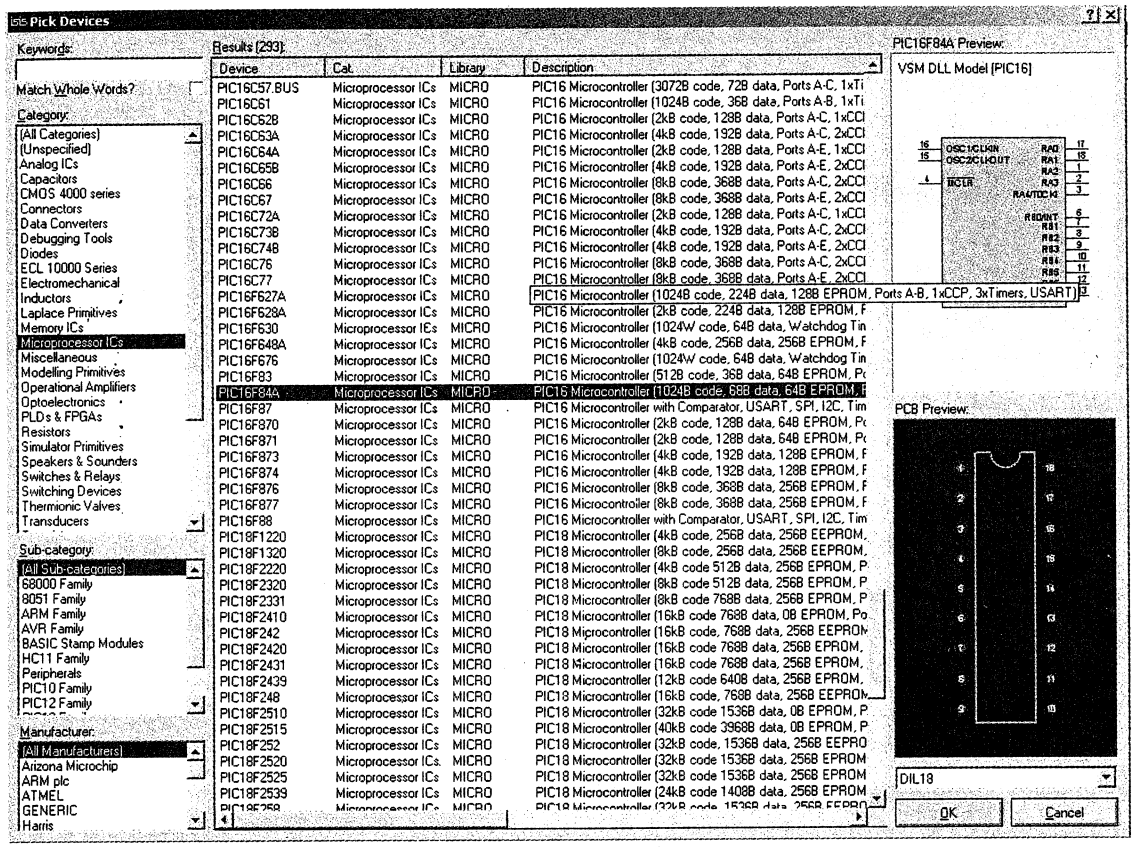
Proteus programı içerisinde iki alt program vardır: ISIS ve ARES. ARES baskı devre çizim programı olduğundan konumuzla ilgisi yoktur. Bu yüzden Proteus programı içerisindeki ISIS seçeneğini tıklayarak çizim,simülasyon ve animasyon kısmını açıyoruz.

Programı açtığınızda View Sample Designs şeklinde bir pencere gelebilir. Burada ISIS programında yer alan örnek çizim uygulamalarını görmek isteyip istemediğiniz sorulmaktadır. Evet diyerek bu örnekleri inceleyebilirsiniz. Hayır seçeneğine basarsanız karşı sayfadaki üst şekilde görüldüğü gibi boş bir çalışma sayfası ekrana gelecektir.

Tasarım alanına eleman çağırmak için Tasarım Araç Çubuğu üzerinde bulunan (component) simgesine tıklayın. Daha sonra sol taraftaki Tasarım Araç Çubuğunun hemen yanındaki P (Pick Devices) butonuna basın. Karşı sayfadaki alt şekilde görülen ekran görüntülenecektir. Sol taraftaki Category kısmında ISIS içerisindeki tüm kütüphaneler listelenecektir. Bu kütüphanelerden örneğin Microprocessors ICs kütüphanesini tıklayın. Results kısmında bu kütüphanedeki tüm elemanlar görüntülenecektir. Kullanmak istediğiniz elemanı çift tıklayarak çalışma sayfanıza alabilirsiniz.

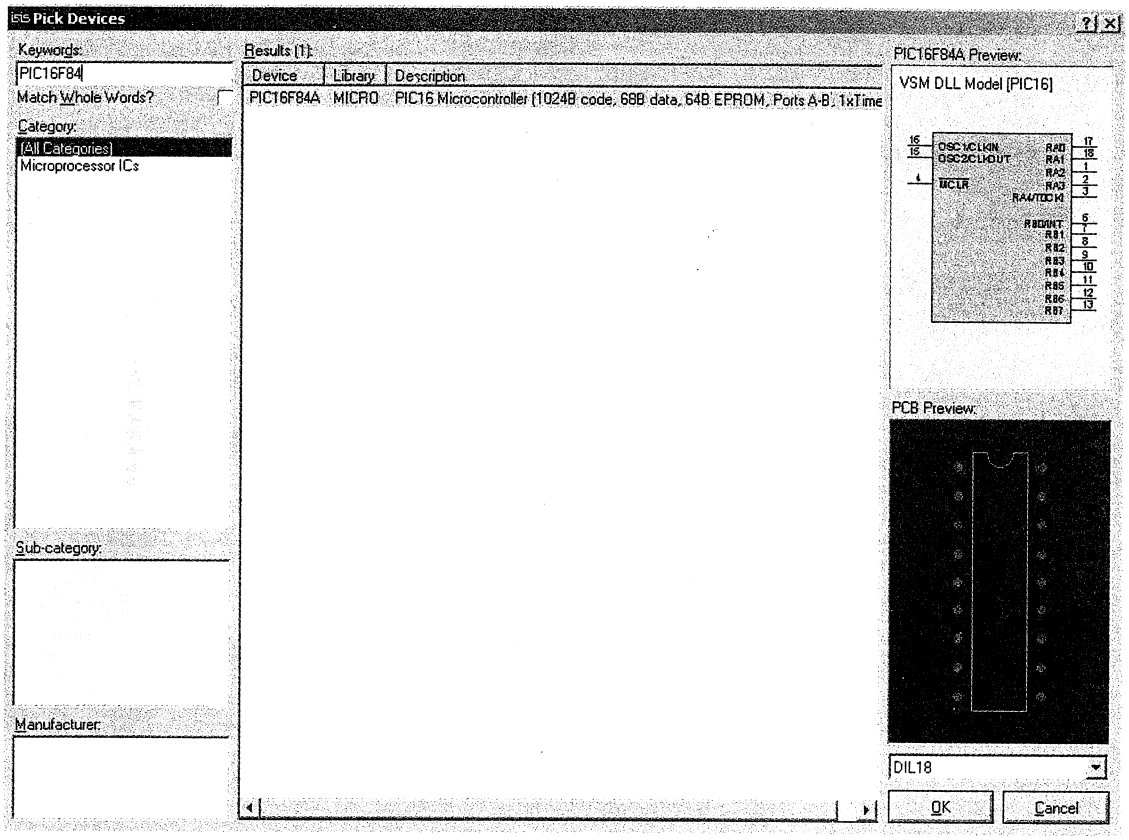

ANA MENÜ
TASARIM ALANI ÖN GÖRÜNÜŞ
ÇALIŞMA SAYFASI
TASARIM ARAÇ ÇUBUĞU
ELEMAN DÖNDÜRME
SİMÜLASYON ARAÇ ÇUBUĞU

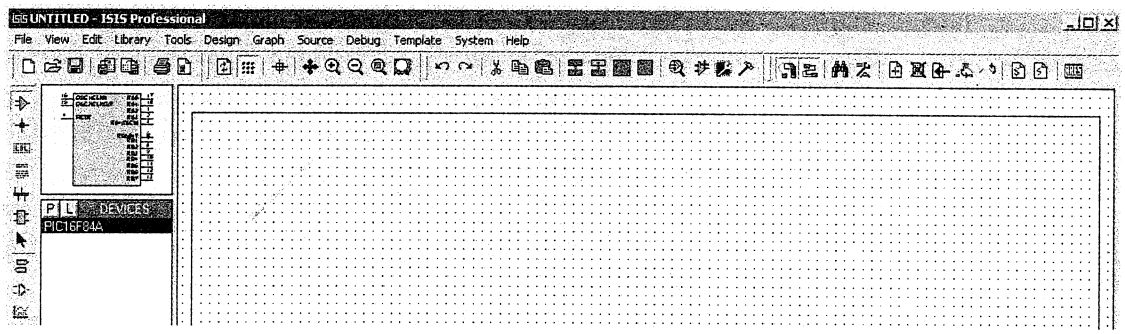

Pick Devices
Keywords:
Match Whole Words?
Category:
(All Categories)
(Unspecified)
Analog ICs
Capacitors
CMOS 4000 series
Connectors
Data Converters
Debugging Tools
Diodes
ECL 10000 Series
Electromechanical
Inductors
Laplace Primitives
Memory ICs
Microprocessor ICs
Miscellaneous
Modelling Primitives
Operational Amplifiers
Optoelectronics
PLDs & FPGAs
Resistors
Simulator Primitives
Speakers & Sounders
Switches & Relays
Switching Devices
Thermionic Valves
Transducers
Sub-category:
Manufacturer:
Results (No Filter):
Device
Cat.
Library
No search criteria
Please enter one or more keywords and/or
select a Category, Sub-category or Manufacturer.
Schematic Preview:
(Nothing selected for preview)
PCB Preview:
OK
Cancel



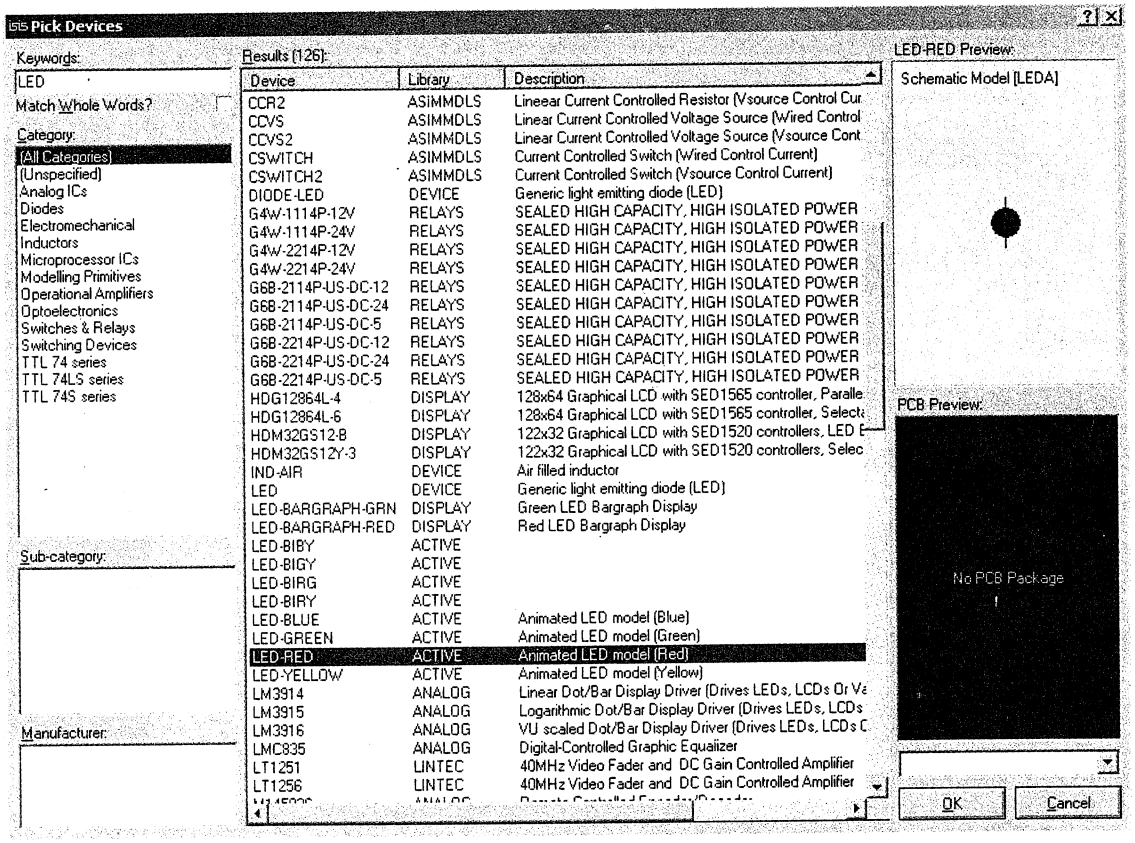
Eleman çağırmanın ikinci yolu P butonuna bastıktan sonra açılan penceredeki Keywords (Anahtar Kelimeler) kısmına istediğiniz elemanın ismini yazmaktır. Keywords kısmına PIC16F84A yazalım.

Orta kısımdaki Results bölümünde elemanın içinde yer aldığı kütüphanenin ismi ve eleman hakkında bilgiler yer almaktadır. PIC16F84'ün MICRO kütüphanesi içinde bulunduğu ve bellek bilgileri, port sayısı, vs. bilgileri ekranda görülmektedir. Sağ tarafta ise PIC16F84'ün pin yapısı ile baskı devre çiziminde kullanılacak PCB görünümü yer almaktadır. Bu ekranda OK butonuna basarak PIC16F84 entegresini çalışma sayfamıza aktarıyoruz. Karşı sayfadaki alt şekilde görüldüğü gibi Devices penceresine PIC16F84A yerleştirilmiş ve hemen üstünde elemanın görüntüsü verilmiştir.

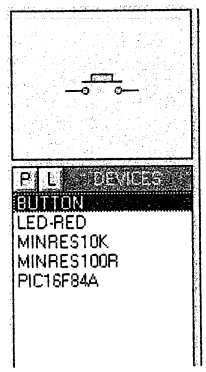




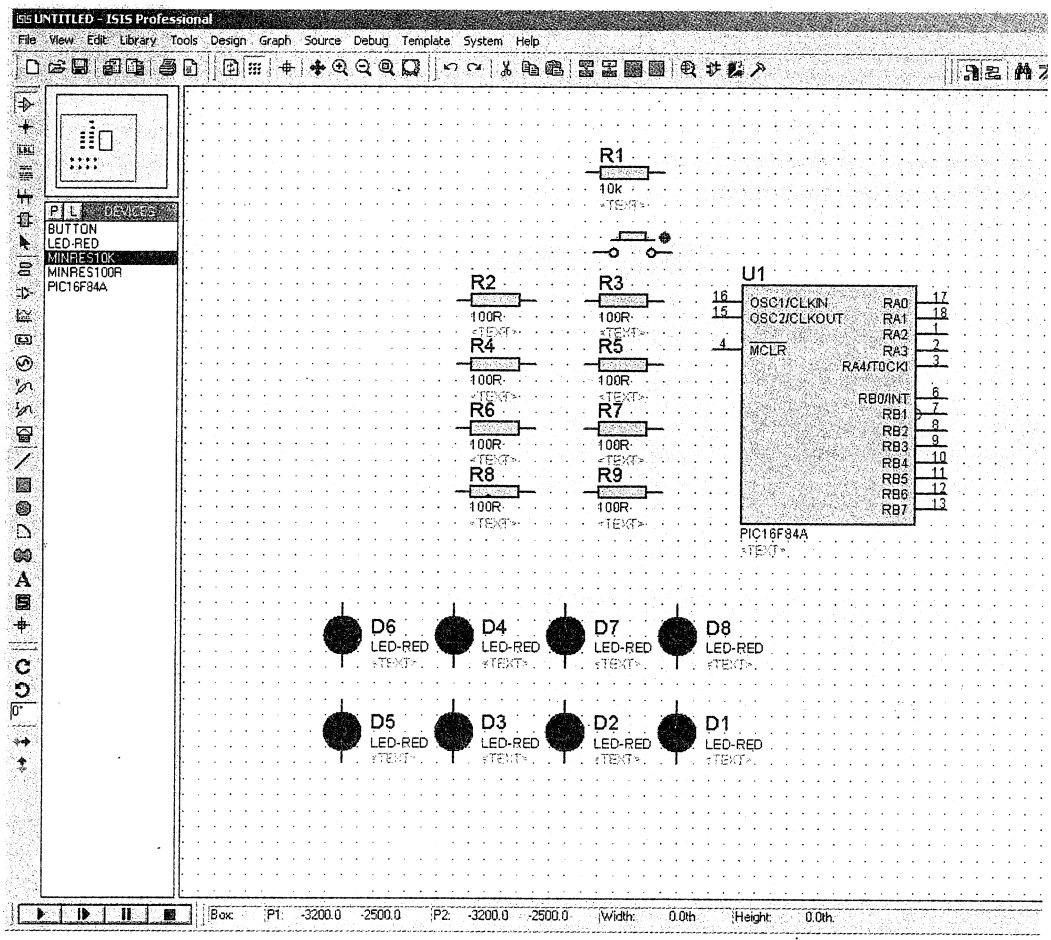
Şimdi de sayfamıza led ekleyelim. Yine P butonuna basarak açılan pencerede Keywords kısmına LED yazalım. Bu kez sağ tarafta birden fazla elemanın listelendiğini göreceksiniz. Çünkü tek çeşit led yoktur; renk, şekil, vs özellikleri bakımından farklı olan tüm ledler gösterilmektedir. Bu durumda Results kısmındaki elemanları tıklayarak her birinin tipini sağ taraftaki Preview kısmından inceleyebilir, böylece size en uygun elemanı aralarından seçebilirsiniz. LED-RED elemanını tıklayalım ve çalışma sayfamıza aktaralım.



Bu şekilde çizimimizde kullanacağımız tüm elemanları teker teker çalışma sayfamıza aktarıyoruz. Çalışma sayfamızdaki Devices kısmında seçtiğimiz tüm elemanlar listelenecektir. Yani devre çizimine başlamadan önce malzeme kutumuza gerekli elemanları topluyoruz.

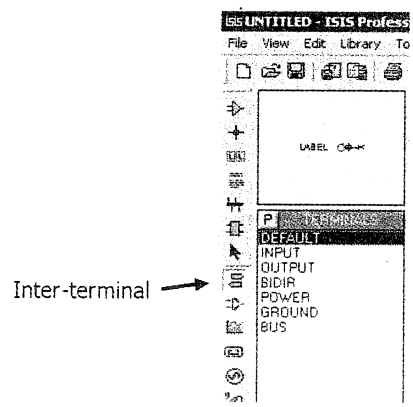


Listedeki 10K'lık direnci bir kez tıklayın. Hemen üstündeki kare içerisinde bu eleman görüntülenecektir. Bu durumda mouse ile çalışma sayfası içerisinde bir kez tıklarsanız 10K'lık direnç sayfa üzerine yerleştirilir. Diğer elemanları da aynı şekilde çalışma sayfanıza getirin.

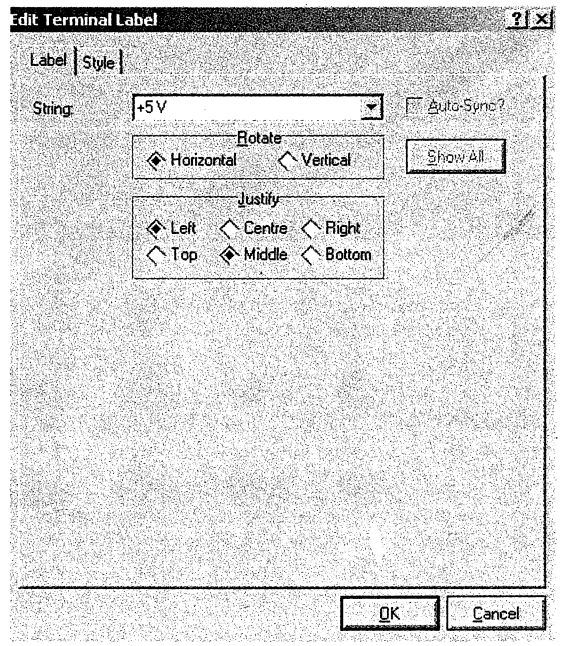


Eğer çalışma sayfanızdan istemediğiniz bir elemanı silmek isterseniz o eleman önce üzerine sağ tıklanarak seçilir. İkinci kez sağ tıklarsanız eleman silinir.

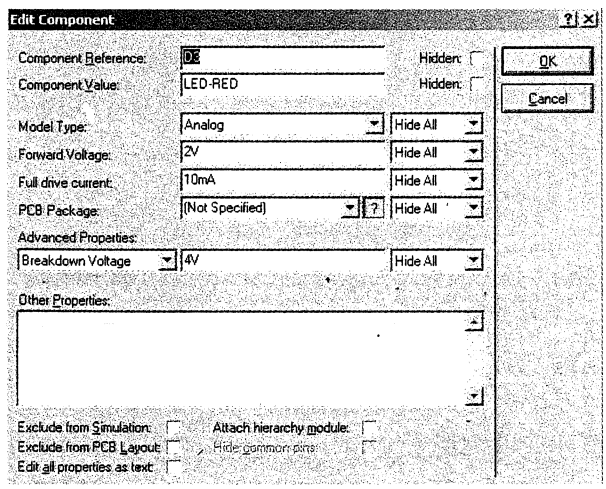
Şimdi de enerji bağlantıları için gerekli elemanları çalışma sayfamıza getirelim. Tasarım Araç Çubuğu üzerindeki simgesine tıkladığımızda terminal listesi gelecektir.



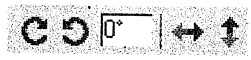
+VCC eklemek için Power'a, şase için GROUND'a bir kez tıklayarak çalışma sayfasına yerleştirin. +VCC'yi ekledikten sonra eleman özelliklerine girerek uygulayacağınız gerilimin değerini belirlemeniz gerekir. Bir elemanın özelliklerine girmek için Tasarım Araç Çubuğu üzerindeki simgesine yani Instant Edit Mode düğmesine tıklayın. Aşağıdaki Edit Component yani eleman düzenleme penceresi açılacaktır. Bu penceredeki string kısmına +5V yazarak 5V'luk besleme kullanmak istediğimizi belirtiyoruz.



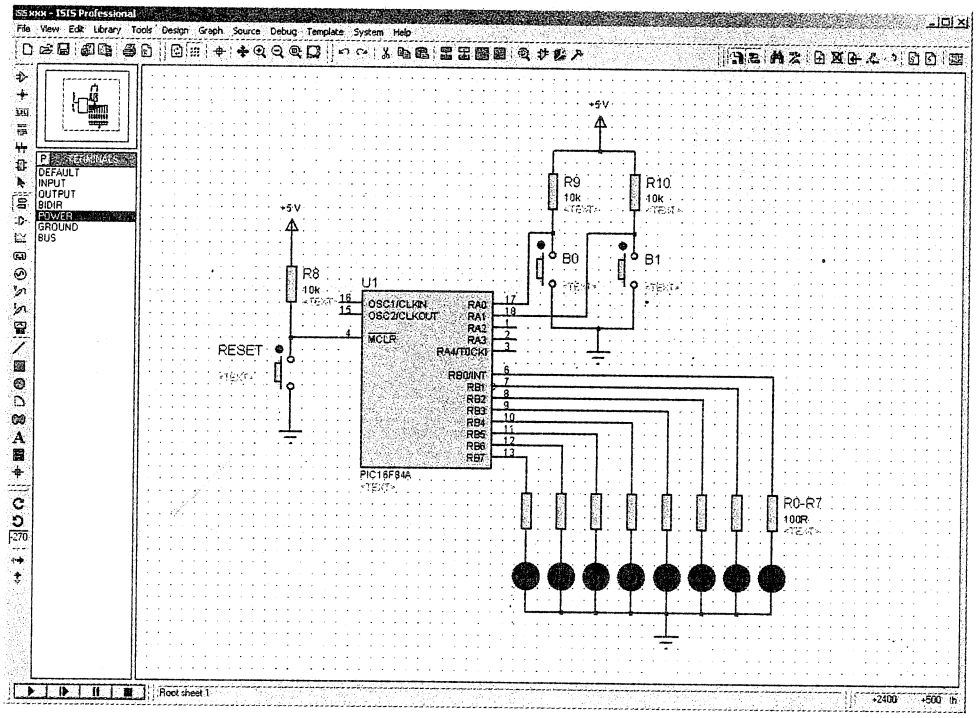
Edit Component penceresinde gelen ekran her eleman için farklı olabilir. Örneğin LED-RED elemanının özelliklerine girersek aşağıdaki ekran gelecektir. Görüldüğü gibi +VCC'nin özellik penceresinden farklı olarak ledin eleman numarası, sürülme akımı, iletime geçme gerilimi, analog/dijital olması gibi farklı özelliklerini değiştirebilmemize olanak sağlayan farklı bir pencere gelecektir. Buradan eleman özelliklerini istediğiniz şekilde ayarlayabilirsiniz.



Şimdi sayfadaki elemanları devremizi oluşturmak üzere birbirine bağlayalım. Bunun için bazı elamanların döndürülmesi gerekecektir. Aşağıda eleman döndürme araç çubuğu görülmektedir. Buradaki butonları kullanarak elemanları istediğiniz gibi döndürebilirsiniz. Bir elemanı döndürmek için önce o elemanın seçilmiş olması gerekmektedir. Seçme işlemi eleman üzerine bir defa sağ tıklayarak yapılır.



Elemanları uygun konuma istediğiniz açıda yerleştirdikten sonra aralarındaki iletkenleri çizmek için bağlamak istediğiniz elemanın ucuna doğru mouse'u yaklaştırın. Elemanın ucunda x harfi belirecektir. Mouse ile sol tıklayın ve birleştirmek istediğiniz diğer noktaya da bir kez tıklayın. İki eleman otomatik olarak bağlanacaktır. Gerekli bağlantıları yaptıktan sonra ekran aşağıdaki gibi olacaktır.

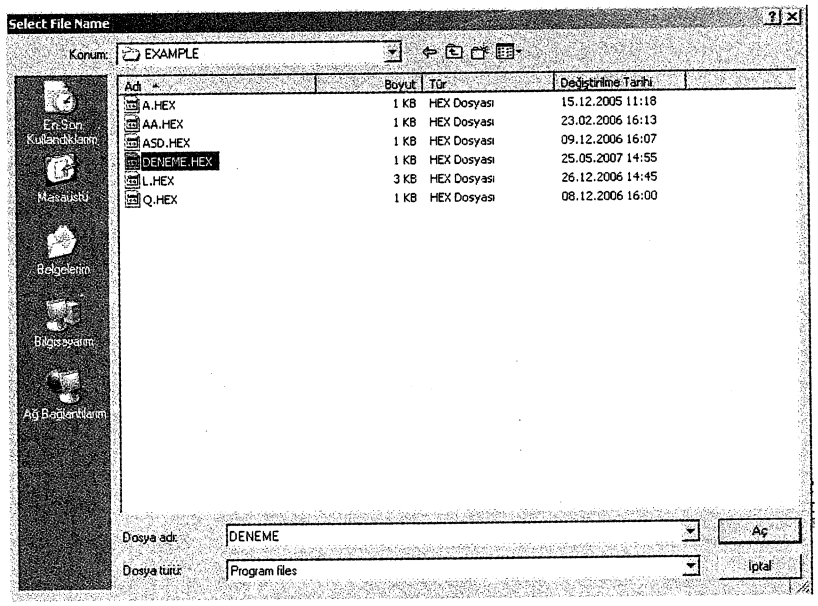


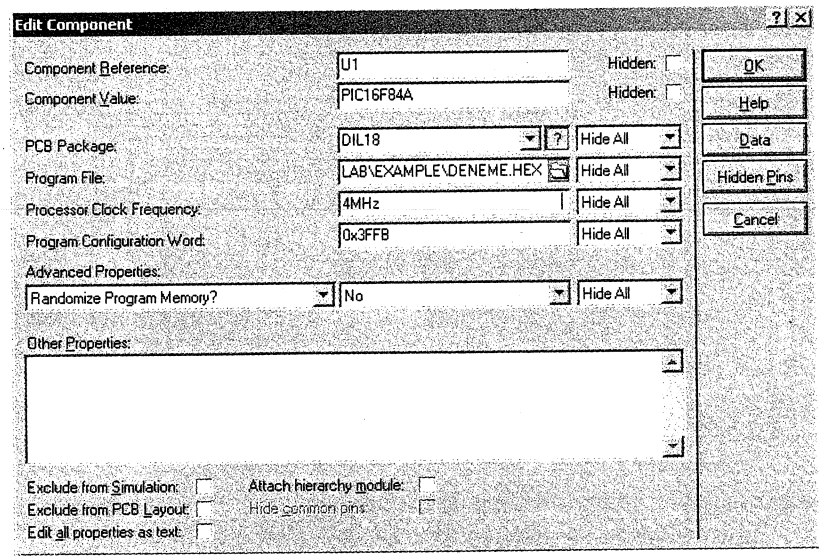
Çizimde osilatör devresinin yer almadığına dikkat ediniz. Proteus programı osilatör devresine ihtiyaç duymaksızın yüklediğiniz programı çalıştırabilir.

Artık sıra PIC'e istediğimiz programı yüklemeye geldi. Bölüm 5'teki Program 8'i deneme.asm adı ile MPLAB'da yazarak kaydedin. Derleme işlemi sonucu **deneme.hex** dosyası oluşturulacaktır.

simgesine tıklayarak devremizdeki PIC16F84 entegresi üzerine sol mouse tuşu ile bir kez tıklayarak Edit Component penceresini ekrana getirin. Burada Program File kısmında PIC'e

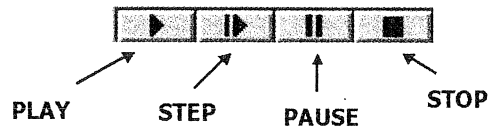
yükleyeceğimiz programı seçiyoruz. simgesine tıklayın. Açılan Select File Name penceresinde hangi dosyayı yüklemek istediğimizi belirtmemiz gerekir. Bunun için deneme dosyasını kaydettiğiniz konumu bulun ve üzerine çift tıklayarak dosyayı seçin ve Edit Component penceresinde Program File kısmına **deneme.hex** dosyasının yazıldığını görün.





Processor Clock Frequency kısmında clock frekansını 4 MHz olarak değiştirin. Ekrandaki görüntü yukarıdaki gibi olacaktır. OK butonuna basarak programı sanal entegremize yüklüyoruz.

Artık yapacağımız tek şey devreyi çalıştırmaktır. Bunun için simülasyon araç çubuğu üzerindeki Play butonuna basalım.



Burada önemli bir hatırlatma yapalım. Simülasyon yapılacağında (component) düğmesinin basılı olması gerekir. düğmesi basılı iken simülasyonu başlatır ve butona basmaya çalışırsanız durum çubuğunda "Real time simulation in progress – Press ESC to stop" mesajıyla karşılaşır ve butona basmanıza izin verilmediğini görürsünüz. Bunun sebebi programın butona basmak değil, butonu düzenlemek (Edit) istediğinizi sanması ve simülasyon yaptığından dolayı buna izin vermemesidir. Bu nedenle STOP düğmesine basarak simülasyonu durdurmanız ve (component) düğmesini aktif hale getirip yeniden PLAY'e basarak simülasyonu baştan başlatmanız gerekmektedir.

Devre çalıştırıldığında önce tüm ledler sönük olacaktır. B0 butonuna bastığınızda alt dört bite bağlı ledlerin, B1'e bastığınızda ise üst dört bite bağlı ledlerin yandığını göreceksiniz. Programı durdurmak için Stop butonuna basın. Simülasyonu adım adım yapmak için Step butonuna, programın çalışmasını o an için durdurmak için ise Pause butonuna basmanız yeterlidir.

Bölüm 17

Cep Telefonu ile Uzaktan Kontrol

Buraya kadar öğrendiğimiz programlama bilgisi basit bir proje yapmaya yetecek düzeydedir. Basit ancak oldukça ilgi çekici olan bu projemiz cep telefonu ile uzaktan kontrol. Kontrol edeceğimiz şey bir cihaz olabileceği gibi bir lamba, bir motor veya buna benzer herhangi bir şey de olabilir.

Projede iki adet cep telefonuna ihtiyacımız var. Bunlardan birini kontrol etmek istediğimiz cihazın üzerine yerleştiriyoruz. Örneğin bir oyuncak araba kontrol edeceksek telefonu araba üzerine yerleştirmeliyiz. Bu telefonun aramalara otomatik olarak cevap verebilme özelliği olması gerekir. Aksi halde elimizde bulunan telefon ile arama yapmamız sabitlediğimiz telefon açılamayacağı için bir işe yaramayacaktır.

Bu projemizi gerçekleştirmek için öncelikle telefonlarda tuş bilgisinin nasıl üretildiğini inceleyelim. Kullandığımız telefonlar tuş bilgisini DTMF kod sistemi kullanarak üretir. DTMF (Dual Tone Multi Frequency) çift tonlu çoklu frekans anlamına gelir. DTMF işareti farklı frekanslı aynı genlikli iki sinüs dalgasının toplamından oluşur. Aşağıdaki tabloda her bir tuş için hangi frekansların toplandığı görülmektedir.

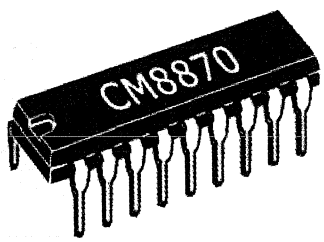
TABLO 17: DTMF Kodlama Sistemindeki Tuş Kombinasyonları ve Frekans Değerleri

	1209 Hz	1336 Hz	1477Hz	1633 Hz
697 Hz	1	2	3	A
770 Hz	4	5	6	B
852 Hz	7	8	9	C
941 Hz	*	0	#	D

Tabloya göre örneğin 1 tuşuna bastığımızda 697 Hz'lik ve 1209 Hz'lik iki sinyal toplanır ve elde edilen sinyal telefon hattı üzerinden santrale gönderilir. Santralde bu sinyal çözülür ve hangi numaraya basıldığı anlaşılır. Aynı şekilde örneğin 9 tuşuna bastığımızda santrale gönderilecek olan frekans 852 Hz ile 1477 Hz'lik sinyallerin toplamıdır.

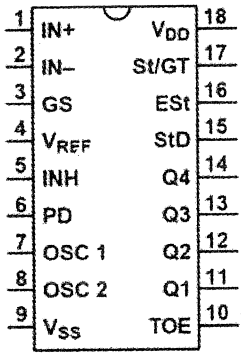
Cep telefonları da sabit telefonlar gibi basılan numarayı DTMF işaretlerini kullanarak gönderir. O halde ilk yapmamız gereken şey cep telefonunun gönderdiği bu DTMF işaretini çözmek ve buna göre hangi tuşa basıldığını tespit etmektir. Yani santralin yaptığı işi bizim için yapacak bir devreye ihtiyacımız var.

Piyasada bu işi yapan DTMF kod çözücü entegreler bulunmaktadır. Bu entegreler girişine uygulanan DTMF işaretini çözer ve binary kod olarak çıkışına aktarır. Projemizde DTMF kod çözücü olarak CM8870 entegresini kullanacağız.



ŞEKİL 53: CM8870 DTMF kod çözücü entegresi.

Aşağıda CM8870 entegresinin pin yapısı görülmektedir. Bu entegre 18 bacaklı bir entegre olup CMOS yapıdadır ve 5 V besleme gerilimi ile çalışır. CM8870 telefon cevaplama, arama kısıtlama, uzaktan kumanda gibi farklı uygulamalarda kullanılmaktadır.



ŞEKİL 54: CM8870 pin görünüşü.

Entegre girişine DTMF işareti uygulandığında Q4 Q3 Q2 Q1 çıkış uçlarında basılan tuş bilgisinin binary karşılığı görülür. Aşağıdaki tabloda hangi frekans çiftinde hangi tuşa basıldığı ve bu tuşa karşılık hangi binary kodun üretildiği görülmektedir.

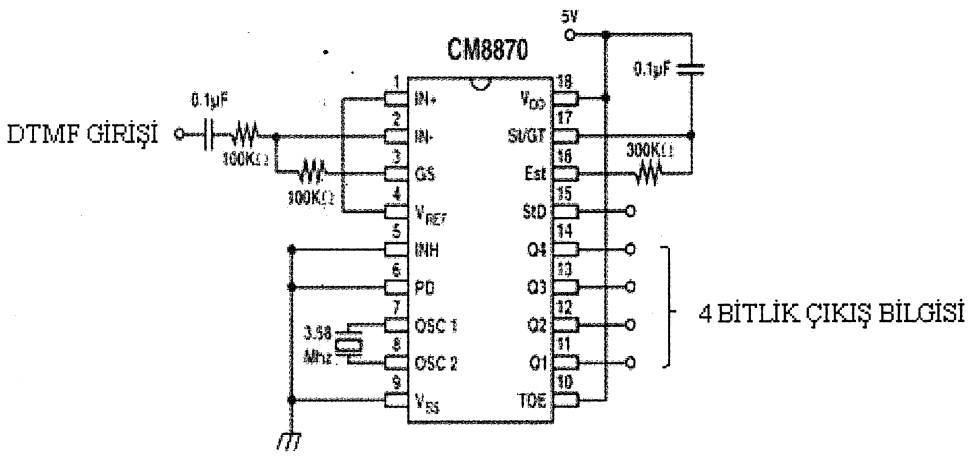
TABLO 18: Basılan Tuşa Göre CM8870 Kodçözücünün Ürettiği Binary Kodlar

F_{LOW}	F_{HIGH}	KEY	TOW	Q_4	Q_3	Q_2	Q_1
697	1209	1	H	0	0	0	1
697	1336	2	H	0	0	1	0
697	1477	3	H	0	0	1	1
770	1209	4	H	0	1	0	0
770	1336	5	H	0	1	0	1
770	1477	6	H	0	1	1	0
852	1209	7	H	0	1	1	1
852	1336	8	H	1	0	0	0
852	1477	9	H	1	0	0	1
941	1209	0	H	1	0	1	0
941	1336	*	H	1	0	1	1
941	1477	#	H	1	1	0	0
697	1633	A	H	1	1	0	1
770	1633	B	H	1	1	1	0
852	1633	C	H	1	1	1	1
941	1633	D	H	0	0	0	0
-	-	ANY	L	Z	Z	Z	Z
L = logic Low, H = Logic High, Z = High Impedance							

Hiçbir tuşa basılmamışsa CM8870'in çıkışları yüksek empedans durumundadır. Çıkışta 16 farklı kod bulunmaktadır. Bunun anlamı her bir kod ile bir cihaz kontrol edeceğimizi varsayarsak 16 farklı cihazı kontrol edebileceğimizdir.

CM8870 entegresi 3.5795 MHz'lik bir kristal ile çalışır. Bu kristal 7 ve 8 numaralı osilatör uçları arasına bağlanır. Entegrenin STD ucu girişine bir tuş bilgisi geldiği sürece 1'dir, tuş bırakıldı-

ğında 0 seviyesine iner. Yani cep telefonundan herhangi bir tuşa basıp basılı tuttuğunuz sürece STD=1, tuşu bıraktığınızda ise STD=0 olacaktır. TOE (Three State Output Enable) ucu İse 0 olduğu sürece çıkış bilgisinin çıkış uçlarından alınmasına izin verir. TOE=1 olursa oluşan tuş bilgisi çıkış uçlarında görülmez. Aşağıdaki şekilde CM8870'in girişteki DTMF işaretini çıkışa 4 bitlik binary kod halinde aktarabilmesi için gereken bağlantı şeması verilmiştir.



ŞEKİL 55: CM8870 entegresinin bağlantı şeması.

Kontrol edeceğimiz aracın üzerine yerleştireceğimiz cep telefonunun kulaklık çıkışına bir kulaklık takarak kulaklığın canlı ucunu DTMF girişi olarak yukarıdaki devrenin girişine bağlıyoruz. Böylece elimizdeki cep telefonundan herhangi bir tuşa bastığımızda üretilen DTMF işareti CM8870'in girişine uygulanacak, böylece bastığımız tuşun binary kodunu elde edebileceğiz. Örneğin telefonun 6 numaralı tuşuna basarsak çıkışta

Q4	Q3	Q2	Q1
0	1	1	0

bilgisi oluşacaktır. Şimdi yapmamız gereken bu binary kodu PIC ile işlemek ve 6 tuşuna basıldığında devremizin ne yapmasını istiyorsak ona göre bir program yazmaktır.

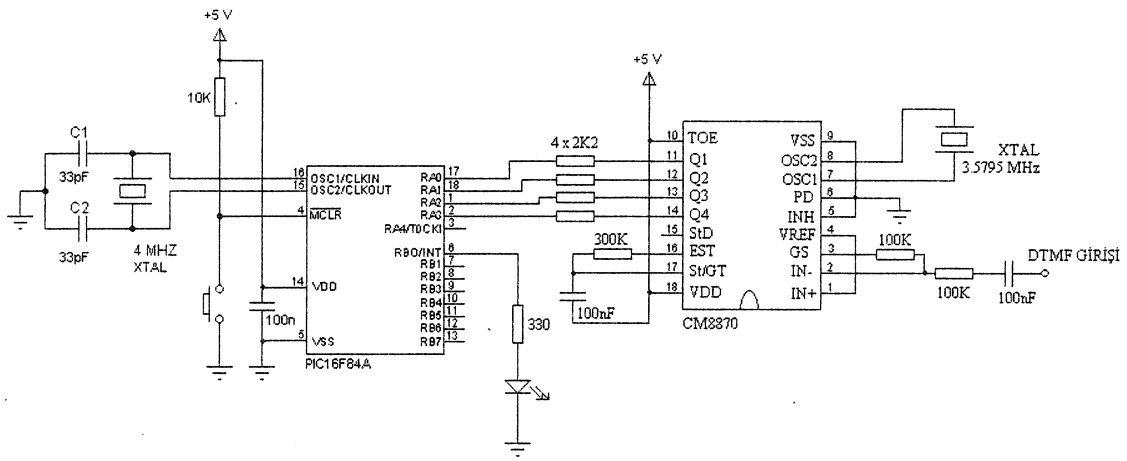
PROGRAM 49: Cep Telefonundan 1 Tuşuna Basıldığında PORTB'nin 0. Bitine Bağlı Ledi Yakan, 7 Tuşuna Basıldığında İse Aynı Ledi Söndüren Program

```
    LIST P=16F84            ; Kullanılacak PIC'i tanıt
    INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
    BSF STATUS,5            ; Bank-1'e geç
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF PORTB              ; PORTB'yi temizle
TUS1
    MOVLW B'00000001'       ; 1 tuşunun binary kodunu W'ya yükle
    XORWF PORTA,W           ; PORTA'dan girilen sayıyı W'daki sayı ile karşılaştır
```

```
    BTFSS STATUS,Z          ; Sayı 1 mi? (Zero flag =1 mi?)
    GOTO TUS1               ; Hayır, 1 tuşunu yeniden kontrol et
YAK
    BSF PORTB,0             ; Evet, ledi yak
TUS2
    MOVLW B'00000111'       ; 7 tuşunun binary kodunu W'ya yükle
    XORWF PORTA,W           ; PORTA'dan girilen sayıyı W'daki sayı ile karşılaştır
    BTFSS STATUS,Z          ; Sayı 7 mi? (Zero flag =1 mi?)
    GOTO YAK                ; Hayır, led yanık kalsın
    BCF PORTB,0             ; Evet, ledi söndür
    GOTO TUS1               ; Yeniden 1'e basılıp basılmadığını kontrol et
    END
```

Bu programda yapmamız gereken şey cep telefonundan 1 ve 7 tuşuna basılıp basılmadığını tespit etmek ve buna göre ledi yakmak veya söndürmektir. CM8870 entegresinin çıkışları Şekil 56'da görüldüğü gibi PIC'e giriş olarak uygulanır. Portlardan veri girişi daha önce anlatılmıştı. Telefondan 1 tuşuna bastığımızda Tablo 18'e göre CM8870 çıkışlarında 0001 bilgisi meydana gelecektir. Programda 1 tuşuna basılıp basılmadığını anlamak için **PORTA**'dan gelen bilgi ile B'00000001' bilgisini karşılaştırılır. Karşılaştırma için ÖZEL VEYA işleminden faydalandık. Bilgiler birbirine eşit ise XOR işleminin sonucunda 0 bilgisi elde edilecek, bu durumda Zero Flag = 1 olacaktır. ZF = 1 olduğu zaman çıkıştaki led yakılır.

Ledi yaktıktan sonra bu kez yine aynı yöntemle 7 tuşuna basılıp basılmadığı kontrol edilir.. Eğer basılmışda ledi söndürmek için **PORTB**'nin 0. bitine 0 gönderilir. Basılmamışsa ledin yanık kalması için program **YAK** etiketine dallandırılır.

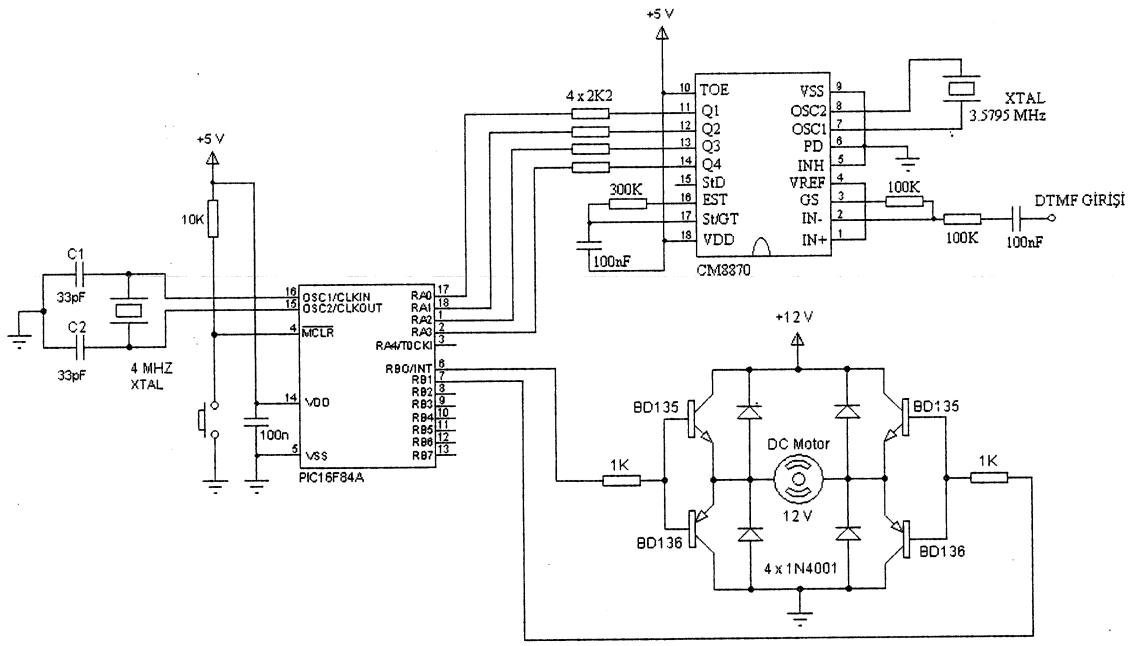


ŞEKİL 56: Cep telefonu ile bir ledi yakıp söndürebilmek için kurulması gereken devre.

PROGRAM 50: Cep Telefonundan 4 Tuşuna Basıldığında bir DC Motoru Sola, 6 Tuşuna Basıldığında Sağa Doğru Döndüren, 0 Tuşuna Basıldığında İse Motoru Durduran Program

```
LIST P=16F84            ; Kullanılacak PIC'i tanıt
INCLUDE "P16F84.INC"    ; INCLUDE dosyasını yükle
BSF STATUS,5            ; Bank-1'e geç
```

```
    CLRF TRISB              ; PORTB'yi çıkış olarak yönlendir
    BCF STATUS,5            ; Bank-0'a geri dön
    CLRF PORTB              ; PORTB'yi temizle
TUS4
    MOVLW B'00000100'       ; 4 tuşunun binary kodunu W'ya yükle
    XORWF PORTA,W           ; PORTA'dan girilen sayıyı W'daki sayı ile karşılaştır
    BTFSS STATUS,Z          ; Sayı 4 mü?
    GOTO TUS6               ; Hayır, 6 tuşunu kontrol et
    MOVLW B'00000010'
    MOVWF PORTB             ; Evet, motoru sola döndür
TUS6
    MOVLW B'00000110'       ; 6 tuşunun binary kodunu W'ya yükle
    XORWF PORTA,W           ; PORTA'dan girilen sayıyı W'daki sayı ile karşılaştır
    BTFSS STATUS,Z          ; Sayı 6 mı?
    GOTO TUS0               ; Hayır, 0 tuşunu kontrol et
    MOVLW B'00000001'
    MOVWF PORTB             ; Evet, motoru sağa döndür
TUS0
    MOVLW B'00000000'       ; 0 tuşunun binary kodunu W'ya yükle
    XORWF PORTA,W           ; PORTA'dan girilen sayıyı W'daki sayı ile karşılaştır
    BTFSS STATUS,Z          ; Sayı 0 mı?
    GOTO TUS4               ; Hayır, 4 tuşunu kontrol et
    CLRF PORTB              ; Evet, motoru durdur
    GOTO TUS4               ; Yeniden 4'e basılıp basılmadığını kontrol et
    END
```



ŞEKİL 57: Cep telefonu ile bir DC motorun yön kontrolünü yapabilmek için kurulması gereken devre.

DC motorun yön kontrolünün nasıl yapılacağı DC Motor Yön Kontrolü bölümünde anlatılmıştı. Bu programdaki farklılık, girişten gelen bilgiye göre motorun sağa veya sola çevrilmesi-

dir. Programda yapmamız gereken ilk şey cep telefonundan 4 veya 6'ya basılıp basılmadığını kontrol etmektir. 4 için CM8870 entegresi 0100, 6 için ise 0110 kodunu üretmektedir. Bu bilgileri W içerisine yükleyip **PORTA**'dan gelen bilginin bu bilgilere eşit olup olmadığını kontrol ediyoruz. Eğer 4 tuşuna basılmışsa motoru sola çevirebilmek için RB0=0 ve RB1=1 yapılır. Daha sonra 6 tuşu ve 0 tuşu kontrol edilir. Her ikisine de basılmamışsa motor sola dönmeye devam eder. 6'ya basılmışsa RB0=1 ve RB1=0 yapılarak motor sağa döndürülmeye başlar, 0'a basılmışsa RB0=0 ve RB1=0 yapılarak motor durdurulur.

Kaynaklar

1. Altınbaşak, Orhan. Mikrodenetleyiciler ve PIC Programlama, Altaş Yayınları, 2004
2. Karakaş, Hakan. İleri PIC16F84 Uygulamaları, Altaş Yayınları, 2003
3. **http://www.mikroe.com/en/books/PICbook/PICbook.htm**
4. **http://ww1.microchip.com/downloads/en/DeviceDoc/35007b.pdf**
5. Bereket, Metin., Tekin, Engin. Atelye ve Laboratuar 2, Mavi Kitaplar, 2004

Dizin

.asm dosyası 4
; işareti 40
__CONFIG ifadesi 52
16 bitlik çıkarma 135
16 bitlik toplama 130
74C922 klavye kod çözücü entegresi 157
7-segment display 82
8 bitlik çıkarma 133
8 bitlik toplama 128
8-bitlik mikrodenetleyici 8
A Portu 10
açık kanal 10
açıklama satırı 40
ADDLW komutu 27, 36, 128
ADDWF komutu 27, 28, 128
adım 102
adım motor 102
adres atanan etiket 41
adres atanmayan etiket 41, 42
adres sayıcı 169
adres yolu 2, 8
adresi okumak 169
akım 8
akış diyagramı 52
alt program 50
ALU 18
ana programın yazılması 49
anakart 2
AND Kapısı 54
ANDLW komutu 27, 37, 87, 108
ANDWF komutu 27, 28
ARES alt programı 196
aritmetik işlemler 2, 128
Aritmetik Lojik Ünite 18
artırma 79
assembler 4
assembly 24
atlama 21
Azaltılmış Komut Seti 8
B Portu 10
bank 16
Bank Değiştirme 47
baskı devre 2
başlangıç adresi 15
BCF 24
BCF komutu 27, 35, 48
bellek 3, 15
bellek ünitesi 2
besleme devresi 11
besleme gerilimi 8
bilgisayar 2, 3
binary sayı 24
binary sayının decimal sayıya çevrimi 56
binary sayının hexadecimal sayıya çevrimi 58
bir register'daki bilgiyi başka bir register'a yüklemek 60
Bit Clear File Register 24
bit kaydırma 97
blok 15
blok diyagram 4
BSF komutu 27, 35, 48, 49
BTFSC komutu 27, 36, 67
BTFSS komutu 27, 36, 67, 142
Build Results penceresi 186
buton arkı 76
çalışma aralığı 8
CALL komutu 27, 37, 50
çarpma işlemi 137
Carry Flag 97, 128
CBLOCK direktifi 42
Central Processing Unit 2
cep telefonu ile uzaktan kontrol 208
Çevre Birimleri Denetleyici 4
çevrim tablosu 83
CGRAM adres ayarı 167
CGRAM'den okumak 169
CGRAM'e yazmak 169
çift döngülü gecikme 75
çıkarma 18
çıkarma işlemi 133
çıkış portunun temizlenmesi 49
çıkış ünitesi 2
çip 2
CISC 8
clock sinyali 12
CLRF komutu 27, 29, 49
CLRW komutu 27, 29
CLRWDT komutu 27, 37
CM8870 entegresi 210
CMOS teknolojisi 8
çok butonlu uygulamalar 68
COMF komutu 27, 29, 133
Complex Instruction Set Computer mikrodenetleyici 8
CP 51
CPU 2, 4
dallanma 21
Darbe Genişlik Modülasyonu 122
DC motor 118
DC motor hız kontrolü 121

DC motor yön kontrolü 119
DDRAM Adres Ayarı 167
DDRAM'den okumak 169
DDRAM'e yazmak 169
DECF komutu 27, 30, 73, 111
DECFSZ komutu 27, 30, 73
decimal sayının binary sayıya çevrimi 55
decimal sayının hexadecimal sayıya çevrimi 56
değişken 15
deneme kartı 4
derleyici 4
display 162
donanım register'ı 8
dot matrix LCD 162
DTMF kod sistemi 208
Dual Tone Multi Frequency 208
düşük enerji modu 9
duty cycle 121
Edit Project penceresi 182, 184
EECON1 register 18
EEIE biti 146
EEIF biti 146
EEPROM bellek 8, 9
EEPROM kesmesi 146
ekran On/Off kontrolü 166
ekranı temizlemek 164
elde bayrağı 128
Electrically Erasable Programmable Memory 9
END direktifi 51
ENDC direktifi 42
entegre 2, 3
EQU direktifi 41
EQU komutu 73
etiket 41
EXOR kapısı 55
FIFO 144
file register 15
First In First Out 144
flash bellek 15, 20
Flash bellek 8
fonksiyon seti 167
gecikme 76
gecikme alt programı 74
Genel Amaçlı Register 16
geri sayıcı 72
GIE biti 141
Giriş Modu 166
giriş ünitesi 2
giriş/çıkış 46
giriş/çıkış portları 9
giriş-çıkış 3
GOTO komutu 27, 37, 50, 63
GPR 16
grafik LCD 162
Harvard mimarisi 8
HD44780 sürücü entegresi 162
hexadecimal sayı 24
hexadecimal sayının binary sayıya çevrimi 57
hexadecimal sayının decimal sayıya çevrimi 57
hız kontrolü 121
HS osilatör 12
I/O 3, 46
ICPROG 4
ICPROG programı 188
ileri sayıcı 78
INCF komutu 27, 79, 111
INCF KOMUTU 31
INCFSZ komutu 27, 79
INCFSZ KOMUTU 31
INCLUDE dosyası 41
Include dosyasının yüklenmesi 43
INCLUDE komutu 46
Input 2
INT kesmesi 142
INTCON register 18
INTCON register'ı 141
INTE biti 142
INTEDG biti 142
Intel 4
interrupt 140
interrupt vektörü 15
INTF biti 142
IORLW komutu 27, 38
IORWF komutu 27, 32
iş çevrimi 121
ISIS alt programı 196
işlemci 2
kaydedici 15
kaydırma 97
kelime uzunluğu 8
kesme 8, 15, 140
kesme alt programı 140
kesme kaynağı 141
kesme sırasındaki bilgileri saklamak 146
kesme türleri 142
kesme vektörü 15
klavye 150
klavye entegresi 157
kod çözücü 82
kod koruması 9, 51
komut 8
komut saykılı 40
komut seti 24
konfigürasyon bitleri 51
kristal osilatör 13
kursör/display kaydırma 167
kursörü başa getirmek 165
LCD 162
LCD komutları 163
LCD'nin kullanıma hazırlanması 170

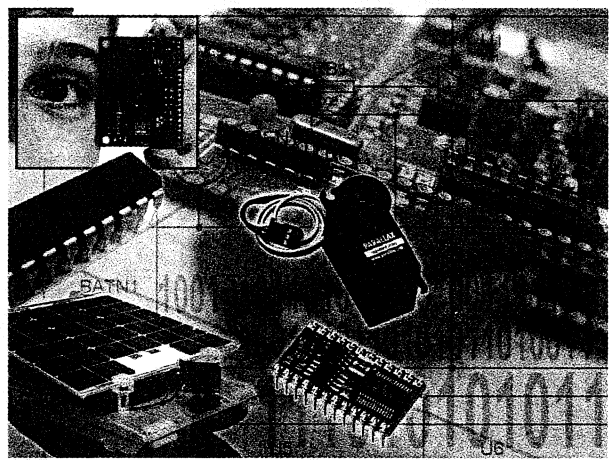
led 82
Likit Kristal Display 162
LIST direktifi 43
lojik işlemler 2
lojik kapı 54
LP osilatör 12
makina dili 24
maskeleme 87, 108
Master Clear ucu 12
MCLR ucu 12
Memory 2
Merkezi İşlem Birimi 2
meşgul bayrağını okumak 169
metin editörüne 4
Microchip 4
mikrodenetleyici 2, 3
mikroişlemci 2
mnemonic 24
motor 102
motor kontrolü 102
Motorola 4
motorun torku 106
MOVF komutu 27, 32, 60
MOVLW komutu 27, 38, 60
MOVWF komutu 27, 32, 60
MPLAB 4
MPLAB programı 180
NOP komutu 27, 33, 158
Not Defteri 4
OPCODE 24
open drain 10
OPERAND 24
OPTION_REG register 18
OPTION_REG register'ı 142
OR kapısı 54
ORG komutu 140
ortak anotlu display 82
ortak katotlu display 82
OSC 51
osilatör 40, 51
osilatör devresi 12
osilatör frekansı 8
osilatör periyotu 40
Output 2
Özel Amaçlı Register 15
ÖZEL VEYA kapısı 55, 91
PC 3
PCL register'ı 21
PCLATH register'ı 21
Peripheral Interface Controller 4
PIC 4
PIC Assembly dili 24
PIC programlama 4
PIC programlama kartı 188
PIC'in tanıtılması 43
PIC16C84 9
PIC16F84 ..... 4, 8
PIC16F84A 9
PORTA 10
PORTB 10
PORTB değişim kesmesi 145
portlara veri gönderme 60
portların giriş-çıkış olarak yönlendirilmesi 46
Power-up Timer 51
program belleği 15, 20
program formatı 42
program sayıcı 21
programın denenmesi 4
programın derlenmesi 4
programın PIC'e yüklenmesi 4
programın sonlandırılması 50
programın yazılması 4
programlama kartı 4
Proteus 4
PROTEUS programı 196
pull-up direnci 10
Pulse Width Modulation 122
PWM 122
PWM metodu 121
PWRT 51
RAM bellek 4, 8, 15, 20
RB0/INT pini harici kesmesi 142
RBIE biti 145
RC ösilatör 13
Reduced Instruction Set Computer 8
register 15
register haritasında 15
register'lara veri gönderme 60
reset devresi 12
reset işlemi 12
RETFIE komutu 27, 38, 140
RETLW komutu 27, 38, 84, 106
RETURN komutu 28, 38, 50
rezonatör 14
RISC mikrodenetleyici 8
RLF komutu 27, 33, 97
RRF komutu 27, 33
RRF komutu 97
sabit veriyi register'a yüklemek 60
sağa kaydırma 97
sayı sistemlerinin birbirine çevrilmesi 55
sayıcı 9, 63
sayıcılar 72
sayıların yazılış formatı 40
saykıl 40
segment 82
seramik rezonatör 14
SFR 15
sıfır bayrağı 81
simülasyon 205

sink akımı 10
SLEEP komutu 28, 39
SLEEP modu 9
sola kaydırma 97
sonsuz döngü 50
source akımı 10
stack register 8
STACK register 50
STACK register'ı 140
STATUS register 18
step motor 102
step motorların sürülmesi 104
step motoru belirlenen adım sayısı kadar döndürme 112
step motoru belirlenen tur sayısı kadar döndürme 115
step motoru durdurmak 112
step motorun 1 fazlı sürülmesi 104
step motorun 2 faz - tam adımlı sürülmesi 105
step motorun 2 faz - yarım adımlı sürülmesi 106
SUBLW komutu 28, 39, 133
SUBWF komutu 27, 81, 133
sürücü devresi 103
SWAPF komutu 27, 34, 147
T0IF bayrağı 146
tarama metodunda 150
taşma 132
tek döngülü gecikme 74
tersleme 92
TMR0 kesmesi 146
toplama 18
toplama işlemi 128
tork 106
TRIS register'ları 47
TRISA register'ı 47
TRISB register'ı 47
tuş takımı 150
üç döngülü gecikme 75
ULN2003A entegresi 103
VE Kapısı 54
veri aktarımı 4
veri belleği 8, 15, 20
veri girişi 4
veri yolu 2, 8
VEYA kapısı 54
Von Neumann mimarisi 8
W register 20
W register'ı 60
Watchdog Timer 9, 37, 51
WDT 9, 37, 51
working register 20
XOR kapısı 91
XORLW komutu 28, 39, 91, 92
XORWF komutu 27, 35, 91
XT osilatör 13
yardımcı ünite 3
yardımcı üniteler 2
yön kontrolü 119
zamanlayıcı 9
zero flag 81
zıt E.M.K 120

Elektronik Hobi

Bilgisayarlar ve mikroişlemciler ile dış dünyaya hükmedin!

- Elektronik hobi dünyasına ilk adımı atın.
- Bilgisayarınızla evinizdeki cihazları kontrol edin.
- Bilgisayar ve mikroişlemci elektroniğini uygulayarak öğrenin.
- Kendi robotunuzu tasarlayın.
- Paralel port uygulamaları
- Adım motor, servo motor ve DC motor kontrolleri
- LCD panel ve röle kontrolleri
- Bilgisayar kontrollü hırsız alarmı, termometre, robot göz ve model araba
- İnternet üzerinden otomasyon
- LINUX'la otomasyon
- PIC programlama ve uygulamaları
- PIC Basic uygulamaları ve BASIC tabanlı mikroişlemciler
- Seri port uygulamaları
- Radyo ve kızılötesi kontrollü uygulamalar
- Basit dedektörler
- Çizgi izleyen robot tasarımı
- Ultrasonik mesafe ölçen robot tasarımı
- Mobil Araştırma Robotu projesi



Güçlü Tuğay

Herkes İçin!®

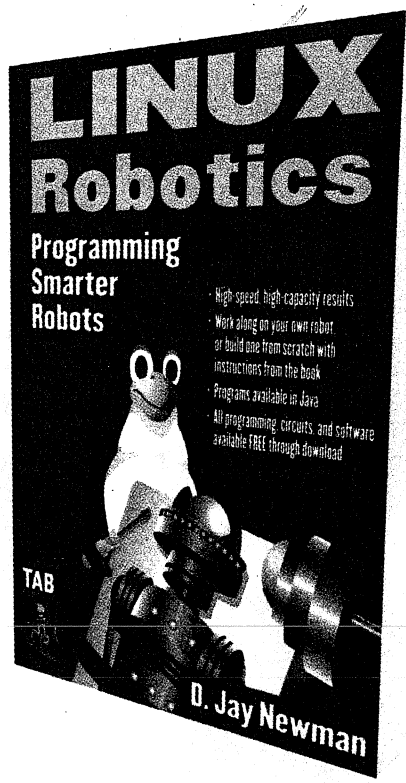
ALFA®

Robotlara meraklıysanız kaçırmamanız gereken bir kitap!

Linux'un gücünü ve esnekliğini kullanarak isterseniz yeni bir robot yapın, isterseniz mevcut robotunuzu geliştirin.

- Robotlarla tanışın.
- Robot elektroniğinin temellerini öğrenin.
- Robotun iç iletişim sistemini ve robot denetim ağını tanıyın.
- Robotun duyu organları olan sensörleri öğrenin.
- Davranışsal programlamaya giriş yapın.
- Konuşmadan metne ve metinden konuşmaya dönüşüm mekanizmalarıyla robotunuzla iletişim kurun.
- Kamera kullanarak robotunuzun dünyayı [illegible] öğrenin.
- Haritа oluşturma algoritmalarını tanıyın.
- Yapay sinir ağlarıyla çalışın.
- Robotla kablosuz iletişimin temellerini öğrenin.

Linux ile Robotik



D. Jay Newman

Herkes İçin!®

OSBORNE
MC Graw Hill

ALFA®